

自然语言处理基础 课程笔记

25-26 学年第 2 学期·Yansong Feng @ PKU

AutoPku

2026-05-17

Contents

自然语言处理基础—课程笔记	6
0. 课程脉络	6
1. 课程概览	6
2. 中英文术语对照（全局精选）	7
Lecture 0: 课程导引与教学大纲（Course Syllabus）	9
0.1 课程基本信息	9
0.2 考核与评分	9
0.3 教学目标	10
0.4 先修知识自查	10
0.5 参考资料	10
0.6 Syllabus（教学路线图）	11
0.7 中英术语对照	12
Lecture 1: NLP 导论与语言的歧义性（Introduction to NLP）	13
1.1 Motivation：为什么研究语言？	13
1.2 什么是语言？	13
1.3 什么是 NLP？	14
1.4 NLP 应用全景	16
1.5 为什么语言难？——歧义无处不在	16
1.6 NLP 的方法论	19
1.7 例题 / 案例练习	20
1.8 概念关系总图	21
1.9 易错点汇总	22
1.10 本节小结	22
1.11 中英术语对照	22
Lecture 2 一词义消歧（Word Sense Disambiguation, WSD）	24
2.1 Motivation：为什么需要 WSD	24
2.2 形式化定义	25
2.3 三种学习范式	26
2.4 核心算法详解	27
2.5 监督 WSD 的特征工程	30
2.6 评测	30
2.7 WSD 的现代化路径	31
2.8 概念关系图	32
2.9 易错点速查	33
2.10 中英术语对照	33
Lecture 3a 一文本分类与概率模型（Classification & Probabilistic Models）	35
3.1 Motivation：从 WSD 到通用分类	35
3.2 形式化：生成式模型	36
3.3 生成式典范：Latent Dirichlet Allocation (LDA)	37
3.4 判别式模型	38

3.5 Perceptron: 第一个判别式线性分类器	38
3.6 特征工程入门	39
3.7 例题: NB vs Perceptron 对比	40
3.8 评测、调参与协议	41
3.9 生成式 vs 判别式的统一对照	42
3.10 易错点速查	42
3.11 中英术语对照	42
Lecture 3b 一对数线性模型 (Log-Linear Models / Maximum Entropy)	44
4.1 Motivation: 从 NB 到判别式的需求	44
4.2 特征表示 (Feature Representation)	45
4.3 模型形式	47
4.4 训练: 极大似然估计 (MLE)	48
4.5 优化算法	49
4.6 正则化 (Regularization)	51
4.7 训练循环 (图示)	53
4.8 与其他模型的关系	54
4.9 评测复盘	54
4.10 例题: log-linear vs NB	55
4.11 概念关系图 (mermaid)	56
4.12 易错点速查	57
4.13 中英术语对照	57
Lecture 4 一神经语言模型 (Neural Language Models)	59
1. Motivation: 从 n-gram 走向“分类式”LM	59
2. 神经网络基础回顾	60
3. NLM-1: Bengio 2003 前馈神经 LM	60
4. NLM-2: 卷积神经网络 LM	63
5. NLM-3: 循环神经网络 LM	63
6. 概念对照与统一视角	65
7. 概念关系图 (mermaid)	66
8. 易错点	68
9. 中英术语对照	68
Lecture 4 一N-gram 语言模型 (N-gram Language Models)	70
4.1 Motivation: 为什么需要语言模型?	70
4.2 模型定义	71
4.3 参数估计: MLE 推导	72
4.4 平滑与插值: 处理未见事件	73
4.5 评测: 困惑度 (Perplexity)	75
4.6 Worked Example	76
4.7 概念关系图	79
4.8 易错点汇总	80
4.9 中英术语对照	80
Lecture 5 一分布式语义 (Distributional Semantics)	82
5.1 Motivation: 词的“意义”是什么?	82
5.2 Distributional Hypothesis (分布假说)	83
5.3 向量空间模型 (Vector Space Model, VSM)	83
5.4 PMI 与 PPMI (信息论加权)	84
5.5 TF-IDF: 另一类加权	85
5.6 维度归约: SVD / LSA	86
5.7 神经预测式: word2vec	87
5.8 GloVe: 加权最小二乘 + 计数	88
5.9 评测	89
5.10 多义性与上下文相关表示 (过渡)	90
5.11 Worked Example: 手算 PPMI	90

5.12 概念关系图 (mermaid)	92
5.13 易错点	93
5.14 中英术语对照	93
Lecture 6 (I): 隐马尔可夫模型与序列标注 (Hidden Markov Models for Sequence Tagging)	95
6.1 序列标注问题 (Sequence Tagging)	95
6.2 词性标注问题的概率建模	96
6.3 隐马尔可夫模型 (HMM)	97
6.4 问题一: Likelihood ——Forward / Backward	98
6.5 问题二: Decoding ——Viterbi 算法	99
6.6 问题三: Learning ——MLE 与 Baum-Welch	101
6.7 Worked Example: 3-state Toy HMM Viterbi	102
6.8 HMM 与同类模型对比	103
6.9 概念关系图	104
6.10 易错点 (Pitfalls)	104
6.11 中英术语对照	105
Lecture 6 (II): Beyond HMM ——MEMM、CRF 与神经序列标注 (Sequence Models beyond HMM)	107
7.1 动机: HMM 的两个“硬伤”	107
7.2 Label Bias 问题 (Lafferty et al. 2001)	108
7.3 线性链 CRF (Linear-Chain Conditional Random Field)	109
7.4 推断 (Inference) ——Forward & Viterbi over Potentials	111
7.5 训练 (Learning) ——Log-Likelihood 梯度	111
7.6 HMM vs MEMM vs CRF 总对照	113
7.7 Neural Sequence Tagging	113
7.8 Worked Example: CRF 训练梯度直觉	114
7.9 Seq2Seq 视角下的序列标注 (一句话总结)	114
7.10 概念关系图	115
7.11 易错点 (Pitfalls)	115
7.12 中英术语对照	116
Lecture 6b: 序列到序列模型与注意力机制 / Seq2Seq & Attention	117
6b.1 Motivation —为什么需要 Seq2Seq	117
6b.2 形式化定义 (Formalization)	118
6b.3 Attention —注意力机制	119
6b.4 Decoder Variants —AT / NAT	120
6b.5 训练 (Training)	121
6b.6 推理 (Inference / Decoding)	122
6b.7 重要技巧 (Tips)	124
6b.8 Worked Example —Beam Search, B=2, 3 步	124
6b.9 概念关系图 (Mermaid)	126
6b.10 易错点 (Common Pitfalls)	126
6b.11 中英术语对照	126
Lecture 7: 短语结构、上下文无关文法与 CKY 句法分析 (Phrase Structure, CFG/PCFG, and CKY Parsing)	128
7.1 为什么需要“树”——超越序列	128
7.2 Constituent (成分)	129
7.3 形式语言 (Formal Languages) 与 文法 (Grammars)	129
7.4 上下文无关文法 (CFG)	130
7.5 Chomsky Normal Form (CNF)	131
7.6 Probabilistic CFG (PCFG)	132
7.7 CKY 算法 (Cocke-Kasami-Younger)	133
7.8 Worked Example: CKY 解析 “I saw the boy with a telescope” (节选)	136
7.9 Inside 算法 (边际概率)	137
7.10 评测: Labeled Precision / Recall / F1 (EvalB)	138
7.11 概念关系图	139

7.12 易错点 (Pitfalls)	139
7.13 中英术语对照	140
Lecture 8: 依存句法分析 (Dependency Parsing)	141
8.1 Motivation: 为什么需要依存结构?	141
8.2 形式化定义	142
8.3 解析算法	143
8.4 评测	147
8.5 Worked Example: Arc-Standard Trace	148
8.6 概念关系图	149
8.7 易错点 & 与同类对比	152
8.8 中英术语对照	152
Lecture 9: 意义表示与组合语义 (Meaning Representation and Compositional Semantics)	154
9.1 Motivation: 从词向量到句子意义	154
9.2 形式语义学 (Formal Semantics)	155
9.3 一阶逻辑 (First-Order Logic, FOL)	156
9.4 λ 演算 (Lambda Calculus)	157
9.5 句法驱动的语义分析 (Syntax-Driven Semantic Analysis)	158
9.6 「看上去 trivial」实际并非: 组合性的反例	161
9.7 Worked Examples	161
9.8 概念关系图	163
9.9 易错点 & 与同类对比	163
9.10 中英术语对照	164
Lecture 10 (II) —IR Embedding (向量检索)	166
1. 为什么需要 Embedding Learning	166
2. Formulation: Representation-centric 视角	167
3. 双塔架构 (Dual Encoders)	167
4. 近似最近邻搜索 (ANNS)	168
5. 训练: Contrastive Learning / Learning to Rank	169
6. 难负例 (Hard Negatives): 本讲的真正主题	170
7. 总结: Embedding Learning 的设计哲学	172
8. 章节内速查表	172
9. 整体关系图	173
10. 期末复习要点	173
11. 参考文献 (课件 Readings)	174
Lecture 10 (Sparse IR): 信息检索之稀疏方法 / Sparse Information Retrieval	175
10.1 Motivation —为什么需要 IR	175
10.2 形式化定义 (Formalization)	176
10.3 关键模型 (Retrieval Methods)	176
10.4 倒排索引 (Inverted Index) —系统视角	180
10.5 评测 (Evaluation)	181
10.6 Worked Example —BM25 手算	182
10.7 概念关系图 (Mermaid)	183
10.8 易错点 (Common Pitfalls)	184
10.9 中英术语对照	184
Lecture 11 —预训练模型 (Pretrained Language Models)	186
1. Motivation: 为什么要预训练	186
2. ELMo: 上下文相关表示的起点	187
3. Pre-training & Fine-tuning 范式	188
4. Transformer 架构核心回顾	188
5. BERT: 双向编码器表示	189
6. RoBERTa: 把 BERT 训得更彻底	192
7. Encoder-only 的局限	192

8. T5: Text-to-Text Transfer Transformer	192
9. GPT 系列: 自回归解码器	193
10. 模型家族 mermaid 图	195
11. 下游适配范式	195
12. 关键模型对比表	196
13. 期末复习要点	196
Lecture 11b —预训练之后: 从 GPT 到 GPT-2 / GPT-3 (Scaling & the Shift to Prompting)	198
1. Motivation: 为什么要继续放大	198
2. GPT-1 / 2 / 3: 参数·数据·时间线对比	199
3. GPT-2: 语言模型是无监督多任务学习者	199
4. GPT-3: In-Context Learning	200
5. Scaling Laws: 缩放律	201
6. Emergent Abilities: 涌现能力	202
7. 为什么 Scaling 选了 Decoder-only	202
8. 范式转变: Fine-tuning → Prompting	203
概念关系	204
记号速查	204
Anki 卡片	205
Lecture 12 —提示学习 (Prompting)	207
1. 什么是 Prompting / Motivation	207
2. Prompt 基本 workflow (Basic Workflow)	208
3. Prompt 设计策略 (Designing Strategy)	209
4. Few-shot Prompting 与 In-Context Learning (ICL)	209
5. Chain of Thought (CoT)	210
6. Prompt Engineering (提示工程)	211
7. Retrieval-Augmented Generation (RAG)	212
8. Take-home Message	214
概念关系	214
记号速查	214
Anki 卡片	215
Lecture 13 —训练大语言模型的数据 (Data for Training LMs)	217
0. 名词预热: 读这篇笔记前需要知道的 5 个词	217
1. Motivation: 为什么单独花一讲讲数据?	218
2. 预训练数据的演进史	218
3. 开源预训练数据集全景	220
4. 数据规模 vs 模型规模: 缩放律的另一半	221
5. 数据治理流水线: 三大步骤总览	223
6. 步骤 ①: 获取 (Acquisition)	223
7. 步骤 ②: 线性化 (Linearization)	224
8. 步骤 ③: 过滤 (Filtering)	225
9. 测验解析: 三类过滤的执行顺序 (高频考点)	227
10. 总结与展望	228
11. 例题 (Worked Examples)	229
12. Anki 记忆卡片	230
13. 记号与术语速查	231

自然语言处理基础—课程笔记

[TIP] 课程定位

本课程是 PKU 计算机/信科类的 NLP 入门核心课。由经典统计 NLP（语言模型、词义消歧、HMM、句法分析、语义表示）一路推进到向量检索与预训练大语言模型范式。期末考会涵盖语言建模、对数线性模型、HMM/CRF、PCFG/CKY、依存句法、语义/统计 MT、向量检索与预训练。

0. 课程脉络



1. 课程概览

周	讲次	主题	关键概念	笔记
0	Lec 0	课程导论	评分、参考书、先修	lec00
1	Lec 1	NLP 总览	任务谱系、五级歧义、Zipf	lec01
2	Lec 2	词义消歧 (WSD)	Lesk、MFS、有监督 NB	lec02
3	Lec 3	分类基础	NB、Perceptron、生成 vs 判别	lec03_classification
3	Lec 3	对数线性 / MaxEnt	softmax 形式、empirical — expected 梯度	lec03_loglinear
4	Lec 4	n-gram LM	MLE、平滑、Kneser–Ney、PPL	lec04_ngram_lm
4	Lec 4	神经 LM	Bengio FFNN、RNN–LM、BPTT	lec04_neural_lm
5	Lec 5	分布式语义	共现/PPMI/SVD、word2vec、GloVe	lec05_distributional
6	Lec 6	HMM	forward / backward / Viterbi / Baum–Welch	lec06_hmm
6	Lec 6	序列标注 CRF	MEMM 标签偏置、线性链 CRF	lec06_seq_crf
6b	Lec 6b	seq2seq + Attention	Bahdanau / dot / beam search	lec06b_seq2seq
7	Lec 7	短语句法	CFG、CNF、PCFG、CKY 算法	lec07_phrase_cfg
8	Lec 8	依存句法	arc–standard / Eisner / MST	lec08_dependency
9	Lec 9	语义表示	FOL、 λ -calculus、Davidson 事件	lec09_meaning
10 (I)	Lec 10	稀疏信息检索	Boolean、TF–IDF、BM25、倒排索引	lec10_ir_sparse

周	讲次	主题	关键概念	笔记
10 (II)	Lec 10	稠密 IR Embedding	双塔、InfoNCE、ANNS	lec10_ir_embedding
11	Lec 11	预训练语言模型	Transformer / BERT / GPT / T5	lec11_pretraining
11b	Lec 11b	GPT 缩放与范式转变	GPT-2/3、缩放律、涌现、ICL	lec11b_gpt_scaling
12	Lec 12	提示学习	Prompting、ICL、CoT、RAG	lec12_prompting
13	Lec 13	训练 LM 的数据	数据治理流水线、C4/Dolma/FineWeb/DCLM、去重/过滤	lec13_data_for_lms

既有 .tex 笔记 (fnlp_notes.tex, fnlp_midterm_review.tex) 是期中知识点压缩版；本目录的 lec*.md 是详细展开版，含推导、worked example、mermaid 图、callout、术语表。

2. 中英文术语对照（全局精选）

中文	English	首次出现
词义消歧	Word Sense Disambiguation (WSD)	Lec 2
朴素贝叶斯	Naïve Bayes	Lec 3
感知机	Perceptron	Lec 3
对数线性模型	Log-linear / MaxEnt	Lec 3
n 元语法	n-gram	Lec 4
困惑度	Perplexity (PPL)	Lec 4
平滑	Smoothing (Add-k / Kneser-Ney)	Lec 4
神经语言模型	Neural Language Model	Lec 4
反向传播时间	BPTT	Lec 4
分布假设	Distributional Hypothesis	Lec 5
共现矩阵	Co-occurrence Matrix	Lec 5
正点互信息	PPMI	Lec 5
词向量	Word Embedding	Lec 5
隐马尔可夫模型	HMM	Lec 6
前向/后向算法	Forward / Backward	Lec 6
维特比算法	Viterbi	Lec 6
Baum-Welch	Baum-Welch (EM)	Lec 6
条件随机场	Conditional Random Field (CRF)	Lec 6
标签偏置	Label Bias	Lec 6
序列到序列	Sequence-to-Sequence	Lec 6b
注意力机制	Attention	Lec 6b
束搜索	Beam Search	Lec 6b
上下文无关文法	Context-Free Grammar (CFG)	Lec 7
Chomsky 范式	Chomsky Normal Form (CNF)	Lec 7
概率上下文无关文法	PCFG	Lec 7
CKY 算法	CKY	Lec 7
依存句法	Dependency Parsing	Lec 8
移进-归约	Shift-Reduce	Lec 8
Eisner 算法	Eisner DP	Lec 8
最大生成树	MST (Chu-Liu-Edmonds)	Lec 8
一阶逻辑	First-Order Logic (FOL)	Lec 9
λ 演算	Lambda Calculus	Lec 9
倒排索引	Inverted Index	Lec 10
词频-逆文档频率	TF-IDF	Lec 10
BM25	Okapi BM25	Lec 10

中文	English	首次出现
双塔模型	Dual Encoder	Lec 10
对比学习	Contrastive Learning / InfoNCE	Lec 10
近似最近邻	ANNS	Lec 10
预训练语言模型	Pre-trained Language Model (PLM)	Lec 11
掩码语言建模	Masked Language Modeling (MLM)	Lec 11
缩放律	Scaling Laws	Lec 11b
上下文学习	In-Context Learning (ICL)	Lec 11b
涌现能力	Emergent Abilities	Lec 11b
提示学习	Prompting	Lec 12
思维链	Chain of Thought (CoT)	Lec 12
提示工程	Prompt Engineering	Lec 12
检索增强生成	Retrieval-Augmented Generation (RAG)	Lec 12
预训练数据	Pre-training Data / Corpus	Lec 13
通用爬虫	Common Crawl	Lec 13
线性化	Linearization (WET / WARC)	Lec 13
去重	Deduplication (MinHash / BFF)	Lec 13
启发式过滤	Heuristic Filtering (C4)	Lec 13
模型法过滤	Model-based Filtering (DCLM / FineWeb)	Lec 13
合成数据	Synthetic Data	Lec 13

各讲正文通过 pandoc 多文件输入自动追加在本 README 之后，目录由 pandoc 自动生成。

Lecture 0: 课程导引与教学大纲 (Course Syllabus)

[TIP] 学习指南

核心：了解本课程定位、考核方式、参考资料与教学路线图，避免在后续学习中“走偏”。前置：概率统计（联合/条件概率、Bayes、Gaussian、Bernoulli、先验/后验/似然/期望）、Python 编程能力、可选的神经网络基础。重点：评分构成（A+L : ME : IP $\approx$ 60 : 30 : 10）、严禁抄袭、Syllabus 涵盖的 10 大主题模块。

0.1 课程基本信息

项目	内容
课程名	Foundations of Natural Language Processing (FNLP)
授课教师	Yansong Feng (冯岩松, Wangxuan Institute of Computer Technology, PKU)
联系邮箱	fengyansong@pku.edu.cn
开课时间	每年第二学期
上课时间地点	周三/周四 10:10 AM, 1 号楼 201
Office Hours	预约制 (By appointment)
课程网站	course.pku.edu.cn

0.2 考核与评分

- **Assignments**: 2 份短报告 (围绕 challenging LLMs 的小型分析)。
- **Labs**: 约 3 个独立完成的实验任务，从简单文本分类器，到由分类器组合而成的稍复杂任务，再到借助 LLM 解决的真实问题。
- **Midterm Exam**: 闭卷笔试。
- **In-class Participation**: 随堂小测、课堂讨论。
- **Bonus**: 自愿做约 5 分钟 voluntary presentation, 加 1%~3%。

权重 (约):

Assignments+Labs : Midterm : In-class $\approx$ 60 : 30 : 10

(0.1)

[WARN] 易错点

No Plagiarism! 抄袭零容忍；与同学讨论思路可以，但代码与报告必须独立完成。借助 LLM 须显式声明、并对结果负责。

0.3 教学目标

面向二年级本科生 (sophomore), 关注三层目标:

1. **Linguistic & Algorithmic Foundations**: 语言现象、核心概念、经验模型与算法、语料与应用。
2. **Conceptual Understanding**
 - Context: NLP 是如何从规则方法演进到 LLM 的 (Rules → Statistics → Neural → PLM → LLM)。
 - Mechanism: 这些模型如何工作、需要什么准备。
 - Intuition: 何时选哪种方法、需注意什么。
3. **关注 vs. 不关注**
 - 关注: 文本与人类语言本身。
 - 不覆盖: 神经网络/LLM 的深层理论、推理模型/多 agent 系统/强化学习等”前沿 LLM 技术”细节。

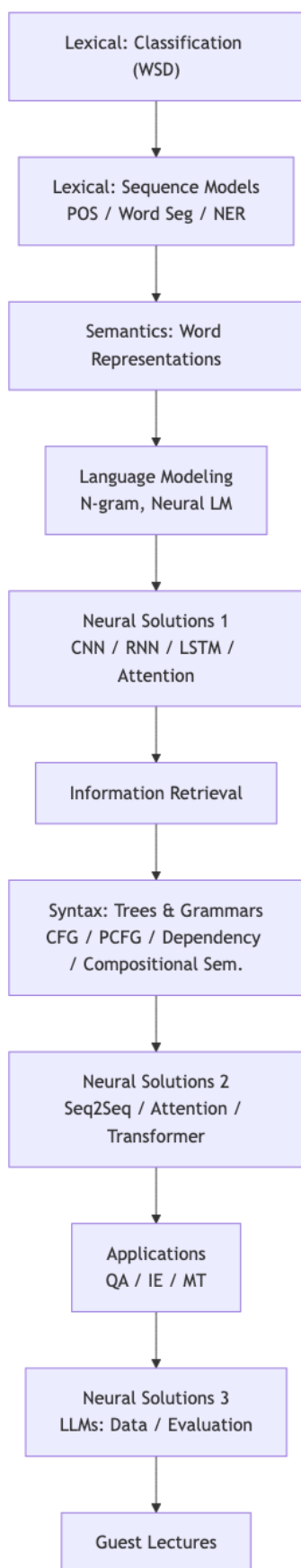
0.4 先修知识自查

- 概率统计: probability、joint/conditional probability、distribution; Bayes、Gaussian、Bernoulli; prior、posterior、likelihood、expectation。
- 语言学常识: Subject / Object / Predicate / Grammar 等基本术语。
- 编程: Python 必备; 不会就”从今天开始学”(可借助 LLM 协助)。
- 神经网络: 非必需, 但若已掌握 TensorFlow / PyTorch / numpy / scikit-learn 等会很加分。

0.5 参考资料

- 教材: Dan Jurafsky & James H. Martin, Speech and Language Processing, 3rd ed. (2026-01-06 release), <https://web.stanford.edu/~jurafsky/slp3/>
- 课程视频: Jurafsky & Manning Coursera NLP; Stanford CS224N (NLP with Deep Learning)。
- 论文与文献: ACL Anthology <http://aclweb.org/anthology/>; arXiv cs.CL <http://arxiv.org/list/cs.CL/rec>ent。
- 顶会/期刊: ACL、NAACL、EMNLP、CL、TACL、NeurIPS、ICML、ICLR、COLING、EACL、AAAI、IJCAI、AIJ、JAIR、T-PAMI 等。

0.6 Syllabus (教学路线图)



0.7 中英术语对照

中文	English
自然语言处理基础	Foundations of Natural Language Processing (FNLP)
词义消歧	Word Sense Disambiguation (WSD)
词性标注	POS Tagging
分词	Word Segmentation
命名实体识别	Named Entity Recognition (NER)
词表示	Word Representations
语言模型	Language Modeling
N 元语言模型	N-gram Language Model
信息检索	Information Retrieval
上下文无关文法	Context-Free Grammar (CFG)
概率上下文无关文法	Probabilistic CFG (PCFG)
依存句法分析	Dependency Parsing
组合语义	Compositional Semantics
序列到序列模型	Sequence-to-Sequence (Seq2Seq)
注意力机制	Attention
大语言模型	Large Language Model (LLM)

[TIP] 期中复习要点

- 记住三档比例：A+L : ME : IP $\approx$ 60 : 30 : 10。- Syllabus 的 10 大模块是后续每讲的“目录”，对新内容先定位再深入。- 课程“不覆盖”的内容（推理模型、多 agent、RL）不会出现在期中。

Lecture 1: NLP 导论与语言的歧义性 (Introduction to NLP)

[TIP] 学习指南

核心：理解 NLP 处理的对象（自然语言 vs. 形式语言）、任务谱系（NLU / NLG / 综合任务）、为什么“难”（多层歧义 + 稀疏 + 变化）、以及主流方法范式（Symbolic → Statistical → Neural → PLM → LLM）。前置：基本语言学常识（morphology / syntax / semantics / pragmatics）、概率统计直觉。重点：语言的 5 个层次、歧义的多层级表现（phonetic / lexical / syntactic / semantic / discourse）、Zipf's Law 与稀疏性、Empirical Methods 的内涵。

1.1 Motivation：为什么研究语言？

语言是人类知识、文化、协作的主要载体。让计算机处理人类语言可以解锁海量数字文本资源（新闻、网页、平行翻译语料、句法树库等），并为信息检索、问答、翻译、对话等真实应用提供基础。本讲建立“问题 → 方法”的全局地图，为后续具体模型铺路。

1.2 什么是语言？

1.2.1 词典定义的多义性

不同词典（Cambridge、Oxford）给出的 Language 定义本身就有多种侧面，关键词共同指向：**communication**（沟通）、**system**（系统）、**word**（词）、**human/people**（人类）。

[NOTE] 直觉理解

一个有效的定义至少包含两个要素：用于人类交流与带有结构与约定的符号系统。这正解释了为何 NLP 需同时关心“意义”和“结构”。

1.2.2 形式语言 vs. 自然语言

Language is the human ability to acquire and use complex systems of communication; a language is any specific example of such a system.

- **Formal language**: 计算机语言（C、C++、Java、Python），语法严格、无歧义。
- **Natural language**: 人类语言（汉语、英语...），语法灵活，充满歧义。
- **Sign language**（手语）：介于两者之间，是开放讨论。

Beamer 中的对照例子：

Computer program (formal)
Com
├ Var Assg Expr

English sentence (natural)
S
├ NP VP

	y = Const Opt Var			DT NN		VBP .
	100 + x			The sun		shone .

1.2.3 “知道一种语言”意味着什么

例句：Jimmy moved to Edinburgh when his parents got new jobs there.

理解该句至少要掌握五个层次：

层次	含义	例
Morphology 词法	词的内部结构	moved = move + -ed
Syntax 句法	词如何组成句子	主谓宾、从句嵌套
Lexical Semantics 词汇语义	单个词义	Edinburgh 是地名
Compositional Semantics 组合语义	由句法决定的意义组合	何时何地谁做了什么
Pragmatics 语用	语境中的意义、推断、动作	“为什么”Jimmy 搬家

后续课程还会扩展 Phonetics/Phonology（语音/音系）、Discourse（篇章）、Language generation（生成）。

1.3 什么是 NLP?

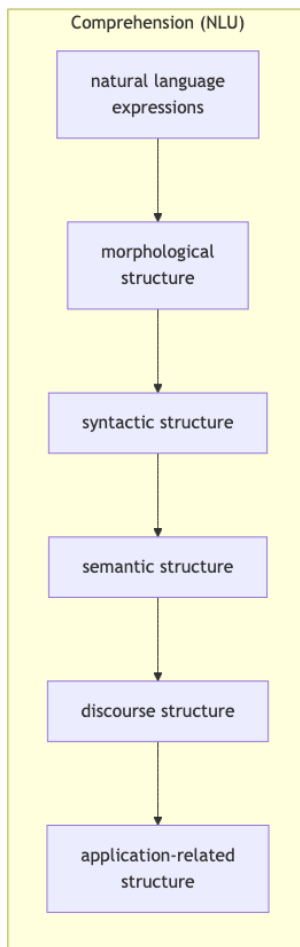
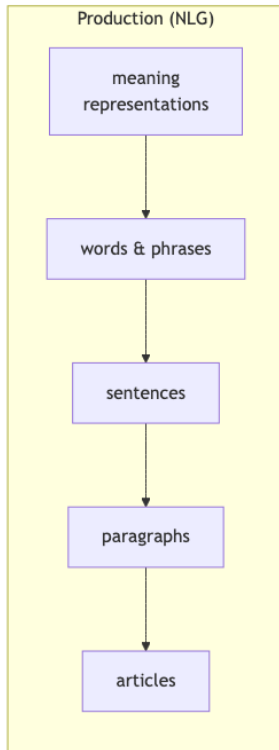
1.3.1 任务定义（M. Collins）

Help computers use human language as input and/or output.

按输入/输出方向划分：

类别	方向	典型任务
NLU (Natural Language Understanding)	human → computer	语音识别、POS tagging、parsing、信息抽取
NLG (Natural Language Generation)	computer → human	内容规划、表层实现、摘要、TTS
复杂任务	双向	问答 QA、对话 Dialogue、机器翻译 MT

1.3.2 Comprehension and Production (双向映射)



1.3.3 计算建模的演化



每一步都不是“丢弃”上一代，而是“吸收 + 增强”：例如 LLM 内部依旧隐含统计与符号成分。

1.4 NLP 应用全景

1.4.1 应用清单

- **ASR & TTS**: 自动语音识别、文本到语音合成 (Nuance、IBM、iFlyTech)。
- **Information Extraction**: 从新闻/生物文献中抽取事件、实体、关系。
- **Sentiment Analysis**: 情感极性判别 (产品评论、社交媒体)。
- **Question Answering / Machine Reading**: IBM DeepQA (Watson@Jeopardy)、Aristo & Euclid (AI2)、Todai、SQuAD 排行榜、ChatGPT / Claude / Gemini / LLaMA / Qwen / Deepseek / Kimi / 豆包.....
- **Dialogue & Smart HCI**: 任务型 (Flight booking、Call center、Customer service) + 闲聊型 (Siri、Alexa、Cortana、智能家居、自动驾驶)。
- **Text Summarization、News Writing、Poem Generation**。
- **Machine Translation**: 海量平行语料训练的中-英/多语翻译。

1.4.2 即便到 2023–2025，仍然不”trivial”

课件反复出现“STILL Not Trivial At ALL –2023/2024/2025”——LLM 进步明显，但语音识别（方言/口音）、复杂语义/语用、长文档篇章理解、跨域 IE 仍有大量失败案例。

[NOTE] 直觉理解

“We need data → 模型学习 → 模型能处理与训练数据相似的挑战”是当前主流范式的简化心法。它解释了为什么 LLM 在新颖、非常见场景仍会失败——训练分布外 (OOD)。

1.5 为什么语言难？——歧义无处不在

Why is Language hard? – Many levels of languages – Ambiguities on many levels – Create rules, but far more exceptions

→ AMBIGUITY EVERYWHERE

1.5.1 语言的多层歧义

层次	关心什么	歧义示例
Phonetics/Phonology	音素、子词、词的边界	“you like your mother”快读 → “you lie cured mother”
Lexical (词汇)	词义、词的边界	“本科”=undergraduate vs. “化验科”
Syntactic (句法)	短语、修饰、依存	Jane ate spaghetti with meatballs 的 PP 挂靠
Semantic (语义)	意义、指代	“I want to buy a muffin with choc chips.”
Pragmatic / Discourse	结论、反应、意图	郭德纲段子里反复”挺好”，最后只泡一包

1.5.2 词汇歧义（一词多义 / 一形多义）

[NOTE] 笑话例子

一个人要去化验科。护士指着前方一牌说：“非本科人员不得入内！”那人大怒，骂道：“我就化验个血，还要本科文凭!?”

同一个“本科”对应两个含义——化验科的“本科室”vs. 学历“本科”。

汉语短语切分歧义：一个半小时左右的 ppt 可以切分为：



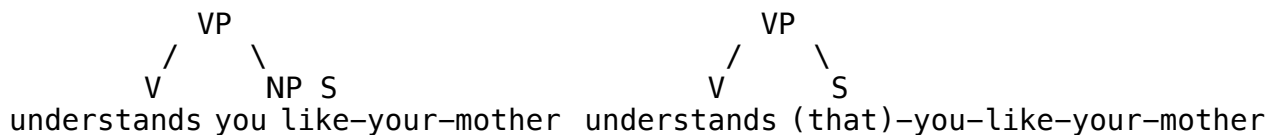
1.5.3 句法歧义（结构 → 解释）

经典例：At last, a computer that understands you like your mother. —L. Lee。

可能解释：

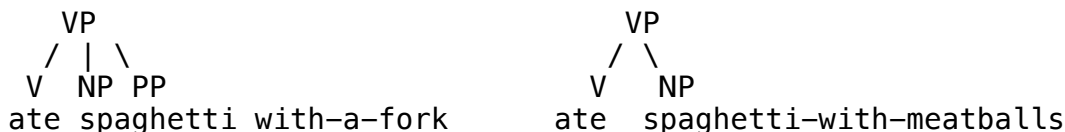
1. Computer understands you **as well as** your mother understands you. (比较)
2. Computer understands (that) you like your mother. (宾语从句)
3. Computer understands you as well as it understands your mother. (并列宾语)

结构对应：



PP-attachment 经典例：

- Jane ate spaghetti **with a fork**. → with-PP 修饰 ate (工具)。
- Jane ate spaghetti **with meatballs**. → with-PP 修饰 spaghetti (共现成分)。



[WARN] 易错点

这类 PP-attachment 不能仅看句法形式判别，需要世界知识 / 常识：“肉丸是 spaghetti 的配料”而“叉子是工具”。这是 statistical / neural parser 引入 lexicalization 的根本动因。

1.5.4 语义 / 语用 / 篇章歧义

例 1 (语义)：> A: I want to buy a **muffin with choc chips**. > B: Em, sorry, we only take cash.

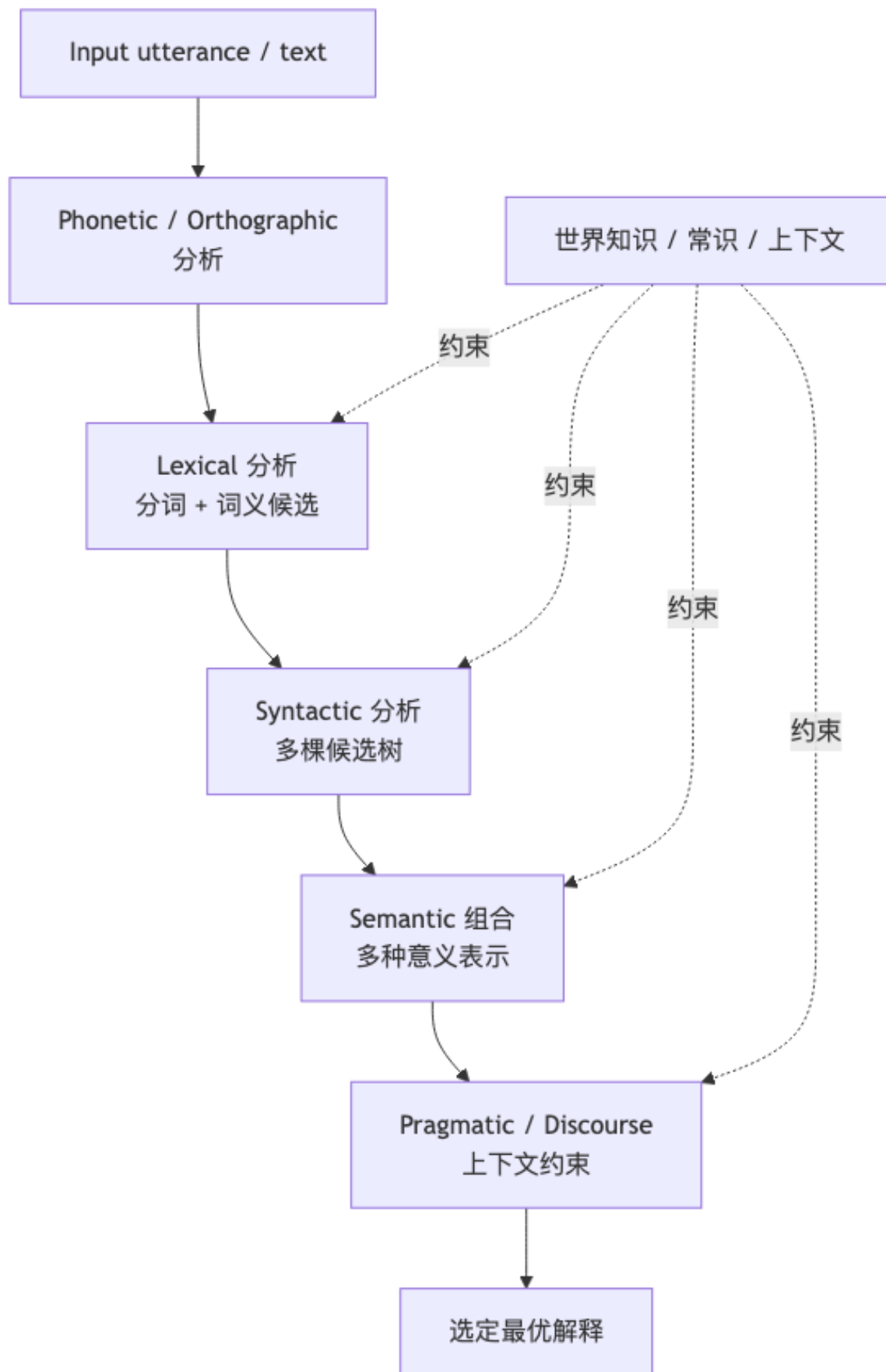
例 2 (指代 + 数量)：> A: Could you buy a bottle of milk? **If they have fresh pomelos get four**. > B: (一小时后) I am back! (with **four bottles of milk**)

例 3 (数量隐含的语用)：> A: We should have a coffee now. > B: How about **10**?

中文经典 (来自郭德纲)：> A: 回家吃点好的，红烧牛肉！—B: 这敢情好！> A: 香菇炖鸡？—B: 这也不错啊 > A: 葱烧排骨！—B: 挺好，是不是有点多？> A: 你说，我**泡哪包**？

A 看似在描绘豪华大餐，最终揭示是“泡面 + 多种调料包”——理解需要篇章级的语用推断。

1.5.5 歧义解析的一般流水线



1.5.6 其他挑战：Variation、Sparsity、Missing

- **Variation**: 中国有数百种方言；全球超过 7000 种语言 (Ethnologue)。
- **Sparsity (Zipf's Law)**: 词频呈幂律分布——少量高频词覆盖语料绝大部分，大量低频词仅出现一次 (hapax legomena)。

$$f(r) \propto \frac{1}{r^s}, \quad s \approx 1 \quad (1.1)$$

其中 r 是词的频率排名， $f(r)$ 是该词在语料中的频次。无论语言或语料规模，always far many infrequent words。

- **Anything missing?**: 语言常省略对人显而易见的信息——“老虎 vs. 猫谁更大?”“怎么描述深蹲 (Squat) 的动作?”——这些需要视觉/物理常识，单纯文本难以覆盖。

[WARN] 易错点

Zipf's Law 直接导致 N-gram 等基于频次的模型严重的**数据稀疏问题**，是后续 smoothing、subword (BPE)、词向量等技术的根本动机。

1.6 NLP 的方法论

1.6.1 Language as Data

It is hard! And we do not exactly know how we humans process languages ! → **Learn from data?**

可用数据规模：

- 数十亿词的新闻文本；
- 数十亿网页文档；
- 几万句标注好句法树的句子（主要在学术界）；
- 数十亿词的双语对照翻译。

核心问题：How can we make use of such data?

1.6.2 Empirical Methods in NLP

- **Data-intensive**: 工作对象是大规模文本。
- **Empirical**: 基于统计或机器学习的数学模型，由数据**修改 / 训练**。
- 主要是概率模型，但不全是；当前流行：**Neural Everything** **Pre-training Everything** **Pre-training THE WORLD**。
- 与之相对：**Computational Linguistics** 更偏理性主义、提供洞察；NLP 更偏数据驱动。

1.6.3 三句”金句”刻画范式演变

“But it must be recognized that the notion of ‘probability of a sentence’ is an entirely useless one, under any known interpretation of this term.”—**Noam Chomsky, 1969** (理性主义 / 计算语言学)

“Every time I fire a linguist, the performance of the speech recognizer goes up.”—**Frederick Jelinek, 1988** (经验主义 / 数据驱动)

“When we train a large neural network to accurately predict the next word in lots of different texts from the Internet, what we are doing is that we are learning a world model.”—**Ilya Sutskever, 2023** (LLM 时代)

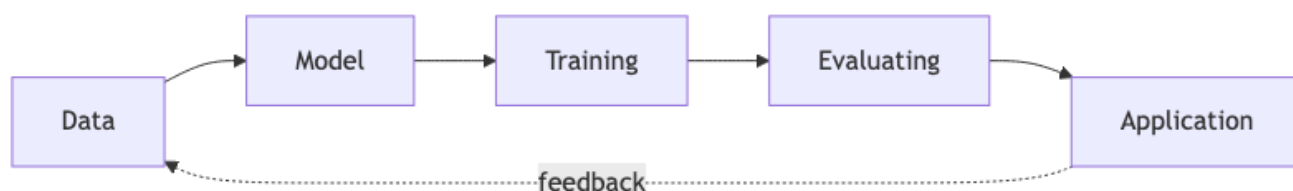
1.6.4 角色冲突与统一

维度	A	B	C（被 LLM 趋同）
角色	Scientist	Engineer / Coder	Scientist + Engineer
视角	解释人类行为（语言）	写程序	写 prompt / scripts
流派	Rationalist	Empiricist	“Play with LLMs”
产出	提供洞察	分析数据	玩转大模型

[NOTE] 直觉理解

LLM 让科研、工程、原型三种角色边界模糊，但不消除对语言现象的本质理解需求——本课程目标正是给你提供“理解 + 动手 + 选择”的基础。

1.6.5 LLM 时代的 NLP 工作循环



1.7 例题 / 案例练习

案例 1：分析下列歧义并指出层次

Jane ate spaghetti with chopsticks.

- 句法层面：with-PP 既可挂在 VP（修饰 ate）也可挂在 NP（修饰 spaghetti）。
- 语义/常识：chopsticks 通常是工具 → 倾向 VP-attachment。
- 启示：单凭句法树枚举不能选出最佳解释，需要 lexicalized / world-knowledge 信号。

案例 2：Zipf's Law 的实际后果

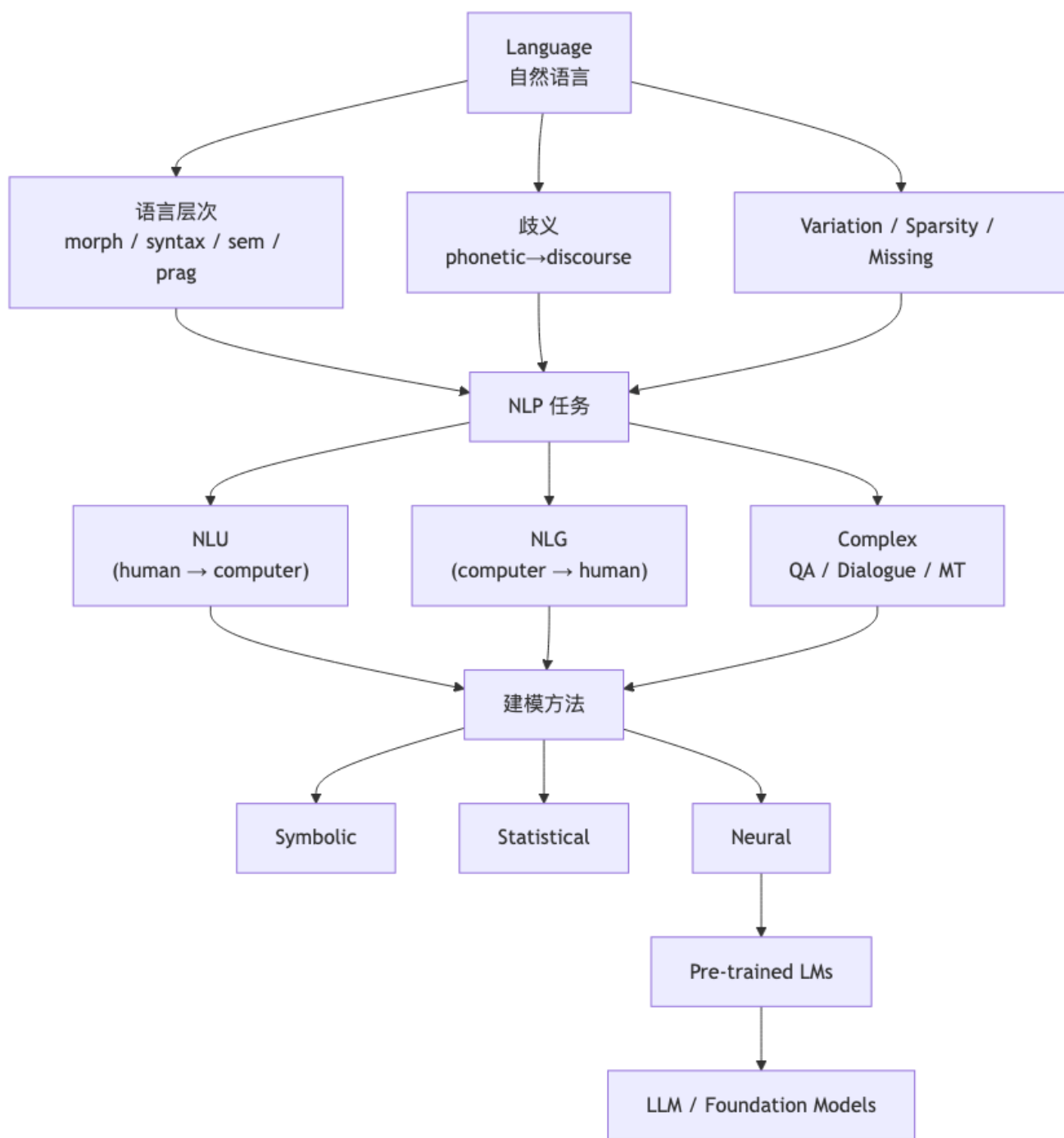
设语料中第 1 名词出现 10^6 次。按 $s = 1$ ，第 10^4 名词约出现 $10^6/10^4 = 100$ 次；第 10^6 名词只出现 ≈ 1 次。

- 含义：约一半词汇是 hapax，N-gram 计数中 $\#(w_{i-1}, w_i) = 0$ 的事件大量出现。
- 解决路径：smoothing (add-k、Kneser-Ney)、回退、subword、分布式表示。

案例 3：篇章歧义需要多轮上下文

复述郭德纲段子的关键：B 在前几轮“挺好”“不错”造成 A “这是一顿大餐”的预设，最后 A 揭示是“泡面 + 调料包”——理解必须维护跨句的世界模型，而不是逐句翻译。

1.8 概念关系总图



1.9 易错点汇总

[WARN] 易错点

1. 不要把”形式语言”与”自然语言”混为一谈：前者无歧义、后者多歧义，所有 NLP 难点几乎都源于自然语言的歧义性。2. 歧义不是只有”一词多义”：从语音到篇章每一层都有歧义，且彼此交互。3. Zipf’s Law 不是冷知识：它是数据稀疏的根本来源，决定后续平滑、subword、表示学习的设计动机。4. PP-attachment 不仅靠句法：常识必不可少；这是 lexicalized parser 和神经 parser 引入 word-level 信号的原因。5. NLP ≠ LLM：LLM 是当前的方法范式之一；课程关注更基础的”问题 + 表示 + 方法”。

1.10 本节小结

- 对象：自然语言（含歧义、稀疏、变化）。
- 任务：NLU / NLG / 复杂任务三大谱系。
- 难点：多层歧义 + Zipf 稀疏 + 跨模态/常识缺口。
- 方法：从 Symbolic → Statistical → Neural → PLM → LLM；统一在”用数据训练数学模型”的经验主义范式下。

1.11 中英术语对照

中文	English
自然语言	natural language
形式语言	formal language
词法/形态学	morphology
句法	syntax
词汇语义	lexical semantics
组合语义	compositional semantics
语用	pragmatics
篇章	discourse
语音/音系	phonetics / phonology
自然语言理解	Natural Language Understanding (NLU)
自然语言生成	Natural Language Generation (NLG)
信息抽取	Information Extraction (IE)
情感分析	Sentiment Analysis
问答	Question Answering (QA)
机器阅读	Machine Reading
对话系统	Dialogue System
机器翻译	Machine Translation (MT)
自动语音识别	Automatic Speech Recognition (ASR)
文本到语音	Text-to-Speech (TTS)
歧义	ambiguity
介词短语挂靠	PP-attachment
齐普夫定律	Zipf’s Law
数据稀疏	data sparsity
经验方法	empirical methods
计算语言学	Computational Linguistics
预训练语言模型	Pre-trained Language Model (PLM)
大语言模型 / 基础模型	Large Language Model (LLM) / Foundation Model
理性主义 vs. 经验主义	Rationalist vs. Empiricist

[TIP] 期中复习要点

- 能复述**语言 5 个层次**及对应的歧义示例（建议每层至少背 1 个例子）。- 能解释 **NLU vs. NLG vs. Complex** 的边界与典型任务。- 能用一段话讲清 **Zipf's Law** 的含义、对 NLP 数据稀疏的影响、以及它驱动了哪些后续方法。- 能画出 **Symbolic → Statistical → Neural → PLM → LLM** 的范式演化箭头，并各举一例。- 记住三句金句的作者（Chomsky 1969 / Jelinek 1988 / Sutskever 2023）及其代表的方法论立场。- 能用 PP-attachment 的例子说明“句法 + 常识”协同的必要性。

Lecture 2 一词义消歧 (Word Sense Disambiguation, WSD)

[TIP] 学习指南

本节核心：给定一个多义词及其上下文，判断它在该上下文中被激活的具体义项。这是 NLP 中最经典的“上下文分类”任务，引出了从基于词典的 **Lesk 算法**、统计基线 **MFS**，到监督学习（朴素贝叶斯为代表）、半监督（Yarowsky bootstrap）、无监督（义项聚类），再到神经网络与预训练模型的完整方法论谱系。**前置知识：**– 概率论：条件概率、贝叶斯公式、独立性假设 – 极大似然估计（MLE）的频次估计 – 一点点信息检索 / 词典学知识（WordNet 的 synset 概念）**重点掌握：**1. WSD 任务的形式化输入输出与“义项库（Sense Inventory）”概念 2. **Lesk 算法**（gloss 与上下文的词重叠）—简化版与扩展版 3. **MFS (Most Frequent Sense)** 基线和 one sense per discourse / collocation 启发式 4. 朴素贝叶斯模型在 WSD 上的推导： $s^* = \arg \max_{s_k} P(s_k) \prod_{v \in C} P(v | s_k)$ 5. 评测：精度 / 召回率 / F_1 ，Macro vs Micro，Lexical-Sample vs All-Words 6. 监督 / 半监督 / 无监督三种学习范式的代价与适用场景

2.1 Motivation：为什么需要 WSD

2.1.1 多义现象无处不在

一个词形 (word form) 往往对应多个义项 (senses)。课程给出的中英文例子：

语言	词形	义项 1	义项 2	...
中文	单位	组织 (organizations)	度量单位 (measurement units)	
中文	镜头	摄像机镜头 (camera lenses)	影片镜头 (frames, pictures)	
英文	can	金属容器	浮标 (buoy)	厕所；保存食物；解雇；情态动词

义项差异可以分两类：

- **同形异义 (homonymy)**：意义完全无关，如 bank（河岸）vs bank（银行），仅形式偶然相同。
- **多义 (polysemy)**：意义有关联（如“book”的“书籍/预定/记录”），来自一个词根的语义扩展。

[NOTE] 直觉理解

同形异义 / 多义边界并不总是清晰。词典学家也常争论某个词究竟该并为一个 synset 还是拆成两个。实际工程中“义项粒度 (sense granularity)”是一个关键设计选择：粒度太细则标注者一致性低、分类器学不动；粒度太粗则下游任务区分不开。

2.1.2 语音合成中的 WSD

中文的多音字直接把 WSD 与 TTS（语音合成）绑在一起。如”长长长长长长”：

句子	“长”含义	拼音
他又长高了？	grow	zhǎng
这条路真长啊。	long	cháng
现在哪个官职都带个长啊！	officer	zhǎng
我长你几岁，你得叫我大哥。	elder than	zhǎng

英文同理：record 在“one of my favorite records”（名词/重音前置）与“record and keep a file”（动词/重音后置）中含义、词性、重音模式都不同。选义项 = 选发音。

2.1.3 命名实体与转喻

The 1965 **Los Angeles Dodgers** finished the regular-season with a 97 –65 record, which earned them the NL pennant by two games over their arch-rivals, **San Francisco**.

这里“Los Angeles”“San Francisco”都是城市名，但实际指代该城市的棒球队（San Francisco Giants），这是典型的 Location Metonymy（地名转喻）。WSD 与命名实体识别 (NER) 在这里交汇。

2.1.4 双关与多模态

- **Pun**: Mr. Robinson tests positive for COVID-19, and his friends...中“positive”同时承载”阳性”与”积极”两个义项，故意保留歧义以营造幽默。
- **Visual-WSD (SemEval-2023 Task 1)**: 给定“andromeda tree”和 10 张候选图，选出对应义项。这把 WSD 从纯文本推进到多模态。

[WARN] 易错点

WSD 不是”找出所有可能含义”，而是”在这个具体上下文中选一个最合适的义项标签”。输出是单一标签而非分布（虽然神经模型内部会算分布）。

2.2 形式化定义

2.2.1 输入 / 输出

对计算机而言，WSD 的形式化：

- **输入**: 目标词 w + 上下文 C (一个句子或若干 token 的窗口) + 该词的可能义项集合 $S = \{s_1, s_2, \dots, s_n\}$
- **模型**: 从 C 中抽取特征，与已有”义项知识”比对
- **输出**: 义项标签 $s^* \in S$

2.2.2 义项库 (Sense Inventory)

计算机必须事先知道 w 的所有候选义项。两种来源：

1. **手工词典资源 (Hand-built lexical resources)**: 每个义项给出定义 (gloss) + 示例
2. **标注样例 (Labeled examples)**: 标注好义项的语料，让模型从样本中学习

主流资源：

资源	语言	特点
WordNet	英文	名/动/形/副词按 synset (同义概念集) 组织成大型层级, 每个 synset 有 gloss 与例句
HowNet	中文	基于义原 (sememe) 的语义网
现代汉语语义词典	中文	北大词典学传统
BabelNet	多语	跨语言语义网, 整合 WordNet + Wikipedia + Wiktionary

WordNet 例子 (“bank”的两个名词义项):

- **S1 (n)**: sloping land (especially the slope beside a body of water). 例: they pulled the canoe up on the bank.
- **S2 (n)**: depository financial institution ...例: he cashed a check at the bank; that bank holds the mortgage on my home.

SensEval / SemEval 系列: WSD 的标准评测, 覆盖多语言, 提供大量标注数据。

2.3 三种学习范式

2.3.1 监督学习 (Supervised)

- 任务定位: **分类任务**
- 前提: 义项库已知 + 有人工标注的训练样本
- 流程:
 1. 收集 (x_i, y_i) 标注样本 (x_i = 带目标词的句子, y_i = 该词的义项)
 2. 抽取上下文特征
 3. 训练分类器 (NB / Logistic Regression / SVM / MaxEnt / NN)
 4. 测试时对未见样本抽特征 → 预测
- 优势: 选择多、性能强
- 代价: 需要人工标注, 标注质量直接决定上限

2.3.2 半监督学习 (Semi-supervised, Yarowsky-style)

- 思路: 从少量标注种子出发, **自举 (bootstrap)** 出更多伪标注样本
- 流程:
 1. 人工标注一小批种子
 2. 用现有分类器对未标注语料打分, 把 top- K 高置信度的样本加入训练集
 3. 也可以用**自然对齐数据** (如双语平行语料里词对齐结果) 反推义项
 4. 重复

Yarowsky 1995 的著名启发式: – I should go to the bank and discuss with the **clerks**. → 银行 (financial) – I enjoy **fishing** at the bank. → 河边 (river)

通过”种子词” (clerks / fishing) 传播标签。

- 优势: 能获得大量训练数据
- 代价: 数据质量难评估, 实践技巧 (trick) 多

2.3.3 无监督学习 (Unsupervised)

- 任务定位: **聚类 (clustering)**
- 前提: 没有义项库, 只有纯文本

- 核心假设：相似义项出现在相似上下文中 (similar senses occur in similar contexts) —与分布式语义假设 (Distributional Hypothesis) 一脉相承
- 流程：
 1. 抽取上下文特征
 2. 对目标词的不同 mention 聚类
 3. 为每个簇生成义项表示
- 优势：无需标注
- 代价：聚类结果难解释，下游使用还需把簇映射回义项库

[NOTE] 三种范式的本质差异

“监督关心”已知义项 → 分类边界；“无监督关心”未知义项 → 发现结构 (sense induction)；“半监督居中”。WSD 历史上三种都被研究得很透，但预训练模型出现后，监督路线（在 WordNet 上微调 BERT）效果最强。

2.4 核心算法详解

2.4.1 Lesk 算法 (Knowledge-based)

Simplified Lesk 是经典的、不需训练数据的词典算法：

对目标词的每个候选义项，把它的 gloss + 例句 当作“袋子里的词”，与目标词上下文窗口里的词求集合重叠 (overlap)，选重叠词数最多的义项。

算法伪代码：

```
function SimplifiedLesk(word, context):
    best_sense = MFS(word)           # fallback: 最常见义项
    max_overlap = 0
    context_set = bag_of_words(context)
    for each sense in senses(word):
        signature = bag_of_words(gloss(sense) u examples(sense))
        overlap = |context_set ∩ signature|
        if overlap > max_overlap:
            max_overlap = overlap
            best_sense = sense
    return best_sense
```

Extended Lesk 的常见改进：

- 把 相关 synset 的 gloss 也并入 signature (上位词、下位词、相似形等)
- 用 IDF / TF-IDF 等权重抑制常见词
- 用 词向量余弦 替代离散交集

2.4.2 Lesk 手算示例 (bank)

输入：> I have deposit at that bank, but they refused my mortgage application.

目标词：bank 上下文词集 (去停用词后)：{deposit, refused, mortgage, application}

候选义项 gloss：– S1 (河岸)：sloping land especially the slope beside a body of water. 例：they pulled the canoe up on the bank. – signature: {sloping, land, slope, body, water, canoe, ...} – S2 (金融机构)：depository financial institution that accepts deposits and channels the money into lending activities. 例：he cashed a check at the bank; that bank holds the mortgage on my home. – signature: {depository, financial, institution, accepts, deposits, channels, money, lending, activities, check, mortgage, home, ...}

重叠计数：– $S1 \cap \text{context} = \{\}$, overlap = 0 – $S2 \cap \text{context} = \{\text{deposit(s), mortgage}\}$, overlap = 2

输出：选 S2 (金融机构义项)。

[NOTE] 直觉理解

Lesk 的本质是把“词典编纂者写的释义”当作每个义项的“主题分布”。当你的上下文与某个义项的“主题词”高度重叠，就投票给它。这是一个**纯静态、零训练数据**的算法，但严重依赖词典质量。

2.4.3 Most Frequent Sense (MFS) 基线

MFS 启发式：对每个目标词，统计训练语料中该词最常出现的义项；测试时不看上下文，直接输出 MFS。

例：在 SemCor 中 `plant:factory` 出现频率远高于 `plant:flora`，于是把所有出现的 `plant` 都标成 `plant:factory`。

MFS 看似粗暴，但在 all-words WSD 中**长期是难以击败的强基线**——多义词的义项分布极度长尾，绝大部分 token 取最常见义项就能蒙对。

2.4.4 两条经典启发式

One Sense per Discourse [Gale et al., 1992]

一个词在同一篇章 (discourse) 中倾向保持单一义项。

实操：如果“plant”在某文档中出现 10 次，只要你能高置信度判断其中一次为 `plant:flora`，就把全部 10 次都标成 `plant:flora`。

One Sense per Collocation [Yarowsky, 1993]

一个词在同一搭配 (collocation) 中倾向保持单一义项。

例：industrial plant 中的 plant 几乎总是“工厂”义项。

[WARN] 易错点

这两条都是**启发式而非绝对真理**。讽刺、双关、同义反复修辞会破坏它们。但作为半监督 bootstrap 的传播规则非常有效。

2.4.5 朴素贝叶斯 (Naïve Bayes)

符号约定 (按课程统一)：- w ：目标词 - $S = \{s_1, \dots, s_n\}$ ： w 的义项库 - C ： w 的上下文 - $V = \{v_1, \dots, v_j\}$ ：上下文特征词表

目标：给定 C ，选义项 s^* ：

$$s^* = \arg \max_{s_k} P(s_k | C) \quad (2.1)$$

由 Bayes 公式：

$$P(s_k | C) = \frac{P(C | s_k)P(s_k)}{P(C)} \quad (2.2)$$

分母 $P(C)$ 与 s_k 无关，可丢弃：

$$s^* = \arg \max_{s_k} P(C | s_k)P(s_k) \quad (2.3)$$

朴素独立假设：上下文中各特征词 $v_x \in C$ 在给定义项 s_k 下条件独立：

$$P(C | s_k) = P(\{v_x | v_x \in C\} | s_k) = \prod_{v_x \in C} P(v_x | s_k) \quad (2.4)$$

代入 (2.3) 得分类器：

$$s^* = \arg \max_{s_k} P(s_k) \prod_{v_x \in C} P(v_x | s_k) \quad (2.5)$$

[WARN] 易错点

“朴素”在于把上下文词当作彼此独立——这显然违反语言事实（句法、共现约束都被忽略），但模型由此变得可估计、可推断。实践中 NB 在 WSD 上经常出乎意料地强。

参数估计 (MLE + 频次)：

$$\hat{P}(v_x | s_k) = \frac{\text{Count}(v_x, s_k)}{\sum_{v \in V} \text{Count}(v, s_k)} \quad (2.6)$$

$$\hat{P}(s_k) = \frac{\text{Count}(s_k)}{\text{Count}(w)} \quad (2.7)$$

测试：对新样本上下文 C' ，逐义项计算 $\prod_{v_x \in C'} \hat{P}(v_x | s_k) \hat{P}(s_k)$ ，取 argmax。

[NOTE] 实现细节

实际实现需要：- 取对数： $\log P(s_k) + \sum_x \log P(v_x | s_k)$ ，避免连乘下溢 - 拉普拉斯平滑 (add-one smoothing)： $\hat{P}(v_x | s_k) = \frac{\text{Count}(v_x, s_k) + 1}{\sum_v \text{Count}(v, s_k) + |V|}$ ，防止未见词把概率压成 0 - 词表 V 通常用 stop-word 过滤 + 词干化 (stemming)

2.4.6 NB 手算示例

设 $w = \text{"bank"}$ ，义项 $\{s_{\text{river}}, s_{\text{financial}}\}$ 。训练语料统计如下：

上下文词	$\text{Count}(\cdot, s_{\text{river}})$	$\text{Count}(\cdot, s_{\text{financial}})$
water	20	1
money	1	30
fishing	15	0
deposit	0	25
总词频	100	200
义项频次	50	150

测试句：“I went to the **bank** to deposit money.” (特征词：deposit, money)

先验：- $P(s_{\text{river}}) = 50/200 = 0.25$ - $P(s_{\text{financial}}) = 150/200 = 0.75$

条件 (加一平滑, $|V| = 4$)：- $P(\text{deposit} | s_{\text{river}}) = (0 + 1)/(100 + 4) = 1/104$ - $P(\text{money} | s_{\text{river}}) = (1 + 1)/(100 + 4) = 2/104$ - $P(\text{deposit} | s_{\text{financial}}) = (25 + 1)/(200 + 4) = 26/204$ - $P(\text{money} | s_{\text{financial}}) = (30 + 1)/(200 + 4) = 31/204$

打分 (取对数比较即可)：

$$\text{score}(s_{\text{river}}) = 0.25 \cdot \frac{1}{104} \cdot \frac{2}{104} \approx 4.6 \times 10^{-5}$$

$$\text{score}(s_{\text{financial}}) = 0.75 \cdot \frac{26}{204} \cdot \frac{31}{204} \approx 1.45 \times 10^{-2}$$

后者远大, 输出 $s^* = s_{\text{financial}}$ 。

2.5 监督 WSD 的特征工程

监督学习 = “特征”+ “分类器”。WSD 常用特征:

- 邻近词 (neighboring words): 窗口内的 bag-of-words
- N-gram / 局部搭配 (local collocations): 如 $(w_{-1}, w_0), (w_0, w_{+1})$
- 目标词的 POS: 动词义项 vs 名词义项
- 邻近词的 POS
- 分布式语义特征: LSA / LDA / 词向量
- 句法特征: 依存关系、主谓宾搭配

分类器选择: – 朴素贝叶斯 (本节主角) – 线性模型: Logistic Regression、Maximum Entropy、SVM – 神经网络 (下两节及 lec11 详述)

2.6 评测

2.6.1 二元 WSD 的列联表

设目标词只有 positive / negative 两个义项:

	Gold Positive	Gold Negative
Sys Positive	tp	fp
Sys Negative	fn	tn

$$\text{Accuracy} = \frac{tp + tn}{tp + fp + fn + tn}$$

$$\text{Precision (针对 positive 类)} = \frac{tp}{tp + fp}$$

$$\text{Recall} = \frac{tp}{tp + fn}$$

F-measure (β 加权调和平均):

$$F_{\beta} = \frac{(\beta^2 + 1) \cdot \text{Pre} \cdot \text{Rec}}{\beta^2 \cdot \text{Pre} + \text{Rec}} \quad (2.8)$$

最常用 F_1 ($\beta = 1$):

$$F_1 = \frac{2 \cdot \text{Pre} \cdot \text{Rec}}{\text{Pre} + \text{Rec}} \quad (2.9)$$

2.6.2 多类聚合: Macro vs Micro

- Macro-F1: 对每个义项分别算 F_1 , 然后取平均。每个类权重相同 → 对少数类敏感。
- Micro-F1: 把所有实例的 tp/fp/fn 先求和再算 F_1 。等价于 Accuracy (多分类下) → 被多数类主导。

[WARN] 易错点

类分布严重不均衡时 (如 MFS 占 80%+), Micro-F1 看上去很高但模型可能根本没学到稀有义项。报告 WSD 结果应同时给 Macro 和 Micro。

2.6.3 评测设置: Lexical-Sample vs All-Words

设置	描述	典型用法
Lexical-Sample (定向 WSD)	限定一小组目标词, 每个目标词单独训分类器; 每句一个目标词	SemEval lexical-sample tracks
All-Words	对文本中所有实词 (名/动/形/副) 做 WSD	SemEval all-words tracks, 更接近实际下游需求

All-Words 需要训练成百上千个分类器 (每个词一个)、或者用一个共享模型 + 词级 embedding。

2.6.4 思考题 (课程原题)

Lexi 是全科医生, 她要听病人主诉并快速判断是否高风险中风以决定转急诊。请问哪个指标最合适?

答案: Recall (针对”中风”正类)。漏诊 (false negative) 代价远高于过度转诊 (false positive)。在医疗筛查、灾难报警等场景, 召回率往往是首要指标, 必要时配合 F_2 ($\beta = 2$, 更看重 Recall)。

2.7 WSD 的现代化路径

2.7.1 IMS (It Makes Sense, Zhong & Ng 2010)

传统监督 WSD 的”集大成者”: - 周围词 POS - 周围词本身 - 局部搭配 (local collocations) - 用 SVM/MaxEnt 分类

长期是传统方法的 SOTA。

2.7.2 神经时代

- Raganato et al., 2017: 基础神经序列模型 (BiLSTM) 做 all-words WSD
- Luo et al., 2018: 把 gloss 编码 引入神经 WSD (GAS, Gloss-Augmented Network)

2.7.3 预训练时代

- GlossBERT / BEM (Blevins & Zettlemoyer 2020): 用 BERT 同时编码上下文与 gloss, 做 bi-encoder 检索
- EWISE / EWISER (Bevilacqua & Navigli 2020): 把 WordNet 图结构嵌入分类层, 突破”80% 玻璃天花板”
- Generationary: 直接生成 gloss, 跳过义项库

预训练几乎”扫平”了所有传统方法。

2.7.4 新挑战

- Multilingual / Cross-lingual WSD: SemEval-2021 Task 2 (MCL-WiC)
- WiC (Word-in-Context): 判断同一个词在两个句子中是否同义 (二分类, 绕开义项库)
- Visual-WSD: 给定词 + 短文本, 从图集中选对应义项的图

- Few-shot WSD: 稀有义项 (long tail of senses)

[WARN] 易错点

WiC 与传统 WSD 的本质区别: WiC 不要求标注义项标签, 只判断“两次出现是否同义”。这绕开了义项库设计这一最大瓶颈, 因此能轻松扩展到多语言、新词。

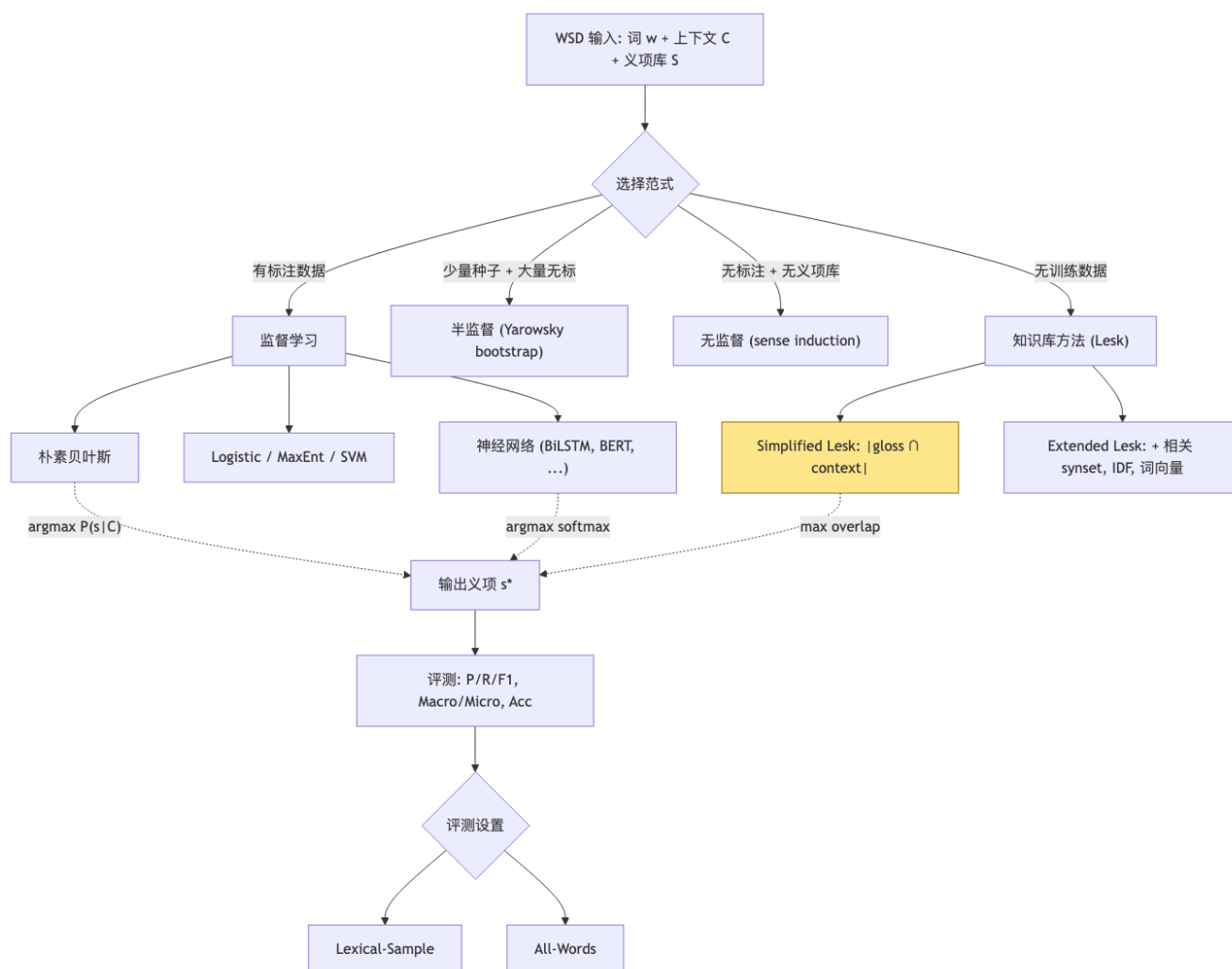
2.7.5 应用案例: 南京”马自达”

南京: 告示牌提醒”严禁黑车及马自达占用停放”(网友吐槽: 开马自达去南京差点出不来)

“马自达”在南京方言中指**载客三轮摩托**(音译”motor”), 与汽车品牌同形异义。纯文本特征很难解决, 需要: - 知识图谱 / 百科信息 (南京地方词汇) - 多模态信息 (告示牌图片) - 大模型常识推理

LLM 时代这类长尾义项可由 GPT 类模型直接处理。

2.8 概念关系图



2.9 易错点速查

易错点	正确认识
“Lesk 需要训练”	否, Lesk 是 零训练 的词典算法, 唯一”训练”是 gloss 本身
“NB 假设词与词独立”	NB 假设的是 在给定类别下 条件独立, 不是边缘独立
“概率连乘可以直接比较”	必须 取对数 避免下溢, 并加平滑防止 $\log 0$
“MFS 是弱基线”	MFS 在 all-words WSD 上能轻松到 60–70%, 是必须报告的对比基线
“WiC 等价于 WSD”	WiC 只判断 同义性 , 不需要义项标签
“Macro-F1 总比 Micro-F1 严格”	不一定, 主要看类分布。两者都要报
“One sense per discourse 是定理”	是 经验启发式 , 遇到讽刺/双关会失效
“BabelNet = WordNet 翻译版”	BabelNet 是融合 WordNet + 维基百科 + Wiktionary 的 多语义网 , 规模和覆盖远超 WordNet

2.10 中英术语对照

中文	英文	缩写
词义消歧	Word Sense Disambiguation	WSD
义项	sense	
义项库	sense inventory	
同形异义	homonymy	
多义	polysemy	
同义集	synset	
词典释义	gloss	
转喻	metonymy	
监督 / 半监督 / 无监督	supervised / semi-supervised / unsupervised	
自举	bootstrap	
朴素贝叶斯	Naïve Bayes	NB
条件独立	conditional independence	
最常见义项	Most Frequent Sense	MFS
简化 Lesk 算法	Simplified Lesk Algorithm	
局部搭配	local collocation	
词性	Part-of-Speech	POS
词义聚类	sense induction / clustering	
精度 / 召回 / F-度量	Precision / Recall / F-measure	
宏 / 微 F1	Macro-F1 / Micro-F1	
定向 WSD / 全词 WSD	Lexical-Sample / All-Words	
词典数据库	lexical database	
跨语言词义消歧	Cross-lingual WSD	MCL-WiC
视觉词义消歧	Visual WSD	V-WSD

[TIP] 期中复习要点

1. **必会推导**: 写出 NB 在 WSD 上的目标函数从 $\arg \max P(s | C)$ 到 $\arg \max P(s) \prod P(v | s)$ 的每一步, 并解释独立性假设的位置。2. **必会算法**: 能写出 Simplified Lesk 伪代码; 能在 3-4 句的上下文上手算重叠并给出预测。3. **必会例子**: bank / plant / 长 / 单位—至少能为每个词举出两个义项及其触发上下文。4. **指标计算**: 列联表给 tp/fp/fn/tn, 会算 P/R/F1, 会算 Macro vs Micro。5. **设置区分**: Lexical-Sample 与 All-Words 各自的数据规模与训练策略差异。6. **范式对比**: 监督 / 半监督 / 无监督 / 知识库 四类方法的输入要求、优劣与典型代表各举一例。7. **现代趋势**: 能说出从 IMS $\rightarrow$ BiLSTM $\rightarrow$ BERT/GlossBERT 的演进逻辑, 理解为什么 gloss 编码能突破长尾。8. **应用思考**: 医疗 / 安全 / 推荐场景下应优先优化 Precision 还是 Recall, 并说明理由。

Lecture 3a —文本分类与概率模型 (Classification & Probabilistic Models)

[TIP] 学习指南

本节核心：把 WSD 抽象为一般的**监督分类任务** $g : x \mapsto y$ ，然后从**两种建模视角**——生成式 (generative) 与判别式 (discriminative)——梳理常用分类器的设计哲学。以 Naïve Bayes 为生成式范例、以 LDA 为完整贝叶斯建模演示，把 lec3 的两条主线（概率模型 / 对数线性模型）连接起来。**前置知识**：– lec2 的 Naïve Bayes 推导与 WSD 评测体系 – 概率论：联合分布、条件分布、Bayes 公式、Multinomial / Dirichlet 分布 – MLE 基本概念 **重点掌握**：1. 监督学习的统一刻画：训练集 $\{(x_i, y_i)\} \rightarrow$ 函数 g 2. **生成式 vs 判别式**： $p(x, y)$ vs $p(y | x)$ ，决策推断、参数效率、灵活性的 trade-off 3. NB 是生成式模型：因为它显式建模 $p(x | y)p(y)$ 4. Bayesian Modeling 的”图模型 + 先验 + 后验”思路；LDA 作为生成式无监督主题模型的典范 5. 判别式模型的优势（任意特征、无强独立假设、通常精度更高）与代价（易过拟合）6. 主流判别式分类器谱系：Perceptron / Logistic Regression / MaxEnt / SVM / CRF / NN

3.1 Motivation：从 WSD 到通用分类

3.1.1 把 WSD 改写成 $g : x \mapsto y$

回顾 lec2 的 NB-WSD 分类器：

$$\text{sense}^* = \arg \max_{\text{sense}} \prod_{\text{word} \in C} P(\text{word} | \text{sense}) P(\text{sense}) \quad (3.1)$$

抽象一层：– x = 数据样本(如带目标词的句子)– y = 标签(如义项 id)– **监督学习** = 已知 $(x_1, y_1), \dots, (x_m, y_m)$ ，要找一个函数 $g : y = g(x)$ ，对任意新 x 给出 y

3.1.2 概率视角

把 g 概率化：

$$g(x) = \arg \max_y p(y | x) \quad (3.2)$$

剩下的全部问题压缩成一个：怎么得到 $p(y | x)$ ？

3.1.3 两条建模路线

视角	建模对象	推断
判别式 (Discriminative)	直接学习 $p(y x)$	对新 x 直接套用模型
生成式 (Generative)	先学联合 $p(x, y) = p(y)p(x y)$, 再用 Bayes 还原	$p(y x) \propto p(y)p(x y)$

这一区分是 Lecture 3a 的灵魂。所有后续模型都可以扣到这两个范畴里：

- 生成式：Naïve Bayes、HMM、Gaussian Mixture、LDA、Bayes Networks、(VAE)
- 判别式：Perceptron、Logistic Regression、MaxEnt、CRF、SVM、判别式神经网络.....

[WARN] 易错点

“Naïve Bayes 是生成式”不是因为它叫“Bayes”，而是因为它建模 $p(x, y) = p(y) \prod_i p(x_i | y)$ 。Logistic Regression 同样用 sigmoid，但只学 $p(y | x)$ ，所以是判别式。

3.1.4 LLM 是哪一种？

课程一道反思题：

Is Naïve Bayes a generative model or discriminative model? What about large language models?

LLM (如 GPT) 在训练目标层面建模 $p(w_t | w_{<t})$ ——这是对文本本身的生成式建模 ($p(x)$ 类型)。但在用于分类 (如 zero-shot prompting) 时，它把 $p(y | x)$ 也实现成了判别式接口。所以 LLM 是“以生成式目标训练、但可被用作判别器”的混合体。这是当代 NLP 与传统监督学习最大的范式差异。

3.2 形式化：生成式模型

3.2.1 联合分布的推导

设要从 $p(y | x)$ 出发，应用 Bayes：

$$p(x, y) = p(y)p(x | y) \quad (3.3)$$

$$p(y | x) = \frac{p(x, y)}{p(x)} = \frac{p(x | y)p(y)}{\sum_{y'} p(y')p(x | y')} \quad (3.4)$$

最大化 $p(y | x)$ 与最大化分子 (联合概率) 等价：

$$g(x) = \arg \max_y p(y | x) = \arg \max_y \frac{p(y)p(x | y)}{\sum_{y'} p(y')p(x | y')} = \arg \max_y p(y)p(x | y) = \arg \max_y p(x, y) \quad (3.5)$$

生成式模型的本质：先建模数据如何生成 (先选类 y ，再由 y 生成 x)，分类只是反向推理。

3.2.2 Bayesian Modeling 模板

把 Bayes 规则套到模型未知量上：

- 数据生成分布： $x \sim p(x | y)$
- 模型先验： $y \sim p(y)$

- 后验目标: $p(y | x) = \frac{p(x | y)p(y)}{p(x)} \propto p(x | y)p(y)$

困难: 1. $p(x | y)$ 通常很复杂 (高维、结构化) 2. 解析求 $p(y | x)$ 经常不可行 (积分难、 $p(x)$ 难算)
→ 必须做假设 (参数族 + 独立性 + 共轭先验)。

3.2.3 Book-Category 例子

设: - Y : 书籍类别 (侦探 / 言情 / 科幻 / ...) - X : 书中的词

直观语义: - $P(X | Y)$: “侦探小说会用哪些词” - $P(Y | X)$: “给定一段文字属于哪个类别”——我们最终要的

加上参数化:

$$Y \sim \text{Multinomial}(\gamma) \quad (3.6)$$

$$X | Y \sim p(X | \theta_Y) \quad (3.7)$$

联合:

$$p(X, Y | \gamma, \Theta) = p(X | Y, \Theta)p(Y | \gamma) \quad (3.8)$$

加上先验 (完全贝叶斯):

$$\gamma \sim \text{Dirichlet}(\beta), \quad \theta_1, \theta_2, \dots \stackrel{\text{iid}}{\sim} p(\theta) \quad (3.9)$$

这就是一个完整的层级贝叶斯模型 (hierarchical Bayesian model) 雏形。

3.3 生成式典范: Latent Dirichlet Allocation (LDA)

[NOTE] 为什么讲 LDA

LDA 是无监督生成式建模的旗舰, 把 lec3 的”贝叶斯建模四件套”(图模型 / 先验 / 后验 / 隐变量) 一次性展示。

3.3.1 生成过程

对语料中每篇文档:

1. 选词数 N (任意 / 从某分布抽)
2. 选主题混合比: $\theta \sim \text{Dirichlet}(\alpha)$
3. 对该文档的每个词位 $n = 1, \dots, N$:
 1. 选主题 $z_n \sim \text{Multinomial}(\theta)$
 2. 由该主题选词: $w_n \sim p(w | z_n, \beta)$

其中 β 是”主题 → 词”的多项分布矩阵 (topic-word matrix)。

3.3.2 解读

- $p(w | z, \beta)$: 第 z 个主题下的词分布
- $p(z | d, \theta)$: 第 d 篇文档下主题分布
- 一个文档 = 主题的混合; 一个主题 = 词的混合

LDA 是无监督模型——主题 z 是潜在变量, 需要通过变分推断 / Gibbs sampling 求后验。

3.3.3 应用

- 大规模语料的主题发现
- 文档相似度（主题分布的距离）
- 社会科学中作为“可解释主题”工具
- 与现代 BERT-Topic 一脉相承

3.4 判别式模型

3.4.1 核心思想

只学 $p(y | x)$ ，不浪费容量去学 $p(x)$ 。

- 优势：
 1. 不强求独立假设：特征之间可以高度相关，模型不必假装它们独立
 2. 任意特征都能塞：哪怕是手工设计的离散特征 + 词向量混搭
 3. 通常精度更高：在分类任务上判别式渐近最优
- 代价：
 1. 易过拟合：高自由度 $\rightarrow$ 必须正则化（见 lec3b）
 2. 无法生成 x ：不能做生成、不能轻易做半监督
 3. 需要更多数据收敛：参数估计与样本量挂钩

3.4.2 常见判别式模型谱系

模型	是否概率式	损失函数 / 目标	备注
Perceptron	否	0-1 mistake driven update	最早的线性判别器 (Rosenblatt 1957)
Maximum Entropy (MaxEnt) / Logistic Regression	是	交叉熵 / 对数似然	课程 lec3b 主角，下一节详细推导
SVM	否	合页损失 (hinge loss) + max margin	几何视角的线性分类器
CRF	是	序列对数似然	把 LR 推广到结构化输出
Neural Networks	是 (含 softmax)	交叉熵 + 复杂特征学习	端到端 + 表征学习

[NOTE] 直觉对比

生成式问“这个 y 类的数据长什么样？”，判别式问“怎样的边界能把不同 y 分开？”。前者更接近“建模世界”，后者更接近“做决策”。Ng 与 Jordan (2002) 的著名论文证明：样本少时生成式更稳，样本多时判别式更准。

3.5 Perceptron：第一个判别式线性分类器

3.5.1 模型形式

二分类 ($y \in \{+1, -1\}$):

$$\hat{y} = \text{sign}(w^T x + b) \quad (3.10)$$

或一般化为多类 / 结构化预测 (采用 $f(x, y)$ 特征):

$$\hat{y} = \arg \max_y w^T f(x, y) \quad (3.11)$$

3.5.2 更新规则 (核心)

对每个训练样本 (x_k, y_k) :

1. 预测 $\hat{y} = \arg \max_y w^T f(x_k, y)$
2. 若 $\hat{y} \neq y_k$ (错了):

$$w \leftarrow w + f(x_k, y_k) - f(x_k, \hat{y}) \quad (3.12)$$

3. 否则不动。

直觉: - 正确标签的特征权重 $\uparrow$ - 错误预测标签的特征权重 $\downarrow$ - 每错一次 = 一次微调

3.5.3 性质

- **收敛性**: 若数据线性可分, Perceptron 在有限步内收敛 (Novikoff 定理), 收敛步数 $O(R^2/\gamma^2)$, R = 数据半径, γ = margin。
- **非可分时震荡**: $\rightarrow$ 实际用 **averaged perceptron** (取所有迭代权重的平均) 以稳定。
- **不是概率模型**: 无 softmax, 无 log-likelihood, 不能输出概率 (但可后接 Platt scaling 转概率)。

3.5.4 与 SVM 的区别

- Perceptron: 只要分对就停手 $\rightarrow$ margin 不保证
- SVM: 在分对基础上**最大化 margin** $\rightarrow$ 泛化更好

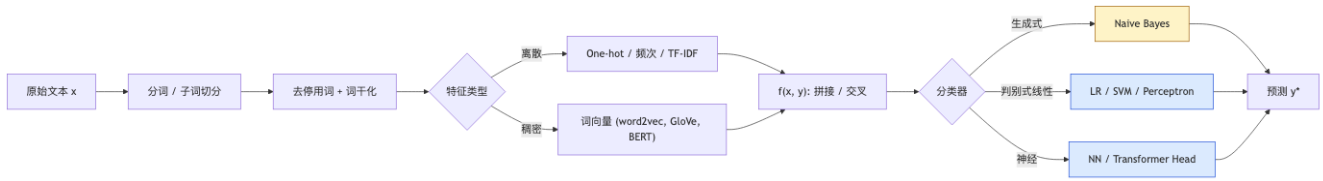
3.6 特征工程入门

无论 NB / LR / SVM / MaxEnt, 都需要把原始文本变成 $f(x, y)$ 向量。Lec3a 的”特征观”在 lec3b 完整展开, 这里先列骨架:

3.6.1 文本分类常见特征

类别	例子
词袋 (Bag-of-Words)	词是否出现 / 频次 / TF-IDF
N-gram	bi-/tri-gram 子串
词性 / 语法	POS 比例、句长
元数据	标题词、作者、发布时间
词典特征	情感词典命中数、命名实体类型
嵌入特征	平均词向量、文档向量

3.6.2 流程图



3.6.3 类不平衡

文本分类（垃圾邮件 / 罕见义项 / 罕见疾病）常遇到极端不平衡：

- 采样：上采样少数类 / 下采样多数类
- 类权重：损失函数里给少数类乘大权重
- 代价敏感学习：明确指定 FP / FN 的代价矩阵
- 指标选择：用 Macro-F1 / AUC-PR 替代 Accuracy
- 阈值调节：分类概率阈值从 0.5 调到更优点

[WARN] 易错点

在 99% 负类 / 1% 正类的数据上，“全部预测为负”也能拿到 99% Accuracy。Accuracy 在不平衡场景里是误导性指标。必须看 Precision / Recall / F1（针对少数类）。

3.7 例题：NB vs Perceptron 对比

数据：4 条短句的情感分类（pos / neg）

样本	文本	标签
1	“good movie great”	pos
2	“bad boring movie”	neg
3	“great great fun”	pos
4	“boring bad”	neg

词表：{good, great, fun, bad, boring, movie}

3.7.1 NB 估计

类先验： $P(\text{pos}) = P(\text{neg}) = 0.5$

类条件（不加平滑，仅示意）：

词	pos 频次	neg 频次	$P(\cdot \mid \text{pos})$	$P(\cdot \mid \text{neg})$
good	1	0	1/6	0
great	3	0	3/6	0
fun	1	0	1/6	0
bad	0	2	0	2/5
boring	0	2	0	2/5
movie	1	1	1/6	1/5

加 Laplace 平滑 ($|V| = 6$) 后再用。

测试句“great movie”: $-P(\text{pos}) \cdot P(\text{great} \mid \text{pos}) \cdot P(\text{movie} \mid \text{pos}) = 0.5 \cdot 4/12 \cdot 2/12 \approx 0.028 - P(\text{neg}) \cdot P(\text{great} \mid \text{neg}) \cdot P(\text{movie} \mid \text{neg}) = 0.5 \cdot 1/11 \cdot 2/11 \approx 0.008$

预测 **pos**。

3.7.2 Perceptron 训练 (示意)

特征 $f(x, y) = \$ \text{词袋} \times \text{类指示器}$; 初始 $w = 0$ 。逐样本扫一遍:

- 样本 1 $\rightarrow$ 预测随机 $\rightarrow$ 若错, 把“good/great/movie | pos”特征加 1, “good/great/movie | neg”特征减 1。
- 样本 2 $\rightarrow$ 类似更新。
- 继续直到收敛。

最终“great”在“pos”列权重 > 0 , 在“neg”列权重 < 0 ; 测试时打分取 argmax 。

[NOTE] 区别小结

NB 用计数直接得到概率; Perceptron 用**犯错驱动**累积权重。NB 在小样本下立刻能用, Perceptron 需要多轮迭代但能利用任意复杂特征。

3.8 评测、调参与协议

虽然 lec3a 主要讲建模, 但 lec3b 反复强调评测细节, 提前总结:

3.8.1 数据划分

集合	用途
训练集 (training)	拟合模型参数
开发集 (development / validation)	超参选择、早停、模型选择
测试集 (test)	最终报告, 不能用于调参

常见比例: 60/20/20 或交叉验证 (k-fold CV)。

3.8.2 评测协议陷阱

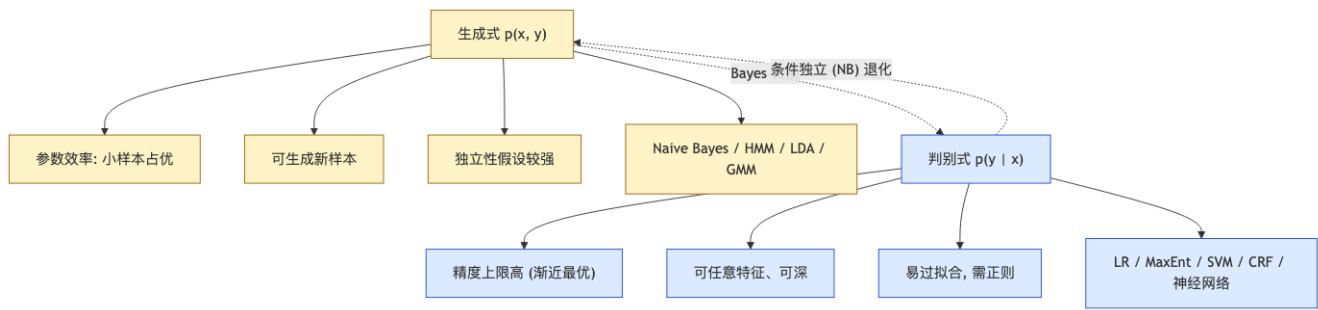
- **测试集泄漏 (test leakage)**: 训练数据中混入测试样本, 结果虚高
- **多次窥视 (peeking)**: 反复在测试集调参, 相当于把测试集变成开发集
- **不平衡指标**: 见 §3.6.3

3.8.3 误差分析

WSD / 文本分类的标准做法:

1. 在开发集上跑模型, 输出混淆矩阵 (confusion matrix)
2. 找出错误最多的“易混淆类对”
3. 抽样错例做语言学分析
4. 反过来设计新特征或修正训练数据

3.9 生成式 vs 判别式的统一对照



3.10 易错点速查

误区	修正
“Naïve Bayes 是判别式” “生成式总是不如判别式” “Perceptron 是 NN”	生成式；它建模 $p(x, y)$ 小样本下生成式更稳；判别式在大样本下渐近最优 Perceptron 是单层无激活的线性模型，常被视为 NN 起源，但本身不是多层 NN
“LDA 是有监督的” “Bayes 公式只能用一次” “判别式不能做半监督”	完全无监督；主题是潜在变量 在层级贝叶斯里反复使用，每一层都引入先验 较难但有方法 (self-training, consistency regularization)；生成式更自然
“特征工程在神经时代死了”	在小数据 / 强领域偏置任务中仍然关键；prompt engineering 是新一代特征工程

3.11 中英术语对照

中文	英文	缩写
监督学习	supervised learning	
训练 / 开发 / 测试集	training / development / test set	
函数逼近	function approximation	
生成式模型	generative model	
判别式模型	discriminative model	
联合分布	joint distribution	
条件分布	conditional distribution	
后验分布	posterior distribution	
先验分布	prior distribution	
朴素贝叶斯	Naïve Bayes	NB
隐 Dirichlet 分配	Latent Dirichlet Allocation	LDA
多项分布	Multinomial distribution	
Dirichlet 分布	Dirichlet distribution	
感知机	Perceptron	
逻辑回归	Logistic Regression	LR
最大熵模型	Maximum Entropy model	MaxEnt

中文	英文	缩写
支持向量机	Support Vector Machine	SVM
条件随机场	Conditional Random Field	CRF
词袋	Bag-of-Words	BoW
词频—逆文档频率	Term-Frequency—Inverse Document Frequency	TF-IDF
类不平衡	class imbalance	
交叉验证	k-fold cross-validation	CV
过拟合 / 欠拟合	overfitting / underfitting	
正则化	regularization	
混淆矩阵	confusion matrix	
极大似然估计	Maximum Likelihood Estimation	MLE
主题模型	topic model	
平均感知机	averaged perceptron	

[TIP] 期中复习要点

1. **必会概念**：用一句话写出生成式 / 判别式各自的建模对象 + 决策公式。2. **必会推导**： $g(x) = \arg \max_y p(y | x) = \arg \max_y p(y)p(x | y)$ 完整三行推导，并指出哪里用了 Bayes。3. **必会分类法**：判别式模型 5 个、生成式模型 4 个、对应损失函数。4. **NB 是生成式**：解释为什么 NB 是生成式（建模 $p(x | y)$ ），并给出“NB 与 LR 在二分类上等价吗？”的对比答案（NO，独立假设不同；但 NB 的判别函数形式与 LR 同为对数线性）。5. **LDA 生成过程**：能背出“每文档选 θ ，每词位选 z 再选 w ”三步。6. **Perceptron 更新规则**：能写出 $w \leftarrow w + f(x, y) - f(x, \hat{y})$ 并解释每一项含义。7. **评测协议**：解释为什么测试集不能用于调参；类不平衡下用什么指标。8. **应用思考**：给出一个 NLP 任务，并讨论该用生成式还是判别式（如：稀有意图分类 → 生成式 / few-shot；高资源文本分类 → 判别式 / Fine-tune BERT）。

Lecture 3b —对数线性模型 (Log-Linear Models / Maximum Entropy)

[TIP] 学习指南

本节核心：把判别式分类的核心算法——**对数线性模型 (log-linear model)**——从特征表示讲到目标函数、再到梯度推导、再到 SGD 训练与正则化。等价名称：**Maximum Entropy (MaxEnt)** / **多类 Logistic Regression** / **Softmax Classifier**。理解为什么“梯度 = 经验特征 - 期望特征”是该家族训练的灵魂。**前置知识**：- lec3a：生成式 vs 判别式 - lec2：NB 的对比标杆 - 微积分：链式法则、log-sum-exp、softmax 求导 - 凸优化基础：梯度上升 / 下降 **重点掌握**：1. 特征函数 $f_i(x, y)$ 的设计思路（指示器特征、特征模板、跨类别共享 vs 类特定）2. 对数线性模型公式 $p(y | x) = \frac{\exp(\theta^T f(x, y))}{\sum_{y'} \exp(\theta^T f(x, y'))}$ 3. 对数似然 $LL(\theta)$ 的展开 = 线性项 - log partition 4. 梯度推导： $\frac{\partial LL}{\partial \theta_i} = \underbrace{\sum_k f_i(x_k, y_k)}_{\text{empirical count}} - \underbrace{\sum_k \sum_{y'} f_i(x_k, y') p(y' | x_k; \theta)}_{\text{expected count}}$ 5. 训练：梯度上升 / 共轭梯度 / SGD 完整步骤 6. 正则化： ℓ_2 (L2, 平滑权重, 防过拟合) vs ℓ_1 (L1, 稀疏权重)；带正则梯度修正项 $-\alpha \theta$ 7. 与 NB 的对比：判别式 vs 生成式、特征独立性的不同假设

4.1 Motivation：从 NB 到判别式的需求

4.1.1 NB 的局限

回顾 NB-WSD：

$$s^* = \arg \max_s P(s) \prod_{w \in C} P(w | s) \quad (4.1)$$

它的问题：- **独立性假设**：上下文词在给定义项下条件独立 → 这是错的，但 NB 没办法表达“两词同时出现”的复杂特征 - **特征类型受限**：只能用“上下文是否包含某词”这类简单指示，不能轻易加 POS、句法、词向量 - **跨特征权重不能调**：每个特征都是 $P(v | s)$ 形式，不能给“主语位置词”特别高的权重

WSD 任务里，我们手头其实有很多有用的提示 (hints)：

- 邻近词
- 邻近词的 POS
- 句法依存
- 命名实体

这些 hints 性质各异，重要性也不同。我们需要：

1. 设计 hints (特征工程)
2. 加权 hints (权重学习)

3. 打分 + 排序 (推断)

这正是对数线性模型要解决的。

4.1.2 为什么是“对数”线性

模型把“分数”建成特征的线性组合：

$$\text{score}(x, y) = \sum_i \lambda_i f_i(x, y) = \lambda^T f(x, y) \quad (4.2)$$

然后通过指数化 + 归一化 (softmax) 得到概率：

$$p(y | x) = \frac{\exp(\text{score}(x, y))}{\sum_{y'} \exp(\text{score}(x, y'))} \quad (4.3)$$

取对数后，log-probability 是线性的：

$$\log p(y | x) = \lambda^T f(x, y) - \log \sum_{y'} \exp(\lambda^T f(x, y')) \quad (4.4)$$

故得名 log-linear。

[NOTE] 直觉理解

log-linear = “用线性模型给每个类别打分，再用 softmax 转概率”。它是 **logistic regression** 的多类推广 (K=2 时退化为 LR)，也是 **MaxEnt** 的等价形式 (最大熵原理给出的解恰好是 softmax 形式)。三者只是观察同一个东西的三种视角。

4.2 特征表示 (Feature Representation)

4.2.1 特征定义

特征 $f_i(x, y)$ 是 (x, y) 的函数，取值 $\in \mathbb{R}$ 。最常用是二值指示器特征：

$$f_i(x, y) = \begin{cases} 1, & \text{某条件满足} \\ 0, & \text{否则} \end{cases} \quad (4.5)$$

WSD-bank 例：

$$f_1(x, y) = \begin{cases} 1, & w_{-1} = \text{transfer} \wedge y = \text{FINANCIAL} \\ 0, & \text{otherwise} \end{cases}$$

语义解读：如果前一个词是“transfer”，那么 bank 的当前义项应当是 FINANCIAL。

4.2.2 特征向量

若设计了 m 个特征，则每个 (x, y) 对都被映射为 m 维向量：

$$f(x, y) = [f_1(x, y), f_2(x, y), \dots, f_m(x, y)]^T \quad (4.6)$$

由于多为指示器，通常是稀疏 0/1 向量，如 $[1, 0, 0, \dots, 1, 0]$ 。

4.2.3 特征模板 (Feature Templates)

实际工程里不会一个个写特征，而是用**模板批量生成**：

WSD 任务（目标词 = bank）的模板：

模板	取值范围
前 i 个词: w_{-i}	所有词形
后 i 个词: w_{+i}	所有词形
是否句首: @1	{YES, NO}
是否大写: Cap.	{YES, NO}

把模板套到句子“I cash a check in that **bank** and transfer 100 dollars to my mom.”上：

$w_{\{-1\}} = \text{that}$, $w_{\{-2\}} = \text{in}$, $w_{\{-3\}} = \text{check}$, ...
 $w_{\{+1\}} = \text{and}$, $w_{\{+2\}} = \text{transfer}$, $w_{\{+3\}} = 100$, ...
@1 = NO, Cap. = NO

考虑候选标签 Fi (financial)：

$f_{\{w_{\{-1\}}=\text{that}, \text{Fi}\}} = 1$
 $f_{\{w_{\{-2\}}=\text{in}, \text{Fi}\}} = 1$
 $f_{\{w_{\{-3\}}=\text{check}, \text{Fi}\}} = 1$
 $f_{\{w_{\{+1\}}=\text{and}, \text{Fi}\}} = 1$
 $f_{\{w_{\{+2\}}=\text{transfer}, \text{Fi}\}} = 1$
 $f_{\{w_{\{+3\}}=100, \text{Fi}\}} = 1$
 $f_{\{@1, \text{Fi}\}} = 0$
 $f_{\{\text{Cap.}, \text{Fi}\}} = 0$

整体得到稀疏特征向量 $f(x, \text{Fi}) = [1, 1, 1, 1, 1, 1, 0, 0, \dots]$ 。

类似地为 Sp, Po, En 等候选标签都构造一组特征。模型会为每个 (,) 组合学一个权重 λ_i 。

[WARN] 易错点

特征是 $f(x, y)$ 的函数——既依赖输入也依赖**候选标签**！这是与 NB（只看 x ）的关键区别。每次推断要对每个候选 y 都算一次 $f(x, y)$ 。

4.2.4 文本分类的 Bag-of-Words

更简单的 BoW 例：新闻分类，文本 = “For many soccer fans around the world, Lionel Messi is the most magical player...”。

把所有词作为特征：– One-hot: $[1, 1, 1, 1, 0, 0, \dots, 1, 0, 0]$ – 频次: $[1, 2, 1, 4, 0, 0, \dots, 90, 0, 0]$ – TF-IDF: $[0.4, 4.5, 1.0, 0.9, 0, 0, \dots, 8.1, 0, 0]$

考虑候选标签 Sp (Sports)：

$f_{\{\text{around}, \text{Sp}\}} = 1$, $f_{\{\text{fans}, \text{Sp}\}} = 1$, $f_{\{\text{game}, \text{Sp}\}} = 1$, $f_{\{\text{Messi}, \text{Sp}\}} = 1$
 $f_{\{\text{soccer}, \text{Sp}\}} = 1$, $f_{\{\text{world}, \text{Sp}\}} = 1$, $f_{\{\text{cup}, \text{Sp}\}} = 1$
 $f_{\{\text{Portugal}, \text{Sp}\}} = 0$, $f_{\{\text{Cristiano}, \text{Sp}\}} = 0$

4.2.5 TF-IDF 权重

词频 (TF)：

$$\text{TF}_{t,d} = \begin{cases} 1 + \log_{10} \text{count}(t, d), & \text{count}(t, d) > 0 \\ 0, & \text{否则} \end{cases} \quad (4.7)$$

逆文档频率 (IDF):

$$\text{IDF}_t = \log_{10} \frac{N}{\text{DF}_t} \quad (4.8)$$

其中 N = 总文档数, DF_t = 含 t 的文档数。

TF-IDF:

$$\text{TF-IDF}_{t,d} = \text{TF}_{t,d} \cdot \text{IDF}_t \quad (4.9)$$

直觉: - TF 高 $\rightarrow$ 词在该文档频繁出现, 重要 - DF 低 $\rightarrow$ 词在整个语料中罕见, 区分力高 - 两者乘积 $\rightarrow$ 兼顾”局部重要”与”全局区分”

[NOTE] 零值的意义

TF-IDF 矩阵里大量为 0: 要么 t 没在文档里 (TF=0), 要么 t 在每篇文档都出现 (IDF=0, 被认为是 stopword)。
stop-word list 与 **stemming** 是与 TF-IDF 配套的预处理。

4.3 模型形式

4.3.1 完整公式

对样本 (x, y) , 对数线性模型定义:

$$p(y | x; \theta) = \frac{\exp(\theta^T f(x, y))}{\sum_{y' \in \mathcal{Y}} \exp(\theta^T f(x, y'))} \quad (4.10)$$

或展开成分量:

$$p(y | x; \theta) = \frac{\exp(\sum_i \theta_i f_i(x, y))}{\sum_{y'} \exp(\sum_i \theta_i f_i(x, y'))} \quad (4.11)$$

其中: - 分子: 对正确类的”正分数指数化” - 分母: 所有候选类分数的归一化项 (**partition function**) - 所有概率为正, 求和为 1

4.3.2 取对数

$$\log p(y | x; \theta) = \theta^T f(x, y) - \log \sum_{y'} \exp(\theta^T f(x, y')) \quad (4.12)$$

两部分: - 第一项**线性**: $\theta^T f(x, y)$ - 第二项**归一化常数的对数** (log partition): $\log Z(x; \theta)$, 其中 $Z(x; \theta) = \sum_{y'} \exp(\theta^T f(x, y'))$

4.3.3 推断

测试时选 $y^* = \arg \max_y p(y | x)$ 。由于分母与 y 无关:

$$y^* = \arg \max_y \theta^T f(x, y) \quad (4.13)$$

→ 推断退化为求 $\theta^T f$ 在所有 y 上的最大值，完全等价于线性分类器。softmax 只在训练时为我们提供概率梯度。

4.4 训练：极大似然估计 (MLE)

4.4.1 似然函数

给定训练集 $\{(x_1, y_1), \dots, (x_K, y_K)\}$ (i.i.d.)，似然：

$$L(\theta) = \prod_{k=1}^K p(y_k | x_k; \theta) = \prod_{k=1}^K \frac{\exp(\theta^T f(x_k, y_k))}{\sum_{y'} \exp(\theta^T f(x_k, y'))} \quad (4.14)$$

取对数得对数似然：

$$LL(\theta) = \sum_{k=1}^K \log p(y_k | x_k; \theta) = \sum_{k=1}^K \left[\theta^T f(x_k, y_k) - \log \sum_{y'} \exp(\theta^T f(x_k, y')) \right] \quad (4.15)$$

目标：找 $\theta^* = \arg \max_{\theta} LL(\theta)$ 。

[NOTE] 凸性

$LL(\theta)$ 是 θ 的凹函数（线性项凹， $-\log \sum \exp$ 也凹 → 和凹）。所以梯度上升可以找到全局最优。这是 log-linear 家族最重要的数学优点。

4.4.2 梯度推导（核心!）

求 $\frac{\partial LL}{\partial \theta_i}$ 。

第一项（线性项）：

$$\frac{\partial}{\partial \theta_i} \sum_k \theta^T f(x_k, y_k) = \sum_k f_i(x_k, y_k) \quad (4.16)$$

第二项（log partition）求导，对单个样本 k ：

$$\frac{\partial}{\partial \theta_i} \log \sum_{y'} \exp(\theta^T f(x_k, y'))$$

设 $Z_k = \sum_{y'} \exp(\theta^T f(x_k, y'))$ 。

$$\begin{aligned} &= \frac{1}{Z_k} \cdot \sum_{y'} \exp(\theta^T f(x_k, y')) \cdot f_i(x_k, y') \\ &= \sum_{y'} f_i(x_k, y') \cdot \frac{\exp(\theta^T f(x_k, y'))}{Z_k} \end{aligned}$$

$$= \sum_{y'} f_i(x_k, y') \cdot p(y' | x_k; \theta) \quad (4.17)$$

合并两项：

$$\frac{\partial LL(\theta)}{\partial \theta_i} = \underbrace{\sum_k f_i(x_k, y_k)}_{\text{Empirical count}} - \underbrace{\sum_k \sum_{y'} f_i(x_k, y') p(y' | x_k; \theta)}_{\text{Expected count}} \quad (4.18)$$

4.4.3 解读：经验 vs 期望

- 经验特征计数 (Empirical count)：在训练数据中，特征 f_i 在 (输入, 真实标签) 对上的总取值。由数据给出，与模型无关。
- 期望特征计数 (Expected count)：模型自己预测的分布下，特征 f_i 的期望发生次数。与模型参数有关，每次更新都会变。

MLE 的最优条件 (KKT)：

$$\text{Empirical Count}_i = \text{Expected Count}_i, \quad \forall i \quad (4.19)$$

[NOTE] 这就是“最大熵原理”的起源

在所有满足“经验特征 = 期望特征”约束的分布里，最大熵分布唯一是 $p(y | x) = \exp(\theta^T f) / Z$ 的指数族形式。这是 MaxEnt 与 log-linear 等价的核心。

4.4.4 与 NB 的统一对比

方面	Naïve Bayes	Log-Linear
范式	生成式	判别式
建模	$p(x, y) = p(y) \prod p(x_i y)$	$p(y x) = \exp(\theta^T f) / Z$
参数估计	闭式计数	迭代优化 (梯度)
特征独立性	必须假设条件独立	不需要
特征类型	通常是“ x_i 是否 = 某值”	任意 $f(x, y)$
计算复杂度	训练 $O(N)$	训练 $O(N \times T \times Y)$
小样本表现	优	一般
大样本表现	受限于独立假设	更优

4.5 优化算法

4.5.1 批量梯度上升 (Batch Gradient Ascent)

Initialize all λ s to 0

Iterate until convergence:

$$\begin{aligned} \Delta &= \partial LL(\lambda) / \partial \lambda && \text{(全数据集梯度)} \\ \beta^* &= \operatorname{argmax}_{\beta} LL(\lambda + \beta \cdot \Delta) && \text{(线搜索, 可选)} \\ \lambda &\leftarrow \lambda + \beta^* \cdot \Delta \end{aligned}$$

特点：- 每一步要扫整个训练集——慢 - 一步走得“准”，收敛圈数少 - 用 conjugate gradient (CG) 或 L-BFGS 可大幅加速

4.5.2 随机梯度上升 (Stochastic Gradient Ascent, SGD)

```

Initialize all  $\lambda$ s, epochs  $T$ , learning rate  $\alpha$ 
For  $t$  in  $1, \dots, T$ :
    随机置换  $\pi$  of  $\{1, \dots, k, \dots\}$     (打乱数据)
    For  $i$  in  $1, \dots, k, \dots$ :
         $\Delta = \partial \text{LL}_{\{\pi(i)\}}(\lambda) / \partial \lambda$     (单样本梯度)
         $\lambda \leftarrow \lambda + \alpha \cdot \Delta$ 
Output  $\lambda$ 

```

单样本梯度 (把 (4.18) 对 k 的求和换成 $k = \text{当前样本}$):

$$\Delta_i^{(k)} = f_i(x_k, y_k) - \sum_{y'} f_i(x_k, y') p(y' | x_k; \theta) \quad (4.20)$$

特点: - 每个样本就更新一次 $\rightarrow$ 收敛快 - 噪声大但能跳出局部小坑 (虽然这里凸函数无局部最优) - 学习率 α 通常衰减: $\alpha_t = \alpha_0 / (1 + t/\tau)$ - 现代变体: **Mini-batch SGD** (折中)、**Adam**、**AdaGrad**

4.5.3 单步 SGD 手算示例

设 2 类问题 $y \in \{A, B\}$, 2 维特征: - $f(x, A) = [1, 0]$, $f(x, B) = [0, 1]$ - 当前 $\theta = [0.5, 0.0]$ - 真实标签 $y = A$ - 学习率 $\alpha = 0.1$

第一步: 算分数 - $\theta^T f(x, A) = 0.5 \cdot 1 + 0 \cdot 0 = 0.5$ - $\theta^T f(x, B) = 0.5 \cdot 0 + 0 \cdot 1 = 0$

第二步: softmax

$$p(A | x) = \frac{e^{0.5}}{e^{0.5} + e^0} = \frac{1.6487}{1.6487 + 1} = 0.6225$$

$$p(B | x) = \frac{1}{1.6487 + 1} = 0.3775$$

第三步: 算梯度 (对每个 θ_i 用 (4.20))

对 θ_1 : - Empirical: $f_1(x, A) = 1$ - Expected: $f_1(x, A) \cdot p(A) + f_1(x, B) \cdot p(B) = 1 \cdot 0.6225 + 0 \cdot 0.3775 = 0.6225$ - $\Delta_1 = 1 - 0.6225 = 0.3775$

对 θ_2 : - Empirical: $f_2(x, A) = 0$ - Expected: $f_2(x, A) \cdot p(A) + f_2(x, B) \cdot p(B) = 0 \cdot 0.6225 + 1 \cdot 0.3775 = 0.3775$ - $\Delta_2 = 0 - 0.3775 = -0.3775$

第四步: 更新

$$\theta_1 \leftarrow 0.5 + 0.1 \cdot 0.3775 = 0.5378$$

$$\theta_2 \leftarrow 0.0 + 0.1 \cdot (-0.3775) = -0.0378$$

直觉: 模型本来已经倾向 A ($p(A) = 0.62$), 但梯度仍把 θ_1 推得更高、把 θ_2 拉得更低, 因为真实标签是 A 且其期望概率还不到 1。模型不断“自信地预测真实标签”才会让梯度趋近 0。

4.5.4 等价表述: 最小化负对数似然

把目标改写为最小化形式:

$$\theta^* = \arg \min_{\theta} \left[- \sum_k \log p(y_k | x_k; \theta) \right] \quad (4.21)$$

$-\log p(y | x)$ 就是 **log loss / cross-entropy loss**。SGD 在最小化形式下变成 **SGD-Descent**。

[NOTE] 三个名字一回事

“最大化对数似然 (MLE)” = “最小化交叉熵” = “最小化对数损失”。三种说法在不同教材里出现，本质完全相同。这也是为什么 PyTorch 里 `nn.CrossEntropyLoss` 直接对应 log-linear 的训练目标。

4.6 正则化 (Regularization)

4.6.1 过拟合问题

考虑特征：

$$f_1(x, y) = \begin{cases} 1, & x \ni \text{NFL} \wedge y = \text{Sports} \\ 0, & \text{otherwise} \end{cases}$$

训练数据中 NFL 出现 100 次，每次类别都是 Sports。

MLE 的最优条件 (4.19) 要求：

$$\underbrace{\sum_k f_1(x_k, y_k)}_{=100} = \underbrace{\sum_k \sum_y p(y | x_k; \theta) f_1(x_k, y)}_{=\sum_k p(\text{Sports} | x_k)}$$

要让右边 = 100，对每个含 NFL 的样本必须 $p(\text{Sports} | x_k) = 1$ ，这要求 $\theta_1 \rightarrow \infty$ 。**MLE 给出无穷大权重。**

后果：测试集上任何含 NFL 的样本都会被强制判为 Sports，哪怕语义上是政治新闻“NFL Players Association sues the government”。模型对训练数据的偶然规律深信不疑。

[WARN] 易错点

“过拟合 (overfitting)”不是说精度 100% 就一定过拟合，而是模型“过于精确地拟合训练数据中的偶然规律，导致泛化能力下降” (牛津词典)。MLE 在判别式模型上没有自我约束，正则化是必需的。

4.6.2 L2 正则化

修改损失函数，加上权重平方惩罚：

$$LL_{\text{reg}}(\theta) = \sum_k \theta^T f(x_k, y_k) - \sum_k \log \sum_{y'} \exp(\theta^T f(x_k, y')) - \frac{\alpha}{2} \|\theta\|_2^2 \quad (4.22)$$

展开 $\|\theta\|_2^2 = \sum_i \theta_i^2$ ：

$$LL_{\text{reg}}(\theta) = \sum_k \theta^T f(x_k, y_k) - \sum_k \log \sum_{y'} \exp(\theta^T f(x_k, y')) - \frac{\alpha}{2} \sum_i \theta_i^2 \quad (4.23)$$

梯度修正：在 (4.18) 上每个分量减 $\alpha\theta_i$ ：

$$\frac{\partial LL_{\text{reg}}}{\partial \theta_i} = \sum_k f_i(x_k, y_k) - \sum_k \sum_{y'} f_i(x_k, y') p(y' | x_k; \theta) - \alpha \theta_i \quad (4.24)$$

效果：每一步更新都把 θ 往零拉一点 $\rightarrow$ 权重不会无限增大。

贝叶斯解读：等价于给 θ 加零均值高斯先验 $\theta_i \sim \mathcal{N}(0, 1/\alpha)$ ，做 MAP 估计。

4.6.3 L1 正则化

把惩罚换成 L1 范数：

$$\theta^* = \arg \min_{\theta} \text{loss}(\theta) + \alpha \|\theta\|_1, \quad \|\theta\|_1 = \sum_i |\theta_i| \quad (4.25)$$

特性：– 在 0 点不可导 $\rightarrow$ 需 subgradient / proximal methods (如 ISTA / OWL-QN) – 倾向把不重要的 θ_i 推到精确 0 $\rightarrow$ 稀疏解 (sparsity) – 适合高维特征空间，做特征选择

贝叶斯解读：相当于给 θ 加 Laplace 先验 $\theta_i \sim \text{Laplace}(0, 1/\alpha)$ 。

4.6.4 L1 vs L2 对比

维度	L1	L2
范数	$\sum_i \theta_i $	$\sum_i \theta_i^2$
解的稀疏性	强 (很多 0)	弱 (小但非零)
在 0 点是否可导	否	是
优化方法	次梯度 / OWL-QN	标准梯度
贝叶斯先验	Laplace	Gaussian
适用场景	高维稀疏特征、特征选择	一般正则化首选

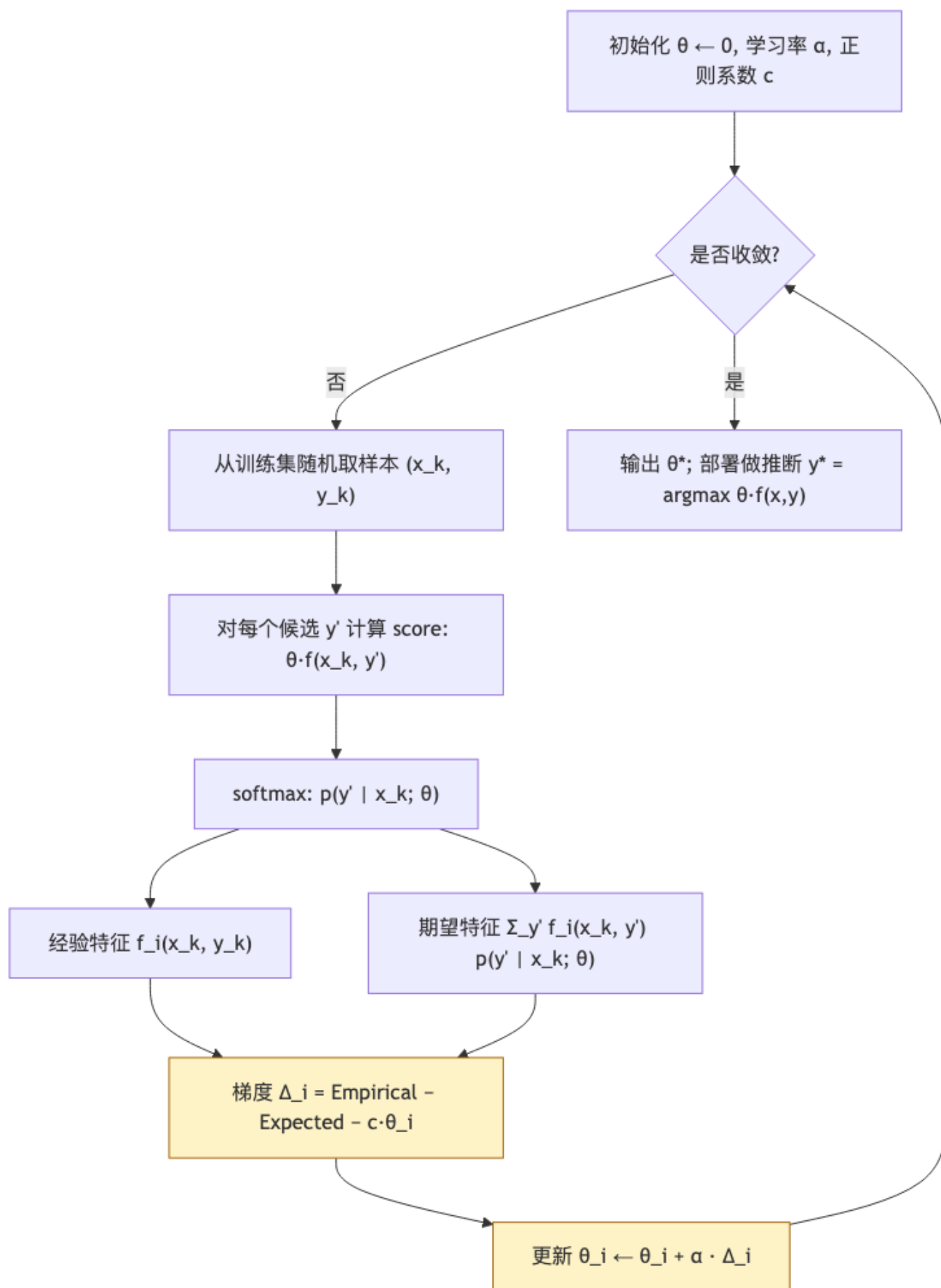
[NOTE] Elastic Net

实际可同时使用： $\alpha_1 \|\theta\|_1 + \alpha_2 \|\theta\|_2^2$ ，称为 Elastic Net，兼顾稀疏与平滑。

4.6.5 正则化系数 α 怎么选

- 在开发集 (dev set) 上网格搜索：典型范围 $\{10^{-4}, 10^{-3}, \dots, 10\}$
- 交叉验证 (cross-validation)
- 早停 (early stopping) 与正则化经常组合使用

4.7 训练循环（图示）



4.8 与其他模型的关系

4.8.1 与逻辑回归 (Logistic Regression)

- 二分类 + 特征不显式依赖 y 时, log-linear 退化为标准 LR:

$$p(y = 1 | x) = \sigma(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}} \quad (4.26)$$

- 多类 LR / Softmax classifier = log-linear (每类一个独立权重向量)

4.8.2 与 MaxEnt

- MaxEnt 的“最大熵 + 特征期望约束”优化问题, 对偶解恰好就是 log-linear 形式
- 历史上 NLP 里更喜欢叫 MaxEnt, 机器学习圈喜欢叫 log-linear / softmax

4.8.3 与 CRF

- CRF = 把 log-linear 推广到结构化输出 (如序列标注)
- 分子分母里的 y 从单个标签换成整条标签序列, partition function 用动态规划算

4.8.4 与神经网络

- log-linear = “线性特征 + softmax”
- 神经网络 = “可学习的非线性特征 + softmax”
- 现代深度模型的最后一层分类头就是 log-linear, 只是底层 $f(x, y)$ 由神经网络自动抽取

4.9 评测复盘

概念	公式 / 含义
Accuracy	正确预测数 / 总数
Precision (per class)	$tp / (tp + fp)$
Recall (per class)	$tp / (tp + fn)$
F_1	$2 \cdot P \cdot R / (P + R)$
Macro- F_1	类级别 F_1 的算术平均
Micro- F_1	实例级别 P/R 合并后算 F_1 (多分类下 = Accuracy)
Cross-Entropy	$-\sum_k \log p(y_k x_k)$, 与训练目标一致
PPL (Perplexity)	$\exp(CE/N)$, 主要用于语言模型

评测协议: - 训练集 → 拟合 θ - 开发集 → 调正则系数 α 、特征模板、学习率 - 测试集 → 报告最终结果, 只看一次

4.10 例题：log-linear vs NB

任务：判断“transfer → bank”上下文中 bank 的义项。

数据：训练集 100 句，特征 $f(x, y)$ 设计如下：

$$f_1 : w_{-1} = \text{transfer}, y = \text{Financial}$$

$$f_2 : w_{+1} = \text{river}, y = \text{Geographic}$$

NB 做法：估计 $P(\text{transfer} \mid \text{Financial})$, $P(\text{transfer} \mid \text{Geographic})$ 等条件概率，再乘上类先验。如果 $P(\text{transfer} \mid \text{Geographic})$ 因为数据稀疏估出 0，整个 Geographic 类的后验立刻为 0（需平滑）。

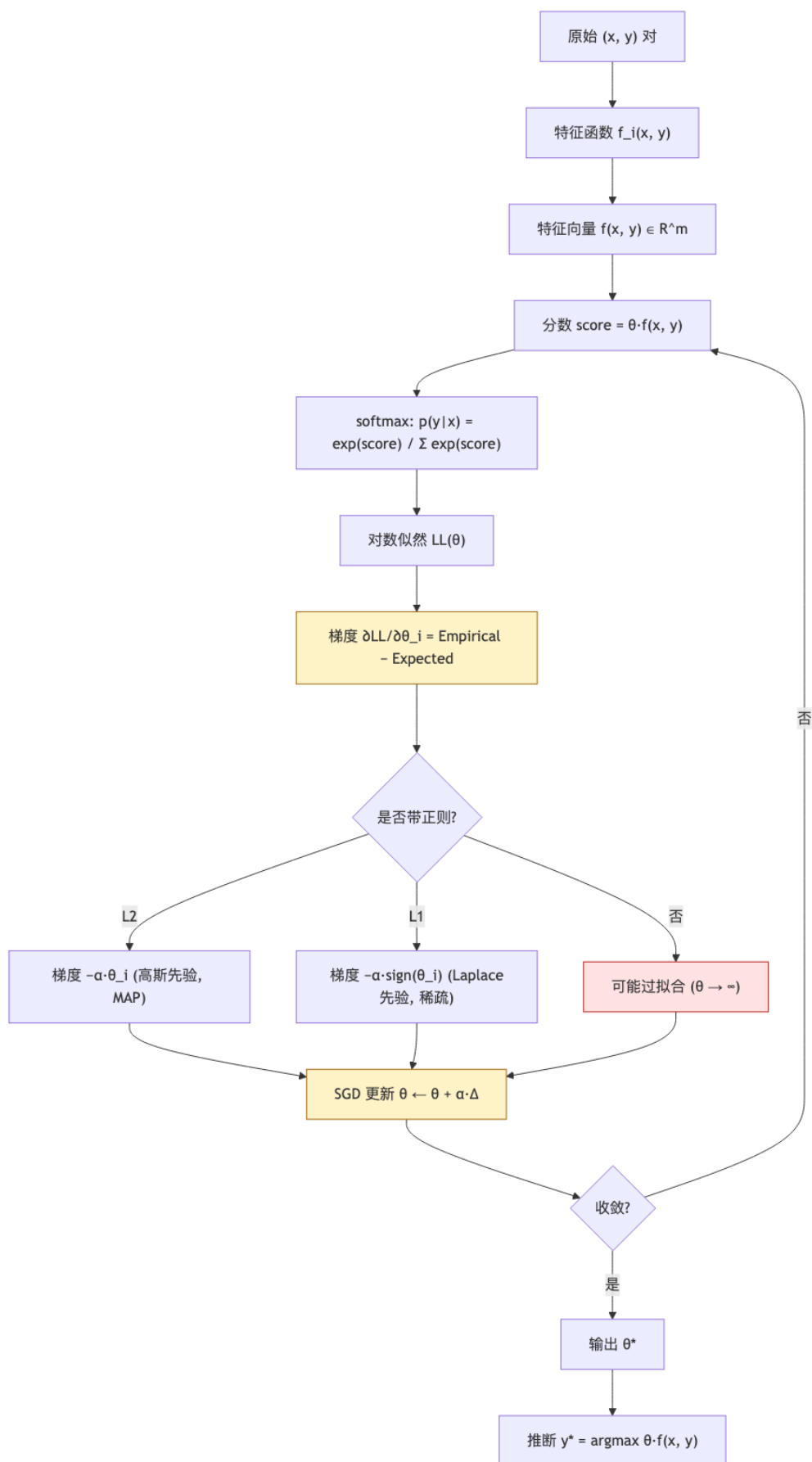
Log-linear 做法：直接学 θ_1 控制“ $w_{-1} = \text{transfer} + \text{Financial}$ ”这条规则的权重。无论 Geographic 类下 transfer 出现多少次， θ_1 都会被推大（如果训练数据强烈支持 Financial）。**不需要担心条件概率零。**

测试句：“I will **transfer** money to the **bank** of the river.”（这里 transfer 是动词、 w_{-1} = “the”）

- 应用模板得到的特征向量 → 计算 $\theta^T f(x, \text{Financial})$ 与 $\theta^T f(x, \text{Geographic})$
- w_{+1} = “of” 但其他位置出现 “river” → θ 学到的更复杂模式可能反过来支持 Geographic
- argmax 得最终义项

→ log-linear 利用**多种特征 + 学习权重**，比 NB 灵活。

4.11 概念关系图 (mermaid)



4.12 易错点速查

误区	修正
“log-linear 与 logistic regression 不同”	二者本质相同；多类版叫 softmax regression / MaxEnt
“softmax 必须在最后输出层”	softmax 可用于任意需要“分数 → 概率”的位置
“梯度 = Empirical - Expected 只是巧合”	这是指数族分布的通用结论，所有 log-linear 模型都满足
“L1 让所有权重变小”	L1 让 部分权重变为精确 0 ，其余几乎不变；L2 才是“所有权重都变小一点”
“Naïve Bayes 与 Log-linear 都是判别式”	NB 是 生成式 ；Log-linear 是判别式
“正则系数越大越好”	太大会 欠拟合 ；要在开发集上选
“log-linear 一定凸”	不带正则就是凸；加 L1 仍凸但不可导；加 L2 严格凸
“训练样本足够多就不用正则”	即使样本多，特征也常超量；正则化几乎总该开
“SGD 一定比 batch 好”	SGD 收敛快但噪声大；批量大小要权衡
“log loss 与 cross-entropy 是两个东西”	二分类下完全相同；多分类下 cross-entropy = log loss = NLL

4.13 中英术语对照

中文	英文	缩写
对数线性模型	log-linear model	
最大熵模型	Maximum Entropy model	MaxEnt
逻辑回归	Logistic Regression	LR
Softmax 分类器	Softmax classifier	
特征函数	feature function	
指示器特征	indicator feature	
特征模板	feature template	
特征向量	feature vector	
分数 / 配分函数	score / partition function	
极大似然估计	Maximum Likelihood Estimation	MLE
经验特征计数	empirical feature count	
期望特征计数	expected feature count	
梯度上升 / 下降	gradient ascent / descent	
共轭梯度	conjugate gradient	CG
随机梯度下降	Stochastic Gradient Descent	SGD
小批量 SGD	mini-batch SGD	
学习率	learning rate	
交叉熵损失	cross-entropy loss	CE
负对数似然	negative log-likelihood	NLL
对数损失	log loss	
正则化	regularization	
L2 / L1 范数	L2 / L1 norm	
高斯 / Laplace 先验	Gaussian / Laplace prior	
最大后验	Maximum A Posteriori	MAP
过拟合 / 欠拟合	overfitting / underfitting	

中文	英文	缩写
词频 / 逆文档频率	term frequency / inverse document frequency	TF-IDF
词袋表示	Bag-of-Words	BoW
独热向量	one-hot vector	
停用词	stop word	
词干化	stemming	
条件随机场	Conditional Random Field	CRF
元素稀疏性	sparsity	

[TIP] 期中复习要点

1. 写出公式：默写 $p(y \mid x) = \frac{\exp(\theta^T f(x, y))}{\sum_{y'} \exp(\theta^T f(x, y'))}$ 。2. 梯度推导：从 $LL(\theta)$ 一步步推到

$\frac{\partial LL}{\partial \theta_i} = \text{Empirical} - \text{Expected}$ ，并解释每一项的物理含义。3. 特征设计：能为 WSD 的 bank 任务写出至少 4 个不同模板的特征 $f_i(x, y)$ 。4. NB vs Log-linear：列表对比”建模对象 / 独立性假设 / 训练复杂度 / 小样本表现”四项。5. SGD 步骤：能写出”初始化 → 随机置换 → 取样本 → 计算梯度 → 更新”全流程。6. 正则化对比：L1 vs L2 的解的稀疏性、贝叶斯先验对应、可导性。7. 手算：给一个 2 类 / 2 特征 / 1 样本，能算 softmax 概率、单步梯度、单步更新。8. 过拟合分析：解释“NFL → Sports”100/100 训练样本下 $\theta \rightarrow \infty$ 的现象，并说明 L2 正则如何阻止。9. 三个等价名：MLE / NLL / Cross-Entropy → 三者是同一个目标，能在卷面上互换。10. 凸性论证：解释为什么 log-linear 的 $LL(\theta)$ 是凹函数，从而梯度上升能找到全局最优。

Lecture 4 —神经语言模型 (Neural Language Models)

[TIP] 学习指南

本节核心：从 log-linear (最大熵) LM 出发，把“用特征模板 + softmax 做下一词分类”逐步换成「词嵌入 + 前馈网络 + softmax」的 Bengio 2003 模型，再用 CNN 拓宽窗口、用 RNN/LSTM 真正建模整段序列。这条线把 LM 从“计数 + 回退”彻底搬到“可学习的特征 + 连续表示”。前置知识：- lec3 Log-linear / softmax 分类 - lec4 n-gram LM、Markov 假设、PPL - 神经网络基础：affine 层、tanh/ReLU、SGD - 链式法则求导
重点掌握：1. NLM 的核心收益：词嵌入 → 同义词泛化 + 突破“计数为零” 2. Bengio FFNN-LM 的 lookup → concat → tanh → softmax 流程及参数清单 3. softmax + cross-entropy 的反向传播形式 4. CNN 在 LM 中的用法 (filter / pooling) 5. RNN-LM 的递推方程、BPTT 一步推导 6. RNN 的梯度消失原因 → LSTM / GRU 的加性连接 + gating

1. Motivation：从 n-gram 走向“分类式”LM

1.1 用分类视角看 LM

n-gram LM 估的是 $p(w_i | w_{i-2}, w_{i-1})$ ——本质上是一个“给定上下文，预测下一词”的多分类问题。能否直接用 log-linear / softmax 分类器去解？

输入：长度可变的历史 $h_{1:i-1}$ 输出： $|\mathcal{V}|$ 类（每个词一类）

1.2 Log-Linear / 最大熵 LM (NLM-0)

把分类器套到 LM 上：

$$p(w_i | h_{1:i-1}) = \frac{\exp(\lambda \cdot f(h_{1:i-1}, w_i))}{\sum_{w'} \exp(\lambda \cdot f(h_{1:i-1}, w'))} \quad (4.18)$$

其中 f 是手工特征模板（前 $n - 1$ 个词、词缀、词性等）， λ 是权重向量。

优点 - 几乎无零事件 (softmax 永远 > 0) - 概率合法 - 可灵活组合特征，超越纯计数

缺点 - 仍需手工 f - 训练贵：分母 $\sum_{w'}$ 要遍历 $|\mathcal{V}|$ - 优化非凸（含特征交互时）- 1990 年代被认为不实用

[NOTE] 历史

Max-Ent LM 与现代神经 LM 在数学骨架上一脉相承：都是“softmax over vocabulary”。区别只是把 $f(\cdot)$ 从手工模板换成了“嵌入 + 神经网络隐藏层”。

2. 神经网络基础回顾

单元 (unit):

$$z = \sum_i w_i x_i + b, \quad y = f(z) \quad (4.19)$$

其中 $f(\cdot)$ 是非线性激活函数:

激活	公式	范围	备注
Sigmoid	$\sigma(z) = \frac{1}{1 + e^{-z}}$	$(0, 1)$	输出概率风格, 易饱和
tanh	$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$(-1, 1)$	零中心, NLP 常用
ReLU	$\max(z, 0)$	$[0, \infty)$	简单、不饱和、可能“死亡”

堆叠多层 (affine $\rightarrow$ 非线性 $\rightarrow$ affine $\rightarrow$ 非线性 $\rightarrow \dots$) 即增加模型容量。

3. NLM-1: Bengio 2003 前馈神经 LM

3.1 词嵌入 (word embeddings)

把每个词 w_i (one-hot $w_i \in \mathbb{R}^V$, 仅第 i 维为 1) 通过查表矩阵 $M \in \mathbb{R}^{V \times d}$ 映射为低维稠密向量:

$$m_{w_i} = w_i^\top M \in \mathbb{R}^d \quad (4.20)$$

- $d \ll V$ (典型 $V \sim 10^4-10^5$, $d \sim 100-300$)
- M 是模型参数, 与 LM 一起 SGD 学
- 不需提前训练或额外标注
- 学完后语义相近的词在 $\mathbb{R}^d$ 中聚集 (car, van, vehicle)

3.2 模型方程

输入: 前 $n-1$ 个词的嵌入 $m_{w_1}, \dots, m_{w_{n-1}}$ 。

$$p(w_n \mid w_1, \dots, w_{n-1}) = \text{softmax}\left(b + \sum_{j=1}^{n-1} m_{w_j} A_j + W \tanh\left(u + \sum_{j=1}^{n-1} m_{w_j} T_j\right)\right) \quad (4.21)$$

参数 (注意维度):

符号	形状	含义
M	$V \times d$	词嵌入矩阵
T_j	$d \times H$	第 j 个位置 $\rightarrow$ 隐层投影
u	H	隐层偏置
W	$V \times H$	隐层 $\rightarrow$ 输出
A_j	$d \times V$	“直连”位置 $j \rightarrow$ 输出 (残差风格)
b	V	输出偏置

直观流程: **lookup** $\rightarrow$ **concat** $\rightarrow$ **tanh** $\rightarrow$ **affine** $\rightarrow$ **softmax** (外加 $\sum A_j$ 直连分支)。

参数量估算 (Bengio 2003): $V \approx 18000, d = 30/60, H = 50/100, n = 6$ 。- 主项: VH (约 1.8×10^6) + Vd (约 5×10^5) + $(n-1)dH$ (约 30000) - 总计 $\approx 460V + 3w$ (讲义记法), 即约几百万参数 - vs 一个 trigram 表 $V^3 \approx 5 \times 10^{12}$ ——参数缩小 6 个数量级, 且能泛化

3.3 参数压缩技巧

- 去掉 A_j (直连分支): 约束 $A_j = 0$, 省掉 $(n-1)dV$ 参数; 训练慢但泛化好
- 共享 T_j + 取平均: 把 $\sum_j m_{w_j} T_j$ 换成 $\bar{m}T$, 即 **CBOW (Continuous Bag of Words)** 的雏形

3.4 训练目标 + softmax + cross-entropy 反向传播

设网络输出 logit $z \in \mathbb{R}^V$, softmax 后得到

$$\hat{p}_k = \frac{\exp(z_k)}{\sum_j \exp(z_j)}, \quad k = 1, \dots, V \quad (4.22)$$

真实下一词为 y , one-hot 标签 y 。Cross-entropy 损失:

$$\mathcal{L} = - \sum_k y_k \log \hat{p}_k = - \log \hat{p}_y \quad (4.23)$$

整个数据集就是

$$\mathcal{L}_{\text{total}} = - \sum_{i=1}^M \log p(w_i | h_{1:i-1}) \quad (4.24)$$

这正是负对数似然, 最小化它等价于 MLE (与 n-gram LM 的目标一致)。

关键梯度: 对 logit z_k 求导:

$$\frac{\partial \mathcal{L}}{\partial z_k} = \hat{p}_k - y_k \quad (4.25)$$

推导:

$$\frac{\partial \mathcal{L}}{\partial z_k} = -\frac{1}{\hat{p}_y} \frac{\partial \hat{p}_y}{\partial z_k}$$

由 $\hat{p}_y = \frac{e^{z_y}}{\sum_j e^{z_j}}$, 链式得

$$\frac{\partial \hat{p}_y}{\partial z_k} = \begin{cases} \hat{p}_y(1 - \hat{p}_y) & k = y \\ -\hat{p}_y \hat{p}_k & k \neq y \end{cases}$$

代入分两种情况合写: $\frac{\partial \mathcal{L}}{\partial z_k} = \hat{p}_k - y_k = \hat{p}_k - \mathbb{1}[k = y]$ 。这就是为什么 softmax+CE 用起来“很干净”——梯度就是预测与 one-hot 之差。

随后用链式法则继续往下传:

$$\frac{\partial \mathcal{L}}{\partial W} = (\hat{p} - y) h^\top, \quad \frac{\partial \mathcal{L}}{\partial h} = W^\top (\hat{p} - y) \quad (4.26)$$

其中 $h = \tanh(u + \sum_j m_{w_j} T_j)$ 。再对 h 用 $\tanh'(z) = 1 - \tanh^2(z)$ 反向传到 T_j 、嵌入 m_{w_j} 和 M 。

[WARN] 易错点

1. softmax 出来已是概率, 不要再套 sigmoid
2. 训练目标常写作 `nn.CrossEntropyLoss`, PyTorch 中它内部做 log-softmax + NLL, 传入 logits 而非概率
3. 别忘了梯度 $\partial \mathcal{L} / \partial M$ 是稀疏的——只更新出现的词的那一行
4. softmax 数值稳定: 先减 $\max_j z_j$ 再 exp

3.5 训练细节

- 优化: MLE + SGD (mini-batch)
- 初始化: 随机小值 (Bengio 时代用均匀分布; 现代用 Xavier / He)
- 学习率: 典型 0.01; 现代用 Adam / AdamW
- 不是凸优化: 可能多个局部最优
- CPU 训练极慢 (Bengio 2003 用 CPU 训练数日, 现代用 GPU 几小时)

3.6 为什么 FFNN-LM 比 n-gram 好?

仍是 n -gram 框架 (只看前 $n - 1$ 个词), 但:

1. 词嵌入: 相似词共享统计强度。例如训练集只见过 the cat is running, 测试集出现 the dog is running, 因为 cat 和 dog 嵌入相近, 新句子也能获得合理概率。n-gram 完全做不到。
2. 特征组合: 多层 tanh 自动学到非线性交互, 超过 log-linear。
3. 维度自由: 可在高维特征空间搜索 conjunctive features。
4. 参数与 V 解耦: trigram 参数随 V^3 爆炸, FFNN-LM 主要 $VH + Vd$ 线性增长。

3.7 仍存在的问题

- 仍只看固定 $n - 1$ 词的窗口
- 无法建模真正的序列: big blue ball 与 blue big ball 在 BoW 风格下可能等价
- 没有可解释参数: 单个权重无直观含义
- 需要更灵活、能记忆长历史的架构 $\rightarrow$ CNN / RNN

4. NLM-2: 卷积神经网络 LM

4.1 动机

希望突破”固定 n “，并对变长历史 $X = [m_{w_1}, \dots, m_{w_{t-1}}]$ (一个 $d \times (t-1)$ 矩阵) 做滑窗特征抽取。

4.2 卷积层

对位置 m 、滤波器 k :

$$X^{(1)}[k, m] = f\left(b_k + \sum_{i=1}^d \sum_{j=1}^{s_w} C^{(k)}[i, j] \cdot X^{(0)}[i, m + j - 1]\right) \quad (4.27)$$

- $C^{(k)} \in \mathbb{R}^{d \times s_w}$: 第 k 个滤波器
- s_w : 滑窗宽度
- f : 非线性 (tanh / ReLU)

$X^{(l)}[:, m]$ 可视为”第 l 层、位置 m 的隐表示”。

超参: 层数、 f 、 s_w 、滤波器个数 k (各层可不同)。

4.3 池化 (pooling)

把可变长度 $X^{(D)}$ ($d \times (t-1)$) 汇聚成定长向量 $z \in \mathbb{R}^d$:

- Max pooling: $z_i = \max_j X^{(D)}[i, j]$
- Average pooling: $z_i = \frac{1}{t-1} \sum_j X^{(D)}[i, j]$

最后 $\hat{p} = \text{softmax}(z)$ 得到下一词分布。

4.4 训练 & 应用

- Cross-entropy 逐词损失、SGD (同 NLM-1)
- Kim (2014) 在文本分类上把 CNN 发扬光大
- Dilated CNN (Yu & Koltun, 2015): 随层深度扩大感受野, 跳过更多词, 模拟长程依赖

[NOTE] CNN 的视角

CNN 把 LM 看作”局部模式 + 全局聚合”。 s_w -gram 由 filter 隐式学到, 比手写 n -gram 特征模板更灵活。但仍是”滑窗”, 对超长依赖不友好。

5. NLM-3: 循环神经网络 LM

5.1 RNN-LM 方程

不再使用固定窗口, 对每个时间步 i 维护一个隐状态 s_i 存储整段历史:

$$s_0 = 0 \quad (4.28)$$

$$s_i = \tanh(m_{w_i} W_w + s_{i-1} W_s + b) \quad (4.29)$$

$$p(w_{i+1} \mid h_{1:i}) = \text{softmax}(s_i W_p) \quad (4.30)$$

- W_w, W_s, W_p : 跨位置共享的参数
- s_i : 定长向量, 理论上可记住任意远的过去

5.2 训练目标

逐位置 cross-entropy:

$$\mathcal{L} = - \sum_{i=1}^M \log p(w_{i+1} \mid h_{1:i}) \quad (4.31)$$

5.3 RNN 展开 + BPTT 一步推导 (Worked Example)

考虑 2-token 序列 $[w_1, w_2]$, 目标预测下一词 w_3 。

前向:

$$s_1 = \tanh(m_{w_1} W_w + 0 \cdot W_s + b)$$

$$s_2 = \tanh(m_{w_2} W_w + s_1 W_s + b)$$

$$\hat{p} = \text{softmax}(s_2 W_p), \quad \mathcal{L} = -\log \hat{p}_{w_3}$$

反向 (BPTT, Back-Propagation Through Time):

记 $\delta_2 = W_p^\top (\hat{p} - y)$ (从 softmax+CE 来), 令 $z_2 = m_{w_2} W_w + s_1 W_s + b$:

$$\frac{\partial \mathcal{L}}{\partial z_2} = \delta_2 \odot (1 - \tanh^2(z_2))$$

对 W_s 的”当前时刻”梯度:

$$\frac{\partial \mathcal{L}}{\partial W_s} \supset s_1^\top \cdot \frac{\partial \mathcal{L}}{\partial z_2}$$

继续传到 s_1 :

$$\frac{\partial \mathcal{L}}{\partial s_1} = \frac{\partial \mathcal{L}}{\partial z_2} \cdot W_s^\top$$

进一步传到 $z_1 = m_{w_1} W_w + b$:

$$\frac{\partial \mathcal{L}}{\partial z_1} = \frac{\partial \mathcal{L}}{\partial s_1} \odot (1 - \tanh^2(z_1))$$

最后对 W_s 的总梯度叠加两次贡献（位置 1 没有 s_0 贡献，因为 $s_0 = 0$ ）。

[WARN] 易错点：参数共享

同一矩阵 W_s, W_w, W_p 在每个时间步出现一次，**梯度要在所有时间步上累加**，不是在每步独立更新。

5.4 梯度消失 / 爆炸

把递推按时间展开 T 步： $s_T \propto \prod_{t=1}^T J_t$ ，其中 J_t 涉及 W_s 与 $\tanh'$ 。 $\tanh'(z) \in (0, 1]$ ，且 $\|W_s\|$ 通常较小：

- 若 $\|J_t\| < 1$ ： T 步连乘 $\rightarrow 0$ ，**梯度消失**，远处的词学不到
- 若 $\|J_t\| > 1$ ： T 步连乘 $\rightarrow \infty$ ，**梯度爆炸**，可用 gradient clipping 缓解

实证：vanilla RNN 难以学到 $> 20-50$ 步的依赖。

5.5 LSTM 与 GRU：长程依赖的”加性 + 门控”

LSTM (Hochreiter & Schmidhuber, 1997) 引入：– **Cell state** c_t ：通过**加性**更新，不易消失 – **Forget gate** $f_t = \sigma(\cdot)$ ：决定保留多少旧记忆 – **Input gate** $i_t = \sigma(\cdot)$ ：决定写入多少新信息 – **Output gate** $o_t = \sigma(\cdot)$ ：决定向外输出多少

核心更新：

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (4.32)$$

$$h_t = o_t \odot \tanh(c_t) \quad (4.33)$$

加性而非乘性传播是关键：梯度沿 c 走时，主项是 $\prod_t f_t$ （接近 1 时几乎不衰减）。

GRU (Cho et al., 2014) 简化版：合并 forget + input 为 update gate，并引入 reset gate；参数更少、计算更快、效果常常持平。

5.6 RNN-LM 总结

- 可在原则上利用任意长历史
- 比 CNN 更贴合”语言是序列”这一事实
- 经过 LSTM/GRU + 堆叠 + 双向化，在 Transformer 出现前是 NLP 的主导架构
- 但训练慢（无法并行化时间步）、长程依然受限——后续被 **Transformer** 取代

6. 概念对照与统一视角

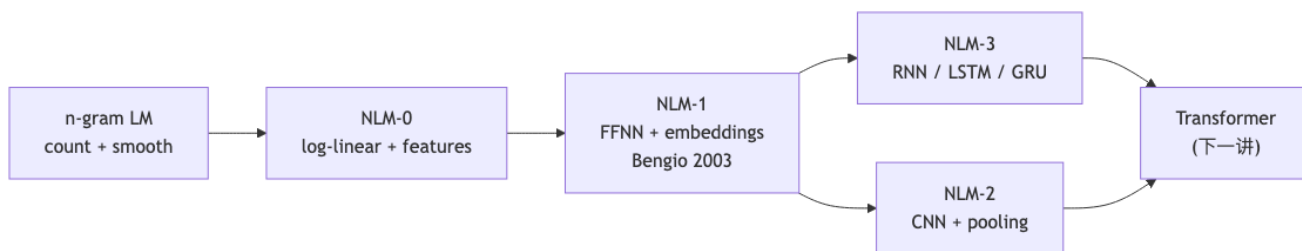
到目前为止，所有 LM 都在算同一个目标 $p(w_i | h_{1:i-1})$ ，差异只在如何参数化条件概率：

模型	公式骨架
传统 n-gram LM	$p(w_i w_{i-n+1:i-1}) = c(\cdot)/c(\cdot) + \text{平滑}$
NLM-0 (MaxEnt)	$\frac{\exp(\lambda \cdot f(h, w_i))}{\sum_{w'} \exp(\lambda \cdot f(h, w'))}$
NLM-1 (FFNN)	$\text{softmax}(b + \sum_j m_{w_j} A_j + W \tanh(u + \sum_j m_{w_j} T_j))$
NLM-2 (CNN)	$\text{softmax}(\text{FFN}(\text{conv}(h_{1:i-1})))$
NLM-3 (RNN/LSTM)	$\text{softmax}(s_i W_p), s_i = \text{LSTM}(s_{i-1}, w_i)$

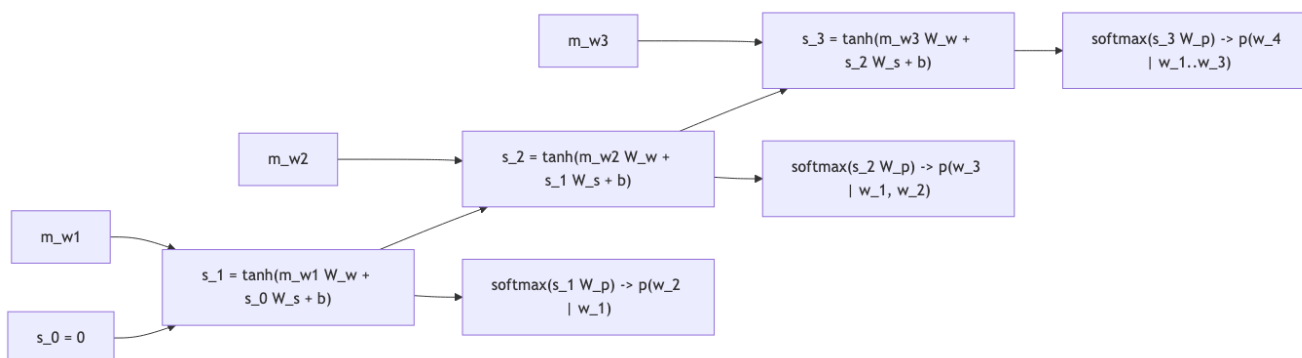
整体趋势：历史越来越长 → 参数越来越多 → 数据需求越来越大 → 性能越来越好。

7. 概念关系图 (mermaid)

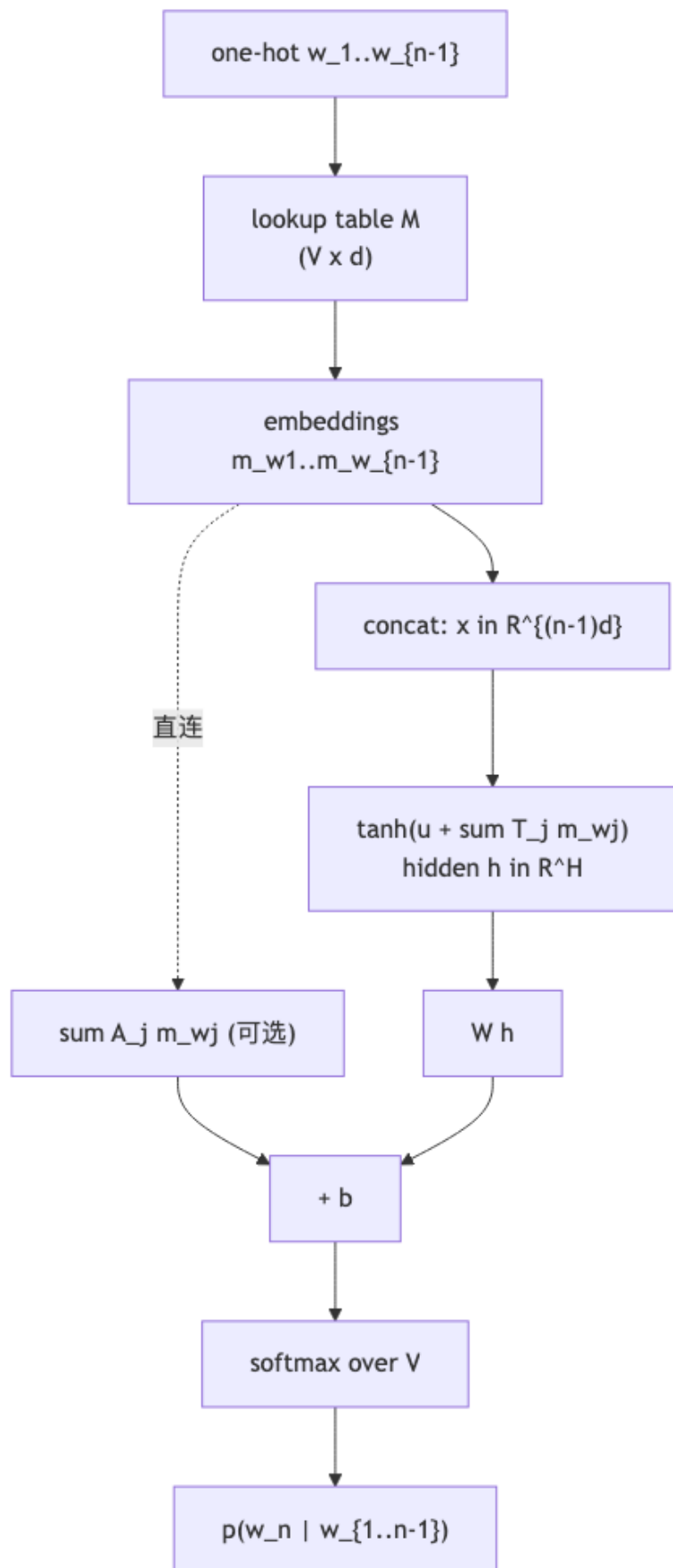
7.1 LM 架构演进



7.2 RNN 按时间展开



7.3 Bengio FFNN-LM 数据流



8. 易错点

[WARN] 期中常见陷阱

1. **NLM 仍是 LM**: 训练目标是 NLL (负对数似然), 与 n-gram MLE 同源; 不是分类任务的 0/1 acc。2. **PPL 计算不要漏 softmax 归一化**: 直接用 logit 算 PPL 是错的。3. **softmax+CE 梯度** $= \hat{p} - y$: 能默写并知道为什么数值稳定 (避免显式分别求 log 和 softmax)。4. **Bengio 模型有“直连 + 残差”分支**: 完整公式两项相加, 不要只写 tanh 那一支。5. **嵌入 M 是参数**: 与 W2V 等“预训练后冻结”不同, NLM-1 训练时一起学。6. **RNN 参数共享**: W_s 在每个时间步是同一个矩阵, 梯度要 sum over t; BPTT 不是把它当 T 个不同矩阵。7. **梯度消失/爆炸的来源**: tanh 导数 $\leq 1 + W_s$ 谱半径; LSTM 的 cell 走加性路径, 但 forget gate 仍可能压低梯度。8. **CNN 池化必须**: 否则可变长度无法接 softmax。9. **n-gram 与 NLM 仍属同一框架**: FFNN-LM 也只看 $n - 1$ 词, 并非“无限上下文”。10. **NLM-1 参数量主导项**: $VH + Vd$, 不是 V^3 ; 这是它“实际可训”的关键。

9. 中英术语对照

中文	English	备注
神经语言模型	neural language model (NLM)	
最大熵语言模型	maximum-entropy LM (Max-Ent LM)	NLM-0 / log-linear LM
前馈神经网络	feedforward neural network (FFNN)	Bengio 2003
词嵌入	word embedding	稠密低维表示
查找表	lookup table	嵌入矩阵 M
隐藏层	hidden layer	tanh / ReLU 输出
仿射变换	affine transformation	$Wx + b$
激活函数	activation function	sigmoid / tanh / ReLU
修正线性单元	rectified linear unit (ReLU)	
双曲正切	hyperbolic tangent (tanh)	
softmax	softmax	$\exp(z) / \sum \exp$
交叉熵损失	cross-entropy loss	$-\sum y \log \hat{p}$
反向传播	back-propagation	链式法则计算梯度
时间反向传播	back-propagation through time (BPTT)	用于 RNN
随机梯度下降	stochastic gradient descent (SGD)	
卷积神经网络	convolutional neural network (CNN)	
滤波器	filter / kernel	$C^{(k)}$
滑窗	sliding window	宽度 s_w
池化	pooling	max / average
空洞卷积	dilated convolution	Yu & Koltun 2015
循环神经网络	recurrent neural network (RNN)	共享参数 + 隐状态
隐状态	hidden state	s_i
梯度消失	vanishing gradient	$\prod J \rightarrow 0$
梯度爆炸	exploding gradient	$\prod J \rightarrow \infty$
梯度裁剪	gradient clipping	抑制爆炸
长短期记忆	long short-term memory (LSTM)	加性 cell + 门控
门控循环单元	gated recurrent unit (GRU)	LSTM 简化版
遗忘门	forget gate	f_t
输入门	input gate	i_t
输出门	output gate	o_t

中文	English	备注
细胞状态	cell state	c_t
连续词袋	continuous bag-of-words (CBOW)	对位置嵌入求平均
参数共享	parameter sharing	RNN 跨时间步同一矩阵

[TIP] 期中复习要点

1. 能写出 NLM-0 (log-linear)、NLM-1 (Bengio FFNN)、NLM-3 (RNN) 的完整方程 2. 能列出 NLM-1 所有参数及形状: $b(V), A(d, V), W(V, H), T(d, H), u(H), M(V, d)$ 3. 能推导 softmax+CE 的梯度 $\partial \mathcal{L} / \partial z_k = \hat{p}_k - y_k$ 4. 能在 2-token 序列上演一遍 RNN 前向 + BPTT 5. 能解释 RNN 梯度消失原因 + LSTM 为什么缓解 (加性 cell + forget gate) 6. 能对比 n-gram vs FFNN-LM vs RNN-LM 的历史长度、参数量、泛化方式 7. 能解释词嵌入带来的核心收益: 相似词共享统计强度, 缓解 n-gram 完全无关联的局限 8. 能解释为什么 NLM 仍然是 LM: 训练目标是 $-\sum \log p(w_i | h)$, 与 n-gram MLE 同源

Lecture 4 —N-gram 语言模型 (N-gram Language Models)

[TIP] 学习指南

本节核心：把“一句话有多自然”形式化为一个概率分布 $p(s)$ ，并用 Markov 假设把指数级的联合分布拆成可计数的 n 元条件概率；MLE 给出最大似然估计，但带来零概率问题，需要 add- k 、Good-Turing、interpolation、Kneser-Ney 等平滑/插值技术；最后用 **困惑度 (Perplexity, PPL)** 来评测。**前置知识：**– 概率链式法则、条件概率 – 多项式分布的 MLE – 交叉熵 / KL 散度 – lec3 判别模型 vs 生成模型 (n-gram LM 属于生成模型)
重点掌握：1. Chain Rule + Markov 假设 $\rightarrow$ bigram / trigram 形式 2. n-gram MLE 的推导 (拉格朗日乘子) 3. 平滑: add- k 、Good-Turing、Absolute Discounting、Kneser-Ney 4. Linear interpolation 与 Stupid back-off 的区别 (后者不是合法分布) 5. PPL 与 cross-entropy / log-likelihood 的等价关系; argmin CE argmax likelihood 6. OOV (Out-Of-Vocabulary) 处理: UNK 伪词、词表截断

4.1 Motivation: 为什么需要语言模型?

4.1.1 直觉

给定一个英文词串，语言模型回答：“这串词作为这门语言的合法表达，有多自然？”

- the dog barks . $\rightarrow$ 自然
- the dog laughs . $\rightarrow$ 不太自然但语法合规
- dog the bark a laugh smile . $\rightarrow$ 不自然

形式化：在词表 $\mathcal{V}$ 上，所有可能的句子构成可数无穷集合 $\mathcal{S}$ ，语言模型是 $\mathcal{S}$ 上的概率分布

$$\sum_{s \in \mathcal{S}} p(s) = 1, \quad p(s) \geq 0 \quad \forall s \in \mathcal{S}. \quad (4.1)$$

希望“自然”的句子获得较大概率，反之概率极小。

4.1.2 应用

LM 是大量任务的“流畅度”评分器 (Fluency)：

$$\text{words}^* = \arg \max_{\text{words}} [\text{Faithfulness}(\text{signals}, \text{words}) + \text{Fluency}(\text{words})] \quad (4.2)$$

典型场景：– 语音识别 (区分 recognize speech vs wreck a nice beach) – OCR、手写识别、中文分词 – 机器翻译、文本生成、对话 – 句子补全 (MS Sentence Completion Challenge) – 心理语言学的 **surprisal theory**: $\text{Surprisal}(w) = -\log_2 p(w \mid h_w)$ ，词在上下文中越可预测越易理解

[NOTE] 直觉理解

“Faithfulness + Fluency”这一拆解贯穿整个统计 NLP：声学/视觉/源语言决定保真度，LM 决定流畅度，二者相加（取 log 即相乘）。LM 不必“懂”内容，只需告诉你哪种词序更符合语言习惯。

4.2 模型定义

4.2.1 朴素方案的失败

若把整句 s 当一个原子事件， $p(s) = \text{count}(s)/|\mathcal{S}|$ 。问题：- $|\mathcal{S}|$ 不可枚举 - 几乎所有 s 出现次数为 0 - 无法泛化到新句子

4.2.2 Chain Rule 分解

把句子视为词序列 $w_1, w_2, \dots, w_n$ （约定末尾加 STOP，可选 START）：

$$P(X_1 = w_1, X_2 = w_2, \dots, X_n = \text{STOP}) = P(X_1 = w_1) \prod_{i=2}^n P(X_i = w_i \mid X_1 = w_1, \dots, X_{i-1} = w_{i-1}) \quad (4.3)$$

仍是指数级条件分布——历史 $h_{1:i-1}$ 可以任意长。

4.2.3 Markov 假设

k 阶 Markov 假设：未来只依赖最近 k 个词。

- 零阶 (Unigram)： $P(X_i = w \mid h) \approx P(X_i = w)$ ，即 bag-of-words，最简单但忽略词序。
- 一阶 (Bigram)： $P(X_i = w_i \mid h_{1:i-1}) \approx P(X_i = w_i \mid X_{i-1} = w_{i-1})$
- 二阶 (Trigram)： $P(X_i = w_i \mid h_{1:i-1}) \approx P(X_i = w_i \mid X_{i-2} = w_{i-2}, X_{i-1} = w_{i-1})$

Trigram LM 的句子概率：

$$p(w_1, w_2, \dots, w_n) = p(w_1) p(w_2 \mid w_1) \prod_{i=3}^n p(w_i \mid w_{i-2}, w_{i-1}) \quad (4.4)$$

[NOTE] 直觉理解

Markov 假设是一种模型偏置 (bias)：用“假设近邻足够”来换取可学习性。 k 太小欠拟合 (unigram 完全无序)， k 太大数据稀疏 (5-gram 几乎没出现过)。实践常用 trigram，4-/5-gram 配 Google N-gram 等大语料库。

4.2.4 一个 trigram 例子

句子 the cat laughs . STOP:

$$\begin{aligned}
p(\text{the cat laughs . STOP}) &= p(\text{the}) \\
&\times p(\text{cat} \mid \text{the}) \\
&\times p(\text{laughs} \mid \text{the, cat}) \\
&\times p(. \mid \text{cat, laughs}) \\
&\times p(\text{STOP} \mid \text{laughs, .})
\end{aligned}$$

STOP 是必要的：它使得 $\sum_{s \in \mathcal{S}} p(s) = 1$ （否则任何长度都会有概率泄漏）。

4.3 参数估计：MLE 推导

4.3.1 直觉计数

对 bigram:

$$p_{\text{ML}}(w_i \mid w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i)}{\text{count}(w_{i-1})} \quad (4.5)$$

对 trigram:

$$p_{\text{ML}}(w_i \mid w_{i-2}, w_{i-1}) = \frac{\text{count}(w_{i-2}, w_{i-1}, w_i)}{\text{count}(w_{i-2}, w_{i-1})} \quad (4.6)$$

4.3.2 MLE 推导（拉格朗日乘子）

记训练语料中 trigram (u, v, w) 出现次数为 $c(u, v, w)$, bigram (u, v) 出现次数 $c(u, v) = \sum_w c(u, v, w)$ 。令参数 $\theta_{w|u,v} = p(w \mid u, v)$, 对数似然:

$$\mathcal{L}(\theta) = \sum_{u,v,w} c(u, v, w) \log \theta_{w|u,v}$$

约束：对每个 (u, v) , $\sum_w \theta_{w|u,v} = 1$ 。引入拉格朗日乘子 $\lambda_{u,v}$:

$$\mathcal{F} = \sum_{u,v,w} c(u, v, w) \log \theta_{w|u,v} - \sum_{u,v} \lambda_{u,v} \left(\sum_w \theta_{w|u,v} - 1 \right)$$

对 $\theta_{w|u,v}$ 求偏导并令为 0:

$$\frac{\partial \mathcal{F}}{\partial \theta_{w|u,v}} = \frac{c(u, v, w)}{\theta_{w|u,v}} - \lambda_{u,v} = 0 \implies \theta_{w|u,v} = \frac{c(u, v, w)}{\lambda_{u,v}}$$

代入约束: $\sum_w \theta_{w|u,v} = 1 \implies \lambda_{u,v} = \sum_w c(u, v, w) = c(u, v)$ 。故

$$\hat{\theta}_{w|u,v} = \frac{c(u, v, w)}{c(u, v)} \quad (4.7)$$

正是经验计数比，与式 (4.6) 完全一致。

4.3.3 模型规模

Bengio et al. (2003) 在 Gigaword 上 (275M 词): – 不同 unigram: 716,706 – 不同 bigram: 12,537,755 – 不同 trigram: 22,174,483 – 理论 trigram 上界: $|\mathcal{V}|^3 \approx 5.15 \times 10^{13}$, 实际只占 1/4,000,000 – 数据稀疏 (data sparsity): 绝大多数 n -gram 计数为 0

4.4 平滑与插值：处理未见事件

[WARN] 易错点

MLE 给未见 n -gram 概率 0 → 整个测试集概率为 0 → 困惑度 ∞ 。任何实际 LM 都必须处理零概率。

总体思路：从已见事件挪一些概率质量给未见事件，使得所有事件 $p > 0$ 且总和仍为 1。

4.4.1 Add- k (Laplace / Lidstone) 平滑

最朴素做法：给每个可能 bigram 加一个虚拟计数 k 。

$$p_{\text{add-}k}(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i) + k}{c(w_{i-1}) + k|\mathcal{V}|} \quad (4.8)$$

$k = 1$ 即 Laplace / Add-One; $0 < k < 1$ 称 Lidstone。

推导直觉：等价于在 MLE 之上加 Dirichlet 先验 $\text{Dir}(k, k, \dots, k)$ 取 MAP。

致命缺陷：把太多概率质量“白白送给”了未见事件。例： $|\mathcal{V}| = 12, p(\text{seen}|\text{the})$ 从 1 暴跌到 0.36, $p(\text{unseen}|\text{the})$ 从 0 涨到 0.64——绝大多数概率给了从未出现的事件。适用于未见事件数量不大的场景（如文本分类），不适用于完整 LM。

4.4.2 Good-Turing 折扣

直觉：用“出现 $r + 1$ 次的 n -gram 个数”来推断“出现 r 次的 n -gram 的真实概率”。

记 N_r 为“在语料中出现 r 次”的不同 n -gram 个数 (count of count)。把真实计数 r 调整为：

$$r^* = (r + 1) \frac{N_{r+1}}{N_r} \quad (4.9)$$

为未见事件 ($r = 0$) 保留的概率质量：

$$P_{\text{GT}}(\text{unseen}) = \frac{N_1}{N_{\text{all}}} \quad (4.10)$$

推导思路：随机从训练集删一个出现 $r + 1$ 次的词 w ，得到伪测试集中 w 的期望计数应为 r 。这样的删除操作对 $r + 1$ 类词共带来 $N_{r+1}(r + 1)$ 次计数转移，分摊给 N_r 个 r 类 n -gram，每个调整后期望为 $\frac{N_{r+1}(r + 1)}{N_r}$ 。

Church & Gale (1991) 在 AP 新闻 22M 词上的实验：

r	r^*	$r - r^*$
0	0.000027	—
1	0.446	0.554
2	1.26	0.74
3	2.24	0.76
4	3.24	0.76
5	4.22	0.78

观察：对 $r \geq 2$ ，折扣量近似常数 $\approx 0.75 \rightarrow$ 启发了 Absolute Discounting。 N_{r-1} 为 0 时用曲线拟合 / 线性回归近似。

4.4.3 Absolute Discounting

直接从每个非零计数中减去一个固定值 d ，剩余概率质量按权重 $\lambda(\cdot)$ 退化到低阶模型：

$$p_{\text{abd}}(w_2 | w_1) = \frac{\max(c(w_1, w_2) - d, 0)}{c(w_1)} + \lambda(w_1) p(w_2) \quad (4.11)$$

参数： $-d \in (0, 1)$ ，常取 $0.75 - \lambda(w_1)$ 定义为使整体仍为概率分布所需的归一化系数： $\lambda(w_1) = \frac{d \cdot |\{w : c(w_1, w) > 0\}|}{c(w_1)}$

4.4.4 Linear Interpolation (线性插值)

把不同阶 LM 凸组合：

$$p_{\text{il}}(w_i | w_{i-2}, w_{i-1}) = \lambda_1 p_{\text{ML}}(w_i | w_{i-2}, w_{i-1}) + \lambda_2 p_{\text{ML}}(w_i | w_{i-1}) + \lambda_3 p_{\text{ML}}(w_i) \quad (4.12)$$

要求 $\lambda_1 + \lambda_2 + \lambda_3 = 1$, $\lambda_i \geq 0$ 。

λ 的估计：留出一份 held-out / 验证集，用 EM 或网格搜索最大化验证集似然。

是否仍是合法分布？是。凸组合保持归一化与非负。

4.4.5 Stupid Back-off (Brants et al., 2007)

Web 规模工程实现，舍弃概率合法性以换速度：

$$s(w_i | w_{i-2}, w_{i-1}) = \begin{cases} \frac{c(w_{i-2}, w_{i-1}, w_i)}{c(w_{i-2}, w_{i-1})} & c(w_{i-2}, w_{i-1}, w_i) > 0 \\ 0.4 \cdot s(w_i | w_{i-1}) & \text{otherwise} \end{cases} \quad (4.13)$$

[WARN] 易错点

Stupid back-off 不是合法概率分布 (s 不归一化), 但在 BLEU 等下游度量上几乎不输 Kneser-Ney。课堂可能会问“它是合法 LM 吗”——答否。

4.4.6 Kneser-Ney 平滑 (核心)

KN 平滑结合绝对折扣 + 关注 continuation 的低阶分布 + 插值。

关键直觉: 对低阶概率, 不该用 $p(w) = c(w)/N$ (unigram 概率), 而要用“ w 作为新 context 续接出现”的概率。例: Francisco 在数据中只在 San Francisco 出现, 普通 unigram 概率高, 但 Francisco 几乎从不作为新 bigram 的右词出现。

定义 continuation 概率:

$$p_{\text{cont}}(w) = \frac{|\{v : c(v, w) > 0\}|}{|\{(v', w') : c(v', w') > 0\}|} \quad (4.14)$$

即以 w 结尾的不同 bigram 类型数 / “所有不同 bigram 类型数”。

插值 KN (bigram 形式):

$$p_{\text{KN}}(w_i | w_{i-1}) = \frac{\max(c(w_{i-1}, w_i) - d, 0)}{c(w_{i-1})} + \lambda(w_{i-1}) p_{\text{cont}}(w_i) \quad (4.15)$$

其中 $\lambda(w_{i-1}) = \frac{d}{c(w_{i-1})} \cdot |\{w : c(w_{i-1}, w) > 0\}|$ 保证归一化。

[NOTE] 直觉理解

普通 unigram $p(w) \propto c(w)$ 反映“被频繁说”; continuation 概率反映“被用在多少不同前缀后”。KN 把“多样性”作为后退分布——很好地解决了 Francisco 这种“高频但孤立”的词被高估的问题。KN 通常被认为是非神经 LM 时代效果最好的平滑方法, 参考 SLP3 第 3 章。

4.4.7 OOV 处理

训练时不可能见过所有词。两种主流做法:

1. **固定词表** $\mathcal{V}$: 将训练集中频次 $< \tau$ (如 5) 的词全部替换为伪词 $\langle \text{UNK} \rangle$, 预测和评测时陌生词均映射到 $\langle \text{UNK} \rangle$ 。
2. **特殊处理 + 字符级回退**: 现代 BPE / subword 分词从根本上缓解此问题 (见 lec on tokenization)。

PPL 比较前提: 词表 $\mathcal{V}$ 必须一致, 否则 PPL 数字没有可比性。

4.5 评测: 困惑度 (Perplexity)

4.5.1 定义

测试集 D 包含若干句子, 共 M 个词。LM 给每句赋概率 $p(s)$ 。整体几何平均概率 $\sqrt[M]{\prod_{s \in D} p(s)}$, 取负对数得到平均交叉熵:

$$H(D) = -\frac{1}{M} \sum_{s \in D} \log_2 p(s) \quad (4.16)$$

困惑度定义为：

$$\text{PPL}(D) = 2^{H(D)} = 2^{-\frac{1}{M} \sum_{s \in D} \log_2 p(s)} \quad (4.17)$$

PPL 越低，LM 在 D 上预测越准。

4.5.2 推导：argmin CE argmax likelihood

测试集对数似然： $\mathcal{L}(D) = \sum_{s \in D} \log_2 p(s)$ 。

$$\arg \max_{\theta} \mathcal{L}(D; \theta) = \arg \max_{\theta} \frac{1}{M} \mathcal{L}(D; \theta) = \arg \min_{\theta} \left(-\frac{1}{M} \mathcal{L}(D; \theta) \right) = \arg \min_{\theta} H(D; \theta)$$

且 $\text{PPL} = 2^H$ 关于 H 单调递增 $\rightarrow$ 最小化 PPL 最小化交叉熵 最大化测试集似然。

更进一步，若 p^* 是真实数据分布，则按大数律

$$H(p^*, p_{\theta}) = -\mathbb{E}_{s \sim p^*} [\log p_{\theta}(s)] \approx -\frac{1}{M} \sum_{s \in D} \log p_{\theta}(s)$$

因此 PPL 近似衡量模型与真实分布的交叉熵， $2^{H(p^*)}$ 为不可降的语言固有不确定度（entropy of language）。

4.5.3 PPL 的直觉：分支因子（branching factor）

均匀 $|\mathcal{V}| + 1$ 上的 trigram: $p(a | b, c) = 1/(|\mathcal{V}| + 1)$ ，代入得 $\text{PPL} = |\mathcal{V}| + 1$ 。即“模型每步在等价于 PPL 个候选中做选择”。PPL 越小，可选项越少，模型越“确信”。

[WARN] 易错点

– PPL 必须在未见测试集上算，训练集 PPL 几乎总是更低（过拟合）– PPL 对 $\mathcal{V}$ 极敏感：同一模型在大词表上 PPL 一定更高 – PPL 与下游任务表现并非线性相关；MT、QA 等需用 BLEU / Accuracy / F1 等任务专用指标

4.6 Worked Example

4.6.1 训练 bigram LM 并算 PPL

训练语料：

```
i love pku .
i like thu .
you love pku .
you do not like thu .
```

总词数 $M_{\text{train}} = 18$ (不含 STOP)。约定每句末加 STOP，并用 START 作为 w_0 。

bigram 计数 (部分):

pair	count
(START, i)	2
(START, you)	2
(i, love)	1
(i, like)	1
(love, pku)	2
(like, thu)	2
(you, love)	1
(you, do)	1
(do, not)	1
(not, like)	1
(pku, .)	2
(thu, .)	2
(., STOP)	4

bigram MLE:

$$p(\text{love} \mid i) = \frac{c(i, \text{love})}{c(i)} = \frac{1}{2} = 0.5$$

$$p(\text{like} \mid i) = \frac{1}{2}, \quad p(\text{love} \mid \text{you}) = \frac{1}{2}, \quad p(\text{thu} \mid \text{like}) = \frac{2}{2} = 1$$

测试句: you like pku . STOP

$$p(s) = p(\text{you} \mid \text{START}) p(\text{like} \mid \text{you}) p(\text{pku} \mid \text{like}) p(. \mid \text{pku}) p(\text{STOP} \mid .)$$

代入: $p(\text{you} \mid \text{START}) = 2/4 = 0.5$, $p(\text{like} \mid \text{you}) = 0$ (因为 you 后只跟 love / do) → 整个测试集概率为 0, PPL = ∞ 。

→ 必须平滑。用 add-1 smoothing (词表 $\mathcal{V} = \{i, \text{you}, \text{love}, \text{like}, \text{do}, \text{not}, \text{pku}, \text{thu}, ., \text{STOP}\}$, $|\mathcal{V}| = 10$):

$$p_{+1}(\text{like} \mid \text{you}) = \frac{0 + 1}{c(\text{you}) + |\mathcal{V}|} = \frac{1}{2 + 10} = \frac{1}{12}$$

平滑后所有概率非零, PPL 可算。练习: 自行计算 $p(i, \text{hate}, \text{thu}, ., \text{STOP})$, 注意 hate 是 OOV, 需先映射为 <UNK>。

4.6.2 Quiz 题 (课堂)

频次表:

w_1	w_2	I	finished	work	saw	the	beautiful	gift
I		0	40	20	30	0	0	0
finished		0	0	10	0	8	5	3
work		0	5	0	0	0	0	0

w_1	w_2	I	finished	work	saw	the	beautiful	gift
saw		0	5	5	0	8	5	3
the		0	10	15	0	0	20	10
beautiful		0	0	5	0	0	0	10
gift		0	5	0	0	0	3	0

Q1: 最可能的以 I 开头的 4 词句?

$$c(I) = 0 + 40 + 20 + 30 = 90$$

$p(\text{finished}|I) = 40/90$, $p(\text{saw}|I) = 30/90$, $p(\text{work}|I) = 20/90$ 。最大: I finished。

接下来 I finished: $c(\text{finished}) = 31$, $p(\text{work}|\text{finished}) = 10/31$ 最大。

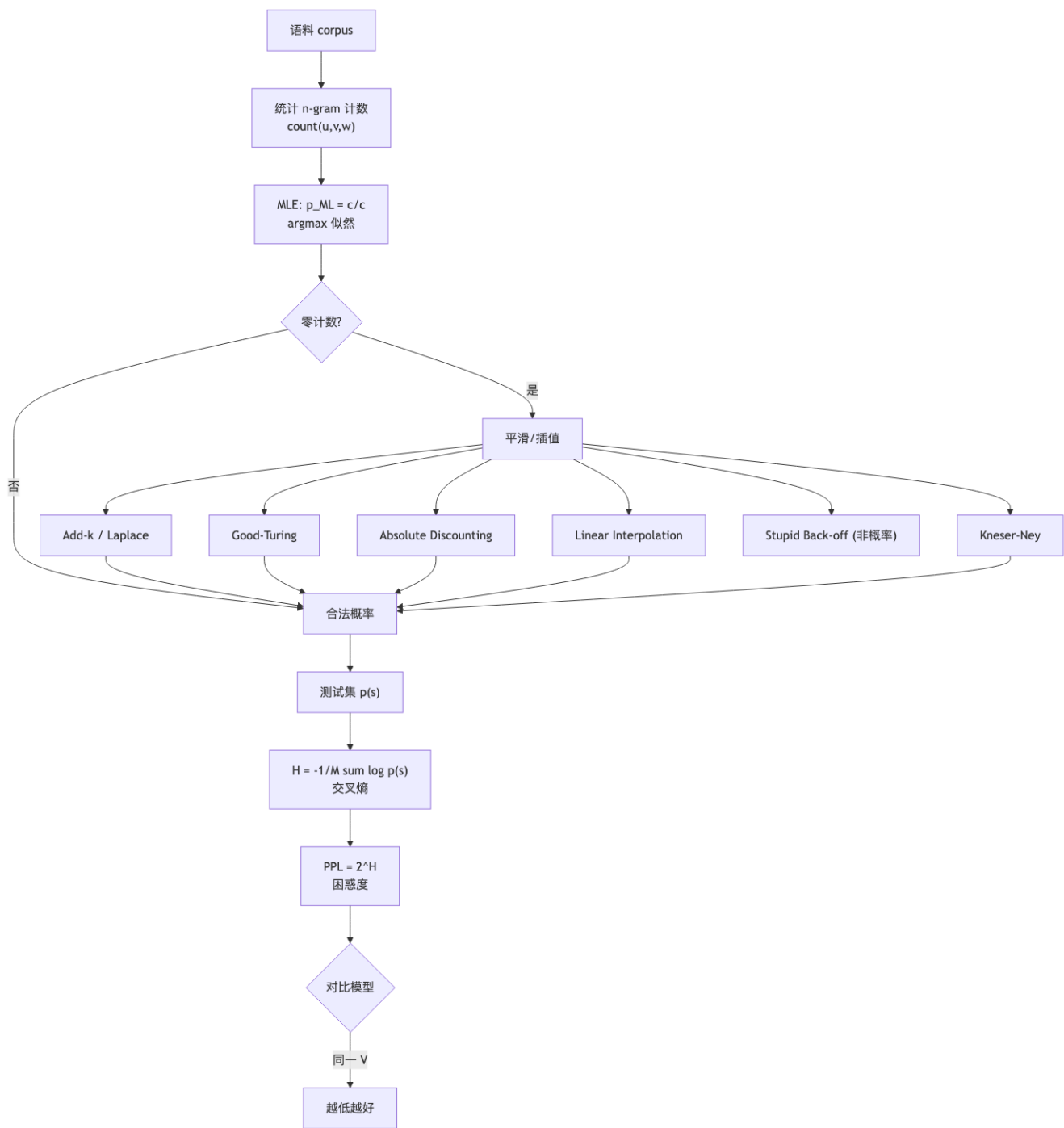
I finished work: $c(\text{work}) = 5$, $p(\text{finished}|\text{work}) = 5/5 = 1$ 。

→ 最可能: **I finished work finished** (注意这是贪心解, 未必全局最优)。

Q2: $p(\text{gift}|\text{the})$?

$$c(\text{the}) = 10 + 15 + 0 + 0 + 20 + 10 = 55, \quad p(\text{gift}|\text{the}) = 10/55 \approx 0.182$$

4.7 概念关系图



4.8 易错点汇总

[WARN] 期中常见陷阱

1. 没加 STOP: 忘记 STOP 后 $\sum_s p(s) \neq 1$, 违反概率公理。2. 混淆 unigram 句子概率与多项式概率: unigram $p(s) = \prod_i p(w_i)$, 要乘 STOP; 不要写成多项式系数。3. PPL 公式底数: 用 2 或 e 均可, 但 $\text{PPL} = b^{H_b}$ 中底数要一致; 课堂用 2。4. PPL 跨词表比较: 词表不同的两个 LM 的 PPL 不能直接比。5. MLE 推导: 必须写出拉格朗日乘子和约束 $\sum_w \theta = 1$, 不能直接“显然 $\theta = c/c$ ”。6. add- k 的分母: 是 $c(w_{i-1}) + k|\mathcal{V}|$, 不是 $c(w_{i-1}) + k$ 。 $|\mathcal{V}|$ 通常含 STOP。7. Good-Turing 的 N_r 是“count of count”, 不是某个 n -gram 的计数。8. Stupid back-off 不是概率分布: 常考“是否合法 LM”陷阱题。9. Kneser-Ney 的 continuation 概率: 分子是“以 w 结尾的不同 bigram 类型数”, 不是 $c(w)$ 。10. OOV: 测试集出现训练集没见过的词时, 若不做 <UNK> 替换, PPL 直接爆炸; 汇报数字时必须说明 OOV 处理方式。

4.9 中英术语对照

中文	English	备注
语言模型	language model (LM)	句子上的概率分布
词表	vocabulary $\mathcal{V}$	通常含 <UNK>、STOP
n 元语法	n -gram	uni- / bi- / tri- / 4- / 5-gram
Markov 假设	Markov assumption	k 阶 = 最近 k 词
链式法则	chain rule	联合 $\rightarrow$ 条件之积
最大似然估计	maximum likelihood estimation (MLE)	计数比
数据稀疏	data sparsity	多数 n -gram 计数为 0
平滑	smoothing	给未见事件分概率
折扣	discounting	从已见事件挪概率
插值	interpolation	多阶 LM 凸组合
回退	back-off	高阶不足时退化到低阶
加 k 平滑	add- k / Lidstone smoothing	$k = 1$ 即 Laplace
Good-Turing 折扣	Good-Turing discounting	用 N_{r+1}/N_r 调计数
绝对折扣	absolute discounting	减常数 d
Kneser-Ney 平滑	Kneser-Ney smoothing	continuation prob
continuation 概率	continuation probability	不同前缀类型计数
Stupid back-off	stupid back-off	非合法分布、工程化
留出集 / 验证集	held-out / validation / dev set	调超参
测试集	test set	不可碰、最终评测
困惑度	perplexity (PPL)	2^H
交叉熵	cross-entropy	$-\frac{1}{M} \sum \log p$
未登录词	out-of-vocabulary (OOV)	通常映射为 <UNK>
分支因子	branching factor	PPL 的直觉解释
Surprisal	surprisal	$-\log_2 p(w h)$
词袋	bag-of-words (BoW)	unigram 等价表达

[TIP] 期中复习要点

1. 能写出 trigram 句子概率公式 (含 START / STOP)
2. 能推导 MLE: 拉格朗日乘子 $\rightarrow \hat{\theta} = c/c$
3. 能写出 add- k 、Absolute Discounting、Linear Interpolation、Stupid Back-off、KN 的公式, 并说出 Stupid Back-off 不是合法分布
4. 能解释 Good-Turing 的 $r^* = (r + 1)N_{r+1}/N_r$ 直觉
5. 能写出 PPL 定义并证明 $\text{argmin PPL} \quad \text{argmax 似然}$
6. 能算简单 worked example: 从频次表算 bigram 概率与最可能续句
7. 能解释 PPL 跨词表不可比、Kneser-Ney 为什么用 continuation prob
8. 能说出 n-gram LM 的优缺点: 易实现、计数即可、上手快; 只能短距、稀疏、参数巨大、无泛化 (同义词无关联)

Lecture 5 — 分布式语义 (Distributional Semantics)

[TIP] 学习指南

本节核心：用 Harris / Firth 的“分布假说”（一个词由它出现的上下文定义）把“词义”从词典/WordNet 这种符号定义，过渡到**向量空间中的几何对象**。从最朴素的共现矩阵 → 加权 (PPMI / TF-IDF) → 降维 (SVD / LSA) → 神经预测 (word2vec / GloVe) → 上下文相关表示 (ELMo / BERT 的起点)。**前置知识：**– 概率基础 (联合 / 边缘 / 条件、独立性) – 信息论：互信息、熵 – 线性代数：内积、向量范数、SVD、低秩近似 – lec3 log-linear + softmax – lec4 NLM、词嵌入概念 **重点掌握：**1. Distributional Hypothesis (Firth / Harris) 的语言学根据 2. 共现矩阵 + 余弦相似度的几何直觉 3. PMI / PPMI 的信息论推导 4. TF-IDF 的两个加权项及其作用 5. SVD 截断给出最优低秩近似 (Eckart-Young 定理直觉) 6. word2vec: CBOW vs Skip-gram 的目标差异；Skip-gram + 负采样的完整损失 7. GloVe 的加权 LSQ 目标及偏置项推导 8. 词义多义性 → 上下文表示的动机

5.1 Motivation: 词的“意义”是什么？

5.1.1 从词典到几何

Q: What's the meaning of life? A: LIFE. —Barbara Partee (1967)

把 meaning(life) = LIFE 这种符号式答案，与 **WordNet** 这种结构化词典对照：

WordNet 把英语词组织为 synsets (同义词集合)，并定义：– **Synonym (同义)**：automobile / car (特定上下文) – **Similarity (相似)**：car / truck / bike (共享部分语义) – **Relatedness / Word association**：coffee / sugar (语义场相关) – **Antonymy (反义)**：hot / cold – **Connotation (情感倾向)**：copy / fake – **Hyponym / Hypernym (上下位)**：dog / animal – **Meronym (部分-整体)**：wheel / car

5.1.2 评测数据集

- **WordSim-353** [Agirre et al., 2009]：353 词对，分别打**相似度**与**相关度**两套分数
- **SimLex-999** [Hill et al., 2015]：只打“严格相似度”（区别于 relatedness）

例：

词对	类型	相似度
tiger —cat	synonym (近义)	7.35
tiger —tiger	identical	10.00
plane —car	antonym	5.77
smart —stupid	反义但相关	5.81
money —cash	synonym	9.15

5.1.3 词典式方法的局限

- 无法跟上新词 (tezgüino, nindin)
- 同义词列表难维护、覆盖差
- 不能直接和数值模型对接
- 难刻画“程度” (adept / expert / proficient 微妙差异)

观察：

Bottle of tezgüino is on the table. Everyone likes tezgüino. Tezgüino makes you drunk. We make tezgüino out of corn.

不查词典也能推断：tezgüino 是一种用玉米酿的酒精饮料。——这就是“上下文定义词义”。

5.2 Distributional Hypothesis (分布假说)

Firth (1957): > You shall know a word by the company it keeps.

Harris (1954): > Distributional statements can cover all of the material of a language without requiring support from other types of information.

核心思想：上下文（共现的词、结构）足以表征词义；不需要外部知识。

5.2.1 两类共现

- First-order (句法/syntagmatic 关联)：常常相邻出现。如 wrote 与 book / poem。
- Second-order (聚合/paradigmatic 关联)：有相似邻居。如 wrote 与 said / remarked——它们都跟“主语 + 引语”搭配。

second-order 共现是分布式向量“相似度”的真正来源。

[NOTE] 直觉

把每个词的“邻居分布”画成一个直方图。两个词的直方图越像，它们的意义越接近。共现矩阵就是把这些直方图整齐排成的矩阵。

5.3 向量空间模型 (Vector Space Model, VSM)

5.3.1 词表示的层级

方案	维度	稀疏度	备注
One-hot	$ \mathcal{V} $	极稀疏	互相正交 → 无语义
Brown clustering	离散簇 ID	—	硬聚类，难做相似度
共现矩阵	$ \mathcal{C} $	稀疏	行向量即词的“邻居指纹”
PMI / PPMI 加权	$ \mathcal{C} $	稀疏	强化“非偶然”共现
SVD / LSA 截断	k (50–300)	稠密	去噪 + 降维
word2vec / GloVe	d (100–300)	稠密	神经/隐式矩阵分解
ELMo / BERT	d	稠密 + 上下文相关	同一词不同句不同向量

5.3.2 共现矩阵构造

1. 选定 **target words** $\mathcal{T}$ 与 **basis / context vocabulary** $\mathcal{C}$
2. 选定窗口宽度 w (左右各 w 个词)
3. 数语料里每个 target 与每个 context 的**共现次数** $\#(t, c)$
4. 第 t 行的向量 $v_t \in \mathbb{R}^{|\mathcal{C}|}$ 就是它的分布式表示

窗口的语义性：– 窗口窄 (1–2) → 偏句法相似 (POS 接近) – 窗口宽 (5–10) → 偏主题/语义相似

实践常加：停用词表、词形还原 (lemma)、考虑词性。

示例：

```
... and the cute kitten purred and then ...
... the cute furry cat purred and miaowed ...
... that the small kitten miaowed and she ...
... the loud furry dog ran and bit ...
```

基词 = {bit, cute, furry, loud, miaowed, purred, ran, small}

- kitten = [0, 1, 0, 0, 1, 1, 0, 1]
- cat = [0, 1, 1, 0, 1, 0, 0, 0] (注：上文示例中 cat 未与 small 共现)
- dog = [1, 0, 1, 1, 0, 0, 1, 0]

$\text{cosine}(\text{kitten}, \text{cat}) > \text{cosine}(\text{kitten}, \text{dog}) \rightarrow$ 反映了语义。

5.3.3 余弦相似度

$$\cos(u, v) = \frac{u^\top v}{\|u\| \cdot \|v\|} \quad (5.1)$$

- 取值 $[-1, 1]$ (PPMI 后通常 ≥ 0)；1 = 同向，0 = 正交，-1 = 反向
- 优点：忽略向量长度 (即词频)，只比“方向”——避免常见词压制
- 替代度量：欧氏距离、Jaccard、KL/JS 散度 (若把行向量当分布)

5.3.4 词-文档矩阵

把“上下文”换成“文档”，行向量是 $|D|$ 维的文档分布——可用于 **信息检索 (IR)**、**文档相似度**。这就是 LSA 的原始输入。

5.4 PMI 与 PPMI (信息论加权)

5.4.1 为什么要加权

原始计数有两大缺陷：– 高频虚词 (the, of) 几乎与所有词共现，但区分度极差 – 低频强搭配 (Francisco 几乎只跟 San) 很重要却被淹没

5.4.2 PMI 定义

对随机变量 X (出现词 x) 和 Y (出现 context y)：

$$\text{PMI}(x, y) = \log_2 \frac{P(x, y)}{P(x)P(y)} \quad (5.2)$$

- $P(x, y) = \#(x, y) / N$ (共现概率)
- $P(x) = \#(x) / N$ 、 $P(y) = \#(y) / N$ (边缘)
- N = 全部共现 token 数

信息论解释：测量 x, y 共现频率比独立假设下多多少。PMI $> 0 \rightarrow$ 比独立更频繁； $= 0 \rightarrow$ 独立； $< 0 \rightarrow$ 比独立更少。

5.4.3 PPMI

PMI 的负值在“很少共现”时不可靠（稀疏数据噪声大），且人类对“不相关”的判断不敏锐：

$$\text{PPMI}(x, y) = \max(\text{PMI}(x, y), 0) \quad (5.3)$$

得到的 PPMI 矩阵 仍稀疏，但所有非零项为正——可直接作为词向量。

5.4.4 直觉与算例

讲义中的 4x5 共现表 (apricot, pineapple, digital, information x computer, data, pinch, result, sugar) :

	computer	data	pinch	result	sugar
apricot	0	0	1	0	1
pineapple	0	0	1	0	1
digital	2	1	0	1	0
information	1	6	0	4	0

总和 $N = 19$ 。

$P(\text{apricot}) = 2/19 \approx 0.105$ (讲义近似为 0.11)； $P(\text{pinch}) = 2/19$ ； $P(\text{apricot}, \text{pinch}) = 1/19$ 。

$$\text{PMI}(\text{apricot}, \text{pinch}) = \log_2 \frac{1/19}{(2/19) \cdot (2/19)} = \log_2 \frac{19}{4} \approx 2.25$$

讲义最终 PPMI 表中 apricot 对 sugar / pinch 都是 2.25，information 对 data 是 0.57（高频词稀释了）。

5.4.5 PMI 的问题与补救

- 稀有事件偏置： N_{xy} 很小时分母也很小，PMI 异常大
- 解 1: Add-one (Laplace) smoothing——把所有计数 +1 再算 PMI，能显著拉平稀有项的 PMI
- 解 2: α -平滑 context 概率 (Levy & Goldberg 2014): $P_\alpha(c) = \#(c)^\alpha / \sum_{c'} \#(c')^\alpha$, $\alpha \approx 0.75$, 与 word2vec 负采样默认一致
- 解 3: 直接用 PPMI (已经丢负值)

5.5 TF-IDF：另一类加权

对词-文档矩阵：

$$\text{TF}_{t,d} = \begin{cases} 1 + \log_{10} \text{count}(t, d) & \text{count}(t, d) > 0 \\ 0 & \text{otherwise} \end{cases} \quad (5.4)$$

$$\text{IDF}_t = \log_{10} \frac{N}{\text{DF}_t} \quad (5.5)$$

$$\text{TF-IDF}_{t,d} = \text{TF}_{t,d} \cdot \text{IDF}_t \quad (5.6)$$

- N = 文档总数, DF_t = 含 t 的文档数
- TF 用 $\log$ 压缩高频
- IDF 压制全集出现的“水词”
- 经典 IR 加权方案; 亦可用于词相似度的弱基线

[NOTE] PMI vs TF-IDF

两者都给“罕见但关键”的事件赋更大权重, 思路相通。区别: – PMI 用于词-词矩阵, IDF 用于词-文档矩阵 – PMI 是对称信息论量; TF-IDF 是 IR 启发式

5.6 维度归约: SVD / LSA

5.6.1 动机

PPMI / TF-IDF 矩阵仍 $|\mathcal{V}| \times |\mathcal{C}|$ 维, 太长且稀疏。**Latent Semantic Analysis (LSA)**: 用奇异值分解 (SVD) 得到稠密低维表示。

5.6.2 SVD

任意实矩阵 $A \in \mathbb{R}^{m \times n}$ 有奇异值分解:

$$A = U \Sigma V^T, \quad U \in \mathbb{R}^{m \times m}, V \in \mathbb{R}^{n \times n}, \Sigma = \text{diag}(\sigma_1, \sigma_2, \dots) \quad (5.7)$$

- U, V 列正交
- $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$
- U 的列 = “左奇异向量”, V 的列 = “右奇异向量”

5.6.3 截断 SVD (rank- k 近似)

只保留前 k 个奇异值/向量:

$$\hat{A}_k = U_k \Sigma_k V_k^T, \quad U_k \in \mathbb{R}^{m \times k}, \Sigma_k \in \mathbb{R}^{k \times k}, V_k \in \mathbb{R}^{n \times k} \quad (5.8)$$

Eckart-Young 定理: $\hat{A}_k$ 是所有秩 $\leq k$ 矩阵中, 在 Frobenius / 谱范数下与 A 距离最小者:

$$\hat{A}_k = \arg \min_{\text{rank}(B) \leq k} \|A - B\|_F \quad (5.9)$$

5.6.4 LSA 输出

把每个词表示为 $U_k \Sigma_k$ 的对应行 (k 维)。在 PPMI / 词-文档矩阵上做 LSA 的好处:

1. 降维 $|\mathcal{C}| \rightarrow k$ (典型 $k = 100-300$)
2. 去噪: 丢弃小奇异值方向, 相当于平滑掉随机共现

3. 泛化：低秩近似让”未直接共现但语义相近”的词获得高相似度
4. 同义词融合：car / automobile 共享潜在维度

5.6.5 PLSA & LDA (一笔带过)

- PLSA (Hofmann 1999): 把文档建模为主题混合 $p(w | d) = \sum_z p_{\text{topic}}(z | d) p_{\text{word}}(w | z)$, 用 EM 训练; 但对训练集外文档不能赋概率
- LDA (Blei et al. 2003): 给主题分布加 Dirichlet 先验, 可对新文档推断主题

[WARN] LSA / PLSA 仍是 BoW

它们丢弃了词序与上下文位置, 只能学到主题级别的语义。

5.7 神经预测式: word2vec

5.7.1 思想转变

LSA 把”共现矩阵分解”作为目标; word2vec (Mikolov 2013) 转而预测:

让模型学会从词预测上下文 (或反过来), 副产品就是嵌入。

理论上等价于隐式因子分解一个 shifted PMI 矩阵 (Levy & Goldberg, 2014)。

5.7.2 CBOW: 从上下文预测中心词

$$p(w_n | w_{n-2}, w_{n-1}, w_{n+1}, w_{n+2}) = \text{softmax}\left(b + \sum_{j \neq n} m_{w_j} A_j + W \tanh\left(u + \sum_{j \neq n} m_{w_j} T_j\right)\right) \quad (5.10)$$

形式上与 lec4 的 Bengio FFNN-LM 完全一致, 差别: - 上下文是**两侧** (不仅是左侧历史) - 实际实现常去掉中间 tanh 层, 只用嵌入平均 → 速度大幅提升

5.7.3 Skip-gram: 从中心词预测上下文

对每个目标 w , 每个上下文 c 用两套嵌入 $w, c \in \mathbb{R}^d$:

$$p(c | w) = \frac{\exp(c^\top w)}{\sum_{c' \in \mathcal{V}} \exp(c'^\top w)} = \frac{1}{Z_w} \exp(c^\top w) \quad (5.11)$$

为什么两套嵌入? 同一词作 target 和 context 的角色不同; 分开建模更灵活。最终通常取 target 嵌入 w 用, 但也可拼接 $[w; c]$ 。

朴素训练目标 (语料 T 个 token, 窗口半径 m):

$$\mathcal{L}_{\text{sg}} = \frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log p(w_{t+j} | w_t) \quad (5.12)$$

问题: 分母 $Z_w = \sum_{c'} \exp(c'^\top w)$ 需要遍历 $|\mathcal{V}|$, 对每个 (target, context) 都要算一次 → $O(|\mathcal{V}|)$ per step, 慢得不可接受。

5.7.4 Negative Sampling (核心技巧)

把“预测正确 context”问题换成“区分正例 vs 噪声样本”的二分类。对每个正对 (w, c) ，采 K 个负样本 $c_1, \dots, c_K \sim P_n(c)$ (噪声分布)。

单步损失：

$$\mathcal{L}_{(w,c)} = -\log \sigma(c^\top w) - \sum_{k=1}^K \mathbb{E}_{c_k \sim P_n} [\log \sigma(-c_k^\top w)] \quad (5.13)$$

其中 $\sigma(x) = 1/(1 + e^{-x})$ 。

全局目标：

$$\mathcal{L}_{\text{sgns}} = - \sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} \left[\log \sigma(c_{w_{t+j}}^\top w_t) + \sum_{k=1}^K \mathbb{E}_{c_k \sim P_n} [\log \sigma(-c_k^\top w_t)] \right] \quad (5.14)$$

实践细节：- $K = 5-20$ (小语料更大) - 噪声分布 $P_n(c) \propto \#(c)^{0.75}$ (unigram 的 0.75 次幂，经验最优) - **Subsampling 高频词**：以概率 $1 - \sqrt{t/f(w)}$ 丢弃 token ($t \approx 10^{-5}$) - 计算量从 $O(|\mathcal{V}|)$ 降到 $O(K + 1)$

5.7.5 Skip-gram + Negative Sampling 的梯度

对 w, c 的偏导 (用 $\sigma'(x) = \sigma(x)(1 - \sigma(x))$, $\partial \log \sigma(x)/\partial x = 1 - \sigma(x)$):

$$\frac{\partial \mathcal{L}_{(w,c)}}{\partial w} = -(1 - \sigma(c^\top w))c + \sum_{k=1}^K \sigma(c_k^\top w) c_k \quad (5.15)$$

$$\frac{\partial \mathcal{L}_{(w,c)}}{\partial c} = -(1 - \sigma(c^\top w))w \quad (5.16)$$

直观：正例时把 w 拉向 c ，负例时把 w 推离 c_k (与对比学习思想一致)。

[WARN] 易错点

SGNS 的目标是“ P_n 下的期望”，梯度公式里负项前的系数 σ (不是 $1 - \sigma$) ——因为 $\log \sigma(-x)$ 求导是 $-\sigma(x)$ 。

5.8 GloVe: 加权最小二乘 + 计数

5.8.1 动机

word2vec 是 预测式 + 隐式计数；GloVe (Pennington et al., 2014) 直接对共现统计 X_{ij} 建模，但仍学习稠密向量。

5.8.2 目标函数

$$J = \sum_{i,j=1}^{|\mathcal{V}|} f(X_{ij}) \left(w_i^\top c_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2 \quad (5.17)$$

- X_{ij} : 词 i 与上下文 j 的共现次数
- $w_i, c_j \in \mathbb{R}^d$: 目标 / 上下文向量
- $b_i, \tilde{b}_j$: 标量偏置
- $f(\cdot)$: 权重函数, 压制极端值

权重函数:

$$f(x) = \begin{cases} (x/x_{\max})^\alpha & x < x_{\max} \\ 1 & x \geq x_{\max} \end{cases}, \quad x_{\max} = 100, \alpha = 0.75 \quad (5.18)$$

- $f(0) = 0$: 不参与计算 ($\log 0$ 不需要处理)
- 对很小的 x 渐减为 0: 噪声不主导
- 对很大的 x 封顶为 1: 常见对不主导

5.8.3 偏置项的推导 (直觉)

理想化目标: $w_i^\top c_j = \log X_{ij}$ 。注意 $\log X_{ij} = \log P(j | i) + \log X_i$ (其中 $X_i = \sum_k X_{ik}$)。把和 j 无关的项 $\log X_i$ 吸入 b_i (目标词偏置), 把和 i 无关的项吸入 $\tilde{b}_j$ (上下文词偏置):

$$w_i^\top c_j + b_i + \tilde{b}_j \approx \log X_{ij} \quad (5.19)$$

更进一步, 从 "ratio of co-occurrence probabilities" 推导: GloVe 论文展示 $w_i^\top c_j - \log X_{ij} - b_i - \tilde{b}_j$ 能让

$$w_i^\top (c_j - c_k) \approx \log \frac{P(j | i)}{P(k | i)}$$

——这就是为什么 $\text{king} - \text{man} + \text{woman} \approx \text{queen}$ 类比关系能在 GloVe 空间中线性成立。

5.8.4 训练

- AdaGrad SGD
- 维度 $d = 50-300$
- 最终向量常用 $w_i + c_i$ (增强稳定性)

5.9 评测

5.9.1 Intrinsic (内在评测)

- 相似度: 与人工打分 (WordSim-353, SimLex-999) 的 Spearman / Pearson 相关
- 类比: $\text{Beijing} : \text{China} :: \text{Paris} : ?$ ——用 $w_{\text{China}} - w_{\text{Beijing}} + w_{\text{Paris}}$ 的最近邻评测
- TOEFL 同义词题: 四选一
- 聚类 / 可视化: t-SNE 投影到 2D 人工检查

5.9.2 Extrinsic (外在评测)

把词向量用到下游任务 (NER、POS、分类、QA)：– **Frozen features**：嵌入冻结 – **Fine-tuning**：嵌入随任务一起更新 看下游精度 / F1 是否提升。

5.9.3 Count-based vs Prediction-based

维度	Count-based (SVD/PPMI)	Prediction-based (w2v/GloVe)
稀疏 / 稠密	稀疏 / SVD 后稠密	稠密
训练速度	快 (一次分解)	与语料线性扩展
统计利用	高 (直接矩阵)	低 (流式采样)
下游性能	中等	通常更好
词序信息	无	局部窗口
多义	不区分	不区分 (仍单向量)

GloVe 是混合的 (用计数 + 学嵌入)，常在两端之间取平衡。

5.10 多义性与上下文相关表示 (过渡)

静态向量给每个词一个向量，但 play 有多种含义：

例句	play 含义
The new-look play area is due to be completed	玩耍区域
representatives who play to the party base	迎合行为
MJ just completed the three-point play for a 66-63 lead	比赛

一个向量必须妥协所有义项的”平均”。**One vector per word is not enough.**

→ 引出 **contextual representations** (ELMo, BERT)：同一词在不同句子有不同向量，由编码器根据整句生成。详见 lec11 预训练模型。

5.11 Worked Example: 手算 PPMI

回顾 4x5 矩阵，总共现 $N = 19$ ：

	computer	data	pinch	result	sugar	row sum
apricot	0	0	1	0	1	2
pineapple	0	0	1	0	1	2
digital	2	1	0	1	0	4
information	1	6	0	4	0	11
col sum	3	7	2	5	2	19

步骤 1: 算 $P(x, y), P(x), P(y)$ (每格除以 19)

例: $P(\text{apricot}, \text{pinch}) = 1/19 \approx 0.053$, $P(\text{apricot}) = 2/19 \approx 0.105$, $P(\text{pinch}) = 2/19 \approx 0.105$ 。

步骤 2: 算 PMI

$$\text{PMI}(\text{apricot}, \text{pinch}) = \log_2 \frac{0.053}{0.105 \times 0.105} = \log_2 \frac{0.053}{0.011} = \log_2 4.81 \approx 2.27$$

(讲义化简后 $\log_2(1 \cdot 19)/(2 \cdot 2) = \log_2 4.75 = 2.25$)

步骤 3: $\text{PPMI} = \max(\text{PMI}, 0)$

$$\text{PMI}(\text{information}, \text{computer}) = \log_2 \frac{1/19}{(11/19)(3/19)} = \log_2 \frac{19}{33} < 0 \Rightarrow \text{PPMI} = 0.$$

完整 PPMI 表 (讲义结果):

	computer	data	pinch	result	sugar
apricot			2.25		2.25
pineapple			2.25		2.25
digital	1.66	0.00		0.00	
information	0.00	0.57		0.47	

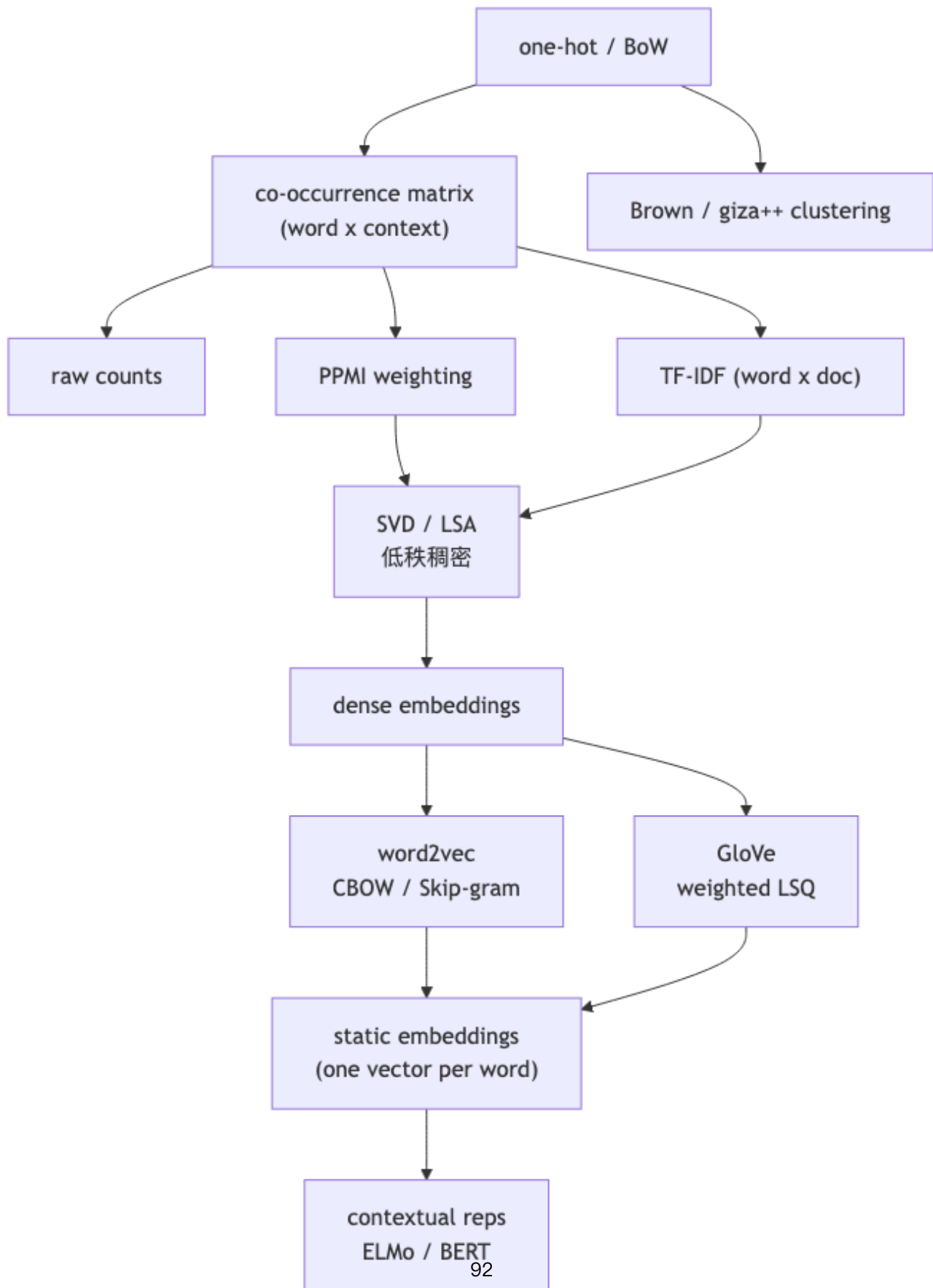
步骤 4: 余弦相似度

$$\cos(\text{apricot}, \text{pineapple}) = \frac{2.25 \cdot 2.25 + 2.25 \cdot 2.25}{\sqrt{2 \cdot 2.25^2} \sqrt{2 \cdot 2.25^2}} = 1 \text{ (完美对齐)}.$$

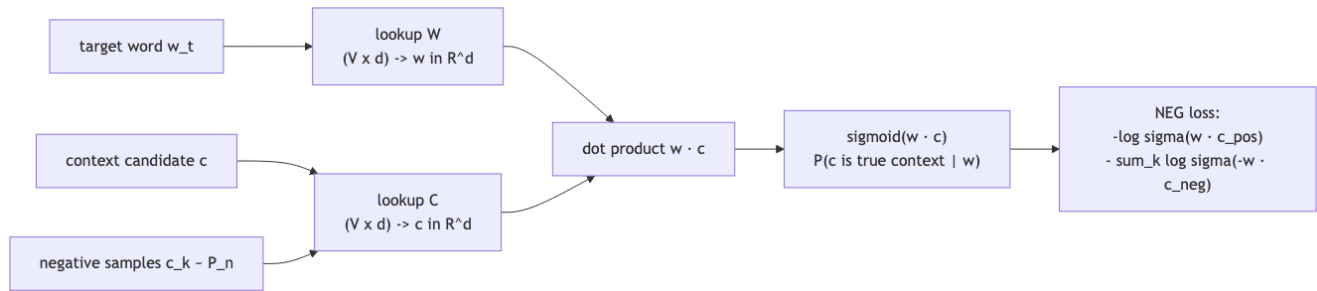
$$\cos(\text{apricot}, \text{digital}) = 0 \text{ (向量正交, 无共同 context)} \rightarrow \text{模型正确分开两个语义簇}.$$

5.12 概念关系图 (mermaid)

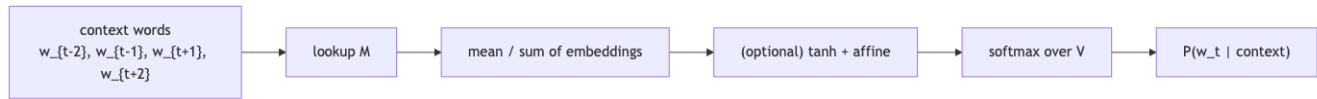
5.12.1 词表示演进树



5.12.2 Skip-gram 架构



5.12.3 CBOW 架构



5.13 易错点

[WARN] 期中常见陷阱

1. PMI 不是对称归一化: $P(x, y) / (P(x)P(y))$ 的对数; 不要写成 $P(x|y)P(y|x)$ 之类。
2. PPMI 阈值是 0: $\max(\text{PMI}, 0)$, 不是 $\max(\text{PMI}, \epsilon)$ 。
3. TF-IDF 的 TF 用 log 压缩: $1 + \log_{10} c$; count = 0 时定义为 0, 不要写成 $\log 1 = 0$ 同义。
4. SVD 截断: 截掉的是最小奇异值的维度 (信息少), 不是最大。
5. Skip-gram 朴素 softmax 太慢: 必须用 negative sampling 或 hierarchical softmax; 负采样不是 softmax 的近似而是另一目标 (NCE 的简化)。
6. SGNS 的负样本分布: $P_n(c) \propto \#(c)^{0.75}$, 不是 $\#(c)$ 也不是均匀。
7. 每个词有两个嵌入 (w 和 c); 用哪个看习惯, 常见做法是只用 w 或 $w + c$ 。
8. GloVe 的权重函数: $f(0) = 0$ 避免 $\log 0$; 不要把 f 写成单调递增到无穷。
9. 类比关系不一定线性: GloVe / w2v 在 king: queen 这种好; 在 Beijing: China 类的国家-首都也行; 但在 cat: kitten 这种关系上不稳定。
10. 分布式 $\neq$ 真正语义: 同义和反义在分布上都很像 (都填同样的槽), 所以 hot / cold 余弦相似度也很高——这正是为什么 SimLex-999 设计来“惩罚 relatedness”。
11. 静态嵌入不解决多义: bank (河岸 vs 银行) 共享一个向量, 是一个折中——动机引出上下文相关模型。

5.14 中英术语对照

中文	English	备注
分布式语义	distributional semantics	Firth / Harris
分布假说	distributional hypothesis	“邻居定义意义”
词义	word sense	一词多义
同义 / 反义 / 相关	synonymy / antonymy / relatedness	
上下位关系	hypernym / hyponym	树状语义结构
部分-整体关系	meronym / holonym	
词典 / 同义词集合	lexicon / synset	WordNet 基本单元
共现	co-occurrence	first-order / second-order
句法共现	syntagmatic	常相邻
聚合共现	paradigmatic	共享邻居

中文	English	备注
向量空间模型	vector space model (VSM)	
词嵌入	word embedding	稠密低维
一热编码	one-hot encoding	$ \mathcal{V} $ 维
余弦相似度	cosine similarity	方向相似度
点互信息	pointwise mutual information (PMI)	$\log \frac{P(x, y)}{P(x)P(y)}$
正点互信息	positive PMI (PPMI)	$\max(\text{PMI}, 0)$
词频-逆文档频率	term frequency-inverse document frequency (TF-IDF)	IR 加权
奇异值分解	singular value decomposition (SVD)	
潜在语义分析	latent semantic analysis (LSA)	SVD on word-doc
概率潜在语义分析	probabilistic LSA (PLSA)	Hofmann 1999
主题模型	topic model	LDA 等
低秩近似	low-rank approximation	Eckart-Young
词袋	bag-of-words (BoW)	忽略词序
连续词袋	continuous bag-of-words (CBOW)	word2vec 之一
跳字	skip-gram	word2vec 之一
负采样	negative sampling (NEG / SGNS)	
噪声对比估计	noise contrastive estimation (NCE)	负采样的理论原型
层次 softmax	hierarchical softmax	树状加速
全局向量	GloVe	加权 LSQ
加权最小二乘	weighted least squares	GloVe 目标
类比	analogy	$a - b + c$
上下文相关表示	contextual representation	ELMo / BERT 的起点
内在评测 / 外在评测	intrinsic / extrinsic evaluation	

[TIP] 期中复习要点

1. 能口头解释 Harris/Firth 分布假说及其与 WordNet 的对比 2. 能写出 PMI、PPMI、TF-IDF 的公式并解释为何加权 3. 能在 4x5 共现表上手算 PPMI 矩阵和两词余弦相似度 4. 能解释 SVD 截断为何给出最优低秩近似 (Eckart-Young) 5. 能写出 Skip-gram + 负采样的完整损失 (式 5.13–5.14) 和梯度 (式 5.15–5.16) 6. 能写出 GloVe 目标 (5.17)、权重函数 (5.18) 并解释偏置项 7. 能对比 count-based vs prediction-based: 稀疏 / 稠密、统计利用、下游表现 8. 能解释为什么静态向量解决不了多义, 从而引出上下文相关表示 9. 能讨论分布式相似度的“反义陷阱”: hot / cold 共现分布相近但语义相反

Lecture 6 (I): 隐马尔可夫模型与序列标注 (Hidden Markov Models for Sequence Tagging)

[TIP] 学习指南

本讲核心：把“给每个 token 一个 tag”的任务 (POS / NER / 分词 / Chunking) 建模为一个概率生成过程——隐马尔可夫模型 (HMM)，并用 **Viterbi 动态规划** 做解码。三大问题 (Rabiner, 1989) 务必背熟：1. **Likelihood (Evaluation)**: 给定 λ 与 O ，求 $P(O | \lambda)$ ——Forward 算法；2. **Decoding**: 给定 λ 与 O ，求最优状态序列 Q^* ——Viterbi 算法；3. **Learning**: 给定 O (可能无标签)，求 $\hat{\lambda}$ ——监督 MLE / 无监督 Baum-Welch (EM)。

前置：lec4_lm (n-gram 语言模型、MLE、smoothing)、Bayes 公式、条件独立、动态规划基础。

复习关注点：HMM 两个独立性假设、Viterbi 与 Forward 的差别 (max vs sum)、Baum-Welch 中 $\gamma_t(i)$ 与 $\xi_t(i, j)$ 的定义与再估公式、复杂度 $O(n|S|^2)$ (bigram) 或 $O(n|S|^3)$ (trigram)。

6.1 序列标注问题 (Sequence Tagging)

6.1.1 任务谱系

NLP 任务按“决策结构”复杂度可分三层：

类型	例子
a single decision	文本分类、词义消歧 (WSD)
a sequence of decisions	词性标注 (POS)、NER、分词、Chunking
decisions with complex structures	句法树、依存图、事件抽取

序列标注 (Sequence Tagging)：给定序列 $x = x_1, \dots, x_n$ ，从预定义标签集 $\mathcal{T}$ 中给每个 x_i 指派一个 y_i ，输出 $y = y_1, \dots, y_n$ 。

典型任务：

- POS Tagging: China/N Mobile/N is/V a/DT communication/N giant/N in/P east/ADJ Asia/N
- NER: [China Mobile/Org] is a communication giant in east [Asia/Loc]
- Word Segmentation: 我爱北京天安门 → 我 爱 北京 天安门
- NP Chunking / Base NP: 找出最浅层的名词短语
- Case / Accent Restoration、Information Extraction.....

6.1.2 把“分段问题”转成“逐 token 标注”——BIO 方案

非标签型的分段任务 (chunking / NER / 分词) 通过引入 BIO 标签集变成纯序列标注:

- B: phrase 的开始 (Beginning)
- I: phrase 内部 (Inside)
- O: 不属于任何 phrase (Other)

可扩展为 BIEO (加 Ending)、BIOES (加 Single) 等更细方案。

例 (NP Chunking):

[China/B Mobile/I] is/O [a/B communication/I giant/I] in/O [east/B Asia/I]

例 (NER, 带类型):

[China/B-Org Mobile/I-Org] is/O a/O communication/O giant/O in/O east/O [Asia/B-Loc]

例 (中文分词, 字一级 BI 标注):

我/B 爱/B 北/B 京/I 天/B 安/I 门/I

6.1.3 难点: 歧义 (Ambiguity)

词性歧义: 同一词形可对应多个 POS。

- They canned the peas. —can 作动词;
- I can do it. —can 作情态动词;
- Throw these beer cans. —can 作名词;
- 经典句: Time flies like an arrow.

分词歧义 (中文):

- 南京市长江大桥 → 南京市 / 长江大桥 或 南京 / 市长 / 江大桥
- 下雨天留客天留我不留 → 下雨 / 天留客 / 天留 / 我不留 或 下雨天 / 留客天 / 留我 / 不留.....

[NOTE] 启示

仅靠 token 自身 (local clue) 不足以消歧, 必须引入上下文 (contextual) 建模——这正是 HMM 之所以胜过“每词独立分类”的根本理由。

6.2 词性标注问题的概率建模

6.2.1 形式化与 Bayes 翻转

记句子 $x = x_1, \dots, x_n$, 对应 tag 序列 $y = y_1, \dots, y_n$ 。我们要找:

$$y^* = \arg \max_y P(y | x) = \arg \max_y \frac{P(x, y)}{P(x)} = \arg \max_y P(x, y). \quad (6.1)$$

(分母 $P(x)$ 与 y 无关, 可去掉。)

进一步用 Bayes:

$$P(x, y) = \underbrace{P(y)}_{\text{先验: tag 序列}} \cdot \underbrace{P(x | y)}_{\text{似然: 词 | tag}}. \quad (6.2)$$

6.2.2 两个核心假设

假设 A: tag 序列服从马尔可夫链。把 $P(y)$ 像语言模型那样链式展开, 再做 k -阶马尔可夫近似:

- Bigram (1 阶): $P(y) \approx \prod_i P(y_i | y_{i-1})$;
- Trigram (2 阶): $P(y) \approx \prod_i P(y_i | y_{i-2}, y_{i-1})$ 。

假设 B: 观测条件独立 (emission independence)。每个 token 只依赖其当前 tag:

$$P(x | y) \approx \prod_{i=1}^n P(x_i | y_i). \quad (6.3)$$

代入 (6.2), 对 bigram HMM:

$$P(x, y) = \prod_{i=1}^n \underbrace{P(y_i | y_{i-1})}_{\text{transition}} \underbrace{P(x_i | y_i)}_{\text{emission}} \quad (6.4)$$

(约定 $y_0 = \text{START}$, 并在末尾乘 $P(\text{STOP} | y_n)$)。

[WARN] 这两个假设都很“粗暴”

– 假设 A 切断了远距离 tag 依赖 (“南京市市长”的歧义靠 trigram 部分缓解); – 假设 B 切断了 x_i 与上下文 $x_{<i}$ 的直接联系 (实际中 apple 在 Apple Inc. 与水果之间的偏好就丢失了)。CRF / 神经标注器正是为了保留特征灵活性而生 (见 lec6_seq_crf)。

6.3 隐马尔可夫模型 (HMM)

6.3.1 五元组定义

一个一阶 (bigram) HMM 形式化为 $\lambda = (S, V, A, B, \pi)$:

符号	含义
$S = \{s_1, \dots, s_N\}$	隐状态集 (hidden states): POS tag
$V = \{v_1, \dots, v_M\}$	观测字典 (vocabulary): 词
$A = [a_{ij}]$	转移矩阵, $a_{ij} = P(y_t = s_j y_{t-1} = s_i)$
$B = [b_j(k)]$	发射矩阵, $b_j(k) = P(x_t = v_k y_t = s_j)$
$\pi = [\pi_i]$	初始分布, $\pi_i = P(y_1 = s_i)$

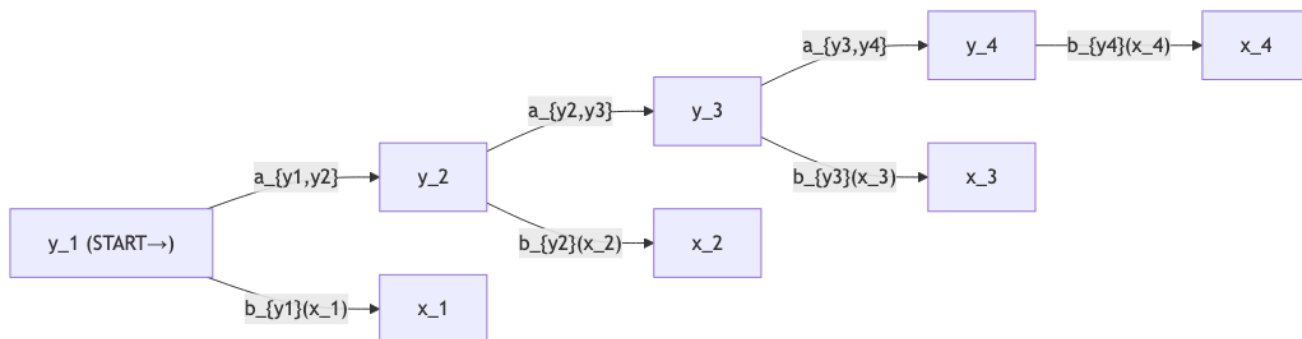
满足 $\sum_j a_{ij} = 1$, $\sum_k b_j(k) = 1$, $\sum_i \pi_i = 1$ 。

6.3.2 生成故事 (Generative Story)

HMM 是一个生成式模型: 先生成隐状态链, 再由状态发射出词。

1. $y_1 \sim \pi$;
2. $x_1 \sim B(\cdot | y_1)$;

3. for $t = 2, \dots, n$:
- $y_t \sim A(\cdot | y_{t-1})$
 - $x_t \sim B(\cdot | y_t)$



[NOTE] “隐”在哪里？

我们只能观测到 x （词），而隐藏链 y （tag）需要根据 x 推断——这就是“Hidden”的含义。

6.3.3 HMM 三大问题 (Rabiner 1989)

#	名称	输入	输出	算法
1	Likelihood (Evaluation)	λ, O	$P(O \lambda)$	Forward / Backward
2	Decoding	λ, O	$Q^* = \arg \max_Q P(Q O, \lambda)$	Viterbi
3	Learning	O (可带 label)	$\hat{\lambda}$	MLE (监督) 或 Baum-Welch / EM (无监督)

后面三节分别展开。

6.4 问题一：Likelihood ——Forward / Backward

6.4.1 朴素计算的爆炸

$$P(O | \lambda) = \sum_Q P(O, Q | \lambda) = \sum_{q_1, \dots, q_n} \pi_{q_1} b_{q_1}(o_1) \prod_{t=2}^n a_{q_{t-1}q_t} b_{q_t}(o_t).$$

直接枚举 Q 有 N^n 项，指数爆炸。用动态规划压到 $O(nN^2)$ 。

6.4.2 Forward 算法

定义前向概率：

$$\alpha_t(j) \triangleq P(o_1, o_2, \dots, o_t, y_t = s_j \mid \lambda). \quad (6.5)$$

含义：观察到前缀 $o_{1:t}$ 且第 t 步落在状态 s_j 的联合概率。

递推：

- 初始： $\alpha_1(j) = \pi_j b_j(o_1)$;
- 递推 ($t \geq 2$):

$$\alpha_t(j) = \left[\sum_{i=1}^N \alpha_{t-1}(i) a_{ij} \right] b_j(o_t) \quad (6.6)$$

- 终止： $P(O \mid \lambda) = \sum_{j=1}^N \alpha_n(j)$ 。

复杂度： $O(nN^2)$ 。

6.4.3 Backward 算法

定义后向概率：

$$\beta_t(i) \triangleq P(o_{t+1}, o_{t+2}, \dots, o_n \mid y_t = s_i, \lambda). \quad (6.7)$$

含义：在第 t 步处于状态 s_i 的条件下，未来观测 $o_{t+1:n}$ 的概率。

递推（从右向左）：

- 初始： $\beta_n(i) = 1$ （边界条件）；
- 递推 ($t = n - 1, \dots, 1$):

$$\beta_t(i) = \sum_{j=1}^N a_{ij} b_j(o_{t+1}) \beta_{t+1}(j) \quad (6.8)$$

- 终止： $P(O \mid \lambda) = \sum_{j=1}^N \pi_j b_j(o_1) \beta_1(j)$ 。

[NOTE] forward/backward 的“边界”

– α 自左向右，包含当前时刻 t 的观测；– β 自右向左，不含当前时刻 t 的观测（只看未来）；– 二者的乘积满足 $\alpha_t(i)\beta_t(i) = P(O, y_t = s_i \mid \lambda)$ ，这一恒等式是 Baum-Welch 的核心，下文 6.6 节会用到。

6.5 问题二：Decoding ——Viterbi 算法

6.5.1 直觉：Forward 求和 $\rightarrow$ Viterbi 取最大

把 Forward 公式 (6.6) 的 $\sum$ 换成 $\max$ ，再加一份回溯指针 ψ ，就得到 Viterbi。

6.5.2 Bigram Viterbi

定义：

$$\delta_t(j) \triangleq \max_{y_1, \dots, y_{t-1}} P(y_1, \dots, y_{t-1}, y_t = s_j, o_1, \dots, o_t \mid \lambda). \quad (6.9)$$

即”以 s_j 结尾、长度为 t 的最优前缀”的联合概率。

递推：

- 初始: $\delta_1(j) = \pi_j b_j(o_1)$, $\psi_1(j) = 0$;
- 递推 ($t \geq 2$):

$$\delta_t(j) = \max_{i=1, \dots, N} [\delta_{t-1}(i) a_{ij}] b_j(o_t) \quad (6.10)$$

$$\psi_t(j) = \arg \max_{i=1, \dots, N} [\delta_{t-1}(i) a_{ij}]. \quad (6.11)$$

- 终止: $P^* = \max_j \delta_n(j)$, $y_n^* = \arg \max_j \delta_n(j)$;
- 回溯: $y_t^* = \psi_{t+1}(y_{t+1}^*)$, $t = n-1, \dots, 1$ 。

复杂度: $O(nN^2)$ 。

6.5.3 Trigram Viterbi (讲义版本)

讲义采用 trigram HMM, 把状态扩成 $(u, v) = (y_{t-1}, y_t)$ 二元 pair:

$$\pi(k, u, v) \triangleq \max_{y_1 \dots y_{k-2}} P(y_1, \dots, y_{k-2}, u, v, o_1, \dots, o_k). \quad (6.12)$$

递推 ($k \geq 1$):

$$\pi(k, u, v) = \max_{w \in S_{k-2}} \{ \pi(k-1, w, u) P(v \mid w, u) P(o_k \mid v) \} \quad (6.13)$$

$$\text{path}(k, u, v) = \arg \max_{w \in S_{k-2}} \{ \pi(k-1, w, u) P(v \mid w, u) P(o_k \mid v) \}. \quad (6.14)$$

初始: $\pi(0, \text{START}, \text{START}) = 1$ 。

终止:

$$P^* = \max_{u \in S_{n-1}, v \in S_n} \{ \pi(n, u, v) P(\text{STOP} \mid u, v) \}. \quad (6.15)$$

回溯: 先得 (y_{n-1}^*, y_n^*) , 再 $y_{k-2}^* = \text{path}(k, y_{k-1}^*, y_k^*)$ 。

复杂度: $O(nN^3)$ 。

6.5.4 为什么 Viterbi 是正确的？

子结构最优性 (Optimal Substructure): 以 s_j 结尾的全局最优路径，其前 $t-1$ 步必然是“以某 s_i 结尾的局部最优路径”。

证：假设存在 s_j -结尾的最优路径 P^* ，其前缀不是任何 $\delta_{t-1}(\cdot)$ 对应的最优前缀。那么用真正的最优前缀替换之，乘上同样的 $a_{ij}b_j(o_t)$ ，得到更大的概率，与 P^* 最优矛盾。□

[WARN] 暴力搜索为什么不行？

长度 n 、tag 数 N 的所有 tag 序列共 N^n 条，每条要 $O(n)$ 计算概率，总开销 $O(nN^n)$ 。对 PTB 的 $N \approx 45$ 、 $n = 20$ ，这是 $20 \cdot 45^{20} \approx 7 \times 10^{34}$ 。Viterbi 把它压到 $O(nN^2) \approx 4 \times 10^4$ 。这是 DP 思想的教科书式胜利。

6.6 问题三：Learning ——MLE 与 Baum–Welch

6.6.1 监督学习：直接计数 + 平滑

若训练数据 $(x^{(d)}, y^{(d)})$ 同时给出词与 tag (如 Penn Treebank)，用极大似然估计：

$$\hat{a}_{ij} = \frac{\#(s_i \rightarrow s_j)}{\#(s_i \rightarrow \cdot)}, \quad \hat{b}_j(k) = \frac{\#(s_j \uparrow v_k)}{\#(s_j)}, \quad \hat{\pi}_i = \frac{\#(\text{首词为 } s_i)}{\#(\text{句子数})}. \quad (6.16)$$

问题：未出现的 (s_i, s_j) 或 (s_j, v_k) 会得到 0 概率，导致整条 Viterbi 路径概率为 0。**解决：**用 lec4_lm 学过的 **smoothing** (add- k 、Witten–Bell、Kneser–Ney、deleted interpolation 等)。

6.6.2 无监督学习：Baum–Welch (EM 在 HMM 上的实例)

只有词、没有 tag，怎么训练？用 EM：把 y 视作隐变量，迭代最大化 $\log P(x | \lambda)$ 。

E-step：计算后验

利用 α, β 算两个关键后验：

单时刻后验 $\gamma_t(i)$ ：

$$\gamma_t(i) \triangleq P(y_t = s_i | O, \lambda) = \frac{\alpha_t(i)\beta_t(i)}{\sum_j \alpha_t(j)\beta_t(j)} = \frac{\alpha_t(i)\beta_t(i)}{P(O | \lambda)}. \quad (6.17)$$

双时刻后验 $\xi_t(i, j)$ (“软对齐”边)：

$$\xi_t(i, j) \triangleq P(y_t = s_i, y_{t+1} = s_j | O, \lambda) = \frac{\alpha_t(i) a_{ij} b_j(o_{t+1}) \beta_{t+1}(j)}{P(O | \lambda)}. \quad (6.18)$$

注意 $\gamma_t(i) = \sum_j \xi_t(i, j)$ (边际化)。

M-step: 再估参数

把”硬计数 #“换成”软计数 $\sum \gamma / \sum \xi$ “; 再做 MLE:

$$\hat{\pi}_i = \gamma_1(i), \quad (6.19)$$

$$\hat{a}_{ij} = \frac{\sum_{t=1}^{n-1} \xi_t(i, j)}{\sum_{t=1}^{n-1} \gamma_t(i)}, \quad (6.20)$$

$$\hat{b}_j(v_k) = \frac{\sum_{t=1, o_t=v_k}^n \gamma_t(j)}{\sum_{t=1}^n \gamma_t(j)}. \quad (6.21)$$

迭代 E/M 直到 $\log P(O | \lambda)$ 收敛。保证单调上升, 但只收敛到局部极值, 对初值敏感。

[NOTE] 直观理解

– $\gamma_t(i)$: 在第 t 步处于状态 s_i 的”软指示器”; – $\xi_t(i, j)$: 在第 $t \rightarrow t+1$ 步发生 $s_i \rightarrow s_j$ 转移的”软指示器”; 把它们当作 fractional count 喂给 MLE, 就是 Baum–Welch 的本质。

6.7 Worked Example: 3-state Toy HMM Viterbi

7.1 模型 (一个 toy weather/ice-cream HMM)

- 状态 $S = \{H, C\}$ (Hot / Cold);
- 观测 $V = \{1, 2, 3\}$ (一天吃几只冰激凌);
- 参数:

$$\pi = \begin{pmatrix} 0.8 \\ 0.2 \end{pmatrix}, \quad A = \begin{pmatrix} a_{HH} & a_{HC} \\ a_{CH} & a_{CC} \end{pmatrix} = \begin{pmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{pmatrix},$$

$$B = \begin{pmatrix} b_H(1) & b_H(2) & b_H(3) \\ b_C(1) & b_C(2) & b_C(3) \end{pmatrix} = \begin{pmatrix} 0.2 & 0.4 & 0.4 \\ 0.5 & 0.4 & 0.1 \end{pmatrix}.$$

观测序列 $O = (3, 1, 3)$ 。求最可能的天气序列。

7.2 Viterbi 表

$t = 1$:

$$\delta_1(H) = \pi_H \cdot b_H(3) = 0.8 \times 0.4 = 0.32; \quad \delta_1(C) = 0.2 \times 0.1 = 0.02.$$

$t = 2$ (观测 $o_2 = 1$):

$$\delta_2(H) = \max\{0.32 \cdot 0.7, 0.02 \cdot 0.4\} \cdot b_H(1) = 0.224 \cdot 0.2 = 0.0448, \quad \psi_2(H) = H;$$

$$\delta_2(C) = \max\{0.32 \cdot 0.3, 0.02 \cdot 0.6\} \cdot b_C(1) = 0.096 \cdot 0.5 = 0.048, \quad \psi_2(C) = H.$$

$t = 3$ (观测 $o_3 = 3$):

$$\delta_3(H) = \max\{0.0448 \cdot 0.7, 0.048 \cdot 0.4\} \cdot b_H(3) = 0.03136 \cdot 0.4 = 0.012544, \quad \psi_3(H) = H;$$

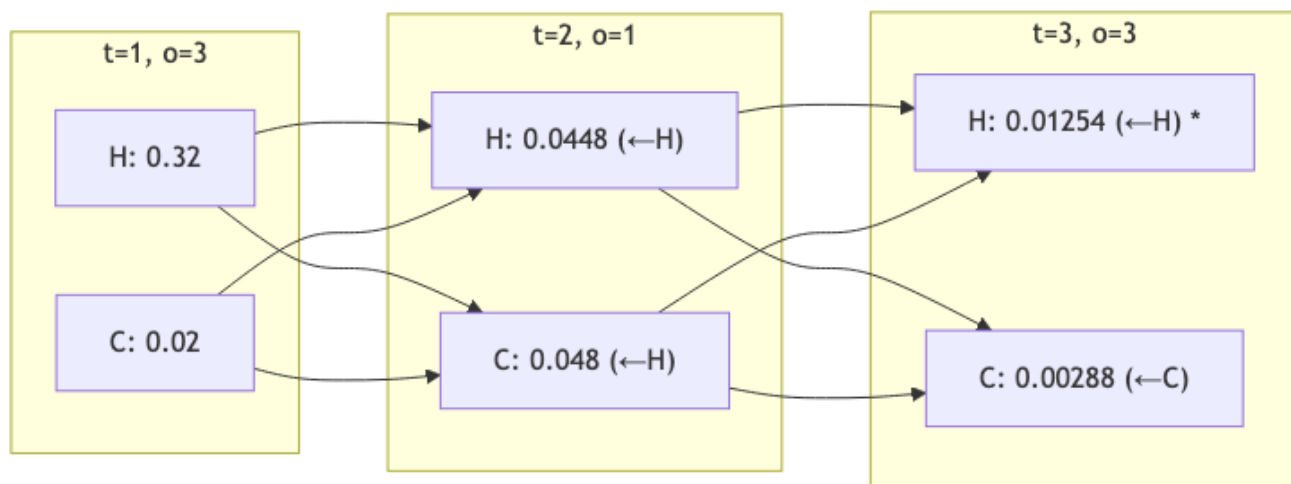
$$\delta_3(C) = \max\{0.0448 \cdot 0.3, 0.048 \cdot 0.6\} \cdot b_C(3) = 0.0288 \cdot 0.1 = 0.00288, \quad \psi_3(C) = C.$$

回溯:

- $y_3^* = \arg \max\{0.012544, 0.00288\} = H$;
- $y_2^* = \psi_3(H) = H$;
- $y_1^* = \psi_2(H) = H$ 。

最优序列 $y^* = (H, H, H)$, 概率 0.012544。

7.3 DP 表流程图



* = 最终回溯起点。

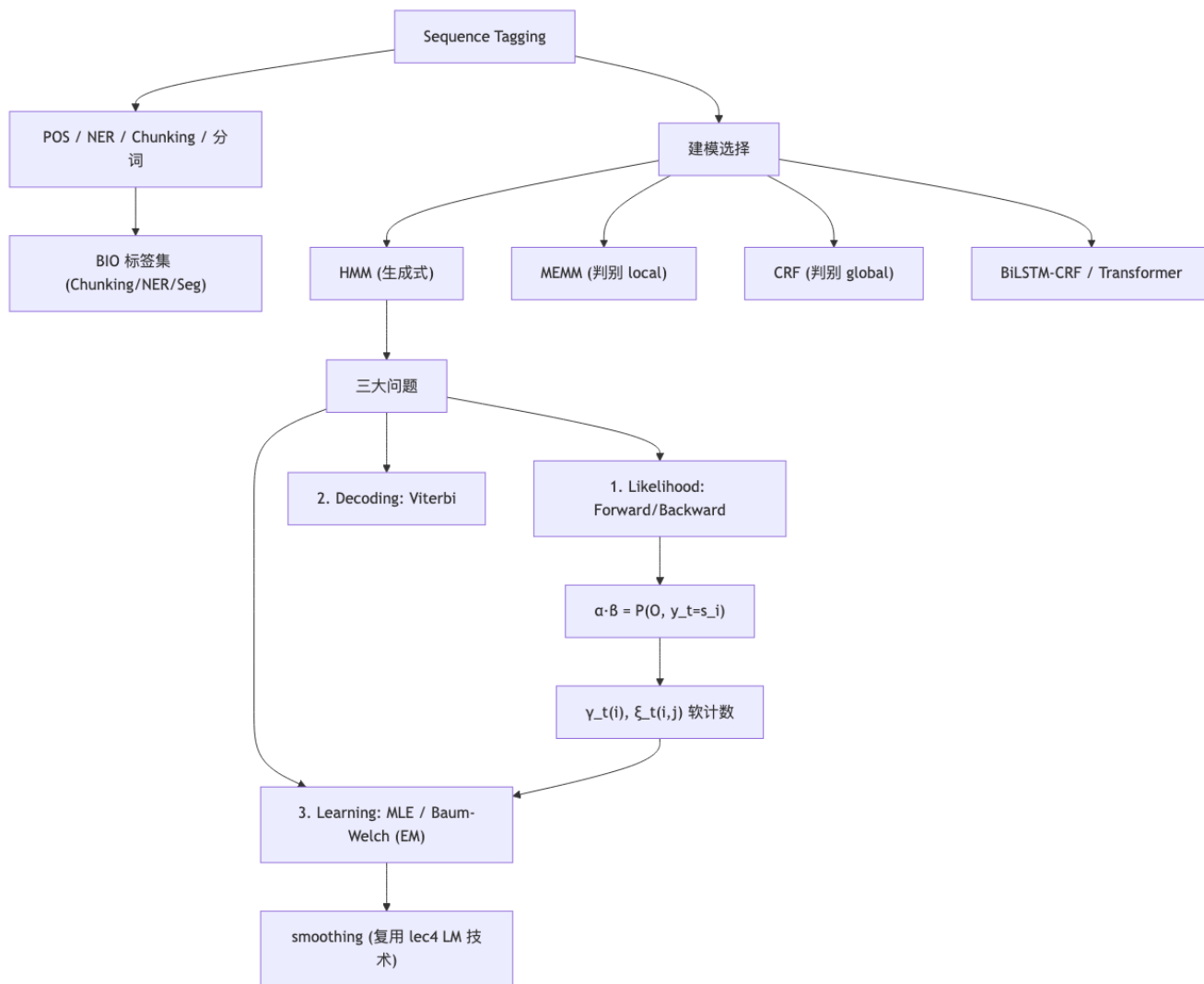
6.8 HMM 与同类模型对比

维度	HMM (生成式)	MEMM (判别式, 局部)	CRF (判别式, 全局)
建模目标	$P(x, y)$	$\prod_t P(y_t y_{t-1}, x_t)$	$P(y x)$
归一化	局部 ($\sum_x b_j(x) = 1$)	局部 (每步 softmax)	全局 (partition $Z(x)$)
特征灵活性	受限于 emission	可任意特征 $\phi(y_t, y_{t-1}, x_{1:n})$	同 MEMM, 但避免 label bias
训练	计数 / Baum-Welch	per-step max entropy	全局 log-likelihood + grad

维度	HMM (生成式)	MEMM (判别式, 局部)	CRF (判别式, 全局)
解码 经典缺陷	Viterbi $O(nN^2)$ 假设强、特征贫乏	Viterbi (over local probs) label bias	Viterbi (over potentials) 训练慢

CRF 与 MEMM 的细节见 `lec06_seq_crf.md`。

6.9 概念关系图



6.10 易错点 (Pitfalls)

1. **Forward vs Viterbi 混淆**: 递推结构相同, 差别只在 $\sum$ vs $\max$; Viterbi 多一份 backpointer。
2. **概率乘法下溢**: 长序列累乘易溢出, **实现时取 $\log$** , 加法替乘法、`logsumexp` 替求和。
3. **trigram Viterbi 复杂度**: 是 $O(nN^3)$ 不是 $O(nN^2)$; 状态升维到 pair (u, v) 。

4. 未见词 (OOV): emission $b_j(x_t) = 0$ 会让整条路径概率为 0。必须有 unk 模型 / 字符级 backoff。
5. Baum-Welch 不保证全局最优: EM 只单调上升 $\log P(O | \lambda)$, 不同初值得到不同局部最优——多次随机初始化取最好。
6. $\xi_t(i, j)$ 与 $\gamma_t(i)$ 的下标范围: ξ 的 t 只到 $n - 1$ (因为要看 o_{t+1}); 分母 $\sum \gamma_t(i)$ 也只算到 $n - 1$ (否则与转移分母不匹配)。
7. bigram 假设 = 1 阶马尔可夫: 意思是 y_t 只依赖 y_{t-1} , 不是只看 y_{t-1} 一个。trigram = 2 阶。
8. 生成式 vs 判别式: HMM 学的是联合 $P(x, y)$, 所以理论上还能”生成”句子 (虽然没人这么用); CRF 只能 score 给定 x 的 y , 不能采样 x 。
9. 特征贫瘠: HMM 的 emission 只能用 token 自身, 不能用前缀 / 后缀 / 大小写 / 词性词典等”工程特征”。这是 CRF 出现的直接动机。

6.11 中英术语对照

中文	英文	缩写 / 备注
序列标注	Sequence Tagging / Labeling	—
词性标注	Part-of-Speech Tagging	POS
命名实体识别	Named Entity Recognition	NER
名词短语分块	Noun Phrase Chunking	NP Chunking
隐马尔可夫模型	Hidden Markov Model	HMM
隐状态	hidden state	S
观测	observation	V
转移概率	transition probability	a_{ij}
发射概率	emission probability	$b_j(k)$
初始分布	initial state distribution	π
前向算法	Forward algorithm	$\alpha_t(j)$
后向算法	Backward algorithm	$\beta_t(i)$
维特比算法	Viterbi algorithm	$\delta_t(j), \psi_t(j)$
鲍姆-韦尔奇算法	Baum-Welch algorithm	EM for HMM
期望最大化	Expectation Maximization	EM
单时刻后验	smoothed marginal	$\gamma_t(i)$
双时刻后验	pairwise marginal	$\xi_t(i, j)$
极大似然估计	Maximum Likelihood Estimation	MLE
平滑	smoothing	add- k , KN, ...
解码	decoding	—
BIO 标记	BIO tagging scheme	Begin-Inside-Other
生成式模型	generative model	$P(x, y)$
判别式模型	discriminative model	$P(y \ x)$

[TIP] 期中复习要点

1. **公式默写**: bigram HMM 联合概率 (6.4)、Forward (6.6)、Backward (6.8)、Viterbi (6.10)、 γ_t, ξ_t (6.17)(6.18)、Baum–Welch 三条再估 (6.19)–(6.21)。2. **算法对比**: Forward (sum) Viterbi (max); 监督 MLE 无监督 Baum–Welch。3. **复杂度**: bigram $O(nN^2)$, trigram $O(nN^3)$; 与暴力搜索 $O(nN^n)$ 对比。4. **手算 Viterbi**: 教材常出 2–3 state、3–4 step 的 toy。务必能列 DP 表 + 回溯。5. **三大问题口诀**: 似然 (Forward)、解码 (Viterbi)、学习 (Baum–Welch)。6. **HMM 假设的 2 个**: 1 阶马尔可夫 (tag) + emission 条件独立 (word); 并能指出它们的不合理之处。7. **与 CRF 的对比**: 生成 vs 判别、局部 vs 全局归一化、特征灵活性、label bias。8. **BIO 方案**: 能把 NER / chunking / 分词翻译成序列标注。9. **EM 的性质**: 单调上升、局部最优、初值敏感。10. **应用现实**: HMM 仍是低资源 / 嵌入式 / 小语种 POS 的强 baseline; 现代 SOTA 是 BiLSTM–CRF / BERT–CRF。

Lecture 6 (II): Beyond HMM ——MEMM、CRF 与神经序列标注 (Sequence Models beyond HMM)

[TIP] 学习指南

本讲核心：HMM 的两个独立性假设让它“做不了特征工程、用不了上下文”，本讲从三个层面递进解决：1. **MEMM (判别 + 局部归一化)**：把“生成式 emission”换成“判别式 softmax”，引入任意特征——但带来 **label bias** 问题；2. **CRF (判别 + 全局归一化)**：在整条序列上算 partition function $Z(x)$ ，从根上去除 label bias，成为统计时代的 SOTA 序列标注器；3. **神经序列标注**：BiLSTM / BiLSTM-CRF / BERT-CRF，把“特征工程”换成“表征学习”，再用 CRF 头做结构化解码。

前置：lec06_hmm (HMM、Viterbi、Forward-Backward)、lec3_loglinear (log-linear / max-ent 分类器、softmax、梯度下降)、lec4_nlm + lec5_dist (embedding、RNN/LSTM)。

复习关注点：线性链 CRF 的势函数与全局归一化、partition function 的 forward 计算、log-likelihood 梯度 = “经验特征计数 - 期望特征计数”、label bias 的直观例子、HMM/MEMM/CRF 三者对比、为何 BiLSTM-CRF 比 BiLSTM-softmax 更好。

7.1 动机：HMM 的两个“硬伤”

回顾 lec06_hmm，HMM 联合概率：

$$P(x, y) = \prod_{t=1}^n P(y_t | y_{t-1}) P(x_t | y_t). \quad (7.1)$$

它隐含两个让人不适的假设：

#	假设	后果
A	1 阶马尔可夫 (tag 仅依赖前 1-2 个 tag)	难以捕捉长距 tag 依赖
B	x_t 仅依赖 y_t ，与上下文 $x_{<t}, x_{>t}$ 条件独立	无法使用任何“句子级特征”：词的前缀/后缀、大小写、是否含数字、词典查询、相邻词、词性 n-gram.....

NLP 里大量“工程特征”对识别命名实体、词性歧义有奇效，比如：

- Mr. 后面大概率是 B-PER；
- 大写开头 + 后缀 -ville 大概率是 Loc；

- 出现在 PER 列表里的词大概是 PER;
- 当前词的 POS、当前词是否在 gazetteer 中.....

HMM 用不上这些——它的 emission 只能依赖 x_t 本身。

7.1.1 一个自然的想法：把”生成 emission”换成”判别 emission”

把式 (7.1) 翻转方向：不再算 $P(x | y)$ ，直接算 $P(y_t | y_{t-1}, x_{1:n})$ ，并用 log-linear / softmax 表达任意特征：

$$P(y_t | y_{t-1}, x_{1:n}) = \frac{\exp(\lambda^\top f(y_{t-1}, y_t, x_{1:n}, t))}{\sum_{y'} \exp(\lambda^\top f(y_{t-1}, y', x_{1:n}, t))}. \quad (7.2)$$

整条序列概率：

$$P(y | x) = \prod_{t=1}^n P(y_t | y_{t-1}, x_{1:n}). \quad (7.3)$$

这就是 MEMM (Maximum Entropy Markov Model) ——McCallum, Freitag & Pereira (2000)。

它解决了”特征贫瘠”，但带来一个更隐蔽的问题：Label Bias。

7.2 Label Bias 问题 (Lafferty et al. 2001)

7.2.1 直观例子

考虑两条状态轨迹 (自动机)：

- 状态 0 $\rightarrow$ r $\rightarrow$ 状态 1 $\rightarrow$ i $\rightarrow$ 状态 2 $\rightarrow$ b $\rightarrow$ 状态 3 (rib)
- 状态 0 $\rightarrow$ r $\rightarrow$ 状态 4 $\rightarrow$ o $\rightarrow$ 状态 5 $\rightarrow$ b $\rightarrow$ 状态 6 (rob)

假设训练语料 95% 是 rib, 5% 是 rob, 但输入观测是 **rob**。理想解码应该给 rob 更高分。但 MEMM 会怎样？

在 MEMM 下，每步是局部 softmax：进入”状态 0”后看到 r，必须做一次 $P(\text{next} | 0, r)$ 归一化。由于训练中 r 后绝大部分进入”状态 1”分支，所以 $P(1 | 0, r) \approx 0.95$, $P(4 | 0, r) \approx 0.05$ 。进入分支 1 之后，无论你看 i 还是 o，下一个状态都几乎只能是 2 (因为状态 1 只有一条出边!) ——softmax 在只有一条出路的节点上”白白消耗 1 的归一化质量”。

最终：解码会偏向”出度低的分支”，因为出度低 $\rightarrow$ 局部 softmax 把 1 全部分给那一条边 $\rightarrow$ ”局部高分”骗过了全局。观测在每一步的影响被局部归一化”稀释”了。

[WARN] 本质

MEMM 在每个时刻 t 强制 $\sum_{y_t} P(y_t | y_{t-1}, x) = 1$ ，所以”路径过去走得有多 confident”无法跨步影响”未来”。低熵的状态 (出度小) 会被偏爱。

7.2.2 解决方案：全局归一化

把”per-step softmax”换成”per-sequence softmax”：所有标签序列一起归一化，只算一个 partition function $Z(x)$ ——这就是 CRF。

7.3 线性链 CRF (Linear-Chain Conditional Random Field)

7.3.1 形式化

定义势函数 (potential) 或得分 (score):

$$\text{score}(x, y) = \sum_{t=1}^n \sum_{k=1}^K \lambda_k f_k(y_{t-1}, y_t, x, t). \quad (7.4)$$

每个 f_k 是一个特征函数 (feature function / template), 仅依赖相邻两个 tag + 整个观测 + 位置。 $\lambda \in \mathbb{R}^K$ 是要学的参数。

条件概率:

$$P(y \mid x; \lambda) = \frac{1}{Z(x)} \exp(\text{score}(x, y)) \quad (7.5)$$

partition function (配分函数 / 归一化常数):

$$Z(x) = \sum_{y' \in \mathcal{Y}^n} \exp(\text{score}(x, y')). \quad (7.6)$$

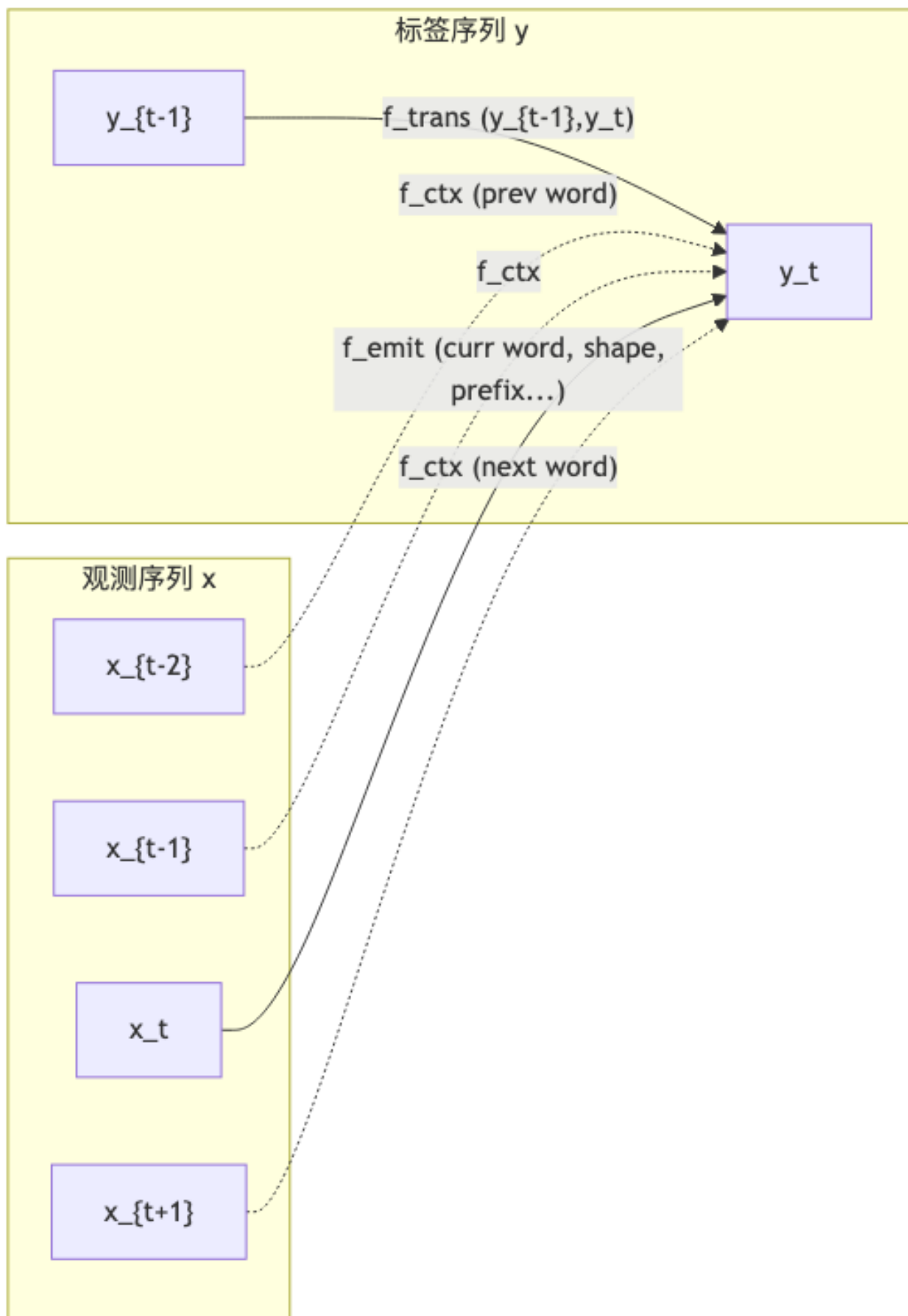
$\mathcal{Y}^n$ 共 $|\mathcal{Y}|^n$ 条序列——直接枚举不可行, 但因 score 是相邻 tag 之和, 可用 **forward 算法** 在 $O(n|\mathcal{Y}|^2)$ 计算。

7.3.2 特征模板的典型例子 (NER)

模板	何时点火	表达力
$f(y_{t-1}, y_t)$	tag 之间的转移	“B-PER 后面接 I-PER 而非 I-LOC”
$f(y_t, x_t)$	当前 token 表面	“当前词 =Obama B-PER”
$f(y_t, x_{t-1})$	上一 token	“上一词 =Mr. B-PER”
$f(y_t, \text{shape}(x_t))$	词形特征	“Xx+ shape 实体”
$f(y_t, \text{prefix3}(x_t))$	前缀	“前缀 Mc- PER”
$f(y_t, x_t \in \text{Gazetteer})$	词典	“在地名表 LOC”

CRF 的关键魅力: 只要能写出 f_k , 就能用, 无需考虑它们的“概率合法性”。

7.3.3 特征模板示意图



注：势函数依赖 (y_{t-1}, y_t) + 整段 x (不仅是 x_t)，这是 CRF 比 HMM 强的核心。

7.4 推断 (Inference) —— Forward & Viterbi over Potentials

7.4.1 partition function 的 Forward 计算

定义 $\alpha_t(j)$ (CRF 版本):

$$\alpha_t(j) = \sum_{y_1, \dots, y_{t-1}} \exp\left(\sum_{s=1}^t \sum_k \lambda_k f_k(y_{s-1}, y_s, x, s)\right) \text{ where } y_t = j. \quad (7.7)$$

记每步的转移-发射联合势:

$$\psi_t(i, j) \triangleq \exp\left(\sum_k \lambda_k f_k(y_{t-1} = i, y_t = j, x, t)\right). \quad (7.8)$$

则递推:

- 初始: $\alpha_1(j) = \psi_1(\text{START}, j)$;
- 递推: $\alpha_t(j) = \sum_i \alpha_{t-1}(i) \psi_t(i, j)$;
- 配分: $Z(x) = \sum_j \alpha_n(j)$ (或 $\sum_j \alpha_n(j) \psi_{n+1}(j, \text{STOP})$).

复杂度 $O(n|Y|^2)$, 结构与 HMM Forward 同构。

[NOTE] 数值稳定性

实现要在 log 空间用 **logsumexp**: $\log \alpha_t(j) = \text{logsumexp}_i(\log \alpha_{t-1}(i) + \log \psi_t(i, j))$ 。

7.4.2 解码: Viterbi over CRF

求 $\arg \max_y P(y | x)$ 等价于 $\arg \max_y \text{score}(x, y)$ (Z 不影响 argmax)。把 Forward 的 $\sum$ 换成 max + backpointer, 即得 Viterbi——与 HMM 完全同构, 区别只是势函数。

$$\delta_t(j) = \max_i [\delta_{t-1}(i) + \log \psi_t(i, j)]. \quad (7.9)$$

7.5 训练 (Learning) —— Log-Likelihood 梯度

7.5.1 训练目标

对训练集 $\{(x^{(d)}, y^{(d)})\}_{d=1}^D$, 最大化条件对数似然 (加 ℓ_2 正则):

$$\mathcal{L}(\lambda) = \sum_{d=1}^D \log P(y^{(d)} | x^{(d)}; \lambda) - \frac{1}{2\sigma^2} \|\lambda\|^2. \quad (7.10)$$

展开第一项：

$$\log P(y | x) = \sum_t \sum_k \lambda_k f_k(y_{t-1}, y_t, x, t) - \log Z(x). \quad (7.11)$$

7.5.2 梯度推导

对 λ_k 求偏导：

$$\frac{\partial}{\partial \lambda_k} \log P(y | x) = \underbrace{\sum_t f_k(y_{t-1}, y_t, x, t)}_{\text{经验特征计数 (empirical)}} - \underbrace{\sum_t \sum_{y', y''} P(y_{t-1} = y', y_t = y'' | x) f_k(y', y'', x, t)}_{\text{模型期望特征计数 (expected under } P_\lambda)}. \quad (7.12)$$

整理为对训练集求和（含正则）：

$$\nabla_{\lambda_k} \mathcal{L} = \mathbb{E}_{\text{empirical}}[f_k] - \mathbb{E}_{\text{model}}[f_k] - \frac{\lambda_k}{\sigma^2} \quad (7.13)$$

[NOTE] 这是 log-linear 模型梯度的通用结构

“empirical – expected”形式在 logistic regression、maxent classifier、CRF、（甚至 RBM）都出现。直觉：让模型生成的特征频率匹配训练数据。

7.5.3 期望特征的计算——Forward-Backward 再次登场

要算 $P(y_{t-1} = y', y_t = y'' | x)$ ，需要 forward α + backward β ：

$$P(y_{t-1} = i, y_t = j | x) = \frac{\alpha_{t-1}(i) \psi_t(i, j) \beta_t(j)}{Z(x)}. \quad (7.14)$$

其中 β 与 HMM backward 同构：

$$\beta_t(i) = \sum_j \psi_{t+1}(i, j) \beta_{t+1}(j), \quad \beta_n(i) = 1. \quad (7.15)$$

训练流程（每个 epoch / mini-batch）：

1. 对每句 x 跑 forward $\rightarrow$ 得 α, Z ；
2. 跑 backward $\rightarrow$ 得 β ；
3. 用 (7.14) 算期望特征计数；
4. 由 (7.13) 累加梯度；
5. 用 L-BFGS / SGD / Adam 更新 λ 。

复杂度：每句 $O(n|y|^2)$ （含 forward + backward）。

7.6 HMM vs MEMM vs CRF 总对照

维度	HMM	MEMM	Linear-Chain CRF
模型类型	生成 (joint $P(x, y)$)	判别 (per-step $P(y_t y_{t-1}, x)$)	判别 (sequence $P(y x)$)
归一化	局部 ($\sum_x b_j(x) = 1$)	局部 ($\sum_{y_t} = 1$)	全局 ($Z(x)$)
特征	x_t 自身	任意 $\phi(y_{t-1}, y_t, x_{1:n}, t)$	同左
Label bias 训练	无 (生成式) 计数 / Baum-Welch	有 per-step max-ent	无 全局 NLL + grad, 需 forward-backward
解码 训练成本	Viterbi 极低	Viterbi (over local probs) 中	Viterbi (over potentials) 高 (每句一次 forward-backward)
适用场景	低资源、嵌入式	历史/教学	经典 SOTA (neural 前)

[WARN] 一个常考点

为什么 CRF 没有 label bias? 因为 $Z(x)$ 是对整条序列归一化, “在某一步把分数全给一条边”不会带来”虚高的局部概率”, 因为这个概率必须与”所有其他完整序列”一起被 Z 拉平。

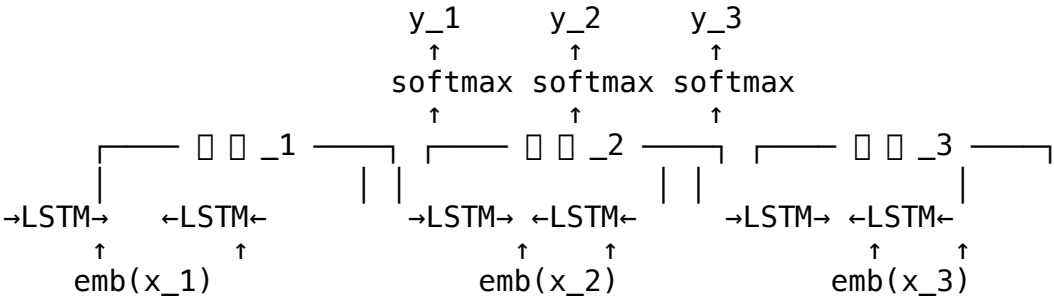
7.7 Neural Sequence Tagging

7.7.1 神经时代的范式转移

维度	统计时代 (CRF)	神经时代
特征	人工模板 ϕ	embedding + RNN/CNN/Transformer 学出来的表征
OOV 上下文 输出层 解码	平滑 + gazetteer 受限于模板窗口 per-step 或全局 CRF Viterbi	character/subword embedding BiLSTM / self-attention “看全句” softmax 或 + CRF 头 Viterbi (若有 CRF 头) / greedy

7.7.2 BiLSTM Tagger

最朴素版本: BiLSTM + per-token softmax。



训练目标: $-\sum_t \log p(y_t | x)$ (per-step CE)。

7.7.3 BiLSTM-CRF (Huang et al. 2015; Lample et al. 2016)

发现：per-step softmax 忽略相邻 tag 的依赖（比如 I-PER 不能跟在 B-LOC 之后）。解决：把 BiLSTM 当成“emission scorer”，再叠一层学习到的转移矩阵 T ，最后用全局 CRF 训练 + Viterbi 解码。

score:

$$\text{score}(x, y) = \sum_{t=1}^n \underbrace{h_t^\top W}_{\text{BiLSTM emission}} y_t + \sum_{t=1}^n \underbrace{T_{y_{t-1}, y_t}}_{\text{learned transition}}. \quad (7.16)$$

训练：依然最大化 $\log P(y | x) = \text{score} - \log Z(x)$ ； Z 用 forward 算；解码用 Viterbi。这是 2015–2018 年 NER 的事实标准。

进一步加 char-CNN/char-LSTM 编码字符级 (Chiu & Nichols 2016; Lample et al. 2016) 来处理 OOV / 形态学。

7.7.4 后来：上下文嵌入 + Transformer

- Flair (Akbik et al. 2018): character LM 提供 contextual string embedding，再喂 BiLSTM-CRF；
- LUKE (Yamada et al. 2020): BERT + entity-aware self-attention + masked entity prediction；
- FLERT (Schweter & Akbik 2020): 把整段 document 作为上下文喂给 transformer NER；
- ACE / Automated Concatenation of Embeddings: NAS 自动找最佳 embedding 组合。

[NOTE] 趋势

CRF 头在 BERT 时代仍然有用——尤其当 tag set 有强结构约束 (BIO 一致性) 时，BERT + CRF 通常比 BERT + softmax 略好；但差距远小于 BiLSTM + CRF vs BiLSTM + softmax，因为 BERT 已经具有更强的相邻一致性归纳偏置。

7.8 Worked Example: CRF 训练梯度直觉

设 2-tag 集合 $\{B, I\}$ ，句子 $x = (x_1, x_2)$ ，特征只有一个：“上一 tag = B 且当前 tag = I” $\rightarrow f_1 = 1[y_{t-1} = B, y_t = I]$ 。

训练样本： $y^{(1)} = (B, I) \rightarrow$ 经验计数 $f_1 = 1$ 。

模型期望计数：

$$\mathbb{E}_{P_\lambda}[f_1] = P(y_1 = B, y_2 = I | x).$$

初始 $\lambda_1 = 0$ ，所有 4 个 tag 序列等概率 = $1/4$ ，所以 $\mathbb{E}[f_1] = 1/4$ 。梯度 $\nabla_{\lambda_1} \mathcal{L} = 1 - 1/4 = 0.75 > 0$ ，于是 λ_1 增大 $\rightarrow P(B, I)$ 上升 $\rightarrow \mathbb{E}[f_1]$ 上升 $\rightarrow$ 梯度变小，直到经验 = 期望。这正是 (7.13) 的几何含义：把模型的特征分布“拉向”数据的特征分布。

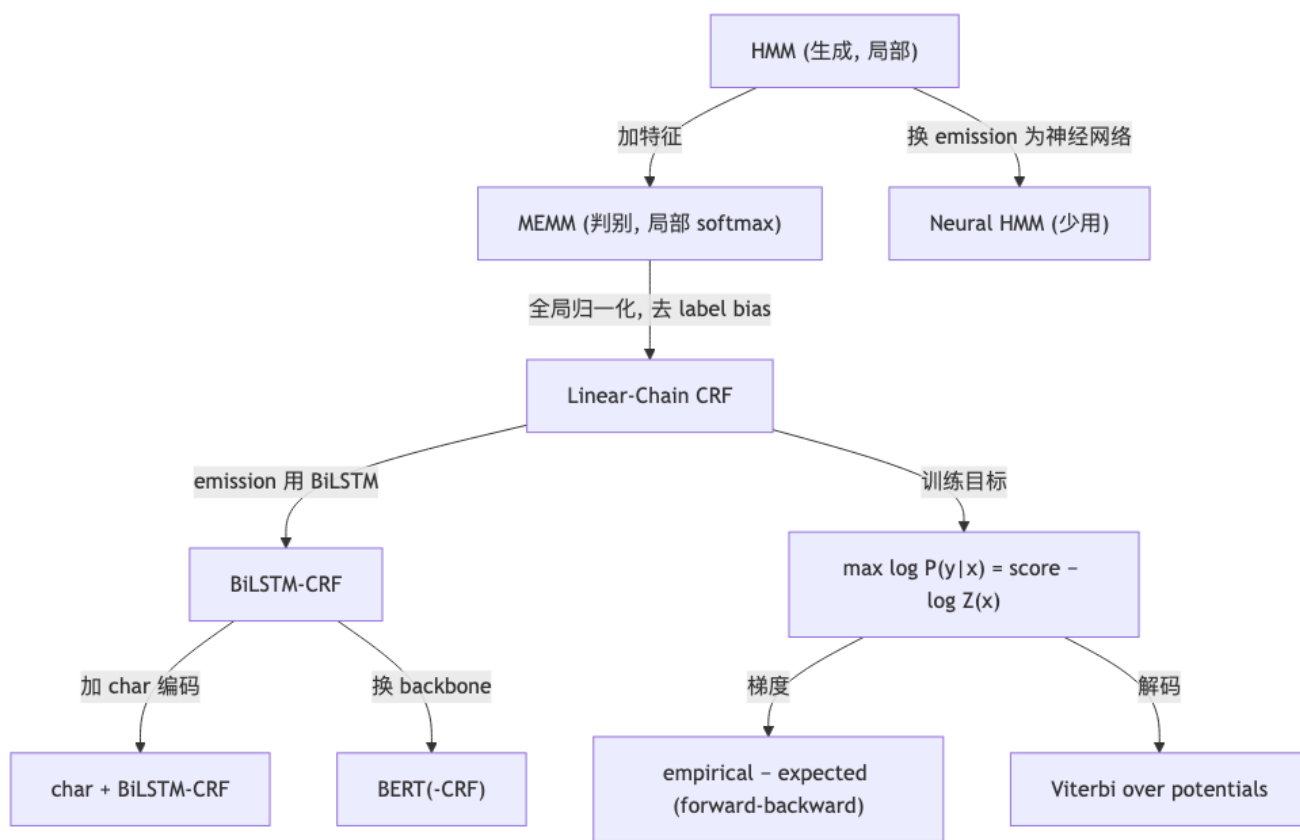
7.9 Seq2Seq 视角下的序列标注（一句话总结）

讲义后半还简介了 encoder-decoder + attention (Sutskever et al. 2014, Bahdanau et al. 2015, Vaswani et al. 2017) 作为更一般的“输入序列 $\rightarrow$ 输出序列”框架。把序列标注当成 $|x| = |y|$ 的特例可以，但工业实践中：

- 长度对齐的任务 (POS / NER / 分词): encoder (BiLSTM/Transformer) + token-level head (softmax / CRF) 更高效;
- 长度不对齐的任务 (MT、摘要、QA 生成): 才需要完整 seq2seq + attention。

Seq2seq / Transformer / Attention 的细节会在 lec08 / lec09 的笔记中展开, 这里只是逻辑链上的承接。

7.10 概念关系图



7.11 易错点 (Pitfalls)

1. MEMM 与 CRF 的归一化范围: MEMM **per step** 归一化 (每步 softmax); CRF **per sequence** 归一化 (一个 $Z(x)$)。这是 label bias 的根因。
2. CRF 梯度的方向: $\nabla = \text{empirical} - \text{expected}$, 不是反过来。记反了就在朝错误方向更新。
3. $Z(x)$ 必须用 **forward** 算, 不能枚举所有序列; 同理 β 用 backward。复杂度 $O(n|y|^2)$ 。
4. $\log Z$ 不参与 **argmax**, 所以解码不需要 Z , 只需要 score (与 HMM Viterbi 同构)。
5. 数值实现一律 **log 空间**: 用 `logsumexp` 替乘加, 避免下溢。
6. 特征模板的结构限制: linear-chain CRF 的 f_k 只能跨相邻两个 tag (y_{t-1}, y_t); 若要跨更远 (如 trigram tag), 需要 higher-order CRF, 复杂度升到 $O(n|y|^3)$ 。
7. CRF 不是“加 CRF 层就一定涨点”: tag 集本身约束弱时 (如普通 POS), BiLSTM-softmax 已够好; 当 tag 集有强结构 (BIO/BIOES), CRF 头收益显著。
8. HMM 是 CRF 的特例: 固定 $f_k = \log P(\cdot|\cdot)$ 时 CRF 退化为 HMM。一句话: “CRF = HMM + 任意特征 + 全局归一化”。

9. 判别 vs 生成的取舍：HMM 可以“生成”假数据做半监督；CRF 不能。判别模型只 score，不 sample x 。
10. 训练慢：每个梯度步要做一次 forward-backward；大规模 corpus + 大 tag 集要用 SGD / L-BFGS + 分布式实现，否则训练日均小时级。

7.12 中英术语对照

中文	英文	缩写 / 备注
最大熵马尔可夫模型	Maximum Entropy Markov Model	MEMM
条件随机场	Conditional Random Field	CRF
线性链 CRF	Linear-Chain CRF	—
标签偏置	Label Bias	Lafferty 2001
势函数	potential function	$\psi_t(i, j)$
配分函数 / 归一化常数	partition function	$Z(x)$
特征函数 / 特征模板	feature function / template	f_k
经验特征计数	empirical feature count	$\mathbb{E}_{\text{data}}[f_k]$
期望特征计数	expected feature count	$\mathbb{E}_{\text{model}}[f_k]$
前向-后向算法	Forward-Backward algorithm	for CRF inference
全局归一化	global normalization	与 per-step 对比
双向长短期记忆网络	Bidirectional LSTM	BiLSTM
神经条件随机场	BiLSTM-CRF	Huang 2015, Lample 2016
字符嵌入	character embedding	char-CNN / char-LSTM
语境化嵌入	contextual embedding	ELMo / Flair / BERT
序列到序列	Sequence to Sequence	seq2seq
注意力机制	attention mechanism	Bahdanau / Vaswani

[TIP] 期中复习要点

1. 公式默写：CRF 条件概率 (7.5)、partition (7.6)、potential (7.8)、梯度 (7.13)、双 tag 后验 (7.14)。2. 算法默写：CRF Forward (7.7)、Backward (7.15)、Viterbi over potentials (7.9)。3. 三大模型对比表 (HMM / MEMM / CRF)：能用一张表完整写出 5 个维度 (模型类型、归一化、特征、label bias、训练/解码方法)。4. Label bias：能给出“局部 softmax 偏爱低出度状态”的直观例子，并说明全局归一化为何能解决。5. 梯度直觉：“empirical - expected”——是 log-linear 模型的通用公式，CRF 是其结构化版本。6. forward-backward 算什么：训练时算 ξ (期望特征计数)，推断时算 Z (partition)。7. BiLSTM-CRF：能画出“BiLSTM emission + learned transition + CRF 头”的架构；说明为何比 BiLSTM-softmax 强。8. 复杂度：linear-chain CRF 训练 / 推断 / 解码均 $O(n|y|^2)$ ；higher-order 升一维。9. HMM 是 CRF 的特例：能解释参数对应关系。10. 何时用 CRF 头：tag 集有强结构约束 (BIO 一致性、依存约束) 时收益最大；纯 POS 时收益小。

Lecture 6b: 序列到序列模型与注意力机制 / Seq2Seq & Attention

课程: 自然语言处理基础 (FNLP, PKU) —Yansong Feng 参考资料: 李宏毅 (Hung-yi Lee) “Transformer / Seq2seq v9”讲义 对应原文件: 李宏毅--seq2seq_v9.pptx.txt

[TIP] 学习指南

本节是从 RNN 语言模型走向 Transformer 的关键过渡。重点掌握: 1. **Encoder-Decoder 范式** 为什么能做“输入输出长度不一致”的任务 (翻译、ASR、TTS、对话、QA、句法分析); 2. **Attention 的三种打分** (dot / general / additive Bahdanau) 与权重的 softmax 归一化; 3. **训练**: teacher forcing 与 exposure bias, scheduled sampling 补救; 4. **推理**: greedy / beam search / sampling 的差异, beam search 的搜索树与长度归一化; 5. **进阶技巧**: copy mechanism、guided attention、AT vs NAT、强化学习直接优化 BLEU。这一讲是**期中必考章节**: seq2seq 是后续 Transformer / BERT / GPT 的母题。

6b.1 Motivation —为什么需要 Seq2Seq

普通分类器只能输出固定长度 / 固定类别; 但很多 NLP 任务的输出**长度由模型决定**:

任务	输入	输出	长度关系
机器翻译 (MT)	源语句 (N 词)	译文 (N' 词)	$N' \neq N$
语音识别 (ASR)	语音帧 (T)	文字 (N)	$N < T$
文语合成 (TTS)	文字 (N)	语音帧 (T)	$T > N$
对话 (Chatbot)	用户句	回复	不定
句法分析	句子	括号化树	$\neq$
问答 (QA)	(q, context)	答案	不定
摘要	长文	摘要	远小于
多标签分类	实例	标签序列	不定
目标检测	图像	bbox 序列	不定

[NOTE] 一句话定义

Seq2Seq (Sequence-to-Sequence): 输入一个序列 $x = (x_1, \dots, x_{T_x})$, 输出一个序列 $y = (y_1, \dots, y_{T_y})$, 其中输出长度 T_y 由模型自身决定 (通常通过预测特殊 EOS 标记停止)。

[WARN] 范式之广

李宏毅讲义反复强调: “Most NLP applications can be done by seq2seq”(arXiv:1806.08730)——把任务全部写成 (input text, output text) 配对是后来 T5 / GPT 的核心思想。

6b.2 形式化定义 (Formalization)

6b.2.1 整体目标

给定输入序列 $x = (x_1, \dots, x_{T_x})$, 模型输出概率:

$$P(y | x) = \prod_{t=1}^{T_y} P(y_t | y_{<t}, x) \quad (\text{seq.1})$$

其中 $y_{<t} = (y_1, \dots, y_{t-1})$ 是已生成前缀。式 (seq.1) 是 **自回归 (Autoregressive, AT)** 分解。

6b.2.2 Encoder

读入 x 并把它编码为一组上下文向量 $(h_1, \dots, h_{T_x})$ 。可用 RNN / CNN / Transformer 实现:

$$h_t = f_{\text{enc}}(h_{t-1}, x_t) \quad (\text{seq.2})$$

[NOTE] Encoder 选型

– **RNN / LSTM / GRU**: 早期 seq2seq (Sutskever 2014, Cho 2014) 的标配; 逐步累积上下文, 最后用 h_{T_x} 作为“句子向量”。– **CNN**: ByteNet / ConvS2S, 可并行但感受野受限。– **Transformer Encoder**: 自注意力 + 残差 + LayerNorm + 前馈 (FFN), 2026 年的事实标准。

6b.2.3 Decoder

在每个时刻 t 根据已生成前缀 $y_{<t}$ 与上下文 c_t 输出下一 token 分布:

$$s_t = g_{\text{dec}}(s_{t-1}, y_{t-1}, c_t) \quad (\text{seq.3})$$

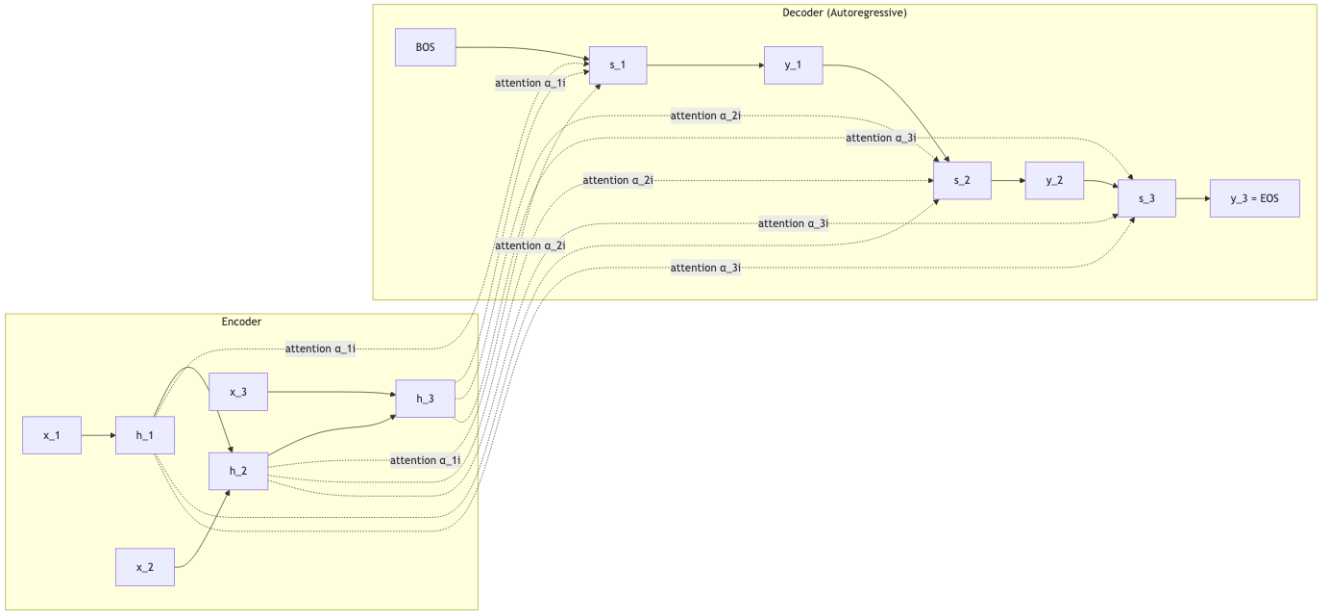
$$P(y_t | y_{<t}, x) = \text{softmax}(W_o [s_t; c_t] + b_o) \quad (\text{seq.4})$$

- s_t : decoder 时刻 t 的隐状态;
- c_t : 从 encoder 端“拿过来”的上下文向量;
- 输出维度 = 词表大小 $|V|$, 用 softmax 归一化为概率。

[WARN] 早期局限

Sutskever 2014 原版只用 $c = h_{T_x}$ ——所有源信息被压缩成一个固定向量, 长句子翻译退化严重。Bahdanau 2015 用注意力把 c 改成与 t 相关的 c_t , 让 decoder 每步“看回去”。这就是 attention 的起源。

6b.2.4 时序图



6b.3 Attention —注意力机制

6b.3.1 直觉

人在翻译时不会一次性记住整个源句，而是”翻一个词就回头瞄一眼对应位置”。Attention 让 decoder 在每个时刻软地选择 encoder 端最相关的位置加权汇总。

6b.3.2 公式

记 decoder 时刻 t 的”查询”为 s_{t-1} (或 s_t , 因实现而异), encoder 端的”键 / 值”为 $\{h_i\}_{i=1}^{T_x}$ 。

1. 打分函数 (alignment score) $e_{ti} = \text{score}(s_{t-1}, h_i)$, 常见三种:

名称	公式	出处
点积 (dot)	$e_{ti} = s_{t-1}^\top h_i$	Luong 2015
缩放点积 (scaled dot)	$e_{ti} = \frac{s_{t-1}^\top h_i}{\sqrt{d}}$	Transformer (Vaswani 2017)
通用 (general)	$e_{ti} = s_{t-1}^\top W_a h_i$	Luong 2015
加性 (additive / Bahdanau)	$e_{ti} = v_a^\top \tanh(W_a [s_{t-1}; h_i])$	Bahdanau 2015

2. softmax 归一化为权重:

$$\alpha_{ti} = \frac{\exp(e_{ti})}{\sum_{j=1}^{T_x} \exp(e_{tj})} \tag{seq.5}$$

显然 $\sum_i \alpha_{ti} = 1$ 。

3. 上下文向量 (context):

$$c_t = \sum_{i=1}^{T_x} \alpha_{ti} h_i \quad (\text{seq.6})$$

4. 与 decoder 状态拼接 → 出 token 分布:

$$\tilde{s}_t = \tanh(W_c[s_t; c_t]), \quad P(y_t | \cdot) = \text{softmax}(W_o \tilde{s}_t) \quad (\text{seq.7})$$

6b.3.3 Cross-Attention (Transformer)

Transformer Decoder 的 cross-attention 把上面打分写成 query / key / value 矩阵乘法:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (\text{seq.8})$$

其中 Q 来自 decoder, K, V 来自 encoder。

[NOTE] 三类 attention 区分

1. **Self-attention**: query / key / value 都来自同一序列 → encoder 内部、decoder 内部 (带 mask)。
2. **Cross-attention**: query 来自 decoder, key/value 来自 encoder → 这是 seq2seq 的“读源端”操作。
3. **Masked self-attention**: decoder 端的自注意力, 时刻 t 只能看到 $\leq t$ 的位置, 防止“偷看未来”。

6b.4 Decoder Variants —AT / NAT

6b.4.1 Autoregressive (AT)

如 §6b.2 所述, 一步一步生成, 第 t 步输出的 token 喂回作为第 $t + 1$ 步输入, 直到产生 EOS。

[START] → 机 → 器 → 学 → 习 → [END]

优点: 质量高、训练稳定。缺点: 必须串行, T_y 步不能并行。

6b.4.2 Non-Autoregressive (NAT)

一次性输出所有位置:

[START] [START] [START] [START] → 机 器 学 习

需要先决定输出长度——两种方案:

1. 额外训练一个长度预测器 $\hat{T}_y$;
2. 输出一个非常长的序列, 遇到 EOS 截断。

优点: 完全并行, 推理快 (TTS 必备)。缺点: 质量通常差于 AT (“multi-modality 问题”——可能学到多个译文的平均, 结果不自然)。

维度	AT	NAT
并行	否	是
质量	高	较低 (因 multi-modality)
长度控制	EOS 自然停止	需显式预测长度
典型应用	MT, Chatbot	TTS, 实时语音

6b.5 训练 (Training)

6b.5.1 Teacher Forcing

训练时把 **ground-truth** 而非模型自己的预测作为下一时刻 decoder 输入。损失为逐时刻交叉熵之和：

$$\mathcal{L} = - \sum_{t=1}^{T_y} \log P(y_t^* | y_{<t}^*, x) \quad (\text{seq.9})$$

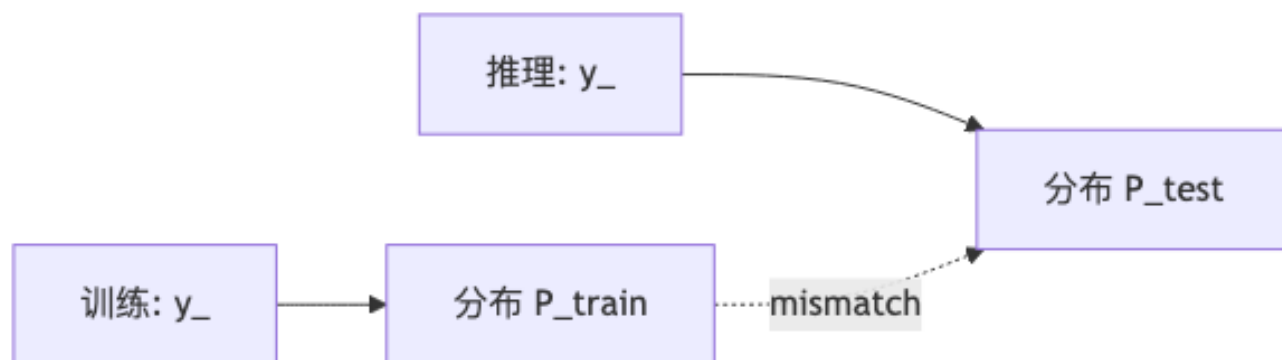
其中 y_t^* 是真实标签。

[NOTE] 为什么用 Teacher Forcing

训练初期模型预测很糟，若用自身预测作为下一步输入，错误会一路放大，梯度极不稳定；用 ground-truth 强制对齐，能让 loss 快速收敛。

6b.5.2 Exposure Bias — 训练-推理失配

问题：训练时每步喂的都是真实前缀；推理时模型只能喂自己的预测——一旦某步出错，后续条件分布 $P(y_t | y_{<t})$ 落到“训练从未见过的前缀”分布外。



6b.5.3 Scheduled Sampling

Bengio et al. 2015: 训练时以概率 ϵ_i 把 ground-truth 替换成模型自己的预测（采样 / argmax）。 ϵ_i 从 0 逐步增大到接近 1，让模型逐渐适应“自己的输出”。

变体：

- Original Scheduled Sampling (RNN, [arXiv:1506.03099](#))
- Scheduled Sampling for Transformer ([arXiv:1906.07651](#))
- Parallel Scheduled Sampling ([arXiv:1906.04331](#))

[WARN] Scheduled Sampling 的副作用

早期被发现会破坏极大似然估计的一致性 (Huszár 2015)；实践中 Transformer 时代用得少，更多靠 MIXER / 强化学习 或 最小风险训练 (MRT) 直接优化 BLEU。

6b.5.4 直接优化评测指标

BLEU / ROUGE 不可微 → 用强化学习 (REINFORCE) 把 BLEU 当作 reward:

$$\nabla_{\theta} \mathcal{J} = \mathbb{E}_{y \sim P_{\theta}} [(R(y) - b) \nabla_{\theta} \log P_{\theta}(y|x)] \quad (\text{seq.10})$$

其中 R 是 BLEU, b 是 baseline 减方差 ([arXiv:1511.06732](https://arxiv.org/abs/1511.06732))。

6b.6 推理 (Inference / Decoding)

6b.6.1 Greedy Decoding

每步选概率最大的 token:

$$y_t = \arg \max_{w \in V} P(w | y_{<t}, x) \quad (\text{seq.11})$$

优点: $O(T_y|V|)$, 最快。缺点: 局部最优 $\neq$ 全局最优——一步错满盘错。

6b.6.2 Beam Search

维护 Top-B 部分序列 (beam size B), 每步扩展所有可能 token、保留累计对数概率最高的 B 个。

累计得分:

$$\text{score}(y_{1:t}) = \sum_{\tau=1}^t \log P(y_{\tau} | y_{<\tau}, x) \quad (\text{seq.12})$$

长度归一化 (length normalization, 避免短序列因负 log-prob 累加更少而占优):

$$\text{score}_{\text{norm}}(y_{1:t}) = \frac{1}{t^{\alpha}} \sum_{\tau=1}^t \log P(y_{\tau} | \cdot), \quad \alpha \in [0.6, 1.0] \quad (\text{seq.13})$$

伪代码:

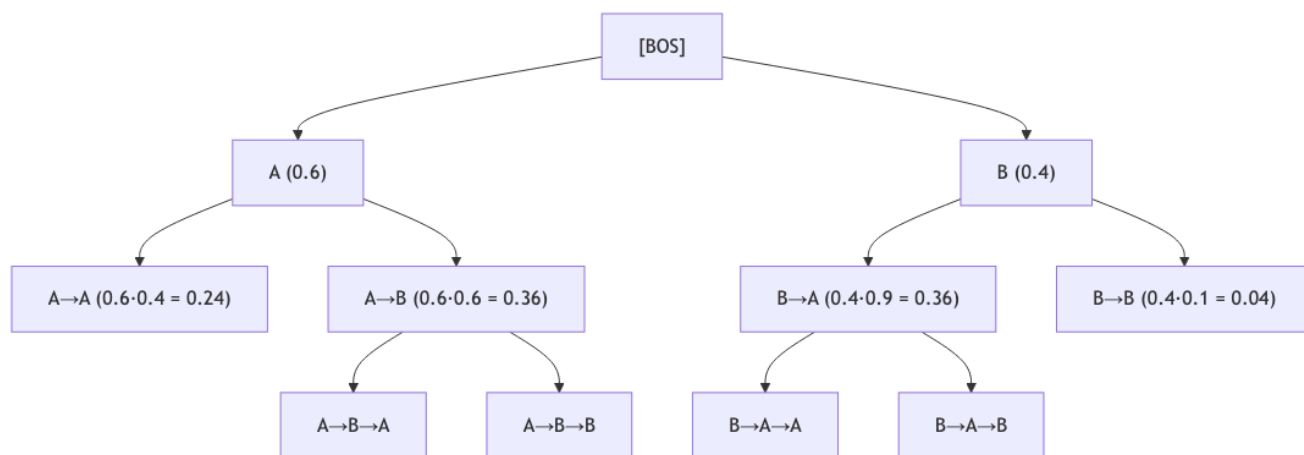
```
def beam_search(x, B, max_len, eos):
    beams = [( [BOS], 0.0 )] # (前缀, 累计 log-prob)
    completed = []
    for step in range(max_len):
        cand = []
        for prefix, score in beams:
            if prefix[-1] == eos:
                completed.append((prefix, score))
                continue
            for w in V:
                cand.append((prefix + [w], score + log P(w | prefix)))
        beams = sorted(cand, key=lambda x: x[1], reverse=True)[:B]
```

```

logp = model(prefix, x) # |V| 维
topB = topk(logp, B)
for token, lp in topB:
    cand.append((prefix + [token], score + lp))
beams = topB_by_score(cand, B) # 仅保留 Top-B
if all(p[-1] == eos for p, _ in beams):
    completed.extend(beams)
    break
return argmax(completed, key=length_norm)

```

Beam Search 搜索树 ($B=2$, $V=\{A, B\}$)



第 1 步：保留 $\{A, B\}$ ($B=2$)。第 2 步：从 4 个候选 $\{AA, AB, BA, BB\}$ 中按 score 取前 2: $\{AB(0.36), BA(0.36)\}$ 。第 3 步：再扩展、再剪枝。最终在所有完成 (含 EOS) 的序列上按归一化得分挑最优。

[WARN] Beam 不是越大越好

大 B 会引入“喜欢生成短而稳的句子”问题 (Murray & Chiang 2018)，需要配合长度归一化、覆盖惩罚 (coverage penalty)、length reward 等技巧；MT 常用 $B = 4 \sim 10$ ，对话 / 摘要常用 $B = 5$ 。

6b.6.3 Sampling

某些任务 (开放对话、创作) 需要随机性。常见策略：

策略	描述
Pure sampling	按 $P(y_t \cdot)$ 直接采样
Temperature τ	$P_\tau(w) \propto P(w)^{1/\tau}$, $\tau \rightarrow 0$ 退化为 greedy
Top- k	仅在概率最高的 k 个 token 中重新归一化采样
Top- p (nucleus)	仅在累计概率达 p 的最小集合中采样 (Holtzman 2019)

[NOTE] Beam vs Sampling 选择

– 机器翻译 / 摘要 / ASR: beam search (追求最可能输出) – 对话 / 故事生成: top- p / temperature (避免重复、提升多样性)

6b.7 重要技巧 (Tips)

6b.7.1 Copy Mechanism

某些 token 需要从源端”复制”而非”生成”——人名、术语、罕见词。CopyNet / Pointer Network 把输出分布写成：

$$P(y_t) = p_{\text{gen}} P_{\text{vocab}}(y_t) + (1 - p_{\text{gen}}) \sum_{i: x_i = y_t} \alpha_{ti} \quad (\text{seq.14})$$

其中 $p_{\text{gen}} \in [0, 1]$ 通过 sigmoid 学得，第二项把”复制概率”按 attention 权重分配。

应用：摘要 (See 2017)、对话（用户名 / 实体复制）、机器翻译稀有词。

6b.7.2 Guided / Monotonic Attention

ASR / TTS 中输入输出单调对齐——但模型有时跳跃、漏字（讲义”發財發財發財”例子）。解决：

- **Monotonic Attention**：约束 attention 仅向右移动；
- **Location-aware Attention**：把上一时刻的 attention 也作为输入 (Chorowski 2015)；
- **Guided Attention Loss**：训练时加正则强制对角对齐。

6b.7.3 Masked Self-Attention (Decoder 内)

位置 t 只能 attend 到 $\leq t$ 的位置

mask =

```
[ [0, -∞, -∞, -∞],  
  [0, 0, -∞, -∞],  
  [0, 0, 0, -∞],  
  [0, 0, 0, 0]]
```

加到 attention score 上再做 softmax，把”未来位置”的权重压成 0。

6b.8 Worked Example —Beam Search, B=2, 3 步

设置：词表 $V = \{A, B, \text{EOS}\}$ ，beam size $B = 2$ ，最大长度 3。模型在每一步给出的条件概率：

步骤	前缀	$P(A)$	$P(B)$	$P(\text{EOS})$
1	[BOS]	0.6	0.3	0.1
2	A	0.4	0.5	0.1
2	B	0.2	0.7	0.1
3	AA	0.3	0.3	0.4
3	AB	0.2	0.2	0.6
3	BA	0.4	0.4	0.2
3	BB	0.1	0.1	0.8

Step 1: 扩展 [BOS] → 候选 $\{A : 0.6, B : 0.3, \text{EOS} : 0.1\}$ 。取 top-2 → beams = $\{A : 0.6, B : 0.3\}$
(按 log-prob: $\{A : -0.51, B : -1.20\}$)。

Step 2: 扩展两条 beam，共 6 个候选：

候选	累计概率	累计 log-prob
AA	$0.6 \cdot 0.4 = 0.24$	-1.43
AB	$0.6 \cdot 0.5 = 0.30$	-1.20
A,EOS	$0.6 \cdot 0.1 = 0.06$	-2.81
BA	$0.3 \cdot 0.2 = 0.06$	-2.81
BB	$0.3 \cdot 0.7 = 0.21$	-1.56
B,EOS	$0.3 \cdot 0.1 = 0.03$	-3.51

取 top-2 $\rightarrow$ beams = $\{AB : 0.30, AA : 0.24\}$ 。注：完成序列 (含 EOS) 移入 completed 池。

Step 3: 扩展 AB 与 AA:

候选	累计概率	累计 log-prob
AB,A	$0.30 \cdot 0.2 = 0.060$	-2.81
AB,B	$0.30 \cdot 0.2 = 0.060$	-2.81
AB,EOS	$0.30 \cdot 0.6 = 0.180$	-1.71
AA,A	$0.24 \cdot 0.3 = 0.072$	-2.63
AA,B	$0.24 \cdot 0.3 = 0.072$	-2.63
AA,EOS	$0.24 \cdot 0.4 = 0.096$	-2.34

完成集合此时含: $\{ABEOS : -1.71, AA EOS : -2.34\}$ (以及 step 2 已结束的 $A EOS : -2.81$ 等)。

长度归一化 $\alpha = 0.7$, score/t^α :

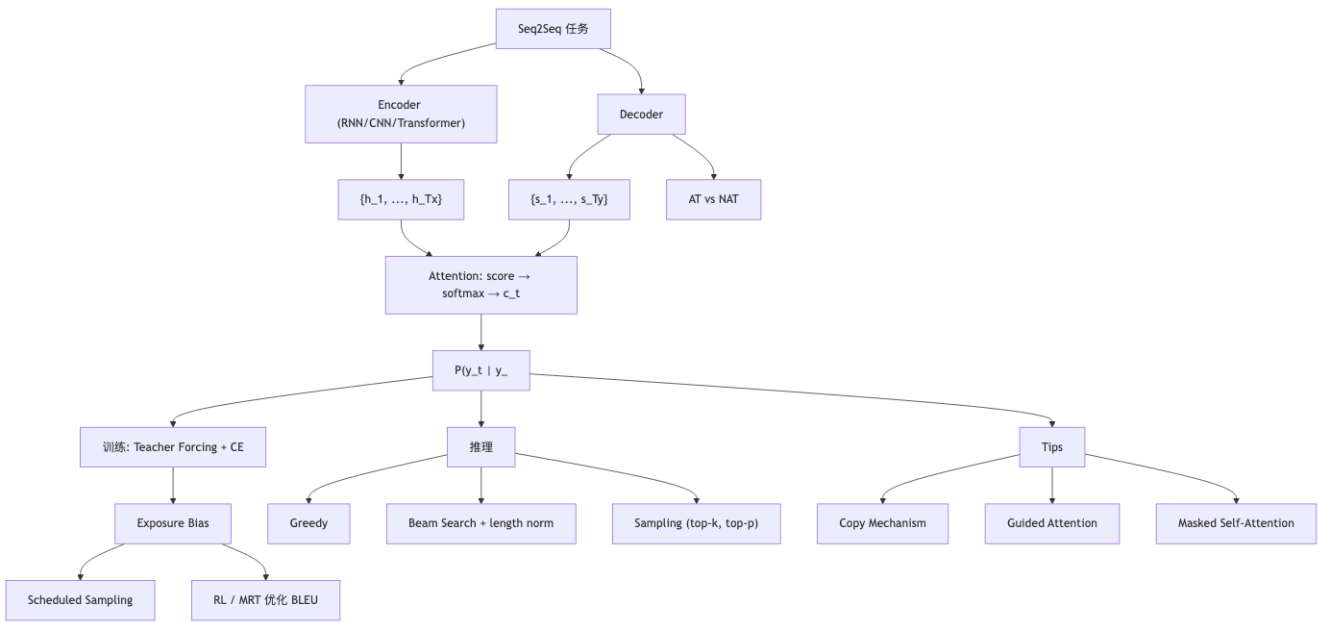
- $ABEOS$ (长度 3): $-1.71/3^{0.7} = -1.71/2.158 \approx -0.793$
- $AAEOS$ (长度 3): $-2.34/2.158 \approx -1.085$
- $A EOS$ (长度 2): $-2.81/2^{0.7} = -2.81/1.625 \approx -1.730$

最终输出: $ABEOS$ —— 序列 A, B 。

[NOTE] 与 Greedy 对比

同样设置下 greedy 会选: step1 $A \rightarrow$ step2 $B \rightarrow$ step3 $EOS \rightarrow$ 也是 $ABEOS$ —— 这次结果相同。但若把 step2 $P(A|B)$ 改成 0.95, greedy 第 1 步选 A 后被锁死在 AB 路径, 可能错过更好的 BA 路径——这正是 beam search 的价值所在。

6b.9 概念关系图 (Mermaid)



6b.10 易错点 (Common Pitfalls)

[WARN] 高频踩坑

1. **Attention 权重的归一化轴**: α_{ti} 是对 encoder 端 i 做 softmax (每个 decoder 时刻一组), 不是对 decoder 时刻做。2. **Bahdanau 与 Luong 的差异**: Bahdanau 用 s_{t-1} 算 attention (“先看再算 s_t ”), Luong 用 s_t 算 (“先算再回头看”)。考试可能问“哪个是 additive”——Bahdanau。3. **Beam 与 Greedy 是 inference 的事**——训练始终是 teacher forcing + CE, 无需 beam。4. **长度归一化指数 α** : 不是简单除以长度; 除以 t^α , $\alpha = 0$ 不归一化, $\alpha = 1$ 完全归一化, $\alpha \approx 0.6 \sim 0.7$ 常用。5. **EOS 是否参与长度计数**: 实现各异, 要按题面规定。6. **Masked self-attention 在 decoder 端永远必要**, 否则训练时会“偷看未来”, 推理时崩盘。7. **AT 不能并行训练?**——能! teacher forcing 下所有时刻的输入都是已知 ground-truth, 可一次性并行计算所有时刻的 loss (这是 Transformer 训练比 RNN 快的关键原因之一)。AT 只在推理时是串行的。8. **Cross-attention 的 query / key / value 来源易记反**: query 来自 decoder (“我现在想知道什么”), key/value 来自 encoder (“源端有什么”)。

6b.11 中英术语对照

中文	英文	简称 / 备注
序列到序列	Sequence-to-Sequence	seq2seq
编码器	Encoder	—
解码器	Decoder	—
自回归	Autoregressive	AT
非自回归	Non-Autoregressive	NAT
注意力机制	Attention Mechanism	—
自注意力	Self-Attention	—
交叉注意力	Cross-Attention	—

中文	英文	简称 / 备注
掩码自注意力	Masked Self-Attention	—
加性注意力	Additive Attention	Bahdanau
点积注意力	Dot-Product Attention	Luong
缩放点积注意力	Scaled Dot-Product Attention	Transformer
上下文向量	Context Vector	c_t
对齐	Alignment	—
教师强制	Teacher Forcing	—
暴露偏差	Exposure Bias	—
计划采样	Scheduled Sampling	—
贪心解码	Greedy Decoding	—
集束搜索	Beam Search	—
集束宽度	Beam Size / Width	B
长度归一化	Length Normalization	—
复制机制	Copy Mechanism	CopyNet / Pointer
引导注意力	Guided Attention	—
单调注意力	Monotonic Attention	—
起始标记	Begin-of-Sentence	BOS / START
结束标记	End-of-Sentence	EOS / END
词表	Vocabulary	V
温度采样	Temperature Sampling	—
顶 k 采样	Top-k Sampling	—
核采样	Nucleus Sampling	Top-p
BLEU 分数	BLEU Score	—
最小风险训练	Minimum Risk Training	MRT

[TIP] 期中复习要点

1. 背公式: seq.5 (attention softmax)、seq.6 (context)、seq.7 (output)、seq.9 (CE loss)、seq.13 (length norm)。2. 画时序图: 能画出 encoder→attention→decoder 的展开图, 标注 h_i 、 s_t 、 α_{ti} 、 c_t 的流向。3. 算 attention: 给 s_{t-1} 与 3 个 h_i , 手算 dot / additive attention 的 α 与 c_t 。4. 走一遍 beam search: 给定模型逐步概率与 B , 能在草稿纸上画出搜索树并算出长度归一化后的最优序列。5. 比较 AT vs NAT: 并行性、质量、长度控制、典型任务。6. 解释 exposure bias 与 scheduled sampling: 训练-推理分布失配为何发生, scheduled sampling 怎样以渐进概率把 ground-truth 换成预测来缓解。7. 三种 attention 评分函数: dot / general / additive 的公式、可学参数与适用场合。8. Copy / Guided / Masked: 知道每种技巧“治什么病”。9. 与 Transformer 的接续: 理解 attention 是 Transformer 的“原子”, self-attention + cross-attention + masked self-attention 共三种用法。

Lecture 7: 短语结构、上下文无关文法与 CKY 句法分析 (Phrase Structure, CFG/PCFG, and CKY Parsing)

[TIP] 学习指南

本讲核心：把“线性序列”升级为“层级树结构”，建立计算语言学最经典的两个工具：1. CFG (Context-Free Grammar) 描述“什么样的句子是合法的” (Search space)；2. PCFG (Probabilistic CFG) 给每条 derivation 打分，回答“哪个树最可能” (Measurement)；3. CKY (Cocke-Kasami-Younger) 算法：在 CNF 形式下做 $O(n^3|G|)$ 的 DP 解析 (Decoding)。

前置：lec06_hmm (DP / Viterbi)、lec4_lm (MLE + smoothing)、基本形式语言 (终结符 / 非终结符 / 推导)。

复习关注点：constituent 的三大测试、CFG 五元组、CNF 归约、PCFG 概率定义、MLE 估计、CKY 表 $\pi[i, j, X]$ 的递推公式、CKY 复杂度推导 $O(n^3|R|)$ 、labeled F1 (EvalB) 评测、PCFG 的两个经典缺陷 (lexical insensitivity & structural frequency)。

7.1 为什么需要“树”——超越序列

7.1.1 三个例子

the guy who fixed the car carefully packed his tools	← OK
carefully the guy who fixed the car packed his tools	← OK
carefully the guy who fixed the car is tall	← OK

carefully 修饰的是 packed 不是 fixed，线性距离最近的不一定是真正的修饰对象。Chomsky 说：

The deepest property of language ...is **structure dependence**. Linear closeness is an easy computation, but here you're doing a much more, what looks like a more complex computation.

→ 语言“依结构而非依线性”。

7.1.2 PP 附着歧义 (PP-attachment)

Jane ate spaghetti with a fork.	(with-fork 修饰 ate)
Jane ate spaghetti with meatballs.	(with-meatballs 修饰 spaghetti)

同一表面线性序列，两个不同树对应两种语义。只靠 sequence model 无法表达这种深层差别。

7.1.3 结构层次：单点 → 序列 → 树 → 图

输入	输出	任务	难度
word	class	WSD, 文本分类	low
word sequence	tag sequence	POS / NER / 分词	mid
word sequence	tree	constituency / dependency parsing	high
word sequence	graph	AMR, 事件抽取	very high

本讲就是从“sequence labelling”迈向“tree-structured prediction”的第一步。

7.2 Constituent (成分)

7.2.1 定义与测试

Constituent: 能像单一单元一样行动的一组词。“单一单元”通过三类操作来检测:

1. **Substitution / Distribution test**: 能否被代词或同类短语替换? Harry the Horse likes ... → He likes ... (NP 行为同一)
2. **Movement test**: 能否整体移到句中其它位置? We will hold the opening ceremony for PKU Cup on April 20th → On April 20th, we will hold the opening ceremony for PKU Cup (PP 整体移动) → *On April we will hold 20th the opening ceremony ... (拆开就 ungrammatical)
3. **Coordination test**: 能否与同类短语并列? ... on April 20th ****and**** in the morning (两个 PP)

典型短语类型: NP (名词短语)、VP (动词短语)、PP (介词短语)、AdjP、AdvP、S (句子)

7.2.2 短语结构 = 嵌套树

NP 内部可以再含 NP, VP 内部可以含 NP/PP/.....→ **递归 (recursion)**。所以构成成分的关系自然是一棵 **constituency tree (短语结构树)**, 叶子是词, 内部节点是 phrase / POS 标签, 根是 S (句子)。

[NOTE] 中文/PKU 校园语料的“递归怪兽”

市发展改革委关于转发省发展改革委关于转发国家发展改革委办公厅关于对真抓实干成效明显地方加大中央预算内投资激励支持力度的通知的通知的通知——一个 67 字的 NP。

7.3 形式语言 (Formal Languages) 与 文法 (Grammars)

7.3.1 形式语言

- **字母表 (alphabet) Σ** : 有限符号集;
- Σ^* : 所有有限长度字符串;
- **形式语言**: Σ^* 的任意子集。

例: $L = \{a^n b^n \mid n \in \mathbb{N}\}$ 是形式语言, 但不能用正则表达式表达 (不是正则语言) ——这一点是 CFG 强于正则的关键。

7.3.2 文法 (Formal Grammar)

$G = (N, \Sigma, R, S)$:

- N : 非终结符集 (NP, VP, S,);
- Σ : 终结符集 (词 / 字符), 与 N 不相交;
- $S \in N$: 起始符;
- R : 产生式规则集, 每条形如 $(\Sigma \cup N)^+ \rightarrow (\Sigma \cup N)^*$ 。

文法的两种用法:

- **Generation**: 从 S 反复展开, 生成字符串;
- **Parsing**: 给定字符串, 回推使用了哪些规则 + 树结构。

7.3.3 Chomsky 层级 (提一下定位)

层级	产生式形式	等价识别机	表达能力
Type-3 Regular	$A \rightarrow aB$ 或 $A \rightarrow a$	有限自动机 (FSA)	最弱
Type-2 CFG	$A \rightarrow \gamma, \gamma \in (\Sigma \cup N)^*$	下推自动机 (PDA)	中
Type-1 CSG	上下文相关	线性有界自动机	强
Type-0 Unrestricted	任意	图灵机	最强

NLP 主流用 **Type-2 (CFG)** ——表达力够建大多数句法, 复杂度还可控 ($O(n^3)$ 可解析)。

7.4 上下文无关文法 (CFG)

7.4.1 定义

CFG = 产生式左侧只能是单个非终结符的文法:

$$A \rightarrow \gamma, \quad A \in N, \quad \gamma \in (\Sigma \cup N)^*. \quad (7.1)$$

“context-free”的含义: 替换 A 时不考虑 A 周围的上下文。

7.4.2 玩具例子 1: $a^n b^n$

$$N = \{S\}, \quad \Sigma = \{a, b\}, \quad R = \{S \rightarrow aSb \mid \varepsilon\}.$$

推导: $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aa\varepsilon bb = aabb$ 。得 $L(G) = \{a^n b^n \mid n \geq 0\}$ 。

7.4.3 玩具例子 2: Chomsky 经典句

$N = \{S, NP, VP, AdjP, AdvP, N, Adj, Adv, V\}; \Sigma = \{\text{colorless, green, ideas, sleep, furiously}\};$
 R :

$S \rightarrow NP \ VP$
 $VP \rightarrow VP \ AdvP \mid V$
 $AdvP \rightarrow Adv$
 $NP \rightarrow AdjP \ NP \mid N$

AdjP → Adj
 N → ideas
 Adj → colorless | green
 Adv → furiously
 V → sleep

推导 colorless green ideas sleep furiously:

$S \Rightarrow NP VP \Rightarrow AdjP NP VP \Rightarrow Adj NP VP \Rightarrow colorless NP VP$
 $\Rightarrow colorless AdjP NP VP \Rightarrow colorless Adj NP VP$
 $\Rightarrow colorless green NP VP \Rightarrow colorless green N VP$
 $\Rightarrow colorless green ideas VP \Rightarrow colorless green ideas VP AdvP$
 $\Rightarrow colorless green ideas V AdvP \Rightarrow colorless green ideas sleep AdvP$
 $\Rightarrow colorless green ideas sleep Adv \Rightarrow colorless green ideas sleep furiously$

7.4.4 文法等价

- **Weak equivalence:** 两文法生成相同的字符串集 $L(G_1) = L(G_2)$ 。
- **Strong equivalence:** weak + 给每个句子分配相同的短语结构。

CKY 要求 CNF, 但 CNF 仅是 **weakly** 等价的形式 (树结构与原文法不同), 最后通常需要”反归约”恢复。

7.5 Chomsky Normal Form (CNF)

7.5.1 定义

CFG G 处于 CNF iff 每条规则形如:

- $A \rightarrow BC$, $A \in N$, $B, C \in N$ (二元非终结符), 或
- $A \rightarrow a$, $A \in N$, $a \in \Sigma$ (词法规则)。

(有时再加 $S \rightarrow \varepsilon$ 一条用于空字符串。)

7.5.2 等价定理

每个 CFG 都可以转换为一个弱等价的 CNF 文法。

7.5.3 归约步骤

1. 去 unit rule $A \rightarrow B$: 复制 B 的所有产生式到 A ;
2. 去 ε rule $A \rightarrow \varepsilon$ (若 $A \neq S$): 把所有用到 A 的右侧拆分成”含 A ”和”不含 A ”两份;
3. 去 long rule $A \rightarrow X_1 X_2 \dots X_k$ ($k \geq 3$): 拆为二元链 $A \rightarrow X_1 A^{(1)}$, $A^{(1)} \rightarrow X_2 A^{(2)}$, $\dots$, $A^{(k-2)} \rightarrow X_{k-1} X_k$;
4. 去混合右侧 $A \rightarrow X_1 a X_2$ (终结混非终结): 替换 a 为新非终结 T_a 并加 $T_a \rightarrow a$ 。

例: $S \rightarrow NP VP PP$ 三元 $\rightarrow$ 引入 X , 变为 $S \rightarrow NP X, X \rightarrow VP PP$ (二元)。

[NOTE] CKY 为什么要 CNF?

CKY 的核心组合规则是”切一刀, 左 = Y , 右 = Z , 合 = X ”。只有 binary rule 才与”二分 span”对应; CNF 把任意 CFG 归约到 binary 形式, 使 CKY 表的尺寸固定为 $n^2 \times |N|$ 。

7.6 Probabilistic CFG (PCFG)

7.6.1 定义

PCFG 给每条规则一个概率 $q(A \rightarrow \beta)$, 满足

$$\sum_{\beta} q(A \rightarrow \beta \mid A) = 1, \quad \forall A \in N. \quad (7.2)$$

一棵树的概率 = 所用规则概率的乘积 (独立性假设: 每次展开非终结符的事件相互独立):

$$p(t) = \prod_{i=1}^{|t|} q(A_i \rightarrow \beta_i). \quad (7.3)$$

一个句子在该 PCFG 下的边际概率:

$$p(s) = \sum_{t \in T(s)} p(t), \quad (7.4)$$

其中 $T(s)$ 是产生 $\text{yield} = s$ 的所有 derivation。

7.6.2 worked 例

文法 (PCFG 节选):

S	$\rightarrow$ NP VP	0.8
S	$\rightarrow$ Aux NP VP	0.15
S	$\rightarrow$ VP	0.05
NP	$\rightarrow$ AdjP NP	0.2
NP	$\rightarrow$ D N	0.7
NP	$\rightarrow$ N	0.1
VP	$\rightarrow$ VP AdvP	0.3
VP	$\rightarrow$ V	0.2
AdvP	$\rightarrow$ Adv	1.0
AdjP	$\rightarrow$ Adj	1.0
Adj	$\rightarrow$ colorless	0.4
Adj	$\rightarrow$ green	0.6
N	$\rightarrow$ ideas	1.0
Adv	$\rightarrow$ furiously	1.0
V	$\rightarrow$ sleep	1.0

ideas sleep furiously 的一条推导:

S	$\Rightarrow$ NP VP	0.8
	$\Rightarrow$ N VP	0.1
	$\Rightarrow$ ideas VP	1.0
	$\Rightarrow$ ideas VP AdvP	0.3
	$\Rightarrow$ ideas V AdvP	0.2
	$\Rightarrow$ ideas sleep AdvP	1.0
	$\Rightarrow$ ideas sleep Adv	1.0
	$\Rightarrow$ ideas sleep furiously	1.0

$$p(t) = 0.8 \times 0.1 \times 1.0 \times 0.3 \times 0.2 \times 1.0 \times 1.0 \times 1.0 = 0.0048.$$

7.6.3 从 Treebank 估计 PCFG

给一棵 treebank (如 Penn Treebank §02–21), 直接 MLE:

$$\hat{q}_{\text{MLE}}(\alpha \rightarrow \beta) = \frac{\text{count}(\alpha \rightarrow \beta)}{\text{count}(\alpha)}. \quad (7.5)$$

性质: 若真实分布也是 PCFG, 则随样本量 $\rightarrow \infty$, $\hat{q} \rightarrow q^*$ (一致性)。

[WARN] 数据稀疏

词法规则 $A \rightarrow w$ 在测试时几乎必然遇 OOV。常见 trick: 删词、训练在 POS 序列上 (这样规则 = $A \rightarrow \text{POS}$, 词由独立 emission 处理)。这样 labeled F1 也能上 70+。

7.6.4 PCFG 的两个经典缺陷

PCFG 假设”展开非终结符的事件互相独立”——但语言并不独立。

1. **Lack of lexical sensitivity**: PP 附着的偏好取决于具体词 (eat with fork vs eat with meatballs)。普通 PCFG 看不到词, 只看 POS / phrase 类别, 做不出区分。
 - 修补: **lexicalized PCFG** (Collins 1999) ——把 head word 标注到每个非终结符上 (如 NP[meatballs]), 再用 backoff 估计。
2. **Lack of structural-frequency sensitivity**: NP PP 的概率与 VP PP 的概率分开估, 看不到 corpus 中”两个 NP 连用”实际频率的不一样。
 - 修补: **parent annotation** (Johnson 1998), 把父节点也编进非终结符 (NP^S)。

7.7 CKY 算法 (Cocke–Kasami–Younger)

7.7.1 任务

给定 PCFG G (CNF 形式) 和句子 $s = w_1 w_2 \dots w_n$, 求

$$t^*(s) = \arg \max_{t \in T(s)} p(t). \quad (7.6)$$

枚举 $T(s)$ 不可行 (树数 = Catalan 级别); CKY 用 DP $O(n^3 |R|)$ 解决。

7.7.2 DP 表

状态: $\pi[i, j, X]$ = 以非终结符 X 覆盖区间 $[i + 1, j]$ (左开右闭) 的所有子树中最大概率。(讲义采用” i 表示词前的空隙”约定, 所以 $w_{i+1}, w_{i+2}, \dots, w_j$ 是被 X 覆盖的词。常见教材也用 $\pi[i, j, X]$ 表示覆盖 $w_i \dots w_j$ 。两种约定区别仅在下标偏移。)

7.7.3 递推 (CNF 假设下)

基底 (lexical rule): 对每个 $i = 0, \dots, n - 1$, 每个 $X \in N$:

$$\pi[i, i + 1, X] = q(X \rightarrow w_{i+1}). \quad (7.7)$$

递推 (binary rule): 对所有跨度长度 $\ell = 2, 3, \dots, n$, 所有起点 i (终点 $j = i + \ell$), 所有 $X \in N$:

$$\pi[i, j, X] = \max_{\substack{X \rightarrow YZ \in R \\ m=i+1, \dots, j-1}} q(X \rightarrow YZ) \cdot \pi[i, m, Y] \cdot \pi[m, j, Z] \quad (7.8)$$

同时记录 backpointer $bp[i, j, X] = (Y, Z, m)$ 用于恢复树。

终止: $p(t^*) = \pi[0, n, S]$; 回溯 bp 即得最优树。

7.7.4 伪代码

INPUT: sentence $w[1..n]$; CNF PCFG G with rules R .

OUTPUT: $\pi[0, n, S]$ and best parse tree.

```

1.  for  $i \leftarrow 0$  to  $n-1$ :
2.      for each rule  $X \rightarrow w[i+1] \in R$ :
3.           $\pi[i, i+1, X] \leftarrow q(X \rightarrow w[i+1])$ 
4.           $bp[i, i+1, X] \leftarrow (w[i+1])$ 

5.  for  $\ell \leftarrow 2$  to  $n$ :                                # span length
6.      for  $i \leftarrow 0$  to  $n-\ell$ :                        # span start
7.           $j \leftarrow i + \ell$ 
8.          for each non-terminal  $X$ :
9.               $\pi[i, j, X] \leftarrow 0$ 
10.             for each rule  $X \rightarrow YZ \in R$ :
11.                 for  $m \leftarrow i+1$  to  $j-1$ :            # split point
12.                     score  $\leftarrow q(X \rightarrow YZ) \cdot \pi[i, m, Y] \cdot \pi[m, j, Z]$ 
13.                     if score  $> \pi[i, j, X]$ :
14.                          $\pi[i, j, X] \leftarrow \text{score}$ 
15.                          $bp[i, j, X] \leftarrow (Y, Z, m)$ 

16. return  $\pi[0, n, S]$ , reconstruct_tree( $bp, 0, n, S$ )

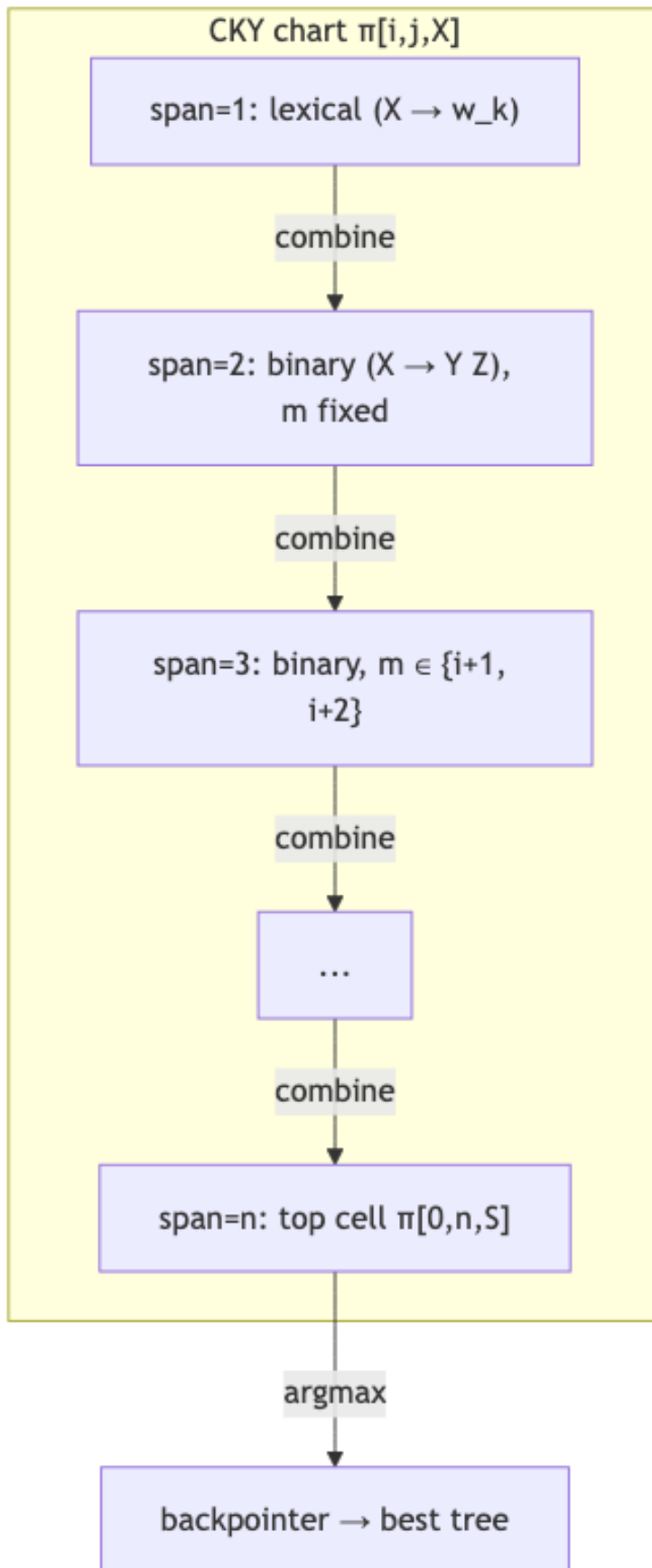
```

recognition 版本: 把所有 q 设为 1, 把 $\max$ 改 $\vee$ 、 $\cdot$ 改 $\wedge$; 若 $\pi[0, n, S] = \text{True}$, 则句子被该 CFG 接受。

7.7.5 复杂度

- 外层三重循环: $O(n^3)$ (ℓ, i, m);
- 内层枚举规则: $O(|R|)$ (每条 binary rule 试一次);
- 总复杂度: $O(n^3 \cdot |R|)$;
- 空间: $O(n^2 \cdot |N|)$ 。

7.7.6 CKY 表结构示意图 (mermaid)



7.8 Worked Example: CKY 解析“I saw the boy with a telescope” (节选)

8.1 文法 (已 CNF)

$S \rightarrow NP VP$	1.0
$NP \rightarrow Det N$	0.6
$NP \rightarrow NP PP$	0.3
$NP \rightarrow I$	0.1
$VP \rightarrow V NP$	0.5
$VP \rightarrow VP PP$	0.5
$PP \rightarrow P NP$	1.0
$V \rightarrow \text{saw}$	1.0
$Det \rightarrow \text{the}$	0.4
$Det \rightarrow \text{a}$	0.6
$N \rightarrow \text{boy}$	0.5
$N \rightarrow \text{telescope}$	0.5
$P \rightarrow \text{with}$	1.0

句子 (7 词): I saw the boy with a telescope, 下标分隔位置 $0, 1, \dots, 7$ 。

8.2 span=1 (词法层)

$\pi[i, i+1, \cdot]$	词	条目
$\pi[0, 1, NP]$	I	0.1
$\pi[1, 2, V]$	saw	1.0
$\pi[2, 3, Det]$	the	0.4
$\pi[3, 4, N]$	boy	0.5
$\pi[4, 5, P]$	with	1.0
$\pi[5, 6, Det]$	a	0.6
$\pi[6, 7, N]$	telescope	0.5

8.3 span=2

- $\pi[2, 4, NP] = q(NP \rightarrow Det N) \cdot 0.4 \cdot 0.5 = 0.6 \cdot 0.2 = 0.12$;
- $\pi[5, 7, NP] = 0.6 \cdot 0.6 \cdot 0.5 = 0.18$;
- $\pi[1, 3, \cdot]: V \text{ the 没有匹配规则} \rightarrow \text{空}$;
- $\pi[4, 6, \cdot]: \text{with a 也没有} \rightarrow \text{空}$ 。

8.4 span=3

- $\pi[1, 4, VP] = q(VP \rightarrow V NP) \cdot \pi[1, 2, V] \cdot \pi[2, 4, NP] = 0.5 \cdot 1.0 \cdot 0.12 = 0.06$;
- $\pi[4, 7, PP] = q(PP \rightarrow P NP) \cdot \pi[4, 5, P] \cdot \pi[5, 7, NP] = 1.0 \cdot 1.0 \cdot 0.18 = 0.18$ 。

8.5 span=4

- $\pi[0, 4, S] = q(S \rightarrow NP VP) \cdot \pi[0, 1, NP] \cdot \pi[1, 4, VP] = 1.0 \cdot 0.1 \cdot 0.06 = 0.006$;
- $\pi[2, 7, NP] = q(NP \rightarrow NP PP) \cdot \pi[2, 4, NP] \cdot \pi[4, 7, PP] = 0.3 \cdot 0.12 \cdot 0.18 = 0.00648$ 。

8.6 span=5

- $\pi[1, 7, VP]$ 有两种切法：
 - $VP \rightarrow V NP$, $m = 2$: $0.5 \cdot 1.0 \cdot \pi[2, 7, NP] = 0.5 \cdot 1.0 \cdot 0.00648 = 0.00324$;
 - $VP \rightarrow VP PP$, $m = 4$: $0.5 \cdot \pi[1, 4, VP] \cdot \pi[4, 7, PP] = 0.5 \cdot 0.06 \cdot 0.18 = 0.0054$;
 - $\max = 0.0054$, $\text{backpointer} = (VP, PP, m=4)$ 。

8.7 span=7 (顶层)

- $\pi[0, 7, S] = q(S \rightarrow NP VP) \cdot \pi[0, 1, NP] \cdot \pi[1, 7, VP] = 1.0 \cdot 0.1 \cdot 0.0054 = 0.00054$ 。

回溯: $S \rightarrow NP VP$ ($m = 1$) $\rightarrow NP = I$; $VP \rightarrow VP PP$ ($m = 4$) $\rightarrow VP \rightarrow V NP$ ($m = 2$) $\rightarrow V = \text{saw}$, $NP \rightarrow \text{Det N (the boy)}$; $PP \rightarrow P NP$ ($m = 5$) $\rightarrow P = \text{with}$, $NP \rightarrow \text{Det N (a telescope)}$ 。

即 PP 附着到 VP (“with telescope”修饰 saw), $p^* = 0.00054$ 。

8.8 部分 CKY 表

span ↓ 起 点 →	0 (I)	1 (saw)	2 (the)	3 (boy)	4 (with)	5 (a)	6 (tel)
1	NP:0.1	V:1.0	Det:0.4	N:0.5	P:1.0	Det:0.6	N:0.5
2	—	—	NP:0.12	—	—	NP:0.18	—
3	—	VP:0.06	—	—	PP:0.18	—	—
4	S:0.006	—	NP:0.00648	—	—	—	—
5	—	VP:0.0054	—	—	—	—	—
6	—	—	—	—	—	—	—
7	S:0.00054	—	—	—	—	—	—

7.9 Inside 算法 (边际概率)

把 CKY 中的 $\max$ 换成 $\sum$, 就得到 PCFG 的 **inside 算法**, 计算句子的边际概率 $p(s) = \sum_{t \in T(s)} p(t)$:

$$\alpha[i, j, X] = \sum_{X \rightarrow YZ \in R} \sum_{m=i+1}^{j-1} q(X \rightarrow YZ) \cdot \alpha[i, m, Y] \cdot \alpha[m, j, Z]. \quad (7.9)$$

基底同 CKY。 $p(s) = \alpha[0, n, S]$ 。

类比 HMM:

HMM 序列	PCFG 树
Forward α	Inside $\alpha[i, j, X]$
Backward β	Outside $\beta[i, j, X]$
Viterbi δ	CKY $\pi[i, j, X]$
Baum–Welch (EM)	Inside–Outside (EM for PCFG)

Inside–Outside 算法可用于无监督学习 PCFG (lec07 未深入, 留作扩展)。

7.10 评测：Labeled Precision / Recall / F1 (EvalB)

把一棵 parse tree 视为一组 **labeled constituents** (每个内部节点对应一个三元组 $(label, i, j)$)。

设黄金树有集合 G ，预测树有集合 P ：

$$\text{Labeled Precision} = \frac{|P \cap G|}{|P|}, \quad \text{Labeled Recall} = \frac{|P \cap G|}{|G|}, \quad F_1 = \frac{2PR}{P + R}. \quad (7.10)$$

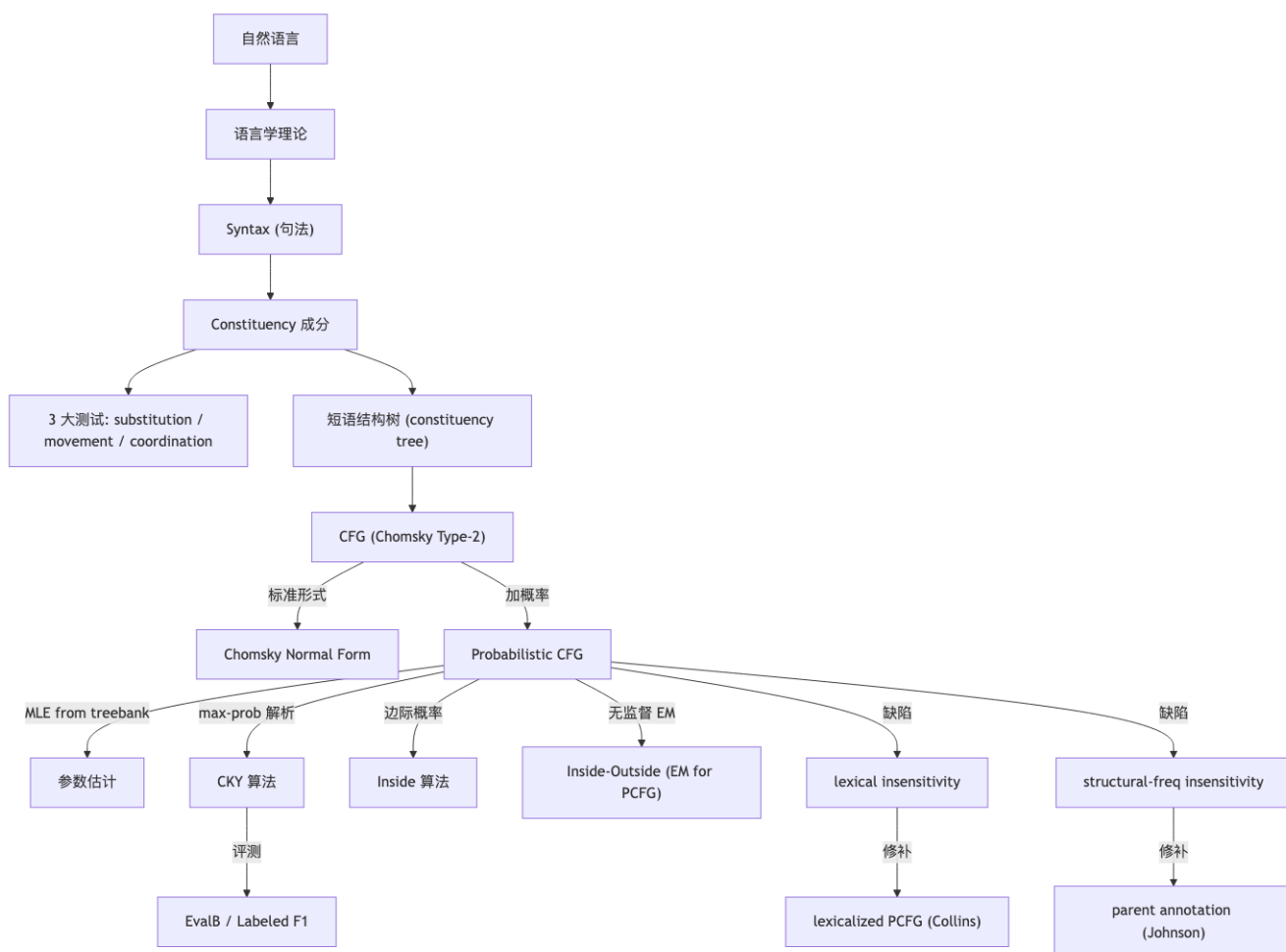
标准实验配置：

- Train: Penn Treebank §02–21;
- Dev: §22;
- Test: §23;
- 工具: **EvalB**;
- baseline PCFG (deleted words, on POS sequence): labeled F1 $\approx$ low-70s。
- 现代神经成分句法分析器: > 95% F1 (lexicalized + 神经表征 + 重排序)。

7.10.1 失败模式举例

1. **PP 附着歧义**: 纯 PCFG 无法区分 eat with fork (VP-PP) vs eat with meatballs (NP-PP);
 2. **NP coordination**: old men and women 的两种分析;
 3. **结构频率不灵敏**: NP→NP PP vs VP→VP PP 实际频率差异 PCFG 看不见 (独立性假设)。
-

7.11 概念关系图



7.12 易错点 (Pitfalls)

1. **CNF 不是 CFG 的“真子集”**: 每条 CFG 都有弱等价 CNF; 但树结构会变 (unit / 长规则被拆掉), 评测前需要反归约或在标注端对齐。
2. **CKY 复杂度**: $O(n^3|R|)$, 不是 $O(n^3|N|)$ 也不是 $O(n^3|N|^3)$ 。原因: 内层只枚举每条 binary rule 一次。
3. **下标约定**: 教材 / 讲义 / 工程实现常用两种: (a) $\pi[i, j, X]$ 覆盖 $w_{i+1} \dots w_j$; (b) $\pi[i, j, X]$ 覆盖 $w_i \dots w_j$ 。手算 / 实现前先把约定写在纸边。
4. **lexical rule 与 binary rule 要分开循环**: 先填 span=1, 再填 span ≥ 2 ; 不要试图统一处理。
5. **max vs $\sum$** : CKY (Viterbi over PCFG) 用 max, inside 用 $\sum$ 。两者递推结构同构。
6. **概率下溢**: 解析长句时 π 极小, 必须在 log 空间运算, max 不需要 logsumexp (max 在 log 空间 = max 本身), inside 才需要。
7. **PCFG 假设的独立性太强**: 每次展开互相独立; 现实是不独立 (lexical / structural / discourse)。这是 PCFG 性能上限只能到 ~70 F1 的根本原因。
8. **OOV 处理**: 测试句中含训练未见词时 $q(X \rightarrow w) = 0$, 整条 derivation 概率归 0。解法: smoothing、UNK 类、字符级 backoff、或者用 POS 序列代替词。
9. **CKY 是 recognition + parsing 二合一**: 把 q 全设 1 + max 改 $\vee$ 即可只做识别。
10. **“correct”在 PCFG 里 = “more probable”**: PCFG 不区分语义对错, 只看 p 排名; 语言学家未必同意。

11. **lexicalized PCFG 复杂度**: head word 注入后非终结符数变 $|N| \cdot |V|$, CKY 退化为 $O(n^5)$ 级别。Collins / Charniak 用各种 backoff + beam 维持工程可行。

7.13 中英语对照

中文	英文	缩写 / 备注
句法	Syntax	—
成分 / 短语	constituent / phrase	NP, VP, PP, ...
短语结构	phrase structure	—
短语结构树	constituency tree / parse tree	—
上下文无关文法	Context-Free Grammar	CFG
终结符	terminal	Σ
非终结符	nonterminal	N
起始符	start symbol	S
产生式	production rule	$A \rightarrow \beta$
推导	derivation	$\Rightarrow$
乔姆斯基范式	Chomsky Normal Form	CNF
概率上下文无关文法	Probabilistic CFG	PCFG
极大似然估计	Maximum Likelihood Estimation	MLE
树库	Treebank	Penn Treebank, CTB
CKY 算法	Cocke-Kasami-Younger algorithm	$O(n^3 \ R\)$
内向算法	Inside algorithm	$\alpha[i, j, X]$
外向算法	Outside algorithm	$\beta[i, j, X]$
内向-外向算法	Inside-Outside algorithm	EM for PCFG
弱等价 / 强等价	weak / strong equivalence	grammar
标记的精确率 / 召回率	Labeled Precision / Recall	LP / LR
F1 分数	F1-score	EvalB
词汇化文法	Lexicalized Grammar	Collins / Charniak
父节点标注	parent annotation	Johnson 1998
介词短语附着	PP-attachment	经典歧义

[TIP] 期中复习要点

1. **Constituent 三大测试**: substitution / movement / coordination; 能举例并指出失败的反例。2. **CFG 五元组** $G = (N, \Sigma, R, S)$; 写出 $a^n b^n$ 与简单英语片段的 CFG。3. **CNF 定义 + 4 步归约** (unit / ε / long / mixed)。4. **PCFG 概率定义** (7.2)(7.3); **MLE 估计** (7.5)。5. **PCFG 两大缺陷**: lexical insensitivity & structural-frequency insensitivity; 对应修补 (lexicalized PCFG, parent annotation)。6. **CKY 递推公式 (7.7)(7.8)**——默写, 包括基底、递推、终止; 写出完整伪代码。7. **CKY 复杂度** $O(n^3 |R|)$ ——能解释外层三重循环 + 内层规则枚举的来源。8. **手算 CKY 表**: 3–4 词、5–6 条规则的 toy, 能填出每个非空 cell 并回溯出最优树。9. **Inside / Viterbi 对比**: $\sum$ vs max; 类比 HMM Forward / Viterbi。10. **EvalB 评测**: labeled precision/recall/F1 的定义; 现代 baseline 数字 (PCFG ~70%, 神经 SOTA > 95%)。11. **PP-attachment 经典例**: eat with fork/meatball; 说明 PCFG 为何区分不开。12. **CKY Viterbi DP 思想统一**: 动态规划在 NLP 中的两条主线 (序列 HMM / 树 PCFG)。

Lecture 8: 依存句法分析 (Dependency Parsing)

[TIP] 学习指南

本讲从语言学动机出发，把句法分析从**短语结构 (constituent)** 转向**依存结构 (dependency)**：用「中心词 → 修饰词」的二元有向关系直接刻画**谓词-论元结构**，对自由语序、长距离依存更友好。重点掌握三件事：1. **形式化**：依存图 $G = (V, A)$ 的四条性质（连通、无环、单头、投射），以及非投射现象。2. **算法**：- 基于转移 (transition-based)：arc-standard / arc-eager 的转移系统、栈缓存配置、oracle、贪心 / beam 解码、复杂度 $O(n)$ 。- 基于图 (graph-based)：边打分 + 最大生成树 (Chu-Liu-Edmonds)，可处理非投射。- 基于动态规划：**Eisner 算法** $O(n^3)$ 处理投射树 (重要!)。3. **评测**：UAS / LAS 的定义与计算。中考点：- 给定句子写出 arc-standard 的转移轨迹 (trace)。- 判断一棵树是否投射，并解释原因。- 写出 Eisner DP 的递推方程并分析复杂度。- LAS / UAS 的计算与差异。

8.1 Motivation：为什么需要依存结构？

8.1.1 Chomsky 的「结构依赖性」

“I think the deepest property of language ...is what is sometimes called **structure dependence**. Linear closeness is an easy computation, but here you're doing a much more complex computation.”——N. Chomsky

例：

- the guy who fixed the car **carefully** packed his tools.
- **carefully** the guy who fixed the car packed his tools.

副词 carefully 的修饰对象（修饰 fixed 还是 packed）不取决于**线性距离**，而取决于**结构距离**。

8.1.2 短语结构 vs 依存结构

以句子 I like eating green apples 为例：

视角	节点	关系
Constituent (短语结构)	S, NP, VP, PP, ...非终结节点	嵌套
Dependency (依存结构)	词本身	二元有向关系：I-like, like-apples, apples-eating, apples-green

依存结构的**优势**：

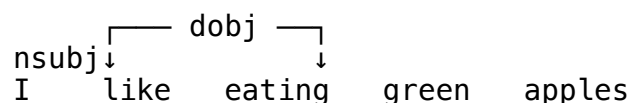
- 直接体现**谓词-论元结构** (predicate-argument structure)，靠近语义；

- 对自由语序语言（如中文、捷克语）更鲁棒；
- 长距离依赖（who/which/that 引导的从句）显式建模。

8.1.3 Tesnière 的依存语言学观

Lucien Tesnière (1959):

- 句子是一个有机整体，由词构成。
- 词与词之间的连接 (connection) 构成句法结构。
- 每条连接联结一个上位词 (superior / head) 和一个下位词 (inferior / dependent)。



8.1.4 头-依赖关系的判定标准 (Collins 1999)

对于一个构造 C 中的 head H 与 dependent D :

1. H 决定 C 的句法范畴 (H 可替代 C);
2. H 决定 C 的语义范畴, D 修饰/限定 H ;
3. H 必现, D 可省;
4. H 选择 (select) D , 并决定 D 是否必现;
5. D 的形式由 H 决定 (agreement / government);
6. D 的线性位置以 H 为参照。

8.2 形式化定义

8.2.1 依存图

依存结构定义为带标号有向图

$$G = (V, A), \quad V = \{0, 1, \dots, n\}, \quad A \subseteq V \times V \times L \quad (8.1)$$

其中:

- V : 节点集合, 0 为虚根 ROOT, $1 \dots n$ 为词;
- A : 弧集合, 每条弧 (i, j, ℓ) 表示 $i \rightarrow j$ 且关系标签为 $\ell \in L$;
- 节点上有线性序 (句子原顺序)。

8.2.2 良构 (well-formedness) 四条件

条件	形式化	含义
(弱) 连通	$\forall i \exists j : i \rightarrow j$ 或 $j \rightarrow i$	没有孤立节点
无环	$i \xrightarrow{*} j \Rightarrow \neg(j \rightarrow i)$	不存在依存环
单头	$i \rightarrow j \Rightarrow \forall k \neq i : \neg(k \rightarrow j)$	每个词只有一个 head
投射	$i \rightarrow j \Rightarrow \forall k \text{ 介于 } i, j : i \xrightarrow{*} k$	弧无交叉

[NOTE] 投射性 (Projectivity)

若把所有弧画在词序之上的半平面内，没有任何两条弧相交，则称该依存树是**投射的**。投射性使得高效 DP 算法 (CKY、Eisner) 可行；但实际语言中常有**非投射**现象 (自由语序、长距离移位)。

8.2.3 非投射示例

A hearing is scheduled on the issue today.

弧 hearing → on the issue 与 scheduled → today 相交，是非投射树。多数理论框架 (如 UD) **不假设**投射性。

8.2.4 依存关系标签 (Universal Dependencies, UD)

UD 项目 (de Marneffe et al., 2021) 定义 **37 种**普适依存关系，例如：

标签	含义	例
nsubj	名词主语	I like ...
obj / dobj	直接宾语	like apples
iobj	间接宾语	give her a book
nmod	名词修饰	financial markets
amod	形容词修饰	green apples
advmod	副词修饰	carefully packed
aux	助动词	is checking
det	限定词	the machine
xcomp	开放从句补语	wants to go
mark	标记词	wants to go
punc	标点	.
root	根	ROOT → 主谓

UD 已经覆盖 >100 种语言、200+ treebank。

8.2.5 依存 vs 短语结构

	依存结构	短语结构
显式	head-dependent (弧)、 grammatical function (标签)	短语 (非终结节点)、结构范畴 (非 终结标签)
隐式 优势	短语结构 接近语义；对语序变化鲁棒	head-dependent 关系 显式短语切分；适合 CKY 一类 DP

依存结构是「谓词-论元关系的代理」(a proxy for predicate-argument structure)。

8.3 解析算法

我们把依存解析视为**结构化预测 (structured prediction)**：

$$y^*(x; \theta) = \arg \max_{y \in \mathcal{Y}(x)} \text{Score}(x, y) \quad (8.2)$$

主要有三大思路：

视角	代表算法	复杂度	投射性
增量搜索 (incremental)	transition-based (arc-standard / arc-eager)	$O(n)$ 贪心	默认投射
离散优化 + DP	Eisner	$O(n^3)$	投射
离散优化 + MST	Chu–Liu–Edmonds	$O(n^2)$	非投射

8.3.1 Transition-Based 依存解析

8.3.1.1 转移系统四元组

$$S = (C, T, c_s, C_t) \quad (8.3)$$

- C : 所有配置 (configuration) 的集合, 配置是「部分解析状态」;
- T : 转移 (transition) 集合, 每个转移 $t: C \rightarrow C$ 是一次解析动作;
- c_s : 初始函数, 把输入句子映射为初始配置;
- $C_t \subseteq C$: 终结配置集合。

8.3.1.2 Stack-based 配置

对句子 $x = w_0, w_1, \dots, w_n$ ($w_0 = \$ \text{ROOT}$), 配置:

$$c = (\sigma, \beta, A) \quad (8.4)$$

- σ : 栈 (stack), 栈顶在右;
- β : 缓存 (buffer), 队首在左;
- A : 已建立的弧集合。

初始 / 终止:

$$c_s(x) = ([0], [1, 2, \dots, n], \emptyset), \quad C_t = \{(\sigma, [], A)\} \quad (8.5)$$

8.3.1.3 Arc-Standard 转移系统

记 $\sigma|i$ 表示栈顶为 i ; $i|\beta$ 表示缓存首为 i 。三种转移:

名称	前置	动作
SHIFT	缓存非空	$(\sigma, i \beta, A) \Rightarrow (\sigma i, \beta, A)$
LEFT-ARC _{ℓ}	栈顶 ≥ 2 且 $i \neq 0$	$(\sigma i, j \beta, A) \Rightarrow (\sigma, j \beta, A \cup \{(j, i, \ell)\})$
RIGHT-ARC _{ℓ}	栈顶 ≥ 2	$(\sigma i, j \beta, A) \Rightarrow (\sigma, i \beta, A \cup \{(i, j, \ell)\})$

直觉:

- SHIFT: 把缓存首压入栈;
- LEFT-ARC: 弧 $j \rightarrow i$ (缓存首作 head), 栈顶 i 出栈;
- RIGHT-ARC: 弧 $i \rightarrow j$ (栈顶作 head), 缓存首 j 出栈后被栈顶替换。

[NOTE] arc-standard 性质

- 每个词正好被 SHIFT 一次: n 次 SHIFT; - 每条弧对应一次 LEFT/RIGHT-ARC: n 次 ARC; - 总转移数 $= 2n$, 线性复杂度 $O(n)$ 。

8.3.1.4 Arc-Eager 转移系统 (对比, 扩展)

arc-standard 的问题: 右弧必须等到右子树全部处理完才能建立 $\rightarrow$ 长右臂句子会被推迟很久。

arc-eager 引入 REDUCE 动作, 提早建立右弧:

名称	动作	前置
SHIFT	$(\sigma, i \mid \beta, A) \Rightarrow (\sigma \mid i, \beta, A)$	缓存非空
LEFT-ARC _{ℓ}	$(\sigma \mid i, j \mid \beta, A) \Rightarrow (\sigma, j \mid \beta, A \cup \{(j, i, \ell)\})$	i 无 head
RIGHT-ARC _{ℓ}	$(\sigma \mid i, j \mid \beta, A) \Rightarrow (\sigma \mid i \mid j, \beta, A \cup \{(i, j, \ell)\})$	j 无 head
REDUCE	$(\sigma \mid i, \beta, A) \Rightarrow (\sigma, \beta, A)$	i 已有 head

差异:

- arc-standard 的 RIGHT-ARC 把缓存首移走; arc-eager 的 RIGHT-ARC 把缓存首压入栈 (之后可继续接其它依赖), 再用 REDUCE 出栈。
- arc-eager 更适合真正的增量解析 (incremental, left-to-right)。

8.3.1.5 Oracle 与学习

定义 oracle $o : C \rightarrow T$, 给出每个配置下的「正确」转移。在训练时:

- 用已标注树生成「金标准」转移序列 (**static oracle**); 对 arc-standard, 给定投射树, 可证明 oracle 唯一。
- **dynamic oracle** (Goldberg & Nivre, 2012): 在偏离金标准的状态下仍给出最优后续, 可与 imitation learning 结合, 缓解 exposure bias。

把 oracle 近似为分类器:

$$\hat{o}(c) = \arg \max_{t \in T} \text{ScoreTrans}(c, t; \theta) \quad (8.6)$$

分类器可选 perceptron、对数线性、SVM、FFN (Chen & Manning 2014)、BiLSTM (Kiperwasser & Goldberg 2016) 等。

8.3.1.6 解码: 贪心 vs Beam

- **贪心**: 每步取 $\arg \max$, 速度极快, 但一旦出错难以挽回 (无回溯);
- **Beam Search**: 保留 top- k 部分解析序列, 按累计分数排序; 常配合全局训练 (如 max-violation perceptron, Zhang & Clark 2008) 以匹配解码目标。

[WARN] 贪心与 garden-path

人类阅读「The old man the boats.»「The horse raced past the barn fell.» 会被引入死胡同 (garden-path), 需要重解析。机器贪心也同理——这正是 beam search 的动机。

8.3.2 Eisner 算法（基于 DP 的投射依存解析）

8.3.2.1 朴素 DP 的问题

若直接在「子树」上 DP，时间复杂度为 $O(n^5)$ ：每个区间 $[i, j]$ 、每个 head 位置 h 都要枚举切分点 k 。

Eisner (1996) 的关键洞见：把子树拆为「半子树 (half-tree)」，保证 head 在区间一端，从而 head 维度消失，复杂度降到 $O(n^3)$ 。

8.3.2.2 Eisner 的两类 item

对区间 $[i, j]$ 和方向 $d \in \{\leftarrow, \rightarrow\}$ 定义：

Item	记号	含义
不完整三角形 (incomplete)	$I[i, j, d]$	区间 $[i, j]$ 内已有 head 在端点 i (若 $d = \rightarrow$) 或 j (若 $d = \leftarrow$)，但尚未接完所有依赖
完整三角形 (complete)	$C[i, j, d]$	区间 $[i, j]$ 已是完整子树，head 在端点 i 或 j

8.3.2.3 递推方程

定义 $s(h, m)$ 为弧 $h \rightarrow m$ 的分数。基例：

$$C[i, i, \rightarrow] = C[i, i, \leftarrow] = 0 \quad (8.7)$$

递推（区间长度 $\ell = j - i$ 从 1 到 n ）：

1. 合并两个完整三角形 $\rightarrow$ 不完整三角形（加一条弧）：

$$I[i, j, \rightarrow] = \max_{i \leq k < j} \left(C[i, k, \rightarrow] + C[k + 1, j, \leftarrow] + s(i, j) \right) \quad (8.8)$$

$$I[i, j, \leftarrow] = \max_{i \leq k < j} \left(C[i, k, \leftarrow] + C[k + 1, j, \rightarrow] + s(j, i) \right) \quad (8.9)$$

2. 不完整三角形 + 完整三角形 $\rightarrow$ 完整三角形（接完一侧依赖）：

$$C[i, j, \rightarrow] = \max_{i < k \leq j} \left(I[i, k, \rightarrow] + C[k, j, \rightarrow] \right) \quad (8.10)$$

$$C[i, j, \leftarrow] = \max_{i < k \leq j} \left(C[i, k, \leftarrow] + I[k, j, \leftarrow] \right) \quad (8.11)$$

最终答案：

$$\text{best} = C[0, n, \rightarrow] \quad (8.12)$$

8.3.2.4 复杂度

- Item 数: $O(n^2)$ 个区间 $\times$ 2 方向 $\times$ 2 类型 $= O(n^2)$;
- 每个 item 的转移: $O(n)$ 个切分点 k ;
- 总时间: $O(n^3)$, 空间 $O(n^2)$ 。

8.3.2.5 与 CKY 的对比

	CKY (CFG)	Eisner (依存)
Item	$[A, i, j]$, 非终结符 A	$C/I[i, j, d]$, 方向 d
切分	$A \rightarrow BC$, 找 k	head 在端点, 找 k
复杂度	$O(n^3 \cdot G)$	$O(n^3)$

8.3.3 Graph-Based 解析与 MST

8.3.3.1 建模

为每条可能的弧 $h \rightarrow m$ 打分 $s(h, m)$ 。一阶模型假设弧之间独立:

$$\text{Score}(x, y) = \sum_{(h, m) \in y} s(h, m) \quad (8.13)$$

寻找 $y^* = \arg \max$ 等价于在完全图上找一棵以 ROOT 为根的最大生成有向树 (MST)。

8.3.3.2 Chu-Liu-Edmonds 算法

1. 每个非根节点 m 选入度最大的弧 $h^* = \arg \max_h s(h, m)$;
2. 若得到一棵树, 结束;
3. 若有环 C :
 - 收缩 C 为单个节点 v_C ;
 - 重定义 C 与外部的弧权 (减去环内被替换弧的权值);
 - 递归求 MST;
 - 展开环: 在环 C 中唯一断开一条弧 (被外部入弧替换的位置)。

复杂度: 朴素 $O(n^3)$, Tarjan 改进 $O(n^2)$ (稠密图) / $O(m \log n)$ (稀疏图)。

8.3.3.3 优势

- 直接处理非投射依存;
- 全局最优, 不受贪心误差累积影响;
- 与神经打分器 (biaffine attention, Dozat & Manning 2017) 结合, 性能 SOTA。

8.4 评测

把依存解析视为「对每个词预测其 head + 关系标签」。两个指标:

$$UAS = \frac{\#\{\text{词被分配了正确 head}\}}{\#\{\text{词}\}} \quad (8.14)$$

$$LAS = \frac{\#\{\text{词被分配了正确 head} \wedge \text{正确关系}\}}{\#\{\text{词}\}} \quad (8.15)$$

- **UAS** (Unlabeled Attachment Score): 只看 head;
- **LAS** (Labeled Attachment Score): head + label 都正确。一般 $LAS < UAS$ 。
- **Exact Match**: 整句完全正确的比例, 更严格。

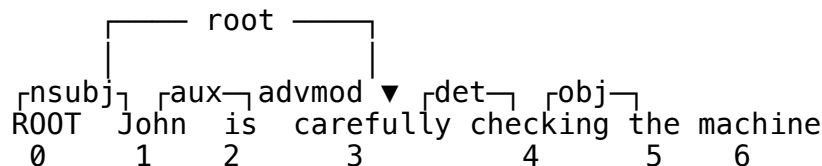
[NOTE] 标点

评测时常排除标点 (“excluding punctuation”), 因为标点的 head 经常约定不一、信号弱。报告分数时要写明。

8.5 Worked Example: Arc-Standard Trace

例句: John is carefully checking the machine (加虚根 ROOT)。

金标准依存:



步骤	转移	Stack (左 → 右栈底 → 栈顶)	Buffer	新弧
0	init	[ROOT]	[John, is, carefully, checking, the, machine]	—
1	SHIFT	[ROOT, John]	[is, carefully, checking, the, machine]	
2	SHIFT	[ROOT, John, is]	[carefully, checking, the, machine]	
3	SHIFT	[ROOT, John, is, carefully]	[checking, the, machine]	
4	LEFT-ARC _{advmod}	[ROOT, John, is]	[checking, the, machine]	checking → carefully (advmod)
5	LEFT-ARC _{aux}	[ROOT, John]	[checking, the, machine]	checking → is (aux)
6	LEFT-ARC _{nsubj}	[ROOT]	[checking, the, machine]	checking → John (nsubj)
7	SHIFT	[ROOT, checking]	[the, machine]	
8	SHIFT	[ROOT, checking, the]	[machine]	
9	LEFT-ARC _{det}	[ROOT, checking]	[machine]	machine → the (det)
10	RIGHT-ARC _{obj}	[ROOT, checking]	[machine]→pop	checking → machine (obj)

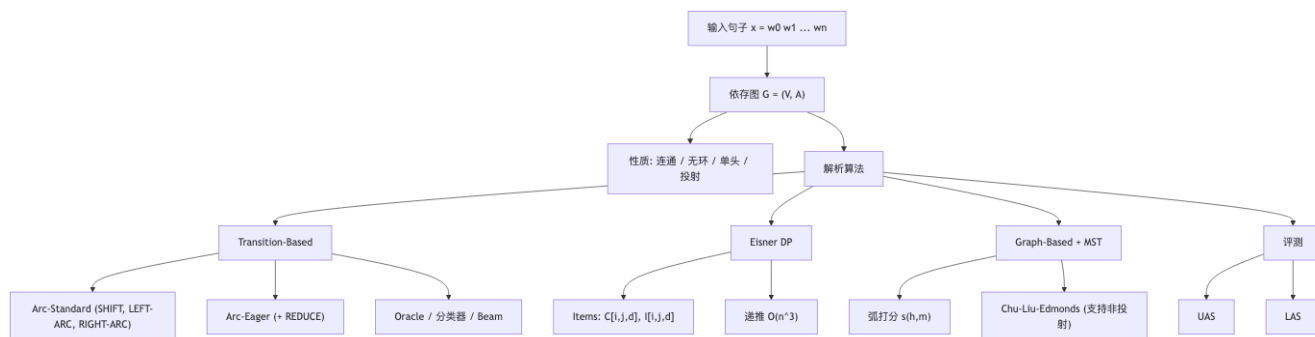
步骤	转移	Stack (左 → 右栈 底 → 栈顶)	Buffer	新弧
11	RIGHT-ARC _{root}	[ROOT]	[]	ROOT → checking (root)

总转移数: $2n = 2 \times 6 = 12$ (与 n 个 SHIFT + n 条弧吻合)。

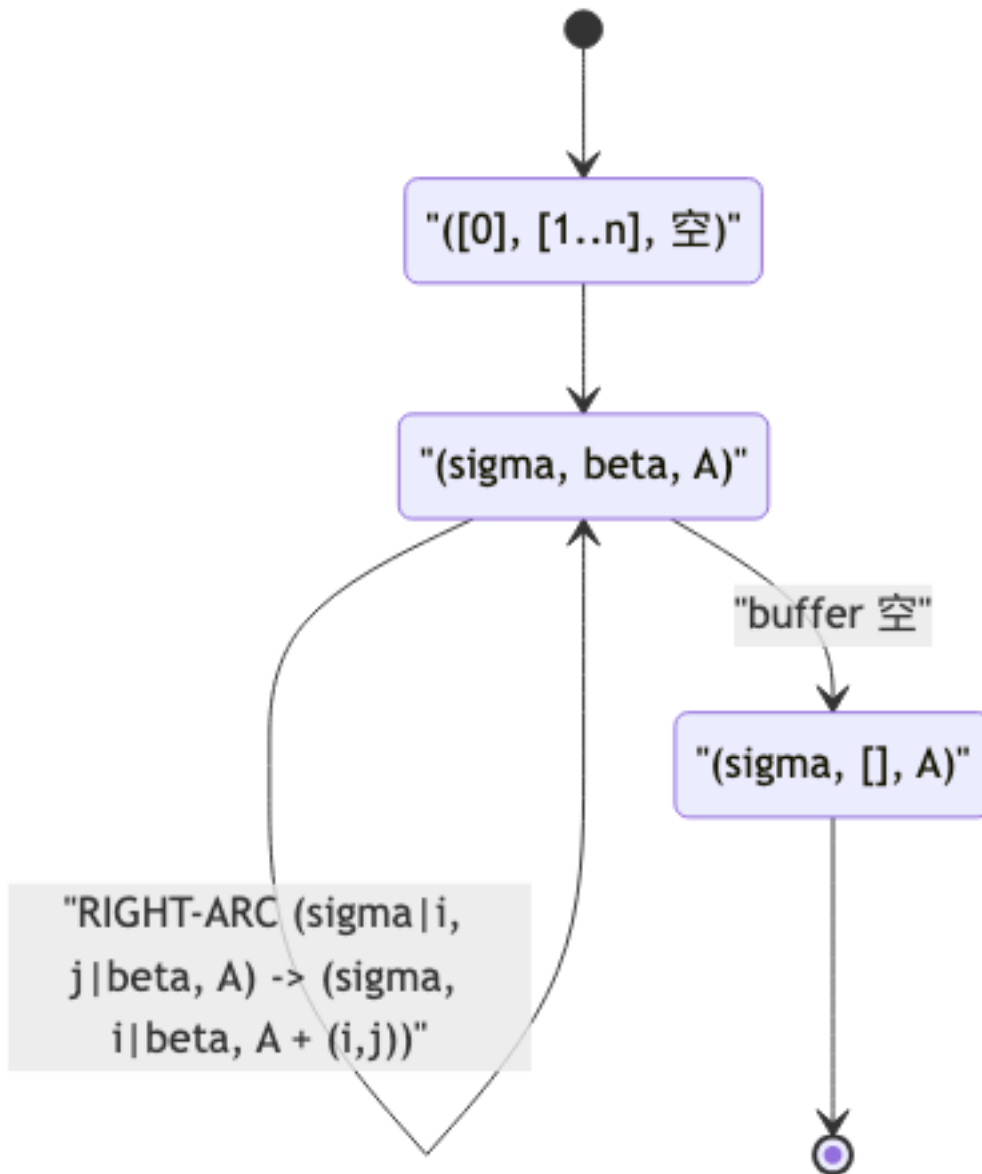
[WARN] LEFT-ARC 的方向陷阱

容易写错方向: LEFT-ARC _{ℓ} 在配置 $(\sigma \mid i, j \mid \beta)$ 上加弧 (j, i, ℓ) (缓存首 j 是 head, 栈顶 i 是 dependent)。「LEFT」指弧从右指向左。

8.6 概念关系图

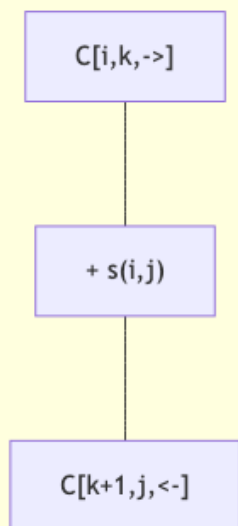


8.6.1 Arc-Standard 转移状态图

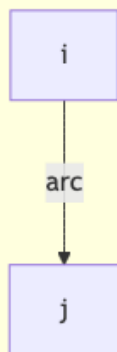


8.6.2 Eisner DP 的 span 结构

合并: $I[i,j,->] = \max_k C[i,k,->] + C[k+1,j,<-] + s(i,j)$



$I[i,j,->]$: 不完整三角形, head=i, 仅一条弧 $i \rightarrow j$



$C[i,j,->]$: 完整三角形, head=i



8.7 易错点 & 与同类对比

8.7.1 常见错误

1. LEFT/RIGHT-ARC 方向：永远是「指向 dependent」。LEFT-ARC = 弧指向左侧的栈顶；RIGHT-ARC = 弧指向右侧的缓存首。
2. arc-standard 没有 REDUCE，arc-eager 才有。
3. 投射性只是「投射树」算法的前提；UD 树本身可以非投射。
4. Eisner 不能直接处理非投射；非投射要 MST。
5. UAS / LAS 计算：分母是「词数」（通常排除标点），不是「弧数」也不是「句数」。
6. Eisner 的 head 必须在 span 端点，这是降复杂度的关键——错记为「head 在中间」会得到 $O(n^5)$ 。

8.7.2 三类算法对比

维度	Transition-Based	Eisner DP	Graph-Based MST
复杂度	$O(n)$ 贪心； $O(nk)$ beam	$O(n^3)$	$O(n^2)$ (CLE)
投射性	默认投射 (arc-standard)；可改造	仅投射	支持非投射
全局最优	贪心否；beam 近似；全局训练可达	是	是
错误传播	有 (贪心)	无	无
特征灵活度	高 (结合丰富特征)	受 DP 限制	一阶最容易；高阶难
解析速度	极快	中	慢

8.7.3 与短语句法的关系

- 短语结构 → 依存结构：用 head-percolation rules 把每条 CFG 规则指定一个 head，再「塌缩」得到依存树 (Penn Treebank → dependency)。
- 依存结构 → 短语结构：信息不全 (短语标签丢失)。

8.8 中英术语对照

中文	英文
依存结构	Dependency Structure
依存图	Dependency Graph
中心词 / 头	Head / Governor / Superior
修饰词 / 依赖词	Dependent / Modifier / Inferior
依存关系	Dependency Relation / Grammatical Relation
投射性	Projectivity
非投射	Non-projective
普适依存	Universal Dependencies (UD)
转移系统	Transition System
配置 (状态)	Configuration
栈 / 缓存	Stack / Buffer
弧标准	Arc-Standard
弧渴求	Arc-Eager
移进	SHIFT
左弧 / 右弧	LEFT-ARC / RIGHT-ARC

中文	英文
归约	REDUCE
神谕	Oracle
贪心搜索	Greedy Search
柱搜索	Beam Search
最大生成树	Maximum Spanning Tree (MST)
Chu—Liu—Edmonds	Chu—Liu—Edmonds Algorithm
未标注依附得分	Unlabeled Attachment Score (UAS)
已标注依附得分	Labeled Attachment Score (LAS)
引诱句	Garden—path Sentence
增量解析	Incremental Parsing
谓词—论元结构	Predicate—Argument Structure
结构依赖性	Structure Dependence

[TIP] 期中复习要点

1. **依存图四性质**：连通 / 无环 / 单头 / 投射；UD 不要求投射。
2. **arc-standard 三转移**：SHIFT, LEFT-ARC_ℓ, RIGHT-ARC_ℓ；写出形式化定义并能在给定句子上写出 trace。
3. **arc-eager**：多了 REDUCE, RIGHT-ARC 后栈顶保留缓存首；适合真正的增量解析。
4. **Eisner DP**：item $C[i, j, d]$ 与 $I[i, j, d]$ 、递推方程 (8.8)–(8.11)、复杂度 $O(n^3)$ 、为什么 head 必须在 span 端点。
5. **Chu—Liu—Edmonds**：1. 每个节点选最大入弧；2. 有环则收缩；3. 递归；可处理非投射。
6. **评测**：UAS (只 head)、LAS (head + label)、排除标点的惯例。
7. **错误传播**：贪心 vs beam；Oracle 训练 vs dynamic oracle。
8. **会写**：给定一句话，手工跑 arc-standard trace；判断给定依存树是否投射；写出 Eisner 的两类 item 与递推。

Lecture 9: 意义表示与组合语义 (Meaning Representation and Compositional Semantics)

[TIP] 学习指南

本讲把语义学从「分布式词向量」推进到句子级别的符号化意义表示。核心思路：Frege 组合性原理——整体的意义是部分意义和组合规则的函数。掌握三块内容：1. 形式语义学 (Formal Semantics)：Montague 传统，将自然语言视为可与逻辑等价处理的形式系统；Wittgenstein 的「意义即真值条件」(Meaning as Truth Conditions)。2. 一阶逻辑 (FOL) + λ 演算：常元、变元、谓词、函数、连接词、量词、 λ 抽象与 β -归约；事件具体化 (reifying events) 以表达副词修饰与论元角色。3. 句法驱动语义分析：CFG 规则配套语义附加 (semantic attachment)，通过 λ -application 自底向上组合；广义量词 (generalized quantifier) 与类型提升 (type raising) 处理「Every kid cries」一类问题。

期中考点：- 把英文句子翻译为 FOL / 带 λ 的 MR。- 写出某棵 CFG 树各节点的 .sem，演示完整 β -归约过程。- 解释为什么「Every kid cries」不能用 $VP.sem(NP.sem)$ ，并写出正确的 NP 类型提升。- 解释存在量词配合 $\wedge$ 而非 $\Rightarrow$ 的语义理由。- 用事件具体化把 John gave Mary a book on Tuesday 转 MR，并演示蕴含 (entailment)。

注：本节主题为「Meaning Representation」，而非统计机器翻译。请勿混淆 Lecture 编号。

9.1 Motivation：从词向量到句子意义

9.1.1 分布式语义的局限

分布式语义 (distributional semantics, Lec 6-7) 已经能很好地表示词，但在两类问题上仍困难：

1. 组合性 (compositionality)：已知 v_{apple} , v_{red} , v_{yellow} ，如何得到 red apple, yellow apple, ... 的向量？简单相加 / 相乘都会丢失语序、否定、量词等结构信息。

2. 推理 (inference)：

Q: Did Poland reduce its carbon emissions since 1989? Docs: Due to the collapse of the industrial sector after 1989, all countries in Central Europe saw a fall in carbon emissions. ... Poland is a country in Central Europe.

要回答这个问题，机器需要跨句子整合事实并执行三段论式推理：

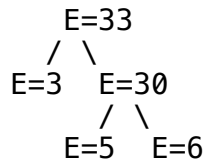
- $\forall x. \text{CentralEurope}(x) \Rightarrow \text{FellEmissions}(x)$
- $\text{CentralEurope}(\text{Poland})$
- $\therefore \text{FellEmissions}(\text{Poland})$

向量无法直接支持这种符号化、可验证的推理。

9.1.2 算术类比： $3 + 5 \times 6$

“What is the meaning of $3 + 5 \times 6$?”

把表达式解析为树并自底向上赋值：



若把 x 视为变量，无法在解析时求值——节点的「意义」不再是数值，而是一段代码 / 一个函数：

$$E_{\text{root}} = \text{add}(3, \text{mult}(5, x)) \quad (9.1)$$

这正是用 λ 表达式 表示「待应用的函数」的雏形。

9.2 形式语义学 (Formal Semantics)

9.2.1 Montague 的纲领

“There is in my opinion no important theoretical difference between natural languages and the artificial languages of logicians...”——R. Montague (1970)

形式语义学 (Montague Grammar) 主张：可以用同一套数学上精确的理论处理自然语言与形式语言的句法和语义。

9.2.2 Wittgenstein 的真值条件观

“To know the meaning of a sentence is to know how the world would have to be for the sentence to be true.”——L. Wittgenstein

「意义」= 「真值条件 (truth conditions)」。词与短语的意义即是其对整句真值条件的贡献。

例：

$$\text{Every student works} = \text{true} \iff \text{student} \subseteq \text{work} \quad (9.2)$$

$$\text{Every student works} \Rightarrow \forall x. \text{stud}'(x) \rightarrow \text{work}'(x) \quad (9.3)$$

9.2.3 「理解」即翻译

系统	输入	输出
编译器	形式语言	形式语言
自然语言理解	自然语言	自然语言 / 形式语言 ★

第二种「翻译为形式语言」的方向，正是语义解析 (semantic parsing) 的核心任务。

9.2.4 意义表示语言 (MRL) 的期望

属性	含义
Compositional 组合性	复杂表达式的意义是其组成部分意义和组合规则的函数
Verifiable 可验证	给定一个世界模型/KB, 可判定 MR 真假
Unambiguous 无歧义	一个 MR 只有一种解读; 歧义句应有多个 MR
Canonical Form 范式	同义句应有相同 MR (被动 / 主动)
Inference 可推理	不仅能查 KB, 还能推出新事实
Expressivity 表达力	覆盖广泛的语义现象

[NOTE] 歧义 vs 模糊

time flies like an arrow — 多种解读 → **歧义 (ambiguity)**, 需多个 MR; I want to eat Italian food — 不指定具体哪家、哪种菜 → **模糊 (vagueness)**, 单一 MR 足矣。

9.3 一阶逻辑 (First-Order Logic, FOL)

9.3.1 基本组件

类别	例	说明
常元 (constants)	Beijing, China, 2021	唯一指代实体; 同一实体可有多名 (Venus $\equiv$ The Evening Star)
变元 (variables)	x, y, z	在实体上取值, 必须被量词约束才有真值
函数符号 (functions)	president(EU), father(John)	间接指代唯一实体; 句法像一元谓词但返回 实体
谓词符号 (predicates)	nation(China), likes(John, Marie)	表示性质 / 关系; arity N 即取 N 个参数
连接词	$\neg, \wedge, \vee, \Rightarrow$	命题连接
量词	$\forall, \exists$	约束变元

9.3.2 谓词的语义

arity 为 N 的谓词 P/N 在世界模型中**指称**一个 N -元组的集合:

$$\llbracket P \rrbracket = \{(a_1, \dots, a_N) \mid P(a_1, \dots, a_N) \text{ 在该模型中为真}\} \quad (9.4)$$

闭式公式 (所有变元被绑定) 有真值; 含自由变元的公式被解读为「集合」:

$$\text{likes}(x, \text{Kim}) \quad \rightsquigarrow \quad \{x \mid x \text{ likes Kim}\} \quad (9.5)$$

9.3.3 量词

全称量词 $\forall$:

Cats are mammals. $\forall x. \text{cat}(x) \Rightarrow \text{mammal}(x)$

存在量词 $\exists$:

Marie owns a cat. $\exists x. \text{cat}(x) \wedge \text{owns}(\text{Marie}, x)$

[WARN] 为什么 $\exists$ 配 $\wedge$ 而非 $\rightarrow$?

比较: $\neg \exists x. \text{cat}(x) \wedge \text{owns}(\text{Marie}, x)$: 「存在一个东西, 它是猫且 Marie 拥有它。」——正确含义。
 $\exists x. \text{cat}(x) \Rightarrow \text{owns}(\text{Marie}, x)$: 「存在一个东西, 如果它是猫, 则 Marie 拥有它。」——任何非猫实体 (如一块石头) 都使蕴含真, 公式总是真, 信息量为零。

类似地, $\forall$ 几乎总是配 $\Rightarrow$, 否则会断言「世界上所有实体都是猫」。

9.3.4 事件具体化 (Reifying Events / Davidsonian Semantics)

为了表达副词、时间、地点等修饰语, 把事件本身具体化为变量 e :

$$\text{I have a car} \Rightarrow \exists e, y. \text{Having}(e) \wedge \text{Haver}(e, \text{Speaker}) \wedge \text{HadThing}(e, y) \wedge \text{Car}(y) \quad (9.6)$$

$$\text{John gave Mary a book} \Rightarrow \exists e, z. \text{Giving}(e) \wedge \text{Giver}(e, J) \wedge \text{Givee}(e, M) \wedge \text{Given}(e, z) \wedge \text{Book}(z) \quad (9.7)$$

$$\text{John gave Mary a book on Tuesday} \Rightarrow \exists e, z. \text{Giving}(e) \wedge \text{Giver}(e, J) \wedge \text{Givee}(e, M) \wedge \text{Given}(e, z) \wedge \text{Book}(z) \wedge \text{Time}(e, \text{Tuesday}) \quad (9.8)$$

蕴含关系 (Entailment)

(9.8) $\Rightarrow$ (9.7): 移除一个合取项后真值条件更宽松。形式化地:

$$A \Rightarrow B \iff \text{每一个使 } A \text{ 为真的世界也使 } B \text{ 为真} \quad (9.9)$$

这正是「事件具体化 + 合取」带来的福利: 副词、时间、地点都成为可独立删除的合取项, 单调推理直接成立。

9.4 λ 演算 (Lambda Calculus)

9.4.1 动机: 组合时的「待定参数」

我们需要在 CFG 自底向上组合时, 允许某些参数暂未提供。 λ 表达式 = 一个匿名函数:

$$\lambda x. \text{run}(x) \quad (9.10)$$

读作「把实体 x 映射到 $\text{run}(x)$ 的函数」。

9.4.2 β -归约

$$(\lambda x. \text{run}(x))(\text{Jane}) \xrightarrow{\beta} \text{run}(\text{Jane}) \quad (9.11)$$

规则: 把形参替换为实参, 去掉 λ 。

9.4.3 嵌套 λ 与柯里化

$$\lambda y. \lambda x. \text{love}(x, y) \quad (9.12)$$

依次应用：

$$\begin{aligned} (\lambda y. \lambda x. \text{love}(x, y))(\text{pizza}) &\xrightarrow{\beta} \lambda x. \text{love}(x, \text{pizza}) \\ (\lambda x. \text{love}(x, \text{pizza}))(\text{Marie}) &\xrightarrow{\beta} \text{love}(\text{Marie}, \text{pizza}) \end{aligned} \quad (9.13)$$

[NOTE] α -转换与变量捕获

在 β -归约前要做 **α -转换 (α -conversion)** 重命名绑定变量，避免「变量捕获」： $(\lambda x. \lambda y. f(x, y))(y) \xrightarrow{\alpha} (\lambda x. \lambda z. f(x, z))(y) \xrightarrow{\beta} \lambda z. f(y, z)$ 。

9.4.4 λ -DCS (扩展)

Liang (2011) 的 **Lambda Dependency-based Compositional Semantics** 是 λ 演算的轻量替代，支持 entity、relation、Join、Intersection、aggregation (argmax/count) 等，常用于 Freebase 问答 (Berant et al., 2013)。

9.5 句法驱动的语义分析 (Syntax-Driven Semantic Analysis)

9.5.1 组合性原理 (Frege / Partee)

“The meaning of an expression is a function of the meanings of its parts and of the way they are syntactically combined.”——B. Partee

9.5.2 朴素方案：Rule-to-Rule 翻译

为每条 CFG 规则附带一个语义构造规则：

$A \rightarrow \alpha_1 \alpha_2 \dots \alpha_n \quad \{ f(\alpha_1.\text{sem}, \dots, \alpha_n.\text{sem}) \}$

即 $A.\text{sem}$ 由其孩子的 sem 通过函数 f 计算。

例：Mary likes Bill

节点	规则	.sem
ProperNoun $\rightarrow$ Mary		mary'
ProperNoun $\rightarrow$ Bill		bill'
NP $\rightarrow$ ProperNoun	unary copy	NP.sem = ProperNoun.sem
V $\rightarrow$ likes		$\lambda y. \lambda x. \text{like}'(x, y)$
VP $\rightarrow$ V NP	V.sem(NP.sem)	$\lambda x. \text{like}'(x, \text{bill}')$
S $\rightarrow$ NP VP	VP.sem(NP.sem)	$\text{like}'(\text{mary}', \text{bill}')$

9.5.3 实例：AyCaramba serves meat

带事件具体化的词汇语义：

$$\text{Verb} \rightarrow \text{serves} : \lambda y. \lambda x. \exists e. \text{Serving}(e) \wedge \text{Server}(e, x) \wedge \text{Served}(e, y) \quad (9.14)$$

组合 VP：

$$\text{VP.sem} = (\lambda y. \lambda x. \exists e. \text{Serving}(e) \wedge \text{Server}(e, x) \wedge \text{Served}(e, y))(\text{Meat}) \xrightarrow{\beta} \lambda x. \exists e. \text{Serving}(e) \wedge \text{Server}(e, x) \wedge \text{Served}(e, \text{Meat}) \quad (9.15)$$

组合 S (规则 $S \rightarrow \text{NP VP } \{\text{VP.sem}(\text{NP.sem})\}$, $\text{NP.sem} = \text{AyCaramba}$):

$$\text{S.sem} = \exists e. \text{Serving}(e) \wedge \text{Server}(e, \text{AyCaramba}) \wedge \text{Served}(e, \text{Meat}) \quad (9.16)$$

9.5.4 量词的难题：Every kid cries

目标 MR：

$$\forall x. \text{Kid}(x) \Rightarrow \exists e. \text{Crying}(e) \wedge \text{Crier}(e, x) \quad (9.17)$$

困难：Every kid 的意义形如 $\forall x. \text{Kid}(x) \Rightarrow Q(x)$ ，其中 Q 是「待填的谓词」；而 cries 的意义是 $\lambda y. \exists e. \text{Crying}(e) \wedge \text{Crier}(e, y)$ 。

如果按 $\text{S.sem} = \text{VP.sem}(\text{NP.sem})$ ：

$$(\lambda y. \exists e. \text{Crying}(e) \wedge \text{Crier}(e, y))(\forall x. \text{Kid}(x) \Rightarrow Q(x))$$

把整个公式塞进 Crier 的位置——这不是合法 FOL，谓词的参数必须是项 (term)，不能是公式。

9.5.5 解决：广义量词 + 类型提升

把规则反过来： $\text{S.sem} = \text{NP.sem}(\text{VP.sem})$ ，让 NP 成为高阶函数 (functor)：

$$\text{Every kid} \rightsquigarrow \lambda Q. \forall x. \text{Kid}(x) \Rightarrow Q(x) \quad (9.18)$$

应用到 VP.sem：

$$\begin{aligned} & (\lambda Q. \forall x. \text{Kid}(x) \Rightarrow Q(x)) (\lambda y. \exists e. \text{Crying}(e) \wedge \text{Crier}(e, y)) \\ & \xrightarrow{\beta} \forall x. \text{Kid}(x) \Rightarrow (\lambda y. \exists e. \text{Crying}(e) \wedge \text{Crier}(e, y))(x) \\ & \xrightarrow{\beta} \forall x. \text{Kid}(x) \Rightarrow \exists e. \text{Crying}(e) \wedge \text{Crier}(e, x) \end{aligned} \quad (9.19)$$

9.5.6 Det / Noun 规则

把 Det 当作「输入两个谓词 P, Q 返回量化公式」的二阶函数：

$$\begin{aligned}
 \text{Noun} \rightarrow \text{Kid} & : \lambda x. \text{Kid}(x) \\
 \text{Det} \rightarrow \text{Every} & : \lambda P. \lambda Q. \forall x. P(x) \Rightarrow Q(x) \\
 \text{Det} \rightarrow \text{A} & : \lambda P. \lambda Q. \exists x. P(x) \wedge Q(x) \\
 \text{Det} \rightarrow \text{No} & : \lambda P. \lambda Q. \neg \exists x. P(x) \wedge Q(x) \\
 \text{NP} \rightarrow \text{Det Noun} & : \text{Det.sem}(\text{Noun.sem})
 \end{aligned} \tag{9.20}$$

推导 Every kid 的 NP.sem:

$$\begin{aligned}
 & (\lambda P. \lambda Q. \forall x. P(x) \Rightarrow Q(x))(\lambda x. \text{Kid}(x)) \\
 & \xrightarrow{\beta} \lambda Q. \forall x. (\lambda x. \text{Kid}(x))(x) \Rightarrow Q(x) \\
 & \xrightarrow{\beta} \lambda Q. \forall x. \text{Kid}(x) \Rightarrow Q(x)
 \end{aligned} \tag{9.21}$$

9.5.7 专有名词的类型提升

为了让 S 的统一规则 $S.\text{sem} = \text{NP.sem}(\text{VP.sem})$ 对专有名词也成立，必须把 Kate 提升为「吃 VP 吐公式」的高阶函数：

$$\text{ProperNoun} \rightarrow \text{Kate} : \lambda P. P(\text{Kate}) \tag{9.22}$$

推导 Kate cries:

$$\begin{aligned}
 & (\lambda P. P(\text{Kate}))(\lambda y. \exists e. \text{Crying}(e) \wedge \text{Crier}(e, y)) \\
 & \xrightarrow{\beta} (\lambda y. \exists e. \text{Crying}(e) \wedge \text{Crier}(e, y))(\text{Kate}) \\
 & \xrightarrow{\beta} \exists e. \text{Crying}(e) \wedge \text{Crier}(e, \text{Kate})
 \end{aligned} \tag{9.23}$$

这就是**类型提升 (type raising)**：把实体 a 提升为 $\lambda P. P(a)$ ，类型从 e 变成 $(e \rightarrow t) \rightarrow t$ 。

9.5.8 一个玩具语法

$$\begin{array}{ll}
 S \rightarrow \text{NP VP} & \{ \text{NP.sem}(\text{VP.sem}) \} \\
 \text{VP} \rightarrow \text{Verb} & \{ \text{Verb.sem} \} \\
 \text{VP} \rightarrow \text{Verb NP} & \{ \text{Verb.sem}(\text{NP.sem}) \} \\
 \text{NP} \rightarrow \text{Det Noun} & \{ \text{Det.sem}(\text{Noun.sem}) \} \\
 \text{NP} \rightarrow \text{ProperNoun} & \{ \text{ProperNoun.sem} \} \\
 \text{Det} \rightarrow \text{Every} & \{ \lambda P. \lambda Q. \forall x. P(x) \Rightarrow Q(x) \} \\
 \text{Det} \rightarrow \text{A} & \{ \lambda P. \lambda Q. \exists x. P(x) \wedge Q(x) \} \\
 \text{Noun} \rightarrow \text{Kid} & \{ \lambda x. \text{Kid}(x) \} \\
 \text{ProperNoun} \rightarrow \text{Kate} & \{ \lambda P. P(\text{Kate}) \} \\
 \text{Verb} \rightarrow \text{cries} & \{ \lambda x. \exists e. \text{Crying}(e) \wedge \text{Crier}(e, x) \} \\
 \text{Verb} \rightarrow \text{serves} & \{ \lambda y. \lambda x. \exists e. \text{Serving}(e) \wedge \text{Server}(e, x) \wedge \text{Served}(e, y) \}
 \end{array}$$

9.5.9 集成 vs 流水线

方案	优点	缺点
集成 (Integrated): 句法 + 语义同步构建	高效; 无错误传播; 可借语义剪枝	实现复杂; 句法器需重写
流水线 (Pipeline): 先句法分析, 再自底向上算 .sem	模块化; 易实现	错误传播; 需手工处理大量例外

9.6 「看上去 trivial」实际并非：组合性的反例

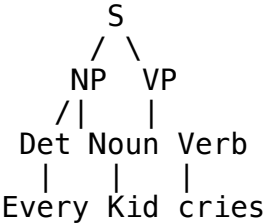
现象	例	难点
代词的语义贡献不均	It barked $\rightarrow \exists x. \text{bark}'(x) \wedge \text{PRON}(x)$; It rained $\rightarrow \text{rain}'$ (it 为 pleonastic)	同样的 it, 语义贡献不同
同结构异义 / 异结构同义	Kim seems to sleep $\equiv$ It seems that Kim sleeps	句法不同但真值条件相同
非组合短语 量词作用域	red tape = "bureaucracy" Every man loves a woman 有 $\forall x \exists y$ 与 $\exists y \forall x$ 两读	整体意义无法从部分推出 作用域歧义
否定与消极性 情态、内涵语境 时态、体貌	I haven't eaten anything yet John believes Mary left John has been running	NPI 需要否定环境 内涵语境, FOL 不够 需引入时间结构

[WARN] FOL 不是万能

FOL 无法直接处理：内涵性 (intensionality)、复数的非分配性 (collective predication)、模糊量词 (many, most)、情态 (may, must)、隐式参数等。后续课程会介绍 **AMR**、**neural semantic parsing**、**Text-to-SQL** 等替代方案。

9.7 Worked Examples

9.7.1 完整 trace: Every kid cries



节点	规则	.sem (化简后)
Det $\rightarrow$ Every	词项	$\lambda P. \lambda Q. \forall x. P(x) \Rightarrow Q(x)$
Noun $\rightarrow$ Kid	词项	$\lambda x. \text{Kid}(x)$
NP $\rightarrow$ Det Noun	$\text{Det.sem}(\text{Noun.sem})$	$\lambda Q. \forall x. \text{Kid}(x) \Rightarrow Q(x)$

节点	规则	.sem (化简后)
Verb $\rightarrow$ cries	词项	$\lambda y. \exists e. \text{Crying}(e) \wedge \text{Crier}(e, y)$
VP $\rightarrow$ Verb	unary	$\lambda y. \exists e. \text{Crying}(e) \wedge \text{Crier}(e, y)$
S $\rightarrow$ NP VP	NP.sem(VP.sem)	$\forall x. \text{Kid}(x) \Rightarrow \exists e. \text{Crying}(e) \wedge \text{Crier}(e, x)$

9.7.2 完整 trace: AyCaramba serves meat

节点	.sem
ProperNoun $\rightarrow$ AyCaramba	$\lambda P. P(\text{AyCaramba})$ (类型提升)
NP $\rightarrow$ ProperNoun	$\lambda P. P(\text{AyCaramba})$
MassNoun $\rightarrow$ meat	$\lambda P. P(\text{Meat})$
NP $\rightarrow$ MassNoun	$\lambda P. P(\text{Meat})$
Verb $\rightarrow$ serves	$\lambda y. \lambda x. \exists e. \text{Serving}(e) \wedge \text{Server}(e, x) \wedge \text{Served}(e, y)$
VP $\rightarrow$ Verb NP	$\text{NP}_{\text{obj}}.\text{sem}(\lambda y. \text{Verb.sem}(y))$ —实际推导见下

实际更简单的做法 (按 9.5.8 玩具语法 $\text{VP} \rightarrow \text{Verb NP} : \text{Verb.sem}(\text{NP.sem})$, 并让宾语 NP 不提升):

$$\text{VP.sem} = \text{Verb.sem}(\text{Meat}) = \lambda x. \exists e. \text{Serving}(e) \wedge \text{Server}(e, x) \wedge \text{Served}(e, \text{Meat})$$

$$\text{S.sem} = \text{NP.sem}(\text{VP.sem}) = (\lambda P. P(\text{AyCaramba}))(\text{VP.sem}) = \exists e. \text{Serving}(e) \wedge \text{Server}(e, \text{AyCaramba}) \wedge \text{Served}(e, \text{Meat})$$

9.7.3 推理示例

KB:

1. $\forall x. \text{cat}(x) \Rightarrow \text{mammal}(x)$ (猫是哺乳动物)
2. $\text{cat}(\text{Sam})$ (Sam 是猫)

由 **Universal Instantiation**: $\text{cat}(\text{Sam}) \Rightarrow \text{mammal}(\text{Sam})$ 。由 **Modus Ponens**: $\text{mammal}(\text{Sam})$ 。

故 KB 蕴含「Sam is a mammal」。

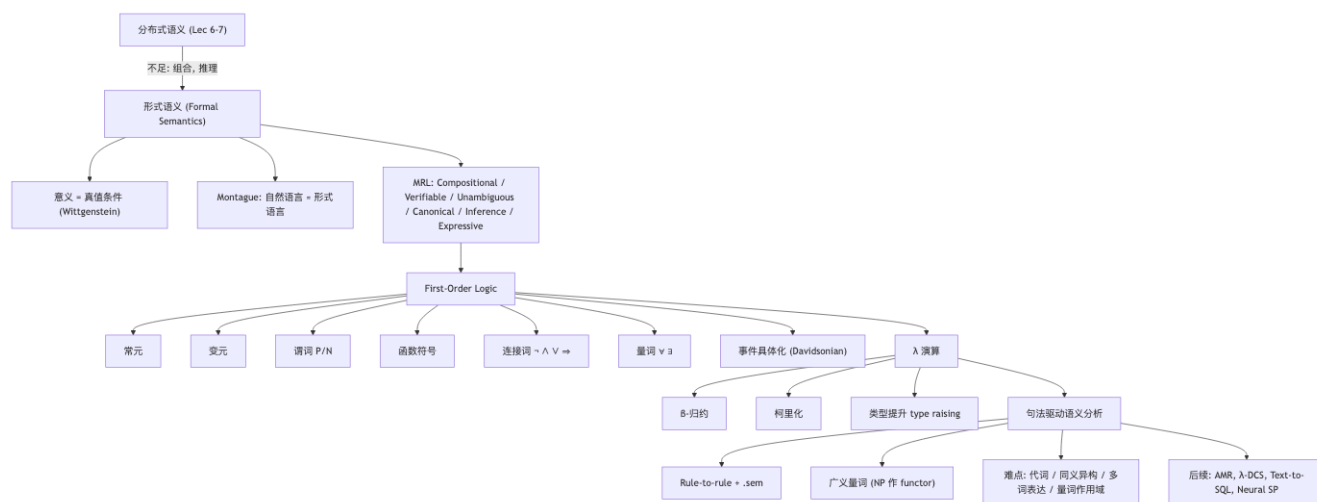
9.7.4 事件具体化下的蕴含

由 (9.8) 删除合取项 $\text{Time}(e, \text{Tue})$ 即得 (9.7):

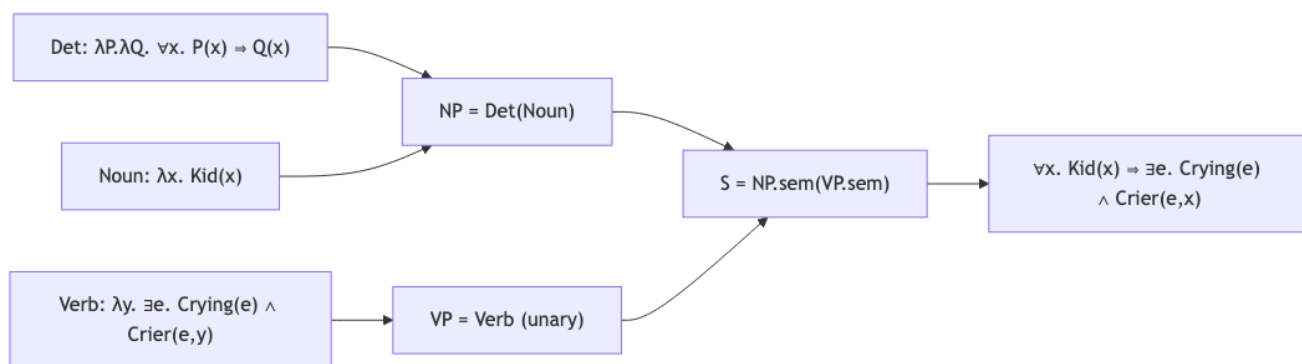
$$\text{John gave Mary a book on Tuesday} \models \text{John gave Mary a book} \quad (9.24)$$

这是 Davidson 事件语义的「免费午餐」。

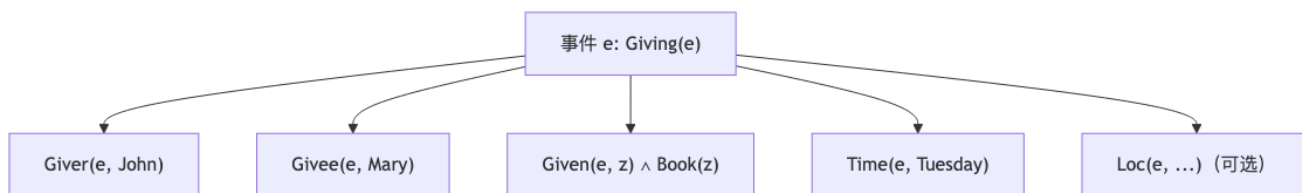
9.8 概念关系图



9.8.1 λ 演算的组合流程



9.8.2 事件具体化下的论元角色



9.9 易错点 & 与同类对比

9.9.1 易错点

1. $\exists$ 必配 $\wedge$; $\forall$ 必配 $\Rightarrow$ (见 9.3.3 警示)。
2. **S.sem = VP.sem(NP.sem)** 在量词 NP 上行不通，必须把 NP 类型提升为高阶函数 (9.5.5)。
3. 专有名词也要类型提升 ($\lambda P. P(Kate)$)，以保证组合规则统一。
4. 谓词参数必须是项 / 实体，不能是公式——这是 FOL 的「first-order」之名所在。
5. β -归约前要做 α -转换，避免变量捕获。

- 6. 歧义 vs 模糊：歧义需多个 MR；模糊一个 MR 即可。
- 7. 量词作用域：Every man loves a woman 有两读，FOL 单一推导无法捕捉，需要 **quantifier raising** 或 underspecified semantics（如 Hole Semantics, MRS）。
- 8. **canonical form**：被动句应与主动句有相同 MR (Beihua serves hotpot $\equiv$ Hotpot is served by Beihua)，这要求语义构造规则正确对齐论元角色。

9.9.2 各种意义表示对比

MR	表达力	可推理	组合性	典型应用
词向量	低（连续）	弱	弱（加和 / 张量）	检索、相似度
FOL	中	强（演绎、SAT 求解）	强	经典 QA、形式验证
λ -DCS	中	中	强	Freebase QA
AMR	中（图结构）	中	中	抽象语义、跨语言
SQL / SPARQL	高（针对 KB / DB）	极强（DB 引擎）	中	Text-to-SQL
自然语言（LLM）	极高	不形式可验证	隐式	通用对话

9.9.3 集成 vs 流水线

见 9.5.9：集成更高效但实现复杂；流水线易于工程但有错误传播。

9.10 中英术语对照

中文	英文
形式语义学	Formal Semantics
意义表示	Meaning Representation (MR)
意义表示语言	Meaning Representation Language (MRL)
组合性	Compositionality
真值条件	Truth Conditions
一阶逻辑	First-Order Logic (FOL)
谓词	Predicate
常元 / 变元	Constant / Variable
函数符号	Function Symbol
连接词	Logical Connective
量词	Quantifier
全称 / 存在量词	Universal / Existential Quantifier
λ 演算	Lambda Calculus
β -归约	β -reduction
α -转换	α -conversion
类型提升	Type Raising
广义量词	Generalized Quantifier
事件具体化	Reifying Events / Davidsonian Semantics
论元角色	Thematic Role / Argument Role
蕴含	Entailment
知识库	Knowledge Base (KB)
语义分析	Semantic Analysis / Semantic Parsing
句法驱动语义分析	Syntax-Driven Semantic Analysis
规则到规则翻译	Rule-to-Rule Translation
语义附加	Semantic Attachment

中文	英文
集成 / 流水线方案	Integrated / Pipeline Approach
多词表达	Multiword Expression (MWE)
多余主语 (it)	Pleonastic Pronoun
量词作用域	Quantifier Scope
抽象意义表示	Abstract Meaning Representation (AMR)
文本到 SQL	Text-to-SQL

[TIP] 期中复习要点

1. 意义 = 真值条件 (Wittgenstein) , 形式语义 = 把自然语言翻译为可验证的形式语言 (Montague) 。
2. MRL 六性质: 组合 / 可验证 / 无歧义 / 范式 / 可推理 / 表达力。
3. FOL 组件: 常元、变元、谓词 (arity $N \rightarrow N$ -元组集合)、函数符号、连接词、量词。 $\exists + \wedge, \forall + \Rightarrow$ 。
4. 事件具体化: 把动词建模为 $\exists e. \text{Event}(e) \wedge \text{Role}_1(e, \cdot) \wedge \dots$; 副词/时间作为额外合取项; 天然支持单调蕴含。
5. λ 演算: $\lambda x. \phi$ 、 β -归约、柯里化、 α -转换。
6. 组合性原理: CFG 规则 + 语义附加 + β -归约。
7. 量词难题与解决: $S \rightarrow \text{NP VP} : \text{NP.sem}(\text{VP.sem})$; Det 是二阶函数 $\lambda P. \lambda Q. \forall x. P(x) \Rightarrow Q(x)$; ProperNoun 类型提升 $\lambda P. P(a)$ 。
8. 易考推导: 手算 Every kid cries、Kate cries、AyCaramba serves meat 的 β -归约。
9. FOL 的局限: 内涵语境、模糊量词、量词作用域、时态体貌——为后续 AMR / Neural SP / LLM 铺垫。

Lecture 10 (II) —IR Embedding（向量检索）

[TIP] 学习指南

本节核心：把文本（query 与 document）编码到同一个向量空间，用向量相似度近似“语义相关性”。重点理清三件事：1. **双塔 (dual-encoder) 架构**——离线编码、在线只做向量运算；2. **对比训练 (contrastive learning) + 难负例采样**——为什么随机负例不够、Sparse Retrieval Negatives 与 Self Negatives (ANCE) 的对照；3. **近似最近邻 (ANNS)**——partition / hash / graph 三类方法如何把检索成本降到 sub-linear。
前置：- lec5_dist (distributional semantics、cosine similarity、word/sentence embedding) - lec10_ir (IR 任务定义、TF-IDF、BM25、Boolean / Vector Space / Probabilistic / Neural Model；DPR 的 in-batch negatives 是本讲的起点)
复习关注点：dual-encoder 的相关性函数、对比损失形式、为什么 corpus 99.9999% trivially irrelevant、ANCE 的“周期性刷新索引”思想、ANNS 三类方法各自代表算法。

1. 为什么需要 Embedding Learning

上一节 (lec10_ir) 走完了 Boolean → Vector Space → BM25 → Neural Model 的脉络，DPR (Dense Passage Retrieval) 让“用稠密向量做召回”成为可行方案。本讲把镜头拉近，专门讨论 **embedding learning** 这一独立的研究主题。

Embedding Learning：把文本（也可推广到用户、商品、图像等）编码到统一向量空间 $\mathbb{R}^h$ ，让“语义关系”对应“向量几何关系” (cosine / 内积)。

它的下游用途横跨 NLP 与推荐系统：

场景	由 embedding 捕获的语义
搜索 / RAG	query-document 相关性
推荐系统	用户兴趣与商品语义
聚类 / 去重	数据间相似性
多模态	跨模态对齐

1.1 三大动机：Efficiency First

课件强调，embedding 之所以被大规模工业系统采用，根本原因是 **efficiency**：

1. **Offline pre-computed embedding**：document 侧的向量可以离线一次性算好，写进磁盘 / 向量库。在线只需要算 query 向量。
2. **Quick similarity calculations**：检索归结为一次向量内积或 cosine，远比走神经网络对每个 (q, d) 配对打分便宜。
3. **Approximate Nearest Neighbor Search (ANNS)**：成熟的近似 KNN 算法把检索复杂度降到 sub-linear，可与倒排索引 (inverted index) 相提并论。

[NOTE] 工业流水线视角

Embedding-based 召回是当今搜索 / 推荐流水线的第一阶段 (“first stage”)。后面通常还有 re-ranking (精排), 用 cross-encoder 或更重的模型在召回的少量候选上再打分。Embedding 的目标是 **recall**, 把相关候选放进来; 精排负责 **precision**。

2. Formulation: Representation-centric 视角

回顾 IR 任务定义: 要计算 query q 与 document d 的相关性 $f(q, d)$ 。Embedding 方法的口号是:

Turn everything into vectors. Only simple vector operations afterwards.

也就是说, 我们只关心两个 **encoder**: query encoder E_q 和 document encoder E_d 。打分函数限制为一种简单的向量运算:

$$f(q, d) = E_q(q)^\top E_d(d). \quad (1)$$

这种把 f 限制成“向量内积”的设计被称为 **representation-centric**, 它牺牲了表达力 (query 与 document 没有任何 token-level 交互), 换来三件事:

- document 向量可离线算;
- query 上来只前向一次;
- 整个 corpus 可建 ANN 索引。

[WARN] 与 interaction-centric 的取舍

Interaction-centric 方法 (如 cross-encoder: 把 $[q; d]$ 一起塞进 BERT 做分类) 允许 token-level 注意力, 效果更好但**无法预先索引**——每来一个 q , 都要对 corpus 中每个 d 重新跑一次 BERT。本讲完全选 representation-centric 一侧。

3. 双塔架构 (Dual Encoders)

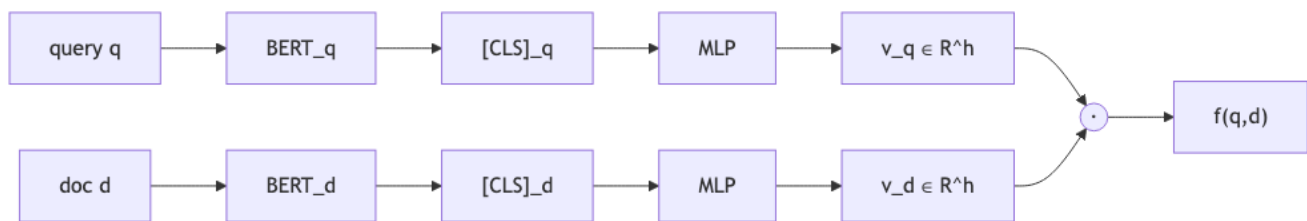
3.1 BERT 版本的实例

课件以 BERT 为骨干给出最简形式:

$$f(q, d) = \text{BERT}(q) \times \text{BERT}(d) = \text{MLP}\left([\vec{\text{CLS}}]_q\right) \times \text{MLP}\left([\vec{\text{CLS}}]_d\right). \quad (2)$$

实现要点:

1. 两个**独立** BERT (也可以共享参数 $\rightarrow$ “siamese”; DPR 用的是**两个独立**的 encoder, 因为 query 与 passage 分布不同)。
2. 取 [CLS] token 的最后一层 hidden state 作为整段文本的表示。
3. 后接一个 MLP 投影到检索空间 (可选, 但常用于降维和归一化)。
4. 相关性函数定为 **dot product** (或先 ℓ_2 -norm 再 dot product, 即 cosine)。



3.2 Inference 流程

```
# 离线：把 corpus 全部编码并建索引
for d in corpus:
    v_d = MLP(BERT_d(d).cls)      # 一次性
    index.add(v_d, doc_id=d.id)
index.build_ann()                # FAISS / HNSW 等

# 在线：每来一个 query
v_q = MLP(BERT_q(q).cls)
top_k = index.search(v_q, k=K)    # 近似 KNN
```

3.3 为什么 inference 仍然“很贵”？

课件特意问了一句 “Inference can be very expensive, why?”。原因有两层：

1. 语料规模：corpus 可能是百万 / 十亿 / 万亿级，朴素地与每个 v_d 算内积是 $O(|D| \cdot h)$ 。
2. 维度诅咒： h 通常 256–768，且要保证检索精度，精确 KNN 不实用。

这就引出下一节的 ANNS。

4. 近似最近邻搜索 (ANNS)

目标：以“略微牺牲 KNN 精度”为代价，把检索做到 **sub-linear** 时间。

ANNS 把 K -nearest 由精确变近似，三大类方法：

4.1 Partition-based

把向量空间划分成若干区域，查询时只搜部分区域。

- 代表算法：hierarchical K-means trees (先做 K-means 把空间分成 K 个 cluster，每个 cluster 再递归 K-means)。
- 查询：先决定 v_q 落入哪个/哪几个 cluster，只在这些 cluster 的成员里做精确 KNN。

4.2 Hash-based

用哈希函数把向量映射成离散 hash code，只检索“同 hash 桶”或“近邻 hash 桶”中的点。

- 代表算法：Locality Sensitive Hashing (LSH) ——满足“距离近的点 hash collision 概率高”。
- 多组 hash 函数 ensemble，召回率随 hash table 数量上升。

4.3 Graph-based

把 corpus 中每个点与其相似邻居用边连起来，查询时贪心地在图上走。

- 代表算法：K-nearest neighborhood graph（每个点与最近的 K 个点连边）、HNSW（层次化跳表式 KNN graph）。
- 查询：从入口点出发，每一步跳到当前节点邻居中与 v_q 最近的一个，直到收敛。

4.4 三类方法速查

类别	思想	代表	优点	弱点
Partition	空间划分	hierarchical K-means	直观；离线建索引快	边界点易丢；不易增量
Hash	离散投影	LSH	理论保证；易并行	高维下 collision 难控
Graph	图上贪心	KNN graph / HNSW	召回精度高；查询快	内存大；建图慢

[TIP] 工业实践

FAISS / ScaNN / HNSWlib 都是常见库。课件结论：ANNS 的成本可以与 inverted index（稀疏检索）持平——这是 dense retrieval 能进入生产管线的关键。

5. 训练：Contrastive Learning / Learning to Rank

5.1 一般形式

训练 dual-encoder 等价于解

$$\arg \min_{\theta} \sum_q \sum_{d^+} \sum_{d^-} \ell(f(q, d^+), f(q, d^-)). \quad (3)$$

其中：

- (q, d^+) 是给定的正例对（“q 与 d 相关”，常来自标注数据集；
- d^- 是负例（“q 与 d 不相关”，需要自己采样；
- ℓ 是 ranking loss，要求 $f(q, d^+) > f(q, d^-)$ 。

5.2 InfoNCE：把负例打包成 softmax

工业最常用的对比损失是 InfoNCE（softmax-form contrastive loss）。给定 query q 、正例 d^+ 和 N 个负例 $\{d_i^-\}_{i=1}^N$ ：

$$\mathcal{L}_{\text{InfoNCE}}(q, d^+, \{d_i^-\}) = -\log \frac{\exp(f(q, d^+)/\tau)}{\exp(f(q, d^+)/\tau) + \sum_{i=1}^N \exp(f(q, d_i^-)/\tau)}. \quad (4)$$

- τ 是 temperature；
- 直观上把 ranking 问题转成“在 $N + 1$ 个候选里挑出 d^+ 的 softmax 分类”。

[NOTE] DPR 的 in-batch negatives (lec10_ir 已讲)

设一个 mini-batch 含 n 对 (q_i, d_i^+) 。把 query embedding 拼成 $Q \in \mathbb{R}^{d \times n}$ 、passage embedding 拼成 $P \in \mathbb{R}^{d \times n}$ ，相似度矩阵

$$S = Q^\top P \in \mathbb{R}^{n \times n}.$$

对第 i 行做 softmax，目标是让对角元 S_{ii} 最大——其余 $n - 1$ 个 passage 自动成为 q_i 的负例。一次前向算 n^2 个 pair，效率极高。

6. 难负例 (Hard Negatives): 本讲的真正主题

6.1 现象：随机负例为什么不够

课件给出关键实验观察：在 MSMARCO 上用**随机负例**训练 dense retriever，loss 早早收敛——模型对随机负例已经几乎完美，但检索效果远未饱和。

根源：retrieval 的独特属性。

- Corpus 巨大：百万至万亿级别；
- 99.9999% 的样本 trivially irrelevant——从语料里随机抽一个 doc，几乎肯定与 query 八竿子打不着；
- 真正考验模型的是少量 hard negatives：那些“看起来很像但实际不答 query”的文档。

[WARN] 关键直觉

Retrieval 的本质不是“判断相关 / 不相关”——而是“在数百万个看似相关的候选里挑出真正相关的那几个”。所以训练信号必须由 hard negatives 提供。

6.2 难负例采样三策略

6.2.1 Sparse Retrieval Negatives

用一个已有的**稀疏检索系统** (BM25 / inverted index) 跑一遍 q ，把它返回的 top-K 中**不含正确答案**的文档当负例。

代表工作：

- Bing Vector Search [Waldburger 2019]: 负例直接来自工业级 inverted index；
- DPR [Karpukhin et al. 2020]: 负例 = BM25 top-K 中不含答案串的 passage；
- Facebook Embedding Search [Huang et al. 2020]: 离线从生产系统中挖 hard negatives。

维度	评价
Pros	可在已有系统上 bootstrap；负例语义有意义
Cons	对 dense retriever 仍然偏 trivial；out-of-domain 泛化弱

核心局限：稀疏检索难以判断的负例（词汇重叠高但语义不同），对稠密检索来说不一定难——反之亦然。两个系统的“难”分布不一致，导致 negative 信号不够锐利。

6.2.2 Self Negatives (ANCE 系列)

Approximate nearest neighbor Negative Contrastive Estimation (ANCE) [Xiong et al. 2020, arXiv:2007.00808] 提出：

让 dense retriever 自己找自己的难负例。

流程：

1. **Warm up**: 先用 sparse retrieval negatives (如 BM25 hard negatives) 训练几个 epoch, 得到一个初步可用的 retriever;
2. **Async refresh**: 定期用当前 retriever 对整个 corpus 重建 ANN 索引;
3. **Self mining**: 对每个 q , 从该索引返回的 top-K 中不含正例的文档作为新一轮 hard negatives;
4. **Continue training**: 继续训 → 回到第 2 步。

```
# ANCE 训练框架 (伪代码)
retriever = warmup_with_bm25_negatives()
for epoch in range(E):
    # 周期性刷新索引
    if epoch % refresh_interval == 0:
        index = build_ann_index(retriever, corpus)

    for batch in data:
        q, d_pos = batch
        # 用当前 retriever 自己挖 hard negatives
        candidates = index.search(retriever.encode(q), top_k=K)
        d_neg = [c for c in candidates if c != d_pos][:n_neg]
        loss = infoNCE(q, d_pos, d_neg)
        loss.backward(); optimizer.step()
```

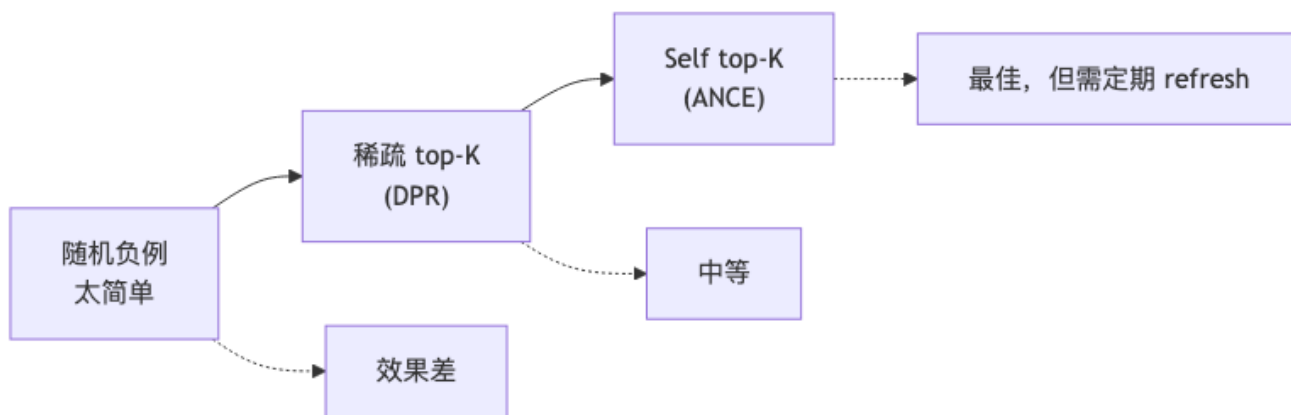
维度	评价
Pros	训练与测试分布对齐; in-domain & out-of-domain 表现都强
Cons	每次刷新索引代价大; 刷新前后训练不稳定 (gradient distribution shift)

[TIP] 对齐 train/test

ANCE 的核心洞察：测试时模型面对的是它自己排出来的 top-K；训练时也应该用它自己排出来的 top-K 作为难负例。如此 train/test gap 才最小。

6.2.3 三种负例策略对照

策略	负例来源	Train/Test 分布	工程代价	典型工作
Random Sparse Retrieval	语料随机抽 BM25 / inverted index top-K	偏差大 部分对齐 (稀疏侧)	极低 低 (已有系统)	早期 baseline DPR, Bing, FB
Self (ANCE)	dense retriever 自 身 top-K	完全对齐	高 (周期刷新)	ANCE



7. 总结: Embedding Learning 的设计哲学

课件结尾的 take-aways:

1. 把所有数据点映射到 **embedding 空间**——文本、图像、用户、商品都同框。
2. 用 **embedding 相似度** 近似语义关系——cosine / dot product 就够用。
3. 借助 **ANN** 实现高效查询——以略损精度换 sub-linear 检索。
4. 训练需要特别处理:
 - 损失函数: **contrastive loss** (InfoNCE);
 - 负例选择: **sophisticated negative mining** (hard / self / cross-batch), 是真正的研究焦点。
5. **trade-off 思维**: representation-centric 牺牲表达力换效率, dense retrieval 牺牲精确 KNN 换 sub-linear, hard negative mining 牺牲训练稳定换分布对齐。

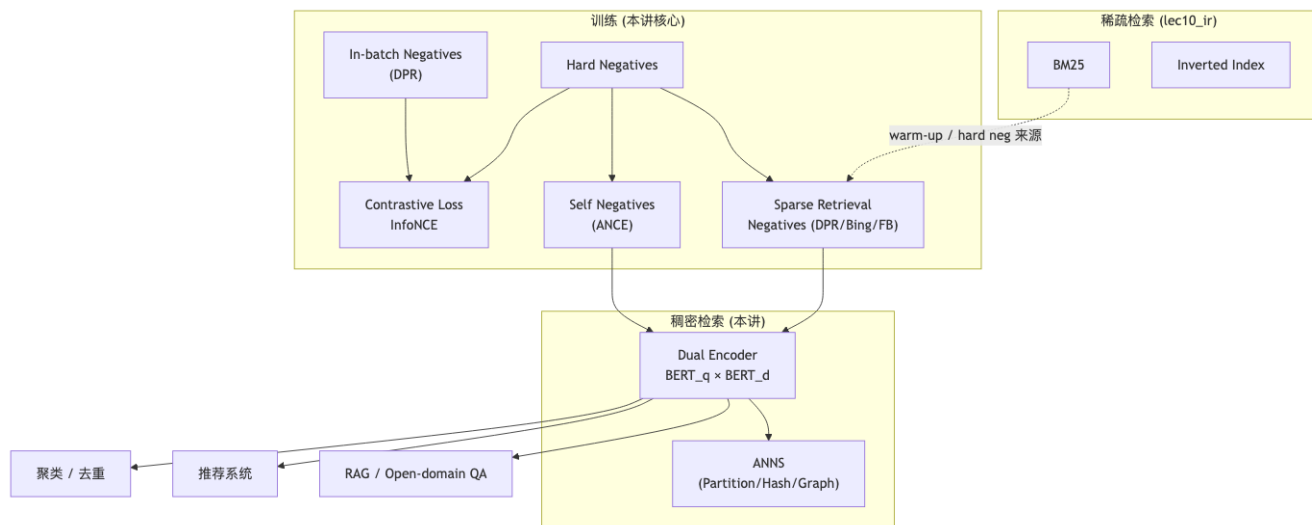
[NOTE] 课件留的 to-do

“To do: survey popular text embedding models, for processing Chinese, and many other languages.”
——课程鼓励大家自行调研 BGE、E5、Contriever、GTE、M3E 等多语 / 中文 embedding 模型, 看它们的训练数据、负采样、模型尺寸如何选择。

8. 章节内速查表

概念	定义 / 关键点
Dual encoder	两个独立编码器, $f(q, d) = E_q(q)^\top E_d(d)$
Representation-centric	只算向量; 可离线索引; 牺牲交互信息
ANNS	近似最近邻, sub-linear 查询
Partition method	空间划分; hierarchical K-means
Hash method	LSH; 同 hash 桶 $\approx$ 相邻
Graph method	KNN graph / HNSW; 图上贪心
In-batch negatives	mini-batch 内其他正例当负例; 相似度矩阵 $S = Q^\top P$
Hard negative	看似相关但实际不相关; 训练 retriever 的关键信号
Sparse retrieval negatives	取 BM25 top-K 中非正例 (DPR)
Self negatives (ANCE)	用 retriever 自己 top-K, 周期刷新
InfoNCE	softmax 形式的对比损失, 温度 τ
MSMARCO	大规模 passage retrieval 数据集; 常用 benchmark

9. 整体关系图



10. 期末复习要点

1. 公式默写

- 双塔相关性: $f(q, d) = \text{MLP}([\text{CLS}]_q)^\top \text{MLP}([\text{CLS}]_d)$;
- 对比训练目标: $\arg \min_{\theta} \sum_{q, d^+, d^-} \ell(f(q, d^+), f(q, d^-))$;
- InfoNCE: 式 (4);
- In-batch negatives 相似度矩阵: $S = Q^\top P \in \mathbb{R}^{n \times n}$, 对角元最大化。

2. 概念辨析

- Representation-centric vs interaction-centric;
- Bi-encoder (本讲) vs cross-encoder (精排, 未本讲展开);
- 三种 ANN 方法 (partition / hash / graph) ——各自代表算法;
- 三种负例策略——pros & cons。

3. 可能考查的开放题

- “随机负例为什么不够?” → 99.9999% trivially irrelevant, retrieval 难度在 hard negatives;
- “BM25 hard negative 与 ANCE self negative 的差异?” → 分布对齐性、refresh 代价、generalization;
- “为什么 dual encoder 比 cross encoder 适合做召回?” → 可离线索引、可上 ANN, 工程效率。

4. 与其他章节的连接

- lec5_dist: cosine 相似度、distributional hypothesis 是 embedding 的语义基础;
- lec10_ir: BM25 / 倒排索引 / DPR / in-batch negatives ——本讲是其 embedding 视角的延伸;
- 后续 RAG / Open-domain QA: dense retriever 是 retriever-reader 流水线的左半边。

11. 参考文献（课件 Readings）

1. Karpukhin et al. Dense Passage Retrieval for Open-Domain Question Answering, EMNLP 2020. — DPR、in-batch negatives。
2. Huang et al. Embedding-based Retrieval in Facebook Search, KDD 2020, pp. 2553–2561. —工业级 hard negative mining。
3. Xiong et al. Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval (ANCE), arXiv:2007.00808. —Self negatives。
4. Waldburger 2019 (Bing Vector Search 报道) ——sparse retrieval negatives 工业案例。

Lecture 10 (Sparse IR): 信息检索之稀疏方法 / Sparse Information Retrieval

课程: 自然语言处理基础 (FNLP, PKU) —Yansong Feng 日期: 2026-05-06 对应讲义: lec10_ir.txt (前半部分: Boolean / VSM / BM25) 配套笔记: lec10_ir_embedding.md (稠密检索 DPR / Bi-encoder / ColBERT)

[TIP] 学习指南

本节聚焦“**稀疏 (sparse) 检索**”——以词项 (term) 计数为基本统计量、以倒排索引 (inverted index) 为加速结构、以词袋表示 (BoW) 为向量化的一类经典 IR 模型。掌握三件事即可在期中拿满: 1. **Boolean / TF-IDF / BM25** 三个评分公式 能默写、能解释每个符号的含义、能比较其优劣; 2. **倒排索引** 的构建流程与查询时的 posting list 求交 (含 skip pointer / positional posting); 3. **评测指标** Precision@k, Recall, MAP, nDCG 的定义与手工计算。

学习路径建议: 先看 §10.1 动机 → §10.2 形式化定义 → §10.3 三个模型层层推进 → §10.4 倒排索引 (系统视角) → §10.5 评测 → §10.6 worked example 自己算一遍 → §10.7 易错点。

10.1 Motivation —为什么需要 IR

Information Retrieval (IR) 的目标: 给定一个用户查询 q 与一个大文档集合 $\mathcal{D} = \{d_1, d_2, \dots, d_K\}$, 返回一个尽可能贴合 q 信息需求的子集 $\tilde{\mathcal{R}}(q) \subseteq \mathcal{D}$, 并按相关性排序。

应用场景 (讲义 p.10):

- 搜索引擎 (Search Engines): query $\rightarrow$ web pages
- 推荐系统 (Recommender Systems): user $\rightarrow$ items
- 电商 (E-commerce): user $\rightarrow$ goods / clients
- 在线广告 (Online Ads): user $\rightarrow$ ads
- 社交媒体: user $\rightarrow$ friends / tweets / videos
- 情报分析 (Intelligence Analysis): claim $\rightarrow$ evidence

[NOTE] IR 与 QA 的关系

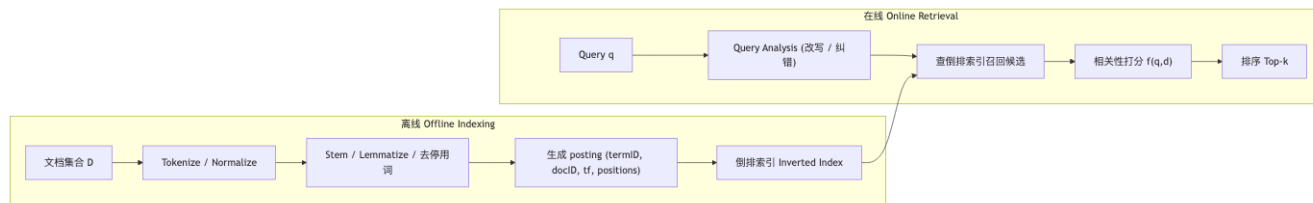
QA (Question Answering) 可视为 IR 的特例——把“答案候选”当作文档。现代 Open-Domain QA / RAG 系统普遍采用 **retriever + reader / generator** 的两阶段结构, retriever 这一步基本就是 IR。

IR 系统理想性质: **Accurate / On time / Comprehensive / Diversity**。这些目标之间常有 trade-off (精度 vs 召回、新鲜度 vs 索引规模)。

10.1.1 主体架构 (Main Architecture)

讲义 p.13–15 给出四大组件:

1. **Document Analysis & Indexer**: 离线把文档清洗、分词、建立倒排索引;
2. **Query Analysis**: 在线把用户 query 改写 (去停用词、词干化、扩展同义词、纠错);
3. **Searching & Ranking**: 用倒排索引快速召回候选 → 用相关性函数 $f(q, d)$ 打分排序;
4. **User Modeling**: 基于点击、停留、历史偏好做个性化重排。



10.2 形式化定义 (Formalization)

给定:

- 词表 $V = \{w_1, \dots, w_N\}$;
- 查询 $q = w_{q_1} w_{q_2} \dots w_{q_m}$;
- 文档集合 $\mathcal{D} = \{d_1, \dots, d_K\}$;
- 真实相关集合 $\mathcal{R}(q) \subseteq \mathcal{D}$ (无法直接获得)。

任务是求一个近似 $\tilde{\mathcal{R}}(q) \approx \mathcal{R}(q)$ 。讲义 p.19–21 给出两类思路:

思路	形式	评论
Solution 1 —绝对相关	$\tilde{\mathcal{R}}(q) = \{d \in \mathcal{D} \mid f(d, q) = 1\}$	“相关 / 不相关”二值判定难定阈
Solution 2 —相对相关 (主流)	$\tilde{\mathcal{R}}(q) = \{d \in \mathcal{D} \mid f(d, q) > \theta\}$, 按 f 排序	只需保证 偏序正确 : $f(q, d_i) > f(q, d_j) \iff P(R q, d_i) > P(R q, d_j)$

通常被分解为 **两步 (two-stage)**:

1. **Retrieval (召回)**: 宽口径找出候选——指标关注 **recall**;
2. **Ranking (精排)**: 在候选上精细排序——指标关注 **precision@k / nDCG**。

[WARN] 关键问题

1. 如何建模相关性 $f(q, d)$ ——Boolean / VSM / Prob / Neural;
2. 如何表示 q 与 d ——one-hot / BoW / TF-IDF / embedding。两件事正交, 可自由组合。

10.3 关键模型 (Retrieval Methods)

10.3.1 Boolean Model —布尔模型

把 query 写成由 AND / OR / NOT 连接的布尔表达式, 把每个文档视作“term 是否出现”的 0/1 向量, 按表达式求值。

Query: like AND information AND retrieval

DocID	内容
1	I like machine learning
2	Machine learning is different with deep learning
3	Information retrieval is important for many applications
4	I like information retrieval
5	Machine learning is useful for ranking in search engine

命中: Doc 4 (同时含三个 term)。

核心问题:

1. **暴力扫描复杂度**: 若文档数 K 、平均长度 L , 单 query 需 $O(KL)$ 时间——百亿网页时不可行;
2. **NOT 处理麻烦**: 必须遍历所有文档才能确定“不含某 term”;
3. **没有排序**: 只能“是 / 否”——无法回答“哪篇更相关”。

[NOTE] 解决思路

必须配合 **倒排索引** 才能高效执行: term 的 AND 两条 posting list 求交集 (线性时间, 配合 skip pointer 可加速); OR 求并; NOT 全集减去命中集 (或在求交时用 not in 跳过)。详见 §10.4。

优势: 精确 (专家可写出极复杂的逻辑表达式, 例如法律、专利、医学检索仍在用)。 **劣势**: 用户不会写表达式; 所有 term 同等权重 (无 IDF); 难以排序。

10.3.2 Vector Space Model (VSM) 一向量空间模型

为解决布尔模型的三大问题——表达难、词等权、不可排——提出把 q 与 d 嵌入到同一向量空间, 以向量之间的距离 / 相似度作为相关性。

(a) Bag-of-Words 表示

文档 d 表示为长度 $|V|$ 的向量 $d \in \mathbb{R}^{|V|}$, 第 i 维记词 w_i 在 d 中的“权重”。最简单的权重就是计数 $\text{count}(w_i, d)$, 但 raw count 有两个明显缺陷:

1. **高频功能词主导**: “the / of / is”在每篇都很多;
2. **未区分稀有词**: 仅在少数文档出现的词反而最有判别力。

(b) Term Frequency (TF) 一词频

$$\text{TF}_{t,d} = \begin{cases} 1 + \log_{10} \text{count}(t, d), & \text{if } \text{count}(t, d) > 0 \\ 0, & \text{otherwise} \end{cases} \quad (10.1)$$

为什么 log-scale? 出现 10 次的词并非比出现 1 次的词“重要 10 倍”, 对数压缩才符合人类对相关性的直觉 (边际效用递减)。

(c) Inverse Document Frequency (IDF) 一逆文档频率

$$\text{IDF}_t = \log_{10} \frac{N}{\text{DF}_t} \quad (10.2)$$

其中 N 为文档总数, DF_t 为含 t 的文档数。

直觉：若 t 在多数文档出现，则其 IDF 趋近 0，无区分度；若 t 罕见，则 IDF 大，是好的相关性线索。

[NOTE] IDF 的平滑变体

常见的平滑版本（避免 $\log 1 = 0$ 与 $DF = 0$ ）：

$$\text{IDF}_t = \log \frac{N + 1}{\text{DF}_t + 1} + 1 \quad (\text{sklearn smooth idf})$$

或 Robertson 概率版本（BM25 用的就是这个）：

$$\text{IDF}_t = \log \frac{N - \text{DF}_t + 0.5}{\text{DF}_t + 0.5}$$

(d) TF-IDF 权重

$$\text{tf-idf}(t, d) = \text{TF}_{t,d} \cdot \text{IDF}_t \quad (10.3)$$

文档与查询均被表示为 $|V|$ 维 tf-idf 向量，记为 d, q 。

(e) 相似度 / 距离函数 (讲义 p.44)

名称	公式	性质
欧氏距离	$f(q, d) = \frac{1}{\sqrt{\sum_i (q_i - d_i)^2}}$	受长度影响大
余弦相似度 (主流)	$\cos(q, d) = \frac{q \cdot d}{\ q\ \ d\ }$	已归一化，长度无关
Dice	$\frac{2q \cdot d}{\ q\ ^2 + \ d\ ^2}$	集合重叠类指标
Jaccard	$\frac{q \cdot d}{\ q\ ^2 + \ d\ ^2 - q \cdot d}$	二值向量常用

为什么默认用余弦？ 文档长度可能差异极大（一句话 vs 一本书）——欧氏距离对此敏感，余弦做了 L2 归一化，等价于把文档投影到单位球面再算夹角。

(f) VSM 的局限

- 仍是“词袋”——丢失词序与句法；
- 词义近似而拼写不同的词被视为不同维度（synonymy / polysemy 问题）；
- 高维稀疏（百万维），存储与计算需要倒排索引。

[NOTE] 改进方向

- 主题模型 (LSI, PLSI, LDA) 一把向量降到隐主题空间；- 神经词嵌入 (word2vec, GloVe) 一用稠密向量替代；
- 稠密检索 DPR / ColBERT 一见 lec10_ir_embedding.md。

10.3.3 Probabilistic Model & BM25 — 概率模型与最佳匹配

(a) 二元独立模型 (Binary Independence Model, BIM)

Robertson & Spärck-Jones (1970s) 的出发点：直接对 $P(R|q, d)$ 建模。在二元独立假设（每个 term 在 d 中只关心“出现/不出现”，且 term 之间条件独立）下，按 $P(R = 1|q, d)/P(R = 0|q, d)$ 排序等价于按下式排序：

$$\sum_{t \in q \cap d} \log \frac{p_t(1 - u_t)}{u_t(1 - p_t)} \quad (10.4)$$

其中 $p_t = P(t \in d | R = 1)$, $u_t = P(t \in d | R = 0)$ 。这就是 IDF 的概率前身。

(b) Okapi BM25 (Robertson, 1994) 一期中必背

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{\text{tf}(t, d) \cdot (k_1 + 1)}{\text{tf}(t, d) + k_1 \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)} \quad (10.5)$$

讲义 p.46 给出的稍简化版本：

$$\text{BM25}(q, d) = \sum_{w \in q} \log \frac{N}{\text{df}_w} \cdot \frac{\text{tf}_{w,d}}{k \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right) + \text{tf}_{w,d}} \quad (10.5')$$

符号详解：

符号	含义	典型取值
N	文档总数	—
df_w	含 w 的文档数	—
$\text{tf}_{w,d}$	w 在 d 中的词频	—
$\ d\ $	d 的长度 (token 数)	—
avgdl	全集平均文档长度	—
$k_1 (= k)$	控制 tf 饱和点	1.2 – 2.0
b	文档长度归一化强度	0.75

两个超参数的角色（必考）：

1. k_1 — **tf 饱和参数**：当 $k_1 \rightarrow 0$ ，公式 $\frac{(k_1+1)\text{tf}}{\text{tf}+k_1(\cdot)} \rightarrow 1$ ，任何出现都等价（退化为布尔权重）；当 $k_1 \rightarrow \infty$ ，公式 $\rightarrow \text{tf}$ （退化为原始词频）。中等 $k_1 \approx 1.5$ 即可让“出现 1 次 $\rightarrow$ 大幅加分”，但“出现 10 次”边际效用递减。
2. b — **长度归一化强度**： $b = 0$ 完全不考虑文档长度； $b = 1$ 完全按 $|d|/\text{avgdl}$ 线性缩放（长文档分数显著降低）； $b = 0.75$ 折衷，长文档不会因为“碰巧含更多词”被高估。

[TIP] 为什么 BM25 战胜 TF-IDF

1. **tf 饱和**: tf-idf 让 tf 线性增长; BM25 让 tf 趋于饱和——更符合“重复 100 次 the 不应该比 5 次更相关”的直觉; 2. **长度归一化**: tf-idf 默认不归一 (除非额外加 cos); BM25 内建长度归一化; 3. **IDF 的概率推导**: BM25 的 IDF 项可由 BIM 严格推出, 非启发式; 4. **强基线**: 在 2026 年仍是稠密检索难以彻底击败的基线, 常作为初步召回 + 重排 (BM25 + DPR)。

(c) Statistical Language Model 一简述

不同视角: 把文档看作生成模型 $P(q|d)$, 即“假设用户在阅读 d 后会查询 q 的概率”。用 unigram 模型 + Dirichlet / Jelinek-Mercer 平滑:

$$P(q|d) = \prod_{w \in q} (\lambda P(w|d) + (1 - \lambda)P(w|\mathcal{D})) \quad (10.6)$$

理论上与 BM25 等价类, 工程上 BM25 更常用。

10.4 倒排索引 (Inverted Index) 一系统视角

[WARN] 期中常考点

倒排索引是 sparse IR 的**唯一加速结构**。问“为什么 Boolean 模型可以在百亿文档秒级返回”——答: 倒排索引把“扫所有文档”变成“扫几条 posting list”。

10.4.1 数据结构

词典 (dictionary / lexicon)
term $\rightarrow$ [df, pointer-to-posting]
posting list
termID $\rightarrow$ [(docID, tf, [pos_1, pos_2, ...]), (docID, tf, [...]), ...]
↑ 通常按 docID 升序排列, 便于求交

- **Dictionary**: 词典, 通常用哈希表 / B-tree / FST 实现, $O(1)$ 或 $O(\log V)$ 查 term;
- **Posting list**: 该 term 命中的所有文档信息; 按 docID 排序方便集合运算;
- **Positional posting**: 保留 term 在文档内的位置, 支持短语 / 邻近查询。

10.4.2 构建流程



复杂度: 若总 token 数 T , 朴素排序 $O(T \log T)$; 大规模下用 BSBI / SPIMI 等外排算法, 按块处理后归并。

10.4.3 查询执行—Posting List 求交

例: query = information AND retrieval

posting(information): [1, 3, 4, 7, 11, 15, 20]

posting(retrieval): [2, 3, 7, 9, 11, 12]

求交 (双指针): [3, 7, 11]

双指针线性时间: $O(|p_1| + |p_2|)$ 。

Skip Pointer 加速: 在 posting list 上每隔 $\sqrt{L}$ 个位置加一个“跳跃指针”，当一边远小于另一边时，长 list 可跳过大段不可能匹配的位置，期望复杂度降至 $O(|p_1| + \sqrt{|p_2|})$ 。

posting(retrieval) with skip @ $\sqrt{L} = 3$:

2 → 3 → 7 → skip → 12 → ...
 ↑ 若 7 < target, 可直接跳到 12

10.4.4 Positional Index —支持短语查询

例: query = "information retrieval" (必须连续)

posting(information): {doc7: [3, 15], doc11: [2]}

posting(retrieval): {doc7: [4, 22], doc11: [3]}

对 doc7: 3+1 == 4 → 命中; 对 doc11: 2+1 == 3 → 命中。一般地, 要求位置序列满足 $\text{pos}(w_{i+1}) - \text{pos}(w_i) == 1$ 。

[NOTE] 工程取舍

位置索引让索引体积膨胀 2—4 倍。生产系统常保留位置只用于 top-k 重排, 召回阶段仍只用 (docID, tf)。

10.4.5 索引压缩 (简述)

- **Variable-Byte / Gamma / Delta 编码**: docID 差值常很小 (gap encoding), 用变长编码大幅压缩;
- **前端字典压缩**: term 字符串共享前缀 (front coding);
- **Block-Max WAND**: top-k 检索时跳过不可能进入 top-k 的 posting, 是搜索引擎标配。

10.5 评测 (Evaluation)

设查询 q 的真实相关集合为 R_q , 系统返回的前 k 个结果为 $\tilde{R}_q^k$ 。

10.5.1 Precision / Recall / F1

$$\text{Precision@}k = \frac{|R_q \cap \tilde{R}_q^k|}{k}, \quad \text{Recall@}k = \frac{|R_q \cap \tilde{R}_q^k|}{|R_q|} \quad (10.7)$$

$$\text{F1} = \frac{2 \cdot P \cdot R}{P + R} \quad (10.8)$$

10.5.2 Mean Average Precision (MAP)

对单条 query:

$$\text{AP}(q) = \frac{1}{|R_q|} \sum_{k=1}^{|\tilde{R}_q|} \text{Precision@}k \cdot \mathbb{1}[d_k \in R_q] \quad (10.9)$$

整体:

$$\text{MAP} = \frac{1}{|Q|} \sum_{q \in Q} \text{AP}(q) \quad (10.10)$$

直觉：奖励“相关文档排得越靠前越好”，且对每条 query 都做了平均。

10.5.3 Mean Reciprocal Rank (MRR)

适合“只有一个正确答案”的场景（如 QA）：

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q} \quad (10.11)$$

其中 rank_q 是第一个相关文档的排名。

10.5.4 nDCG (Normalized Discounted Cumulative Gain)

允许“分级相关” ($\text{rel} \in \{0,1,2,3,4\}$ 等)，并对排序位置打折扣：

$$\text{DCG}@k = \sum_{i=1}^k \frac{2^{\text{rel}_i} - 1}{\log_2(i + 1)} \quad (10.12)$$

$$\text{nDCG}@k = \frac{\text{DCG}@k}{\text{IDCG}@k} \quad (10.13)$$

其中 $\text{IDCG}@k$ 是理想排序（按 rel 降序）的 DCG。 $\text{nDCG} \in [0, 1]$ ，越接近 1 越好。

[NOTE] 指标选择

| 场景 | 推荐指标 | |——| | 单一答案 (QA / NavQuery) | MRR, Hit@k | | 多答案、平等相关 | MAP, Recall@k | | 多答案、分级相关 (网页搜索) | nDCG@k |

10.6 Worked Example —BM25 手算

设置：- 文档集 ($N = 2$)：- d_1 = “machine learning is fun” ($|d_1| = 4$) - d_2 = “machine learning machine translation systems are useful” ($|d_2| = 7$) - $\text{avgdl} = (4 + 7)/2 = 5.5$ - query q = “machine learning”
- 超参数： $k_1 = 1.5$, $b = 0.75$ - 统计：- $\text{df}(\text{machine}) = 2 \rightarrow$ 用讲义版 $\text{IDF} \log \frac{N}{\text{df}} = \log \frac{2}{2} = 0$ - $\text{df}(\text{learning}) = 2 \rightarrow \log \frac{2}{2} = 0$

为了得到非平凡示例，改用 Robertson 平滑 IDF：

$$\text{IDF}(w) = \log \frac{N - \text{df}_w + 0.5}{\text{df}_w + 0.5} = \log \frac{2 - 2 + 0.5}{2 + 0.5} = \log \frac{0.5}{2.5} = \log 0.2 \approx -1.609$$

由于负值不便于演示，进一步把 d_2 替换为“natural language processing is great” ($|d_2| = 5$, machine、learning 都不出现)：

- 重新计算：
 - $df(\text{machine}) = 1, IDF = \log \frac{2-1+0.5}{1+0.5} = \log 1 = 0$ — 仍为 0!

[WARN] 小数据病

当 N 很小时 Robertson IDF 容易为 0 或负，这是教学示例的常见坑。实际系统中 N 通常 $\geq 10^4$ ，不会出现该问题。

为了演示，改用讲义 p.40 的 $IDF \log_{10}(N/df)$ 并把 N 设为 1000 (假想总集)：

修正后设置：- $N = 1000, df(\text{machine}) = 50, df(\text{learning}) = 100 - d_1 = \text{"machine learning is fun"} \rightarrow tf(\text{machine})=1, tf(\text{learning})=1, |d_1| = 4 - d_2 = \text{"machine learning machine translation systems are useful"} \rightarrow tf(\text{machine})=2, tf(\text{learning})=1, |d_2| = 7 - avgdl = 5.5, k_1 = 1.5, b = 0.75$

IDF: - $IDF(\text{machine}) = \log_{10}(1000/50) = \log_{10} 20 \approx 1.301 - IDF(\text{learning}) = \log_{10}(1000/100) = \log_{10} 10 = 1.000$

长度归一化因子 $K = k_1(1 - b + b \cdot |d|/avgdl)$: - $K_1 = 1.5 \cdot (0.25 + 0.75 \cdot 4/5.5) = 1.5 \cdot (0.25 + 0.5455) = 1.5 \cdot 0.7955 \approx 1.193 - K_2 = 1.5 \cdot (0.25 + 0.75 \cdot 7/5.5) = 1.5 \cdot (0.25 + 0.9545) = 1.5 \cdot 1.2045 \approx 1.807$

单 term 打分 $score(t, d) = IDF(t) \cdot \frac{tf(t,d)(k_1+1)}{tf(t,d)+K_d}$:

对 d_1 : - machine: $1.301 \cdot \frac{1 \cdot 2.5}{1+1.193} = 1.301 \cdot \frac{2.5}{2.193} \approx 1.301 \cdot 1.140 \approx 1.483$ - learning: $1.000 \cdot \frac{1 \cdot 2.5}{1+1.193} \approx 1.140 - BM25(q, d_1) \approx 1.483 + 1.140 = 2.623$

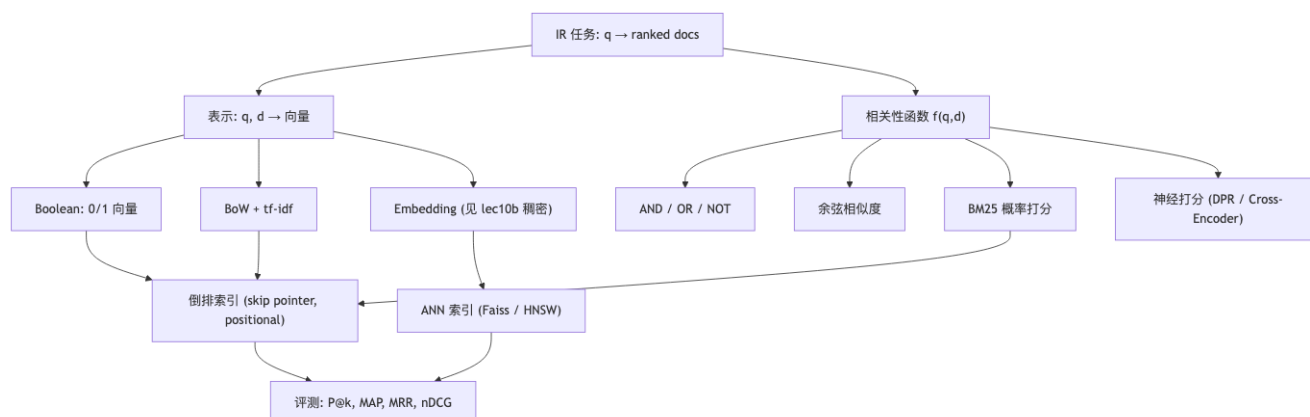
对 d_2 : - machine: $1.301 \cdot \frac{2 \cdot 2.5}{2+1.807} = 1.301 \cdot \frac{5}{3.807} \approx 1.301 \cdot 1.313 \approx 1.708$ - learning: $1.000 \cdot \frac{1 \cdot 2.5}{1+1.807} = 1.000 \cdot \frac{2.5}{2.807} \approx 0.891 - BM25(q, d_2) \approx 1.708 + 0.891 = 2.599$

排序结果: d_1 (2.62) d_2 (2.60)。

[NOTE] 直觉验证

d_2 中 machine 出现 2 次 (tf 高)，但因文档长 (长度归一化扣分)，最终被更“紧凑”的 d_1 险胜——这正是 BM25 长度归一化的设计意图。如果把 $b = 0$ (关闭长度归一化) 重算， d_2 会反超。

10.7 概念关系图 (Mermaid)



10.8 易错点 (Common Pitfalls)

[WARN] 高频踩坑

1. IDF 公式底数与平滑：讲义用 $\log_{10}$ ，sklearn 用 $\ln$ ，BM25 用 Robertson 平滑——考试要按题面给的版本算。2. BM25 是按 term 累加的，不是对整个查询做一次打分。漏求和最常见。3. k_1 与 b 的方向： k_1 越大 $\rightarrow$ tf 越不易饱和（越接近原始 tf）； b 越大 $\rightarrow$ 越严厉惩罚长文档。容易记反。4. 倒排索引保存的是 docID 列表——不是文档内容；位置信息是“加料”，不是必需。5. 求交时 posting list 必须有序——否则双指针不成立。6. MAP 与 nDCG 都包含“位置折扣”思想，但 nDCG 允许多级相关；MAP 只支持二值相关。7. VSM 与 BM25 共用倒排索引：BM25 不是另一套数据结构，只是另一种打分函数。

10.9 中英术语对照

中文	英文	简称 / 备注
信息检索	Information Retrieval	IR
查询	Query	q
文档集合	Corpus / Document Collection	$\mathcal{D}$
词典	Dictionary / Lexicon	—
词项	Term	t
词袋表示	Bag-of-Words	BoW
词频	Term Frequency	TF
文档频率	Document Frequency	DF
逆文档频率	Inverse Document Frequency	IDF
倒排索引	Inverted Index	—
倒排表	Posting List	—
跳跃指针	Skip Pointer	—
位置索引	Positional Index	—
布尔检索	Boolean Retrieval	—
向量空间模型	Vector Space Model	VSM
余弦相似度	Cosine Similarity	—
最佳匹配 25	Best Matching 25	BM25
二元独立模型	Binary Independence Model	BIM
语言模型检索	Language Model for IR	LM4IR
召回	Recall	—
准确率	Precision	—
准确率 @k	Precision at k	P@k
平均准确率	Average Precision	AP
平均的平均准确率	Mean Average Precision	MAP
倒数排名	Reciprocal Rank	MRR
归一化折损累积增益	normalized Discounted Cumulative Gain	nDCG
稠密检索	Dense Retrieval	DPR/ColBERT
稀疏检索	Sparse Retrieval	BM25/TF-IDF

[TIP] 期中复习要点

1. **背公式**: TF-IDF (10.3) 与 BM25 (10.5)。能默写、能解释每个符号、能说出 k_1, b 的作用与极端取值的退化形式。2. **画索引**: 给 3—5 篇短文档, 能手工构造倒排索引并对一个 AND 查询走完 posting 求交 (含 skip pointer)。3. **算指标**: 给定一条 query 的真实相关集合与系统返回的排序, 能手算 P@k、Recall、AP、nDCG。4. **比较模型**: Boolean vs VSM vs BM25 vs 神经检索——优缺点、何时选谁。5. **两阶段思想**: Retrieval (recall 导向) + Ranking (precision 导向); BM25 常作召回基线, 神经模型常用于重排。6. **长度归一化**: 理解为什么“长文档 = 更可能命中 = 不一定更相关”, BM25 的 b 项怎样修正。7. **与稠密检索的关系**: 稀疏 BM25 + 稠密 DPR 互补, 常做“加权融合 / 级联”——参见配套笔记。

Lecture 11 —预训练模型 (Pretrained Language Models)

[TIP] 学习指南

本节核心：从「为每个任务单独训模型」过渡到「先用海量无标注语料预训练，再用任务数据微调」。围绕三类架构 (encoder-only / decoder-only / encoder-decoder) 讲清楚 ELMo → BERT → RoBERTa → T5 → GPT-1/2/3 这条主线，并理解 MLM、Causal LM、Span Corruption 三种自监督目标在数学上的区别。前置知识：- lec4_lm —— n -gram 语言模型、困惑度 (PPL) - lec4_nlm —— FFNN-LM、RNN/LSTM 语言模型 - lec5_dist —— word2vec、分布式表示 - 注意力与 Transformer (lec9 / lec10) —— attention、multi-head、位置编码 重点掌握：1. MLM 与 Causal LM 的目标函数形式及梯度来源差异 2. BERT 的 15% 掩码细则 (80/10/10 比例) 与 NSP 的设计动机 3. ELMo 是「特征注入」、BERT/GPT 是「权重复用」这一范式区分 4. 三类架构与对应自监督目标的对应关系 5. GPT-1 → GPT-2 → GPT-3 的参数 / 数据 / 算力 scaling 轨迹及范式从 fine-tuning 转向 prompting 的动因

1. Motivation：为什么要预训练

1.1 静态词向量的局限

word2vec、GloVe 等方法对每个词只学习一个向量，无法处理一词多义。考虑英文 play 的四种用法（区域 / 行动 / 比赛 / 戏剧），静态向量无法区分：

句子片段	play 含义
Chico Ruiz made a spectacular play on Alusik's grounder	比赛动作
The new-look play area is due to be completed	玩耍区域
representatives who play to the party base	迎合行为
Mike was praised for his all-round excellent play last night	比赛表现
John signed to do a Broadway play for Garson	戏剧

只有第 (C) 句的 play 与原句语义最接近，但 word2vec 给出的全是同一个向量。上下文相关表示 (contextual representations) 应运而生。

1.2 表示学习 + 数据效率

预训练-微调 (Pre-training & Fine-tuning) 的核心论证：

在大规模任务 X 上预训练得到的模型，可以为相关下游任务 Y 提供可泛化的表示能力；找到一个对广泛 Y 都有用的 X，下游只需少量数据 + 少量任务参数即可达到优秀效果。

这意味着：– 大规模无标注语料一次性吃下，得到通用表示 – 下游任务训练数据再少也能用上海量预训练知识 – 模型权重几乎全部复用，只在顶层添加任务头

1.3 历史脉络

课程给出的演进路线：

VSM → PLSA (2003) → LDA (2003) → Neural LM (2003) → word2vec (2013/14) → RNN-LM / LSTM-LM (2015) → **Transformer (2017)** → **ELMo (2018-02)** → **GPT (2018-06)** → **BERT (2018-10)** → GPT-2 (2019) → XLNet (2019) → RoBERTa (2019) → T5 (2019) → GPT-3 (2020) →

2. ELMo：上下文相关表示的起点

2.1 模型结构

ELMo (Deep Contextualized Word Representations, Peters et al., 2018-02) 训练两条独立的多层 LSTM 语言模型：

- 前向 LM: $p(t_k | t_1, \dots, t_{k-1})$
- 后向 LM: $p(t_k | t_{k+1}, \dots, t_N)$

对每个 token k 与每个 LSTM 层 j ，分别得到隐状态 $\vec{h}_{k,j}$ 与 $\overleftarrow{h}_{k,j}$ ，拼接为 $h_{k,j} = [\vec{h}_{k,j}; \overleftarrow{h}_{k,j}]$ 。

最终 ELMo 表示是各层的任务相关加权平均：

$$\text{ELMo}_k^{\text{task}} = \gamma^{\text{task}} \sum_{j=0}^L s_j^{\text{task}} h_{k,j}$$

其中 s_j^{task} 是 softmax 归一化的层权重， γ^{task} 是任务相关的缩放因子。

[NOTE] 为什么要前向 + 后向？

单向 LM 只看到一侧上下文，无法消歧出现在句子开头的歧义词。前向 + 后向分别建模能让 token 获得双侧信息（但仍是两个独立模型，不是真正双向）。

为什么不只取最顶层？ELMo 实验表明：低层 LSTM 更偏句法信息，高层 LSTM 更偏语义信息。任务相关的加权可以让每个下游任务自行选择重要层。

2.2 预训练细节

- 数据：1B Word Benchmark 的单句
- 训练：10 个 epoch，3 张 GTX 1080 训练约 2 周
- 困惑度：前向 / 后向平均 PPL 约 39.7
- 范式：预训练只跑一次，任务无关

2.3 下游使用方式

[预训练 ELMo (冻结)] → 上下文向量 → [下游任务专属模型] → 任务输出

下游任务训练时 ELMo 参数冻结，只调整任务模型。可用于：句子分类（情感分析）、序列标注、问答、翻译.....

2.4 ELMo 的本质与局限

- 优势：预训练一次、各任务复用 contextual embeddings
- 限制：只增强词表示，下游模型架构不变——这是与下一代预训练（BERT/GPT）的根本区别

[Q] 提示

ELMo 只产出嵌入向量，能否做到「连模型权重都直接复用」？这正是 BERT/GPT 给出的答案。

3. Pre-training & Fine-tuning 范式

更现代的范式：

1. 在大规模任务 X 上预训练整个模型
2. 在下游任务 Y 上微调（fine-tune）所有权重，仅添加少量任务相关参数（如分类头）

“Fine-tuning is the process of taking the network learned by these pre-trained models, and further training the model, often via an added neural net classifier that takes the top layer of the network as input, to perform some downstream task.”

任务 X 与 Y 之间存在语义相关性，使得 X 学到的表示能力可迁移到 Y。

3.1 三类架构

Architecture	Pretraining Objective	Examples
Encoder-only	Masked Language Model (MLM)	BERT, RoBERTa, ELECTRA
Encoder-Decoder	Span Corruption	T5, BART
Decoder-only	Autoregressive Language Model	GPT-2, GPT-3, Llama

后续三章分别展开。

4. Transformer 架构核心回顾

本节预训练模型全部基于 Transformer，所以先回顾关键公式。

4.1 Self-Attention

输入向量序列 $X \in \mathbb{R}^{n \times d}$ ，通过线性投影得到 query / key / value：

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

缩放点积注意力：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

其中 $\sqrt{d_k}$ 是温度缩放，避免点积进入 softmax 饱和区导致梯度消失。

4.2 Multi-Head Attention

将 attention 并行计算 h 次，每次用不同投影矩阵：

$$\text{head}_i = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V)$$

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

直觉：不同头可以关注不同位置 / 不同关系（句法、共指等）。

4.3 因果掩码 (Causal Mask)

decoder 的自注意力要求位置 t 不能看到 $t + 1, \dots, n$ ，于是在 softmax 之前加掩码 M ：

$$M_{ij} = \begin{cases} 0 & j \leq i \\ -\infty & j > i \end{cases}$$

$$\text{Attention}_{\text{causal}}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V$$

GPT 系列使用因果掩码以维持自回归性质；BERT encoder 不使用掩码（双向）。

4.4 Encoder / Decoder Block

- Encoder block: MultiHead-Self-Attention $\rightarrow$ Add&Norm $\rightarrow$ FFN $\rightarrow$ Add&Norm
- Decoder block: Masked-Self-Attention $\rightarrow$ Add&Norm $\rightarrow$ Cross-Attention (over encoder output) $\rightarrow$ Add&Norm $\rightarrow$ FFN $\rightarrow$ Add&Norm

三种预训练架构因此可以表述为：– Encoder-only：只堆 encoder block（双向）– Decoder-only：只堆 decoder block 的「masked-self-attn + FFN」部分（单向因果）– Encoder-Decoder：完整 encoder + decoder

5. BERT：双向编码器表示

BERT: Bidirectional Encoder Representations from Transformers (Devlin et al., 2018–10)

5.1 设计要点

- 基于深层 双向 encoder-only Transformer
- 遵循 pre-training & fine-tuning 范式
- 核心目标：基于双向上下文学习表示
- 两个新的预训练目标：
 - Masked Language Modeling (MLM)
 - Next Sentence Prediction (NSP)（后续工作发现弊大于利）

5.2 Masked Language Modeling (MLM)

定义

在输入序列中随机遮蔽 $k\%$ 的 token，用模型预测被遮蔽 token：

原句：the man went to the store to buy a gallon of milk 输入：the man went to [MASK] to buy a [MASK] of milk 目标：第一个 [MASK] → “store”，第二个 [MASK] → “gallon”

BERT 取 $k = 15\%$ 。

形式化

设原句 token 为 $x = (x_1, \dots, x_n)$ ，被遮蔽位置集合为 $\mathcal{M}$ ，遮蔽后输入为 $\tilde{x}$ 。MLM 损失：

$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in \mathcal{M}} \log p_{\theta}(x_i | \tilde{x})$$

其中 $p_{\theta}(x_i | \tilde{x}) = \text{softmax}(Wh_i + b)$ ， h_i 是 encoder 在位置 i 的最终隐向量。

80 / 10 / 10 替换策略

预训练阶段会出现 [MASK] token，但下游微调时输入不含 [MASK]，造成 train-test mismatch。BERT 的缓解方案：被选中的 15% token 中：

- 80% 替换为 [MASK]：went to the store → went to the [MASK]
- 10% 替换为词表中随机词：went to the store → went to the running
- 10% 保持原样：went to the store → went to the store

这样模型必须对每个 token 的表示都保持高质量（无法判断当前位置是不是被「干扰」过）。

5.3 Next Sentence Prediction (NSP)

动机

许多下游任务需要理解两个句子之间的关系：– Natural Language Inference (NLI) – Paraphrase detection – Question Answering – Dialogue

输入格式

[CLS] Sentence A tokens ... [SEP] Sentence B tokens ... [SEP]

- [CLS]：句首特殊 token，其最终隐向量用作 NSP 分类输入
- [SEP]：句段分隔符

采样规则

- 50% 时间： B 是 A 在语料中真正的下一句 (label = IsNext)
- 50% 时间： B 是语料中随机抽的另一句 (label = NotNext)

NSP 设计动机是「缩小预训练和微调之间的差距」。但后续工作 (RoBERTa、Joshi et al. 2020、Liu et al. 2019) 指出 NSP 没用甚至有害，最终被移除。

5.4 输入表示

- 词表：30,000 WordPieces (Wu et al., 2016)
- 输入嵌入 = Token Embedding + Segment Embedding + Position Embedding (三者逐位相加)

5.5 模型规模

模型	层数 L	隐藏维度 H	注意力头数 A	参数量
BERT-base	12	768	12	110M
BERT-large	24	1024	16	340M

5.6 预训练细节

- 语料: Wikipedia (2.5B) + BooksCorpus (0.8B)
- 最大序列长度: 512 WordPiece (两段约各 256)
- 训练步数: 1M steps, 约 40 epoch, batch size 128k
- MLM 和 NSP 联合训练: [CLS] 隐向量训练 NSP, 其它 token 隐向量训练 MLM

5.7 微调

「Pre-train once, fine-tune many times.」

- 在下游数据上更新全部参数
- 不同任务接不同的轻量任务头:
 - 句子分类 → [CLS] 上加分类头
 - 序列标注 → 每个 token 上加分类头
 - 问答 (SQuAD) → 预测 answer span 的 start / end 位置
 - 句对任务 → [CLS] 上加分类头 (输入用 [SEP] 拼接)

5.8 实验结果

- GLUE: BERT-large 大幅超越当时所有方法
- SQuAD: BERT-large 在抽取式 QA 上达到 SOTA

5.9 消融实验

预训练任务

- MLM left-to-right LM: 双向上下文的关键作用
- NSP 在某些任务上有提升, 但后续工作 (Joshi 2020、Liu 2019) 认为整体无益

规模

「The bigger, the better」——增加层数 L 、隐藏维度 H 、注意力头数 A 都能稳定提升下游表现。

训练效率

- MLM 收敛比 left-to-right LM 慢, 因为 每步只预测 15% 的 token (每个 token 提供的梯度信号更少)

5.10 BERT 在学什么?

Clark et al., 2019 What Does BERT Look At? An Analysis of BERT's Attention 指出: BERT 的不同 attention 头会聚焦不同语言现象 (指代、句法依存等), 证明预训练学到了结构化语言知识。

6. RoBERTa: 把 BERT 训得更彻底

RoBERTa: A Robustly Optimized BERT Pretraining Approach (Liu et al., 2019)

核心观察: BERT 严重欠训练。RoBERTa 的改动:

1. 移除 NSP: 实验表明 NSP 引入的噪声多于收益
2. 更长训练 + 10x 数据 + 更大 batch
3. 动态掩码: 每个 epoch 重新选择遮蔽位置 (BERT 是静态预生成)
4. 2019 年在 1024 张 V100 上训练约 1 天

[NOTE] 核心启示

RoBERTa 没有改架构、没有加新目标, 只是把 BERT 的训练做扎实, 就把性能再推一截。这预示了 scaling 的重要性: 数据 + 算力 + 训练时长 经常比新算法更管用。

7. Encoder-only 的局限

“Why not use pre-trained encoders for everything?”

If your task involves **generating sequences**, BERT and other pre-trained encoders don't naturally lead to nice **autoregressive (1-word-at-a-time) generation** methods.

也就是说, encoder-only 的 BERT 强在「读」(NLU), 但不擅长「写」(NLG)。生成任务需要 decoder。

8. T5: Text-to-Text Transfer Transformer

Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer (Raffel et al., 2019)

8.1 设计哲学

“Every task ...is cast as feeding our model text as input and training it to generate some target text. This allows us to use the same model, loss function, hyperparameters, etc. across our diverse set of tasks.”

所有任务统一为 **text-to-text**: 翻译、分类、QA、摘要、回归通通转写成「输入文本 → 输出文本」。例:

translate English to German: That is good.	→ Das ist gut.
cola sentence: The course is jumping well.	→ not acceptable
stsb sentence1: ... sentence2: ...	→ 3.8
summarize: <article>	→ <summary>

8.2 架构

完整 **encoder-decoder Transformer**: encoder 双向读输入, decoder 自回归生成输出。这样兼顾「双向理解」和「自回归生成」。

8.3 自监督目标: Span Corruption

T5 不做单 token MLM, 而是 **整段 (span) 掩盖**:

- 掩码比例: 15%
- 掩码单位: **span** (连续若干 token)

- 用 sentinel token (<X>、<Y>、<Z>) 替换 span
- 目标：decoder 顺序生成「sentinel + 对应内容」

举例：

原文： Thank you for inviting me to your party last week

输入： Thank you <X> me to your party <Y> week

目标： <X> for inviting <Y> last <Z>

形式上，记被掩盖的 spans 集合为 $\{s_1, \dots, s_K\}$ ，每个 s_k 由 sentinel z_k 替换。损失为标准的自回归交叉熵：

$$\mathcal{L}_{T5} = - \sum_{k=1}^K \log p_{\theta}(z_k, s_k \mid z_{<k}, s_{<k}, \tilde{x})$$

8.4 数据与训练

- C4 (Common Crawl)：T5 团队专门做了数据清洗
- 在 NLU (GLUE/SuperGLUE) 和 NLG (CNN/DailyMail summarization、WMT 翻译) 上都拿到强势成绩

9. GPT 系列：自回归解码器

回到三类架构表的最后一栏：decoder-only + autoregressive LM。

9.1 GPT-1

Improving Language Understanding by Generative Pre-Training, OpenAI (2018-06)

设计要点：– 用 Transformer decoder (单向、左到右) 替代 LSTM – 预训练目标：标准语言模型 – 训练文本片段长度 512 BPE token，而非单句

目标函数

$$\mathcal{L}_{LM}(\theta) = - \sum_{i=1}^n \log p_{\theta}(x_i \mid x_1, \dots, x_{i-1})$$

其中 p_{θ} 由带因果掩码的 Transformer decoder 给出。

下游适配

微调阶段对全部参数继续训练，输入 / 输出格式按任务转写（例如分类拼成「<start> 文本 <extract>」→ 在 <extract> 位置取 hidden state 接分类头）。

模型规模

项目	数值
层数	12
隐藏维度	768
注意力头数	12
参数量	110M (与 BERT-base 相当)

项目	数值
训练语料	BooksCorpus (0.8B)
最大序列	512 wordpiece
训练	100 epoch, batch size 64

GLUE 实验

GPT-1 在 GLUE 上「promising, but not exciting」——与同期 BERT 双向 + 更多数据相比稍逊。

9.2 LM vs Masked LM 的目标对比

维度	Causal LM (GPT)	Masked LM (BERT)
每次前向训练信号	每个 token 都参与预测 （密度更高）	仅 15% token 提供梯度
文本生成	天然支持（自回归）	不自然
信号质量	「更嘈杂」——只能用左侧上下文	双向上下文，质量更高
微调后表现	一般而言略逊	通常更好
收敛速度	较快	较慢

要点：LM 每个 token 都给梯度，MLM 只有 15%；但 MLM 的双向上下文让单个梯度信号更强——两条路线各有所长。

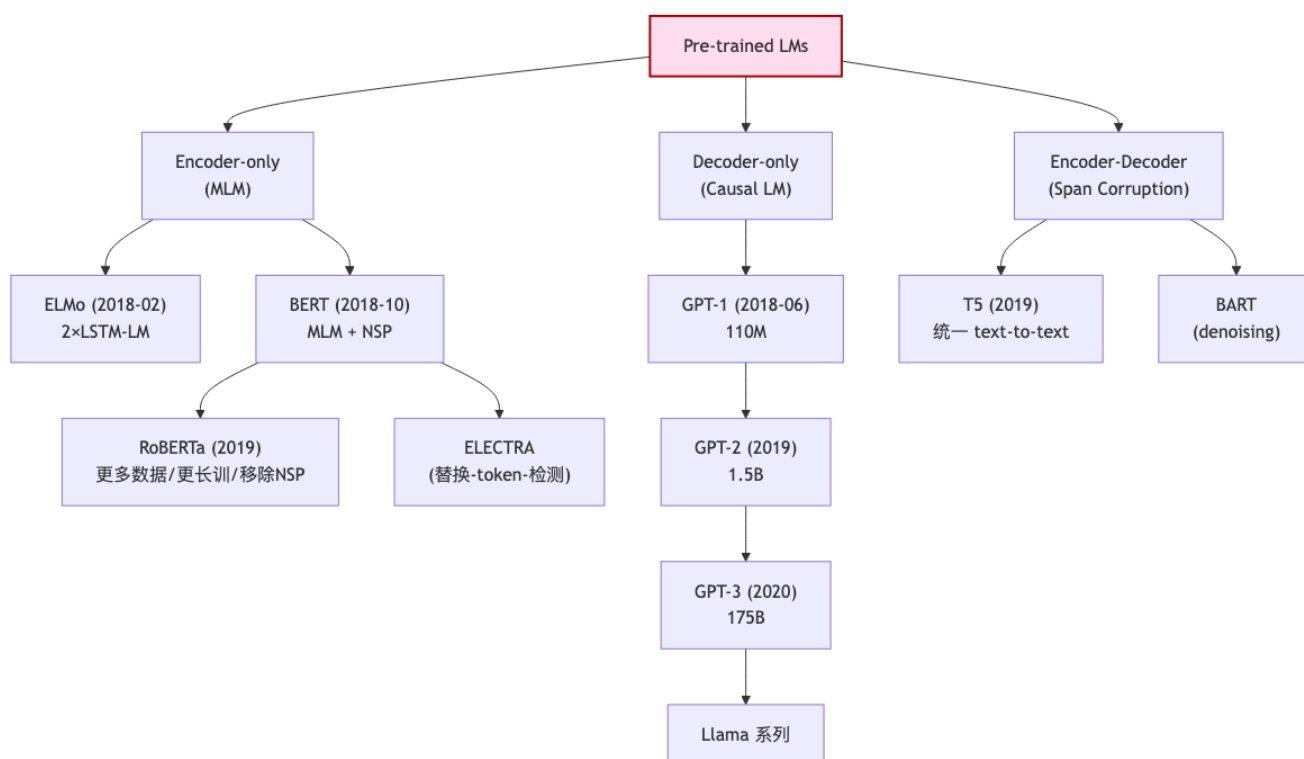
9.3 GPT-2、GPT-3: Scaling Up

模型	参数量	训练词数	发布时间
GPT-1	117 M	~1 B	2018-06
GPT-2	1.5 B	~14 B	2019-02
GPT-3	175 B	~300 B	2020-05

三件事在同步放大：– 更多参数 – 更多（更高质量）训练数据 + 更长文本 – 更多算力

到 2020 年，GPT-3 第一次大规模展示了「模型大到一定程度，不用 fine-tune 也能做新任务」的能力。

10. 模型家族 mermaid 图



注：BART / ELECTRA 仅作为家族成员列出，本课件未展开。

11. 下游适配范式

11.1 Feature Extraction (ELMo 式)

预训练模型权重冻结，只提取 contextual embedding 喂给下游任务模型。下游模型结构与预训练无关。

11.2 Fine-tuning (BERT / GPT 式，2020 之前的默认范式)

预训练模型权重全部参与下游训练，外加少量任务头。

“(Before 2020) Fine-tuning was the default.”—BERT, T5, GPT, ...

特点与代价：– 每个任务单独微调一份模型 – 任务数据量需求：SST-2 有 67k 样本，SQuAD 有 88k 三元组 – 需要计算梯度 + 全量参数更新——对大模型而言代价昂贵

11.3 范式转移的前夜

“What about the Next? Can we somehow try a new paradigm?”

GPT-3 的诞生提示了答案：**prompting / in-context learning** 不更新权重也能执行新任务，从而引出后续课程的 prompt-based learning / instruction tuning 等主题（本节不展开）。

12. 关键模型对比表

模型	年份	架构	预训练目标	参数	训练语料	是否支持生成	主要贡献
ELMo	2018-02	双向 LSTM-LM	前向 LM + 后向 LM	94 M	1B Word Benchmark	否 (仅 embedding)	上下文相关表示, 首次大规模预训练 + 任务无关
GPT-1	2018-06	Transformer decoder	Causal LM	110 M	BooksCorpus (0.8B)	是	首次用 Transformer 做生成式预训练 + 全权重微调
BERT-base	2018-10	Transformer encoder	MLM + NSP	110 M	Wiki + Books (3.3B)	否 (生成不自然)	双向 MLM, 预训练-微调成为通用范式
BERT-large	2018-10	Transformer encoder	MLM + NSP	340 M	同上	否	更大模型, GLUE / SQuAD SOTA
GPT-2	2019	Transformer decoder	Causal LM	1.5 B	~14 B 词	是	显示 scaling 的潜力, zero-shot 初见
RoBERTa	2019	Transformer encoder	MLM (无 NSP)	355 M	160 GB (10x BERT)	否	证明 BERT 被严重欠训练
T5	2019	Encoder-Decoder	Span Corruption	11 B (最大)	C4 (清洗后的 Common Crawl)	是	统一 text-to-text 框架
GPT-3	2020	Transformer decoder	Causal LM	175 B	~300 B 词	是	Few-shot prompting; 推动范式转向 in-context learning

13. 期末复习要点

13.1 概念辨析

1. 静态词向量 vs 上下文向量: word2vec 一词一向量; ELMo / BERT 同词在不同语境下向量不同
2. ELMo 范式 vs BERT/GPT 范式: ELMo 冻结预训练模型, 只把 embedding 注入; BERT/GPT 整体微调
3. 三类架构: encoder-only / decoder-only / encoder-decoder 对应 MLM / Causal LM / Span Corruption
4. MLM 80/10/10 替换: 缓解 [MASK] 在下游不出现的 train-test 不匹配
5. NSP 命运: BERT 引入 → RoBERTa 移除 → 现今被普遍认为弊大于利

13.2 必背公式

- 缩放点积注意力: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$
- MLM 损失: $\mathcal{L}_{\text{MLM}} = -\sum_{i \in \mathcal{M}} \log p_\theta(x_i | \tilde{x})$
- Causal LM 损失: $\mathcal{L}_{\text{LM}} = -\sum_{i=1}^n \log p_\theta(x_i | x_{<i})$
- ELMo 加权: $\text{ELMo}_k^{\text{task}} = \gamma^{\text{task}} \sum_j s_j^{\text{task}} h_{k,j}$

13.3 数字记忆点

事项	数值
BERT-base 参数 / 层 / 头 / 隐藏	110M / 12 / 12 / 768
BERT-large 参数 / 层 / 头 / 隐藏	340M / 24 / 16 / 1024
BERT 词表	30,000 WordPieces
BERT 训练语料	Wiki 2.5B + Books 0.8B
BERT 最大序列	512
MLM 掩码比例	15% (80 / 10 / 10 拆分)
T5 span 掩码比例	15%
GPT-1 / 2 / 3 参数	117M / 1.5B / 175B
GPT-3 训练词数	~300B

13.4 高频追问

- 为何 MLM 比 left-to-right LM 在 NLU 任务上更强? 双向上下文使每个位置都能用左右两侧信息, 对理解任务 (分类、抽取式 QA) 更有利
- 为何 GPT 仍然选择 Causal LM? 自回归性质天然支持文本生成; 同时每个 token 都贡献梯度, 训练信号密度更高
- 为何 ELMo 用前后向独立 LM 而不是真双向? 真双向 LM 会让目标 token 在条件中「泄露」自己, 无法形成有效语言模型目标。BERT 用 MLM 解决了这个问题
- 为何 ELMo 加权多层而非取顶层? 不同层捕获不同语言信息 (低层句法、高层语义), 任务相关加权比硬选某层更灵活
- 为何 BERT 训练时 MLM 收敛慢? 每步只对 15% 的 token 计算 loss, 单次前向获得的梯度信号少
- RoBERTa 启示是什么? 训练时长、数据规模、batch size、动态掩码这些「工程细节」对最终性能极为关键; 不要急于发明新目标
- 2020 前后预训练范式发生了什么转移? 从「每任务都 fine-tune」转向「一次预训练 + prompt / in-context learning 处理多任务」, 由 GPT-3 推动

Lecture 11b —预训练之后：从 GPT 到 GPT-2 / GPT-3 (Scaling & the Shift to Prompting)

[TIP] 学习指南

本节核心：当 fine-tuning 成为 2020 年前的默认范式后，OpenAI 把 decoder-only 语言模型沿「参数 × 数据 × 算力」三条轴一路放大——GPT-1 (117M) → GPT-2 (1.5B) → GPT-3 (175B)。规模带来质变：GPT-2 初见 **zero-shot 多任务**，GPT-3 凭 **in-context learning (few/one/zero-shot)** 不更新一个参数就能做新任务，从而把整个领域从 fine-tuning 推向 prompting 范式。前置知识：– lec11_pretraining ——ELMo / BERT / RoBERTa / T5 / GPT-1，三类架构与三种自监督目标，Causal LM vs MLM 的梯度信号对比 – lec4_lm / lec4_nlm ——语言模型目标函数、困惑度 – Transformer 的因果掩码与 KV cache (lec9 / lec10) 重点掌握：1. GPT-1/2/3 的参数 / 数据 / 时间线三档对比，以及「同步放大三件事」的含义 2. GPT-2 作为 unsupervised multitask learner 的核心主张与其局限（zero-shot 有潜力但 fine-tuning 仍必要） 3. In-context learning 的形式化：zero-shot / one-shot / few-shot 三种 prompt 构造，**无梯度更新** 4. 缩放律（scaling law）的幂律形式，以及「更多参数 + 更多示例 → 更好表现」的经验曲线 5. 涌现能力（emergent abilities）：能力随规模出现的相变现象 6. 为什么 scaling 选择了 decoder-only：密集训练信号、消融成本、KV cache 推理效率 7. fine-tuning → prompting 的范式转变及其动因

1. Motivation：为什么要继续放大

1.1 Fine-tuning 是 2020 前的默认范式

到 2020 年之前，下游任务的标准做法是 **fine-tuning**：拿预训练好的 BERT / T5 / GPT，在每个任务上单独再训一遍。

“(Before 2020) Fine-tuning was the default.”——BERT, T5, GPT, ...

这套范式的代价非常具体：

- **逐任务单独微调**：每个任务都得到一份独立的模型权重副本
- **依赖足量标注数据**：SST-2 有 67k 样本，SQuAD 有 88k 个 (passage, answer, question) 三元组——没有这些就微调不动
- **必须计算梯度并更新全部参数**：对越来越大的模型，这一步越来越昂贵

[NOTE] 核心矛盾

模型越大，表示能力越强，但 fine-tuning 的「为每个任务跑一次全参数梯度更新」的代价也随之爆炸。是否存在一条路，让一个模型不经微调就直接服务多任务？这正是 GPT-2 / GPT-3 要回答的问题。

1.2 放大三件事

预训练的「scaling」同时沿三条轴推进：

- 更多参数 (more parameters)
- 更多、更高质量的训练数据 + 更长文本 (more high-quality data, longer texts)
- 更多算力 (more compute)

这三者必须协同放大，单独堆某一项收益有限——这一直觉在第 7 节的缩放律里会被量化。

2. GPT-1 / 2 / 3：参数·数据·时间线对比

模型	参数量	训练数据	发布时间	相对 GPT-1
GPT-1	117 M	~1 B words	2018-06	1×
GPT-2	1.5 B	~14 B words	2019-02	参数 ~13×, 数据 ~14×
GPT-3	175 B	~300 B tokens	2020-05	参数 ~100× (相对 GPT-2), 数据 ~20×

[NOTE] 数量级对照

课件特意追问：「ELMo, BERT, RoBERTa, GPT, GPT-2 各消耗多少 token？」 – ELMo / GPT-1: ~1 B 量级 – BERT: Wiki 2.5B + Books 0.8B ≈ 3.3 B – RoBERTa: 160 GB (约 BERT 的 10×) – GPT-2: ~14 B – GPT-3: ~300 B tokens GPT-3 相对 GPT-2 参数放大约 100×、数据放大约 20×——参数增长远快于数据增长。

decoder-only 的语言模型目标贯穿整个 GPT 系列：

$$\mathcal{L}_{\text{LM}}(\theta) = - \sum_{i=1}^n \log p_{\theta}(x_i \mid x_1, \dots, x_{i-1}) \quad (1)$$

其中 p_{θ} 由带因果掩码的 Transformer decoder 给出。GPT-2、GPT-3 没有改变目标函数 (1)，只是把 θ 的维度、训练 token 数与算力同步放大。

3. GPT-2：语言模型是无监督多任务学习者

Language Models are Unsupervised Multitask Learners (Radford et al., OpenAI 2019)

3.1 核心主张

GPT-2 的标题本身就是论点：一个仅用语言建模目标训练的模型，在无须任何任务标注数据的情况下，已经能在多种任务上展现 zero-shot 能力。

- zero-shot learning：不使用任何该任务的训练样本
- 跨越不同类型的任务 (cross different types of tasks) 都能见到这种潜力

直觉：互联网级别的语料里本身就自然包含各类任务的演示。例如语料中存在「英文：...中文：...」这种段落，模型在学语言建模时顺带学到了翻译；存在「文章...TL;DR: ...」就顺带学到了摘要。语言模型把这些任务统一编码进了下一个 token 的预测分布里。

[EX] zero-shot 任务框架 (task framing)

不再加任务专属的分类头，而是把任务写成一段自然语言前缀，让模型自回归续写：

摘要： <— 长篇文章> TL;DR: → 模型续写出摘要
翻译： translate to French: cheese => → 模型续写 "fromage"
问答： Q: 长颈鹿有几只眼睛? A: → 模型续写 "两只"
关键在于：任务被编码进 prompt，模型权重一字未改。

3.2 局限：fine-tuning 仍然必要

课件诚实地标注 GPT-2 的现状为 “Limited success”：

- zero-shot 表现「promising」但还远不能取代专门训练的系统
- 在多数基准上，fine-tuning 仍然是必要的

GPT-2 论文的结语恰好点出方向：

“These findings suggest a promising path towards building language processing systems which learn to perform tasks from their **naturally occurring demonstrations**.”

即：从语料中自然出现的演示里学会执行任务——这条路在 GPT-3 上被进一步放大验证。

4. GPT-3: In-Context Learning

Language Models are Few-Shot Learners (Tom B. Brown et al., arXiv:2005.14165, 2020)

4.1 规模

- 参数：175 B (约为 GPT-2 的 100x)
- 数据：300 B tokens (约为 GPT-2 的 20x)

GPT-3 在各类任务上展现了「promising performance」，并暴露出大模型有趣的鲁棒性与常识边界：

[EX] GPT-3 的常识与「编造」

Human: How many eyes does a giraffe have? GPT-3: A giraffe has two eyes.
Human: How many eyes does my foot have? GPT-3: Your foot has two eyes.
Human: How many eyes does a spider have? GPT-3: A spider has eight eyes.
Human: How many eyes does the sun have? GPT-3: The sun has one eye.
Human: How many eyes does a blade of grass have? GPT-3: A blade of grass has one eye.
真常识 (长颈鹿两眼、蜘蛛八眼) 答对，但对无意义问题 (脚 / 太阳 / 草有几只眼) 也会一本正经地编造答案——这反映了语言模型「永远会续写」的特性，而非具备真正的世界模型。

4.2 In-Context Learning 的形式化

GPT-3 最重要的贡献：in-context learning——把若干「示例」直接放进 prompt 的上下文里，模型在单次前向推理中「现学现用」，不进行任何梯度更新、不更新任何参数。

设任务为从输入 x 预测输出 y 。给定 k 个示范对 $\{(x_1, y_1), \dots, (x_k, y_k)\}$ 、一段任务描述 $\mathcal{T}$ 和一个查询 x_{query} ，模型直接对下式做条件生成：

$$\hat{y} = \arg \max_y p_{\theta}(y \mid \mathcal{T}, (x_1, y_1), \dots, (x_k, y_k), x_{\text{query}}) \quad (2)$$

注意 (2) 中 θ 固定不变——「学习」完全发生在前向激活里，而非权重里。按示例个数 k 分为三档：

设定	示例数 k	prompt 形态	是否更新权重
Zero-shot	0	任务描述 + 查询	否
One-shot	1	任务描述 + 1 个示例 + 查询	否
Few-shot	K (如 10~100)	任务描述 + K 个示例 + 查询	否

[EX] few-shot in-context prompt (英 → 法翻译)

Translate English to French: ← 任务描述 T
sea otter => loutre de mer ← 示例 (x_1, y_1)
peppermint => menthe poivrée ← 示例 (x_2, y_2)
plush giraffe => girafe peluche ← 示例 (x_3, y_3)
cheese => ← 查询 x_{query} , 模型续写 "fromage"
三个示例只是被「读进」上下文, 模型据此推断出「=> 左边译成右边」的任务模式。换成 0 个示例即 zero-shot, 换成 1 个即 one-shot。

4.3 Fine-tuning vs In-Context Learning

维度	Fine-tuning (传统)	In-Context Learning (GPT-3)
是否更新参数	是, 全参数梯度更新	否, 权重冻结
每个任务的产物	一份独立模型权重	同一个模型, 换 prompt 即可
数据需求	需要成千上万标注样本	0 ~ 几十个示例放进 prompt
适配开销	计算梯度, 昂贵	仅一次前向推理
适用前提	任意规模模型	模型须足够大才有效

4.4 「更多参数, 更多示例」

GPT-3 的经验曲线给出两个正交的提升方向:

- **More parameters:** 同一设定下, 模型越大, 准确率越高
- **More examples:** 示例数从 $0 \rightarrow 1 \rightarrow \text{few-shot}$, 准确率随之单调上升

二者叠加: **最大模型 + few-shot** 组合给出最佳表现, 且**模型越大, few-shot 相对 zero-shot 的增益越显著**——大模型「更会从上下文里学」。

5. Scaling Laws: 缩放律

GPT-3 之所以敢一路放大, 背后是 **scaling laws** 的支撑: 模型的测试损失 L 随参数量 N 、数据量 D 、算力 C 呈**幂律 (power-law)** 下降, 在 log-log 坐标上近似为直线。其经验形式 (Kaplan et al., 2020):

$$L(N) \approx \left(\frac{N_c}{N}\right)^{\alpha_N}, \quad L(D) \approx \left(\frac{D_c}{D}\right)^{\alpha_D}, \quad L(C) \approx \left(\frac{C_c}{C}\right)^{\alpha_C} \quad (3)$$

其中 N 为参数量、 D 为数据量、 C 为算力, N_c, D_c, C_c 为尺度常数, 指数 $\alpha_\bullet$ 为正的小常数。取对数后:

$$\log L \approx -\alpha_N \log N + \text{const} \quad (4)$$

即「损失对规模的对数」呈线性下降。

[NOTE] 缩放律的实践含义

1. **可外推预测**：在小模型上拟合幂律，即可预测更大模型的表现，从而在烧钱前判断「值不值得放大」。2. **三轴需协调**：(3) 表明 N 、 D 、 C 任一项成为瓶颈都会让损失停滞——这解释了第 1.2 节「三件事同步放大」的必要性。3. **平滑而非饱和**：在课件考察的规模范围内，损失随规模持续平滑下降，尚未见到明显平台——这是「继续放大」的信心来源。

6. Emergent Abilities：涌现能力

缩放律描述的是预训练损失的平滑下降，但在具体下游任务上，能力常常呈现相变式的跃升：当模型规模越过某个阈值，某项能力（如多步算术、复杂推理、指令遵循）从「接近随机」骤然跃升到「显著有效」。

这种「小模型几乎不会、大模型突然会」的现象称为 **emergent abilities**（涌现能力）。

[WARN] 平滑 vs 突变

不要把缩放律的「平滑」与下游能力的「涌现」混为一谈：– **预训练 loss**：随规模幂律平滑下降（公式 (3)）– **下游任务准确率**：可能在某个规模阈值附近非线性跃升 二者并不矛盾——loss 的微小连续改善，可以在某些以阈值/全对全错衡量的任务上表现为指标的突然起跳。

涌现能力是「能否开启新范式」的关键：正因为大到一定程度会出现质变能力，prompting / in-context learning 这类不微调的用法才真正变得可行。

7. 为什么 Scaling 选了 Decoder-only

放大时为什么是 GPT 式的 decoder-only 胜出，而非 encoder-only 或 encoder-decoder？课件给出几条理由。

7.1 三类架构回顾

Architecture	Pretraining objective	Examples
Encoder-only	Masked Language Models	BERT, RoBERTa, ELECTRA
Encoder-Decoder	Span Corruption	T5, BART
Decoder-only	Autoregressive Language Models	GPT-2, GPT-3, Llama

7.2 Encoder-only 为何掉队

- 为表示学习而设计，天然不擅长生成
- 需要任务专属 fine-tuning
- 当大模型靠 (0/few-shot) prompting 就能工作时，而 prompting 需要生成——encoder-only 在这条新范式里用处变小

7.3 Encoder-Decoder 为何没继续放大

encoder-decoder 支持生成、且 encoder 侧有双向上下文，但放大它有几重摩擦：

- 「Ablation Tax」（消融税）：架构里多出一堆需要调的超参——掩码比例该多少？encoder 与 decoder 各分多少参数？每个选择都要做昂贵的消融实验
- decoder-only 则简单得多：只是一摞相同的 block 堆叠，超参空间小、易于放大

7.4 Decoder-only 的两大优势

(a) 密集训练信号 (dense training signals)

decoder-only 对序列里**每一个 token** 都产生预测信号；而 MLM 只对约 **15%** 被掩码的 token 计算 loss。

$$\frac{\# \text{signals}_{\text{Causal LM}}}{\# \text{signals}_{\text{MLM}}} \approx \frac{1.0}{0.15} \approx 7 \times \quad (5)$$

即在相同算力下，decoder-only 获得约 **7 倍** 的训练信号——放大时单位算力的「学习效率」更高。

(b) KV cache 与推理效率

decoder-only 自回归生成时可缓存历史 token 的 key / value (**KV cache**)，新 token 只需与缓存交互，避免重复计算，使长序列生成的推理高效——这对「服务多任务、长 prompt」的实际部署至关重要。

[NOTE] 一句话总结

Decoder-only 之所以成为 scaling 的赢家：**训练信号最密集（每 token 都学，~7x of MLM）、架构最简单（只堆同构 block，免消融税）、推理最高效（KV cache）**——三点叠加让它在大规模下性价比最高。

8. 范式转变：Fine-tuning → Prompting

8.1 旧范式：Pre-training + Fine-tuning

[大规模无标注语料] → 预训练 → [任务 A 数据] 全参数微调 → 模型 A
[任务 B 数据] 全参数微调 → 模型 B
[任务 C 数据] 全参数微调 → 模型 C

每个任务一份模型，逐一付出梯度更新的代价。

8.2 新范式：Pre-training + In-Context Learning

[大规模无标注语料] → 预训练 → 单一大模型（权重冻结）
├ prompt A → 任务 A 输出
├ prompt B → 任务 B 输出
└ prompt C → 任务 C 输出

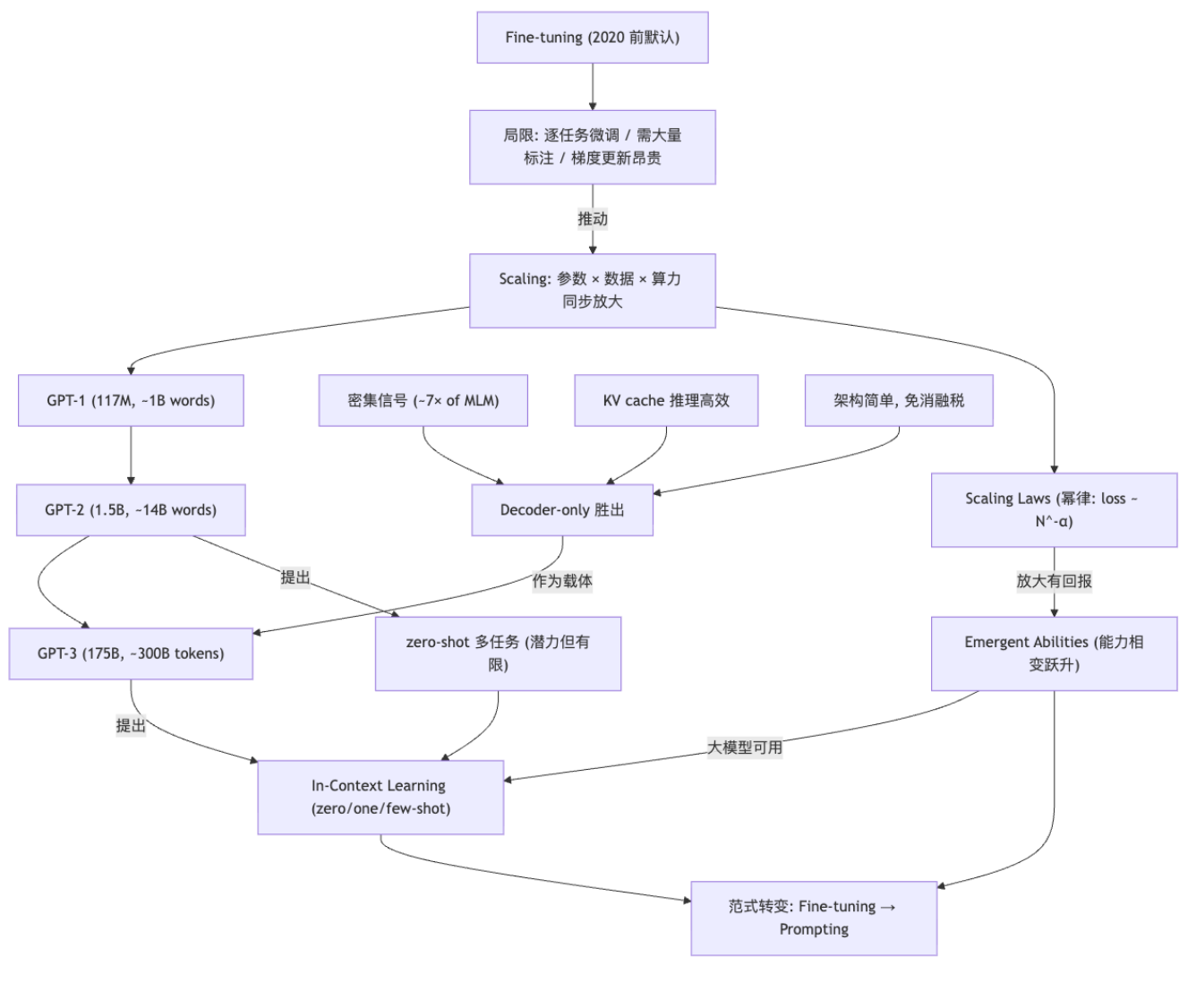
同一个冻结的大模型，**换 prompt 即换任务**，无须任何梯度更新。

8.3 转变的动因与下一步

“What about the Next? Can we somehow start a new paradigm?”

驱动这次转变的，正是前面几节累积的事实：**缩放律保证放大有回报** → 涌现能力让大模型具备 **zero/few-shot 可用性** → **in-context learning 让适配零成本**。于是「prompt」取代「fine-tune」成为与模型交互的主入口，引出后续课程的 **prompt engineering / instruction tuning / RLHF** 等主题（本节不展开）。

概念关系



记号速查

符号	含义
N	模型参数量
D	训练数据量 (token 数)
C	训练算力 (compute)
L	测试损失 (test loss / cross-entropy)
$\alpha_N, \alpha_D, \alpha_C$	缩放律幂律指数 (正小常数)
N_c, D_c, C_c	缩放律尺度常数
k / K	in-context 示例个数 (few-shot 中的 shot 数)
$\mathcal{T}$	prompt 中的任务描述
(x_i, y_i)	prompt 中第 i 个示范输入-输出对
x_{query}	待预测的查询输入
θ	模型参数 (in-context learning 中保持冻结)

符号	含义
$p_{\theta}(\cdot)$	decoder 给出的 (条件) 概率分布

[TIP] 期末复习要点

数字记忆点

事项	数值
GPT-1 参数 / 数据 / 年份	117M / ~1B words / 2018-06
GPT-2 参数 / 数据 / 年份	1.5B / ~14B words / 2019-02
GPT-3 参数 / 数据 / 年份	175B / ~300B tokens / 2020-05
GPT-3 相对 GPT-2	参数 ~100x, 数据 ~20x
Causal LM vs MLM 信号比	~7x (100% vs 15%)
MLM 掩码比例	15%

概念辨析 1. **GPT-2 主张**: 语言模型即无监督多任务学习者; zero-shot 有潜力但 fine-tuning 仍必要 (“limited success”) 2. **In-context learning**: zero / one / few-shot 三档, 靠 prompt 现学, **不更新任何参数** 3. **Fine-tuning vs ICL**: 前者全参数梯度更新、逐任务一份模型; 后者权重冻结、换 prompt 换任务 4. **Scaling law**: loss 随 $N/D/C$ 幂律下降, 可外推预测; 三轴需协调放大 5. **Emergent abilities**: 下游能力随规模出现相变跃升 (与 loss 的平滑下降不矛盾) 6. **Decoder-only 胜出三因**: 密集训练信号 (~7x)、架构简单免消融税、KV cache 推理高效

高频追问 – 为何 GPT-3 不微调也能做新任务? 规模带来涌现能力, 使其能在 prompt 上下文中现学示例 (in-context learning), 单次前向即可完成任务适配。– **为何 scaling 选 decoder-only 而非 encoder-decoder?** decoder-only 每 token 都学、信号密度约为 MLM 的 7 倍; 架构只是同构 block 堆叠, 免去 encoder-decoder 的「消融税」; 且 KV cache 让自回归推理高效。– **为何 encoder-only 在新范式里掉队?** 它为表示学习设计、不擅生成, 需任务专属微调; 而 prompting 范式要求生成能力。– **缩放律的「平滑」和涌现的「突变」矛盾吗?** 不矛盾: 预训练 loss 平滑下降, 但以阈值/全对全错衡量的下游任务指标可在某规模附近非线性跃升。– **范式为何从 fine-tuning 转向 prompting?** 缩放律保证放大有回报 → 涌现能力让大模型 zero/few-shot 可用 → in-context learning 让任务适配零成本, 于是 prompt 取代 fine-tune 成为主入口。

Anki 卡片

问	答
GPT-1 / GPT-2 / GPT-3 的参数量分别是多少?	117M / 1.5B / 175B
GPT-1 / GPT-2 / GPT-3 的训练数据量分别约为多少?	~1B words / ~14B words / ~300B tokens
GPT-3 相对 GPT-2 在参数和数据上各放大了多少?	参数约 100x, 数据约 20x
GPT-2 论文标题的核心主张是什么?	“Language Models are Unsupervised Multitask Learners”——纯语言建模即可获得 zero-shot 多任务能力
GPT-2 的 zero-shot 表现如何评价?	promising 但 limited success, fine-tuning 仍然必要
GPT-3 论文标题强调什么能力?	“Language Models are Few-Shot Learners”——few-shot in-context learning
in-context learning 与 fine-tuning 最本质的区别?	in-context learning 不更新任何参数 (权重冻结), 仅靠 prompt 上下文现学; fine-tuning 做全参数梯度更新
zero-shot / one-shot / few-shot 如何区分?	按 prompt 中放入的示例数 k : 0 / 1 / 多个 (如 10~100), 均不更新权重
写出 GPT 系列的语言模型目标函数。	$\mathcal{L}_{\text{LM}} = - \sum_i \log p_{\theta}(x_i x_{<i})$

问	答
Scaling law 的幂律形式是怎样的？	$L(N) \approx (N_c/N)^{\alpha_N}$, loss 随参数/数据/算力呈幂律下降, log-log 下近似直线
Scaling law 的最大实践价值是什么？	可在小模型上拟合幂律, 外推预测大模型表现, 从而在烧钱前判断是否值得放大
什么是 emergent abilities (涌现能力)？	某项下游能力随模型规模越过阈值时从近随机骤升到显著有效的相变现象
缩放律的「平滑」与涌现的「突变」为何不矛盾？	预训练 loss 平滑下降, 但以阈值/全对全错衡量的下游指标可在某规模附近非线性跃升
Causal LM 相对 MLM 的训练信号多多少？为什么？	约 7x: decoder-only 对每个 token 都产生信号, MLM 只对约 15% 被掩码 token 计算 loss
三类预训练架构及其目标？	Encoder-only / MLM (BERT); Encoder-Decoder / Span Corruption (T5); Decoder-only / Autoregressive LM (GPT)
为什么 scaling 时 decoder-only 胜出？	训练信号最密集 (~7x of MLM)、架构只是同构 block 堆叠免「消融税」、KV cache 使自回归推理高效
encoder-decoder 放大时的「ablation tax」指什么？	掩码比例、encoder/decoder 参数分配等额外超参都需昂贵消融, 而 decoder-only 超参空间小
为什么 encoder-only 在 prompting 范式里用处变小？	它为表示学习设计、不擅生成且需任务专属微调, 而 prompting 需要生成能力
2020 年前后预训练范式发生了什么转变？	从「逐任务 fine-tuning」转向「单一冻结大模型 + prompt / in-context learning 处理多任务」
GPT-3 对「太阳有几只眼睛」类问题的回答说明了什么？	语言模型总会续写、会对无意义问题一本正经地编造, 反映其缺乏真正的世界模型

Lecture 12 —提示学习 (Prompting)

[TIP] 学习指南

本节核心：进入 GPT-3 之后的新范式——不再为每个任务单独微调，而是把任务用一段「文本提示 (prompt)」描述出来，让冻结的预训练大模型直接补全答案。围绕 prompt 的标准 workflow (模板填充 → 答案搜索 → 答案映射)、few-shot 提示与上下文学习 (ICL)、思维链 (CoT)、提示工程 (prompt engineering) 以及检索增强生成 (RAG) 五条主线展开。**前置知识：**– lec11_pretraining ——预训练-微调范式、MLM / Causal LM、BERT / GPT – lec11b_gpt_scaling ——GPT-1/2/3 的参数 / 数据 / 算力 scaling, next-token prediction 目标 – 注意，prompting 的「能力」直接建立在 next-token prediction 这一预训练目标之上 **重点掌握：**1. Prompting 的形式化定义：把任务改写为 $\arg \max_y P(y \mid \text{prompt}(x))$ 2. 标准 workflow 四步：模板填充 (template) → 答案预测 (answer search) → 后处理 (output selection) → 答案映射 (answer mapping) 3. Few-shot / In-Context Learning 的构造方式与「反直觉」实验 (Min et al. 2022) 4. 如何挑选与排序 in-context examples (相关性 / 多样性 / 自生成；顺序敏感) 5. Chain-of-Thought (含 zero-shot CoT)、Program-aided / Program-of-Thoughts、Tree-of-Thoughts 6. Prompt 设计 (手工 vs 自动搜索：离散 / 连续空间) 7. RAG 三阶段流水线 (检索 → 拼接 → 生成) 及其形式化边缘化 $P(y \mid x) = \sum_z P(z \mid x)P(y \mid x, z)$

1. 什么是 Prompting / Motivation

1.1 定义

Prompting：通过提供一段说明任务的文本提示 (textual prompt)，引导一个预训练好的模型做出特定预测。[Pengfei Liu]

换言之，prompt 是一种「通过向输入添加额外文本，更好地调用预训练模型已有知识」的技术。

关键在于：模型权重**不更新**，我们只是把任务「翻译」成模型在预训练时见惯的「补全 (completion)」形式，让它顺着 next-token prediction 的惯性把答案续写出来。

1.2 Basic Prompting：把任务写成补全

最朴素的做法：在序列前面拼一段文本，让模型续写。

[EX] 续写式提示 (Radford et al. 2018)

输入 $x = \text{When a dog sees a squirrel, it will usually - GPT-2 Small 续写: be afraid of anything unusual...}$ (语义不连贯) – GPT-2 XL 续写: lick the squirrel. It will also touch its nose to the squirrel... (明显更合理)

这一能力完全来自神经语言模型的 next-token prediction 训练目标——模型本就擅长「给定前缀续写最可能的后续」。

1.3 形式化

设输入 x ，提示函数 $\text{prompt}(\cdot)$ 把 x 改写成提示串，模型按条件概率选出答案 y ：

$$\hat{y} = \arg \max_{y \in \mathcal{V}_z} P_{\theta}(y \mid \text{prompt}(x)) \quad (1)$$

其中 $\mathcal{V}_z$ 是「答案候选词表」（如情感分类里 $\{\text{fantastic}, \text{terrible}\}$ ）。参数 θ 来自预训练且冻结，这是与 fine-tuning 范式的根本区别。

2. Prompt 基本 workflow (Basic Workflow)

标准流程分四步：

1. 分析任务 → 写出任务描述 (task description)
2. 设计提示模板 → 可引入 chain / tree / program of thought
3. 填充模板 → 配合示例 (demonstrations)
4. 预测答案，并对输出做后处理得到最终答案

2.1 Prompt 模板 (Template)

模板是一个带空槽的文本，把真实输入 $[x]$ 填进去，并留出答案槽 $[z]$ ：

[EX] 情感分析模板

– 输入: $x = \text{"I love this movie"}$ – 模板: $[x]$ Overall, it was $[z]$ – 提示串: $x' = \text{"I love this movie. Overall it was [z]"}$

2.2 答案预测 (Answer Prediction / Search)

给定提示串，让模型补全 $[z]$ ，再用「词表字符串匹配」读出答案：

Prompting: "I love this movie. Overall it was $[z]$ " Predicting: "I love this movie. Overall it was fantastic"

补全 $[z]$ 可用任意推理算法（贪心、束搜索、采样等）。

2.3 后处理 (Post-Processing)

根据补全结果选出真正要的输出，常见做法：– 原样取用 (taking the output as-is) – 格式化输出 (json / markdown / code，便于展示) – 把输出片段映射为动作（如关键词 → 调用工具）– 甚至用另一个 LLM 抽取答案

[EX] 用另一个 LLM 抽答案

user: Does the following sentence imply a positive sentiment? "Overall it was fantastic." LLM output: "Yes, it ..."

2.4 输出选择 (Output Selection)

从较长的回答里挑出指示答案的信息：

Predicting: "... Overall it was a movie that was simply fantastic" ↓
Extraction: fantastic

不同任务的抽取方法：– 分类：识别关键词 – 回归 / 数值问题：识别数字 – 代码：抽取可执行代码片段 – LLM 法：用另一个（小）模型来总结答案

2.5 答案映射 (Output Mapping)

把抽出的答案映射到类别标签或连续值，通常多个词映射到同一类：

Extraction: fantastic $\Rightarrow$ Mapping: Positive – Positive $\Rightarrow$ {Interesting, Fantastic, Happy, ...} – Negative $\Rightarrow$ {Boring, 1-star, ...}

这一步对应公式 (1) 中候选词表 $\mathcal{V}_z$ 到标签空间 $\mathcal{Y}$ 的映射 $v: \mathcal{V}_z \rightarrow \mathcal{Y}$ (即 verbalizer):

$$\hat{\text{label}} = v\left(\arg \max_{z \in \mathcal{V}_z} P_{\theta}(z | x')\right) \tag{2}$$

3. Prompt 设计策略 (Designing Strategy)

什么是好的 prompt:

要素	说明
清晰的任务指令	明确的任务描述、必要背景、输出格式、约束条件
精选的信息	尤其是 LLM 可能不知道的：最新信息、新闻、领域知识
精心设计的示例	给出清晰示例让模型模仿
针对难任务的更好指令	指导 / 演示如何处理复杂任务
角色扮演 (role playing)	常能得到更对路的回复，如 "You are a cardiologist, ..."

4. Few-shot Prompting 与 In-Context Learning (ICL)

4.1 定义

Few-shot prompting: 在指令之外再给几个任务示例。一个完整的 few-shot 提示由三部分构成：1. 自然语言形式的任务指令 (instruction) 2. 几个示例 / 演示 (**demonstrations**)，用你想要的风格写出 3. 你当前的输入

模型据此输出答案。这种「读几个例子就会做新任务」的能力常被称为**指令遵循 (instruction following)** 或上下文学习 (**In-Context Learning, ICL**)。[Brown et al. 2020]

[EX] Few-shot 情感分类提示

Review: A heartwarming masterpiece. Sentiment: Positive
Review: Dull and far too long. Sentiment: Negative
Review: The acting saved an otherwise weak plot. Sentiment: Positive
Review: I love this movie. Sentiment: ← 模型补全
模型从前 3 个 (输入, 标签) 对里「读懂」任务格式，再给最后一行补 Positive。

4.2 形式化

设有 k 个示例 (x_i, y_i) 与测试输入 x_{test} ，ICL 把它们拼接为上下文 C ，再求条件概率最大的答案：

$$\hat{y} = \arg \max_y P_\theta \left(y \mid \underbrace{(x_1, y_1), \dots, (x_k, y_k)}_{\text{demonstrations}}, x_{\text{test}} \right) \quad (3)$$

注意全程不更新 θ ——所谓「学习」只发生在前向传播的上下文里，故称 in-context learning。

4.3 选择 In-Context Examples

4.3.1 LLM 对示例高度敏感

- 示例顺序 (ordering) [Lu et al. 2021]
- 标签均衡 (label balance) [Zhang et al. 2022]
- 标签覆盖 (label coverage) [Zhang et al. 2022]

4.3.2 反直觉的发现 [Min et al. 2022]

[WARN] In-Context Learning 的「反直觉」结论

- 把正确标签换成随机标签，几乎不损害准确率——说明 demonstrations 主要让模型「识别任务格式 / 标签空间」，而非真正从 (x,y) 配对里学映射。
 - 示例更多有时反而拉低准确率。
- ACL 2023 进一步把 ICL 拆为「任务识别 (task recognition)」与「任务学习 (task learning)」两种成分。

4.3.3 如何挑选示例

策略	做法	出处
相关性排序 (Relevance)	选与测试样本最近的 K 个邻居 (KNN, 按嵌入相似度)	ACL 2022
多样集合 (Diverse Set)	选有代表性、高覆盖的样本	ACL 2023
自生成 (Self-generated)	让 LLM 自己生成相关示例	NAACL 2022

4.3.4 顺序也重要 (Order matters)

- 相关性排序：把最相似的示例放在最后（离输入最近）[ICML 2021]
- 试运行 (dry-run)：用一小批样本找最佳顺序 [NAACL 2022]

5. Chain of Thought (CoT)

5.1 思维链提示 [Wei et al. 2022]

让模型在给出答案之前先解释推理过程。直觉上这相当于给模型提供了自适应的计算时间 (adaptive computation time) ——把多步推理摊开到更多 token 上逐步完成。

[EX] CoT 提示 (数学应用题)

示例 (demonstration): Q: Roger has 5 tennis balls. He buys 2 cans of 3 balls each. How many does he have now? A: Roger started with 5 balls. 2 cans $\times$ 3 balls = 6 balls. 5 + 6 = 11.

测试输入: Q: The cafeteria had 23 apples. They used 20 and bought 6 more. How many apples now? A: They started with 23. After using 20, 23 - 20 = 3. Then bought 6, 3 + 6 = 9.

关键在于示例答案里写出了中间步骤，模型学会先推理再作答，复杂算术 / 常识推理的准确率显著提升。

后续工作把推理能力在 SFT / RL 阶段直接注入模型，成为热门方向 (reasoning models)。

5.2 无监督 / Zero-shot CoT [Kojima et al. 2022]

只需在提示后加一句鼓励解释的话就能诱导推理：

[EX] Zero-shot CoT

Q: ... (题目) A: **Let's think step by step.** ← 仅此一句即可触发分步推理

论文还发现，甚至加几个点号 ... 都可能起作用。如今 GPT 系列经过 instruction tuning 后，无需特定指令也能自发推理。

5.3 把输出结构化为程序 (Program-structured Output)

[Madaan et al. 2022] 指出：当输出是结构化的，用编程语言（如 Python）表达往往提升准确率。原因：– 程序高度结构化，且大量出现在预训练数据里（⇒ GitHub!）– 让模型生成 JSON 有助于格式化答案，尤其在处理表格 / 成对的实体-属性时

5.4 Program-aided Language Models (PAL) [Gao et al. 2022]

用程序生成输出比直接让 LM 算更精确，尤其对数值问题。模型负责把题目翻译成代码，真正的计算交给解释器执行——这正是后来「工具调用 / coding agent」的雏形。

5.5 Program of Thoughts (PoT) [Chen et al. 2023]

用 Codex 同时生成自然语言文本 + 编程语句，最后由程序算出答案，把「推理」与「计算」解耦。

5.6 Tree of Thoughts (ToT) [Yao et al. 2023]

进一步从「链」升级到「树」：– 同时考虑多条推理路径，对各选择自我评估 (self-evaluate) – 必要时前瞻 (look ahead) 或回溯 (backtrack) – 可与各种搜索算法结合：BFS / DFS / MCTS

[NOTE] 推理范式的演进

Chain (链) → Program (程序辅助) → Tree (树搜索)。核心思想一脉相承：给模型更多的「思考预算」与「自我纠错」机会，把一次性作答拆成可检验的多步过程。

6. Prompt Engineering (提示工程)

6.1 Prompts Matter: 措辞决定成败

[EX] 同一任务、不同模板，准确率天差地别 (BERT 零样本情感分析)

模板 A: <a movie review> The review is _____. 答案集 {positive, negative} → 准确率 0.534
模板 B: <a movie review> The movie is _____. 答案集 {fantastic, terrible} → 准确率 0.749

仅仅把「review is positive/negative」换成「movie is fantastic/terrible」，准确率从 0.534 跳到 0.749。提示的措辞与答案词的选择对结果影响巨大。

6.2 Prompt 设计的两条路线

- 手工设计 (Manual design): 依据任务特性配置模板
- 自动搜索 (Automated search):

- 在离散空间 (discrete space) 搜索 (搜真实的词 / token)
- 在连续空间 (continuous space) 搜索 (搜可训练的向量, 即 soft prompt)

6.3 手工指令的要点

指令应当清晰、简洁、易懂:

[EX] 模糊 vs 精确 (promptingguide.ai)

较模糊: Explain the concept Prompt in LLMs. Keep the explanation short, only a few sentences, and do not be too descriptive. **更精确:** Use 2–3 sentences to explain the concept Prompt in LLMs to a high school student.

越大的模型越「像对人介绍一样」对待它; 指令越含糊, 结果往往越不准。

6.4 自动提示工程方法

方法	空间	说明
Prompt paraphrasing	离散	改写已有提示得到候选
Gradient-based discrete search (如 AutoPrompt)	离散	用梯度信号在 token 空间搜索触发词
Prompt tuning	连续	只训练拼在输入前的少量软提示向量
Prefix tuning	连续	在每层注入可训练前缀向量

6.5 Prompt Paraphrasing [Jiang et al. 2019]

把一个提示改写成多个候选, 再挑表现最好的:

[EX] 关系探测提示改写

[X] shares a border with [Y] ↓ (Paraphrasing Model) [X] has a common border with [Y]. [X] adjoins [Y]. ...

改写手段: 回译 (back-translation)、用同义词典替换短语、用神经重写器改写; 还可迭代改写 [Zhou et al. 2021].

[NOTE] 离散 vs 连续提示

离散提示是**真实文本** (可读、可迁移、便于人理解); 连续提示 (prompt / prefix tuning) 是**可训练向量** (不可读, 但优化空间更大、参数高效)。两者都属于「参数高效迁移学习 (PEFT)」家族。

7. Retrieval-Augmented Generation (RAG)

7.1 为什么需要 RAG

视角	动机
对 LLM	缓解幻觉 (hallucination); 补充 最新信息 ; 注入 领域知识 ; 恰当地结合自身知识与外部知识
对应用	在众多领域给出 精确答案 ; 维护 最新信息 ; 可回溯 (backtracking) 生成内容的来源

7.2 三阶段流水线 [Gao et al. 2024]

1. 信息检索 (Information Retrieval): 从外部数据存储 (datastore) 中检索信息
2. 拼入提示 (Insert into Prompt): 过滤检索结果, 构造新的提示
3. 生成 (Generation): 用新提示生成答案

7.3 形式化

把检索文档 z 视为隐变量, 对其边缘化 (marginalize), RAG 的生成概率为:

$$P(y | x) = \sum_{z \in \mathcal{Z}} \underbrace{P(z | x)}_{\text{retriever}} \underbrace{P(y | x, z)}_{\text{generator}} \quad (4)$$

其中检索器 $P(z | x)$ 通常用双塔 (dual-encoder) 对查询和文档分别编码, 取相似度的 softmax; 实践中只对 top- k 检索结果求和近似:

$$P(y | x) \approx \sum_{z \in \text{top-}k} P(z | x) P(y | x, z) \quad (5)$$

7.4 改进 RAG: 两个方向

整体可改进两处: 改进 IR 步骤、改进 IR 与 Prompt 的结合。

7.4.1 改进 IR 步骤

- 改进数据存储的组织: 为原始条目构造数据摘要 (data digest); 为 LLM 需求生成新索引
- 改进查询表示:
 - 查询扩展 (query expansion) [NAACL 2025]: 补全缺失信息; 把复杂查询分解为小查询再组合
 - 查询重写 (query re-writing) [EMNLP 2023]: 突出关键信息、消歧; 用小 LLM 重写; 翻译成多语言
- 重排序 (Re-ranking): 用 LLM 重新排序检索列表

7.4.2 改进结合步骤

- 压缩检索信息: 用文本摘要 / 信息抽取找关键信息; 识别重要 token [EMNLP 2023]
 - $\Rightarrow$ 上下文工程 (context engineering, 2024)
 - $\Rightarrow$ agentic context engineering (ACE, 2025)
- 迭代检索 (Iterative retrieval):
 - 评估模型输出后反复执行检索
 - 与 Chain-of-Thought 结合 [ICLR 2024]
 - 自适应决定何时检索 [ICLR 2024]
 - 交织推理与搜索 (Search-R1、Search-O1, 2025); agentic search (2025)

7.5 开放问题

[WARN] 我们真的需要 RAG 吗?

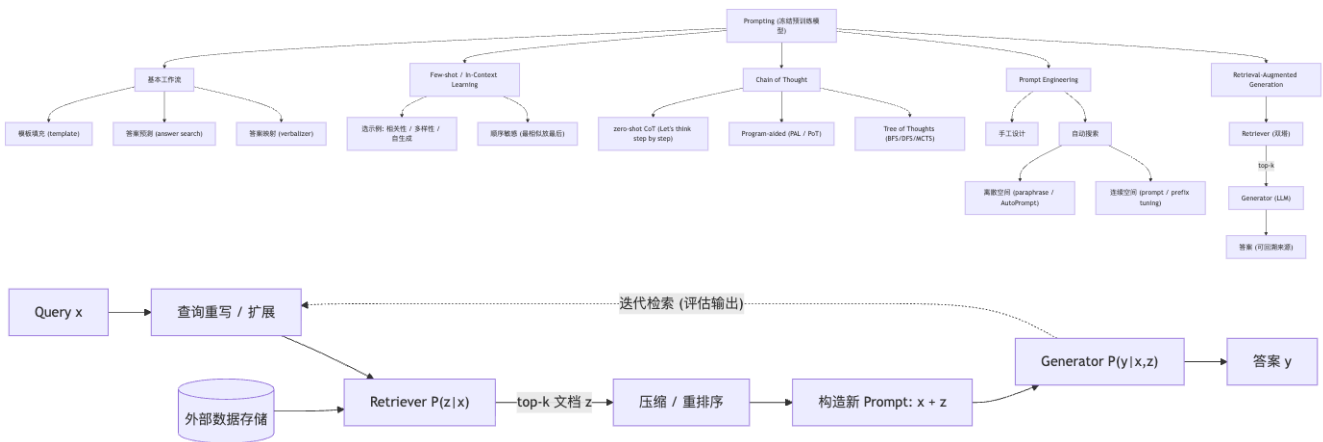
– IR 并不完美——检索噪声会污染上下文。– LLM 是否已足够强——参数化知识可能已覆盖部分需求。– 何时检索? 先检索后生成 (retrieval-then-generation), 还是先生成后检索 (generation-then-retrieval)?

8. Take-home Message

[TIP] 本节核心结论

– **Prompt 很关键**, 措辞与格式会显著改变结果(0.534 → 0.749)。– **小心你带进来的信息**: 示例(demonstrations)与检索到的内容都会影响输出。– **管好上下文 (context) 与记忆 (memory)** ——这正通向 context engineering / agentic 方向。

概念关系



记号速查

记号	含义
$\text{prompt}(x)$	把输入 x 改写成提示串的函数
x'	填好模板的提示串 (含答案槽 $[z]$)
$[x] / [z]$	模板中的输入槽 / 答案槽
$\mathcal{V}$	答案候选词表 (如 $\{\text{fantastic}, \text{terrible}\}$)
$v(\cdot)$	verbalizer, 候选词 $\rightarrow$ 标签的映射
(x_i, y_i)	第 i 个 in-context 示例 (demonstration)
k	few-shot 示例个数 / 检索 top- k
z	RAG 中检索到的文档 (隐变量)
$P(z \mid x)$	检索器 (retriever) 打分
$P(y \mid x, z)$	生成器 (generator) 条件概率
θ	预训练模型参数 (prompting 时冻结)

[TIP] 期末复习要点

概念辨析 1. **Prompting vs Fine-tuning**: prompting 冻结权重、把任务写成补全 (公式 1); fine-tuning 更新全部权重。2. **Zero-shot / Few-shot / ICL**: few-shot = 指令 + 几个示例 + 输入; ICL 指「在上下文里学习、不更新参数」(公式 3)。3. **CoT vs zero-shot CoT**: 前者在示例里写出推理步骤; 后者只加一句“Let’s think step by step”。4. **离散 vs 连续提示**: 离散是真实文本 (paraphrase / AutoPrompt); 连续是可训练向量 (prompt / prefix tuning)。5. **RAG 三阶段**: 检索 → 拼接 → 生成; 形式上是对文档 z 边缘化 (公式 4)。
必背公式 – Prompting: $\hat{y} = \arg \max_y P_\theta(y \mid \text{prompt}(x))$ – ICL: $\hat{y} = \arg \max_y P_\theta(y \mid (x_1, y_1), \dots, (x_k, y_k), x_{\text{test}})$ – RAG 边缘化: $P(y \mid x) = \sum_z P(z \mid x) P(y \mid x, z)$
高频追问 – **为什么 prompting 能 work?** 因为预训练的 next-token prediction 让模型擅长按前缀续写; 把任务改写成模型熟悉的补全形式即可调用其知识。– **为什么换随机标签几乎不掉点 (Min et al. 2022)?** demonstrations 主要让模型识别任务格式与标签空间 (task recognition), 而非真学 (x, y) 映射。– **为什么 CoT 有用?** 提供自适应计算时间, 把多步推理摊到更多 token 上逐步完成。– **PAL / PoT 为什么对数值题更准?** 把计算交给程序解释器执行, 避免 LM 直接算数出错。– **为什么需要 RAG?** 缓解幻觉、补充最新 / 领域知识、可回溯来源。– **RAG 的两个改进方向?** 改进 IR 步骤 (数据组织 / 查询重写扩展 / 重排序) 与改进结合步骤 (压缩 / 迭代检索)。

Anki 卡片

问	答
Prompting 的一句话定义	通过提供说明任务的文本提示, 引导一个冻结的预训练模型做出特定预测; 不更新权重
Prompting 形式化目标	$\hat{y} = \arg \max_y P_\theta(y \mid \text{prompt}(x))$
基本 prompting 能 work 的根源	神经语言模型的 next-token prediction (续写) 训练目标
标准提示 workflow 四步	分析任务 → 设计模板 → 填模板 (配示例) → 预测并后处理答案
Prompt 模板举例	输入 I love this movie, 模板 [x] Overall, it was [z] → I love this movie. Overall it was [z]
verbalizer (答案映射) 作用	把抽取出的词映射到类别, 如 fantastic → Positive; 常多词映射到一类
好 prompt 的要素	清晰指令、精选信息 (LLM 不知道的)、清晰示例、难任务的更好指导、角色扮演
Few-shot prompting 的三部分	自然语言任务指令 + 几个示例 (demonstrations) + 当前输入
In-Context Learning 形式化	$\arg \max_y P_\theta(y \mid (x_1, y_1) \dots (x_k, y_k), x_{\text{test}})$, 全程不更新 θ
Min et al. 2022 反直觉结论	把正确标签换成随机标签几乎不掉点; 示例更多有时反而更差
选 in-context 示例的三种策略	相关性排序 (KNN)、多样高覆盖集合、LLM 自生成
In-context 示例顺序的经验	把最相似的示例放在最后 (离输入最近); 可 dry-run 找最佳顺序
Chain-of-Thought 的核心思想	让模型先解释推理再作答, 提供自适应计算时间 (adaptive computation time)
Zero-shot CoT 触发句	“Let’s think step by step” (Kojima et al. 2022)
PAL / Program-of-Thoughts 优势	把计算交给程序解释器执行, 对数值问题比 LM 直接算更精确
Tree of Thoughts 特点	多条推理路径 + 自评估 + 前瞻/回溯, 结合 BFS/DFS/MCTS

问	答
「Prompts Matter」实验数字	BERT 零样本情感：模板 A(review is positive/negative)=0.534, 模板 B(movie is fantastic/terrible)=0.749
Prompt 设计两条路线	手工设计；自动搜索（离散空间 / 连续空间）
离散 vs 连续自动提示方法	离散：paraphrasing、gradient-based(AutoPrompt)； 连续：prompt tuning、prefix tuning
为什么需要 RAG	缓解幻觉、补充最新信息与领域知识、可回溯来源
RAG 三阶段	信息检索 → 拼入提示 (过滤后) → 生成
RAG 形式化 (边缘化)	$P(y x) = \sum_z P(z x)P(y x, z)$, 实践取 top- k 近似
改进 RAG 的两大方向	改进 IR 步骤（数据组织/查询重写扩展/重排序）；改进结合步骤（压缩/迭代检索）
查询表示的两种改进	query expansion（扩展/分解）与 query re-writing（消歧/突出关键/多语言）
迭代检索的代表方向	与 CoT 结合、自适应决定何时检索、 Search-R1/Search-O1、agentic search

Lecture 13 —训练大语言模型的数据 (Data for Training LMs)

[TIP] 学习指南

本节核心：大语言模型 (LM) 的能力一半来自模型结构与算力，另一半——也是越来越被重视的一半——来自**预训练数据**。本讲不讲网络结构，专讲”喂给模型的文本从哪来、怎么清洗、清洗得好不好直接决定模型多强”。主线有两条：(1) **历史演进**——从 GPT-2 之前用 Wikipedia+ 书籍，到 GPT-2 的 WebText、Common Crawl、C4、GPT-3 的分类器过滤，再到 Pile / RefinedWeb / Dolma / FineWeb / DCLM 等开源数据集的”军备竞赛”；(2) **数据治理流水线**——任何预训练语料都要经过**获取 (Acquisition)** → **线性化 (Linearization)** → **过滤 (Filtering)** 三步，其中过滤又细分为去重、启发式过滤、模型法过滤。**前置知识**：- lec04_ngram_lm ——什么是 token、n-gram、语言模型；本讲的”数据量”以 token 计 - lec11_pretraining ——预训练-微调范式、什么叫”预训练语料”- lec11b_gpt_scaling ——**缩放律 (Scaling Laws)**：模型性能随参数量、数据量、算力幂律提升。本讲反复强调”数据规模和模型规模同等重要”正是缩放律的直接推论 **重点掌握**：1. 预训练数据演进的三个关键里程碑：WebText (GPT-2)、C4 (T5)、分类器过滤 (GPT-3) 各解决了什么问题 2. 数据治理三步流水线 Acquisition → Linearization → Filtering，每一步在做什么、为什么不能跳过 3. **线性化 (linearization)** 为何”看似平凡却极其重要”：WET vs WARC 的区别 4. 三类过滤的原理：去重 (精确 vs 模糊、MinHash / 后缀数组 / BFF)、启发式过滤 (T5 的规则)、模型法过滤 (DCLM 的 bigram 分类器、FineWeb 的 LLM 标注 + 蒸馏) 5. 期末高频考点：为什么标准做法是”先启发式过滤 → 再去重 → 最后模型法过滤”这个顺序 (见 §9 测验解析)
本讲属于偏”科普 + 工程”性质，数学公式不多，但概念密集；下面所有专业名词都会从零解释，无任何前置背景也可读懂。

0. 名词预热：读这篇笔记前需要知道的 5 个词

为了让”完全没接触过大规模数据工程”的读者也能顺畅阅读，先把全篇反复出现的基础名词一次讲清。

名词	一句话解释
Token (词元)	模型处理文本的最小单位，约等于”半个英文单词”或”一个汉字/词”。数据规模用 token 数衡量，如”2 万亿 (2T) tokens”。 $1T = 10^{12}$ 。
语料 / Corpus	一大堆用来训练模型的文本集合。“预训练语料”就是预训练阶段喂进去的全部文本。
Common Crawl (CC)	一个非营利组织，每月免费抓取整个互联网网页并公开，约 20TB/月的纯文本规模。是几乎所有大模型数据的”原料矿”。
网页抓取 / Scraping 去重 / Deduplication	用程序自动把网页内容抓下来。抓下来的是带 HTML 标签的”脏”数据。把语料里重复或近似重复的文本删掉，避免模型反复看同一句话。

[NOTE] 为什么数据是“大模型的石油”？

把 LM 想象成一个只会“接下一个词”的超级补全器（见 lec11b）。它学到的所有知识、语感、推理模式，**全部**来自它在预训练时读过的文本。文本质量差（充斥广告、乱码、重复），模型就学坏；文本覆盖广、干净、知识密度高，模型就强。所以“数据工程”在今天和“模型结构设计”同等重要——甚至更重要，因为结构早已趋同（都是 Transformer），数据才是各家拉开差距的秘密武器。

1. Motivation：为什么单独花一讲讲数据？

1.1 数据是行业最高机密

一个反常的现象：模型权重越来越开放，但训练数据越来越保密。

[EX] Llama 2 技术报告的“挤牙膏”式披露（2023）

> “我们的训练语料包含来自公开可用来源的新混合数据，不包含 Meta 产品或服务的数据。我们努力移除了已知含大量个人隐私信息的网站的数据。我们在 2 万亿 (2T) tokens 上训练，因为这能取得性能-成本的良好折中，并对最具事实性的来源做了上采样 (up-sampling)，以增强知识、抑制幻觉。”

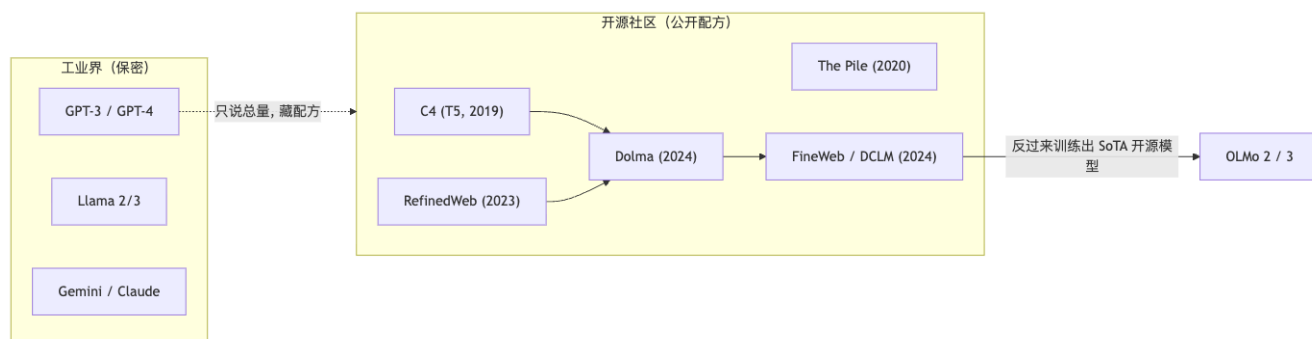
注意它说了什么、没说什么：给了总量 (2T) 和模糊原则（上采样事实性来源），但**具体是哪些网站、各占多少比例、怎么清洗的，一概不说**。这就是“open-weight（开权重）但不 open-data（不开数据）”的典型。

到了 Llama 3、Mistral、Gemma 时代，连这点都不说了。课件原话：

“即便对 Llama、Mistral、Gemma 这类开权重模型，关于训练集的细节也越来越罕见。”“Llama 3 和 Mixtral 这类 SoTA 开源 LLM 的预训练数据集并不公开，关于它们如何构建知之甚少。”

1.2 于是有了开源社区的“逆向复刻”运动

既然工业界藏着，学术界和开源社区就自己动手，公开“配方”。本讲后半段的 Dolma、FineWeb、DCLM 正是这场运动的代表作——它们把“如何从 Common Crawl 造出高质量数据”的每个细节、每个消融实验都写进论文，供全世界复现。



2. 预训练数据的演进史

按时间线梳理，每个阶段都在解决上一阶段暴露的问题。

2.1 GPT-2 之前（≤2018）：精选小语料

早期预训练只用人工精选、相对干净的小语料：

模型	数据	规模
ELMo	1B Word Benchmark	约 10 亿词
GPT-1	BooksCorpus	8 亿词
BERT	BooksCorpus + 英文 Wikipedia	8 亿 + 25 亿词

特点：单一领域、规模小、干净。问题：覆盖面窄，学不到多样化的”任务示范”。

2.2 GPT-2: WebText ——用 Reddit 点赞当质量信号

GPT-2 的核心洞察（课件原话）：

“此前大多工作只在单一领域的文本上训练……我们要构建一个尽可能大且多样的数据集，以收集尽可能多领域、多语境下的自然语言任务示范。”

WebText 的构造技巧：– 抓取网页，但只抓 Reddit 上被外链且获得 ≥ 3 个赞 (karma) 的页面——用”人类点赞”作为廉价的质量代理信号 – 从 HTML 中抽取正文（课件用 ***** 强调这步很难，见 §6 线性化）– 去重 + 启发式清洗 – 最终 800 万 + 文档 – OpenAI 没放出原始数据，但社区复刻了开源版 **OpenWebText (2019)**

[NOTE] 关键思想：用”人类行为”代替”人工标注”

直接雇人标注哪些网页质量高，太贵。WebText 用”Reddit 点赞数 ≥ 3 ”这个免费、规模化的信号近似”这页内容值得一看”。这是”用代理信号做质量过滤”思想的开端，后面 GPT-3、DCLM 的分类器过滤都是它的延续。

2.3 Common Crawl 登场：无限量但极脏

要再往上扩规模，精选语料不够用了，人们盯上了 **Common Crawl (CC)**：

“网络抓取（如 Common Crawl）是多样且近乎无限的文本来源。”——但**质量是大问题**：– “Common Crawl 的内容大多不可读。” (Trinh & Le, 2018) – “它大量由乱码或样板文字组成，如菜单、报错信息、重复文本。” (T5) – “大部分对我们关心的任务无用（脏话、占位文本等）。” (T5)

于是核心矛盾出现：**CC 量大但脏 → 必须重度清洗**。后续所有数据集本质都在回答”怎么把 CC 洗干净”。

2.4 C4: 第一个开源的 CC 清洗方案 (T5, 2019)

C4 = Colossal Clean Crawled Corpus (巨型清洁爬取语料)，T5 论文提出，用一套纯启发式规则清洗 CC：

[EX] C4 的启发式清洗规则（经典，需记忆）

1. 只保留以句末标点（句号、问号等）结尾的行——滤掉残缺片段 2. 丢弃少于 3 句话的页面；只保留 ≥ 5 个词的行 3. 移除含”脏话黑名单 (List of Dirty, Naughty, Obscene or Otherwise Bad Words)”的页面 4. 含”JavaScript”（提示”请启用 JS”的报错）的行 → 删 5. 含”lorem ipsum”（排版占位文）的页面 → 删 6. 含花括号 {（编程语言常见、自然语言罕见）的页面 → 删 7. 删除 Wikipedia 来源的引用标记 [1] [2] 8. 删除含”terms of use / privacy policy / cookie policy / uses cookies ...”的样板行 9. **去重**：任何出现超过一次的”连续 3 句话片段”，只保留一份 10. 用 langdetect 过滤掉非英语页面

结果：**过滤掉了 90% 的数据**，剩下约 175B tokens。课件评价：

“不仅比多数预训练数据集大几个数量级，而且是相当干净、自然的英文文本。”

[WARN] 记住这个 90%

C4 的启发式规则把原始 CC 砍掉了九成。这反复印证一个事实：**Common Crawl 绝大部分是垃圾**，高质量预训练数据的本质是”大规模筛选”而非”大规模收集”。

2.5 GPT-3：从“写规则”升级到“训分类器”

GPT-3 不再纯靠手写规则，而是训练一个分类器来打质量分：

[EX] GPT-3 的分类器过滤（模型法过滤的雏形）

– 训练一个逻辑回归 (logistic regression) 分类器区分高质量 vs 低质量 – 正例：WebText、Wikipedia、若干书籍语料（公认高质量）– 负例：未过滤的原始 Common Crawl – 阈值：在“质量分”和“数据量”间权衡——主要保留高分文档，但仍故意混入一些低分文档（保持多样性、避免分类器自身偏见放大）– 模糊去重 (Fuzzy Deduplication)：移除与其他文档高度重叠的文档 – 再掺入已知高质量参考语料：扩展版 WebText、两个书籍语料 (Books1/Books2)、英文 Wikipedia

采样策略（重要细节）：过滤后约 400B tokens，但不是均匀采样——

来源	采样频率
高质量来源 (Wikipedia 等)	训练中被多次采样 (2–3 次)
Common Crawl、Books2	少于一次 (即只用一部分)

[NOTE] “上采样”是什么意思？

同一份高质量数据在一个训练周期里被重复喂多次叫上采样 (up-sampling)；低质量数据只喂一部分甚至不重复，叫下采样。这样在不改变总 token 预算的前提下，提高了模型“接触优质内容”的比例。Llama 2 报告里“对最具事实性来源上采样”说的就是这个。

3. 开源预训练数据集全景

GPT-3 之后，开源社区造出了一大批数据集，规模从百亿冲到 30 万亿 tokens。下面两张表是课件核心（需对“代表数据集 → 代表模型 → 规模 → 来源”的对应关系有印象）。

3.1 第一波（2019–2023）

数据集（发布）	代表模型	Tokens	主要来源
C4 (2019.10)	T5, FLAN-T5	175B	Common Crawl
The Pile (2020.12)	GPT-J, GPT-NeoX, Pythia	387B	CC, arXiv, PubMed, Books3, Gutenberg, Wikipedia...
The Stack v1 (2022.11)	StarCoder	200B	Software Heritage (代码)
RedPajama v1 (2023.04)	INCITE	1.2T	CC, C4, GitHub, arXiv, Gutenberg, Books3, Wikipedia, StackExchange
RefinedWeb (2023.06)	Falcon	580B*	Common Crawl
Dolma (2023.08)	OLMo	3.1T	CC, C4, Semantic Scholar, Reddit, Gutenberg, the Stack, Wikipedia...

两个“第一次”里程碑（课件标注）：– C4：第一个基于 CC 的开源预训练数据集 – The Pile：第一个规模匹配 GPT-3 训练数据的开源数据集；其卖点是多样性 (Diverse) (Gao et al. 2020)，混入 arXiv 论文、PubMed 医学、代码、书籍等多领域

3.2 第二波（2023–2024）：冲向 10T+

数据集（发布）	代表模型	Tokens	主要来源
OpenWebMath (2023.10)	Llama	15B	CC（数学）
RedPajama v2 (2023.10)	—	30T	Common Crawl
Amber (2023.12)	Amber	1.3T	C4, RefinedWeb, the Stack, RedPajama v1
Dolma 1.7 (2024.04)	OLMo 0424	2.3T	Dolma, RefinedWeb, StackExchange, Flan, OpenWebMath...
FineWeb (2024.05)	—	15T	Common Crawl
Matrix (2024.05)	MAP-Neo	4.7T	RedPajama v2, Dolma, CulturaX, Amber, Falcon, 爬取的中文网页
DCLM (2024.06)	DCLM-Baseline	4T	Common Crawl

课件结论：许多开源数据集已超过 1T tokens。趋势是“领域专门化”（数学、代码、中文各有专门集）+ “层层叠加”（新数据集复用旧数据集）。

3.3 三个标杆数据集

- **LLaMA（数据观）**：“大部分情况下我们复用其他 LLM 用过的数据源，但只用公开可得且兼容开源的数据。”(Touvron et al. 2023) ——强调合规。
- **Dolma**：“3 万亿 tokens 的开放预训练研究语料”(Soldaini et al. 2024)，获 **ACL 2024 最佳资源论文奖**。
- **DCLM & FineWeb**：当前开源数据工程的最前沿，专注“只用 Common Crawl 能榨出多强的模型”(详见 §5–§8)。

[EX] DCLM 的标志性成果

DCLM 只用 Common Crawl + 大量消融实验，做出 7B 模型，在“训练数据公开”的模型里达到 SoTA。一句口号总结全讲精神：“更少但更高质量的数据，反而性能更好。”(Less data with higher quality, better performance.)

4. 数据规模 vs 模型规模：缩放律的另一半

课件用一句话点题：“扩大数据规模和扩大模型规模同等重要（缩放律）。”

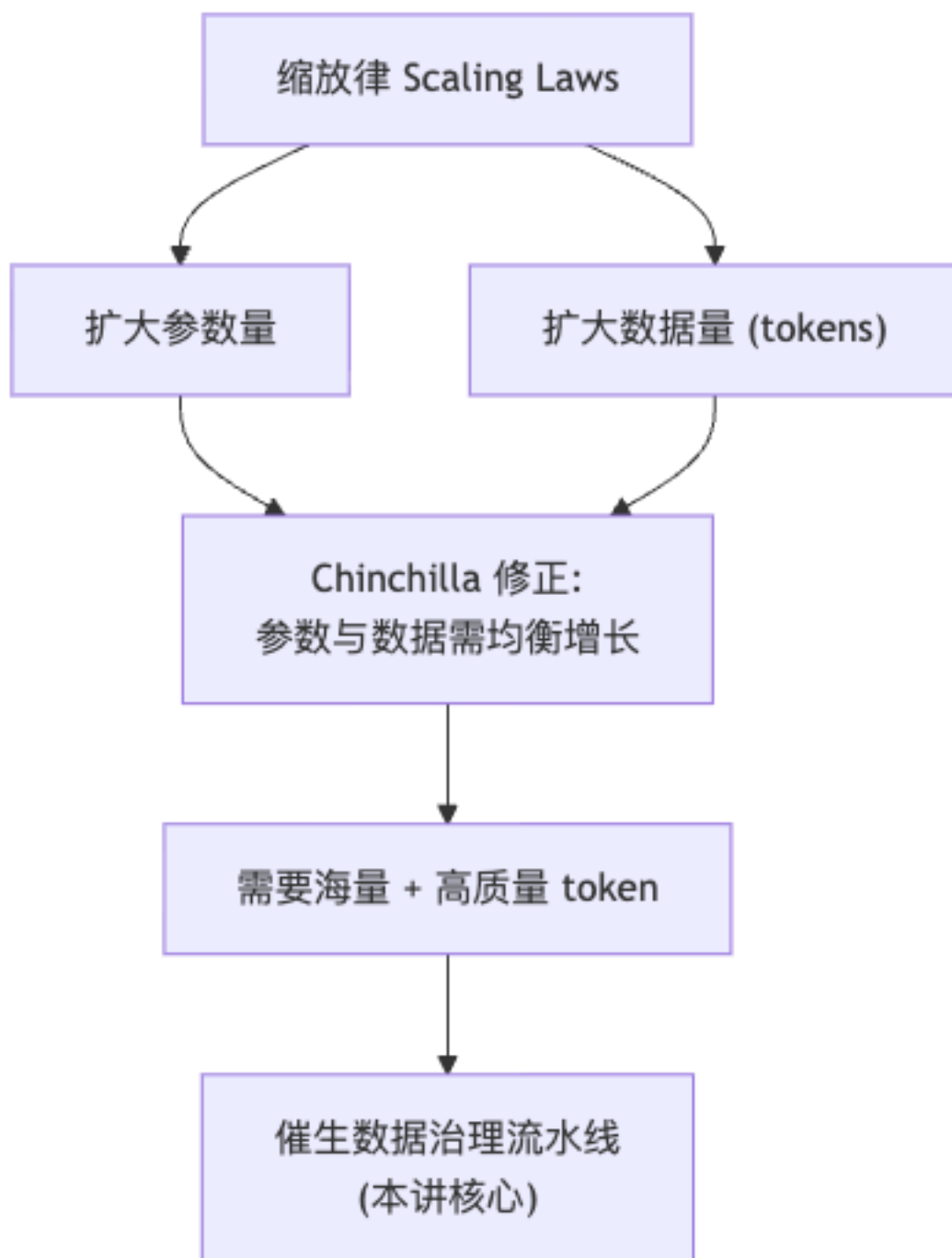
下表是历代模型的“参数量 vs 训练 token 量”对照（注意趋势）：

年份	2020	2021	2022	2023	2024	2025	2025
模型	GPT-3	Gopher	Chinchilla	LLaMA1	Llama3	Qwen3	OLMo2
参数	175B	280B	70B	65B	70B	235B	32B
Tokens	240B	300B	1.4T	1.4T	15T	36T	6T

[NOTE] 从这张表读出的两个趋势

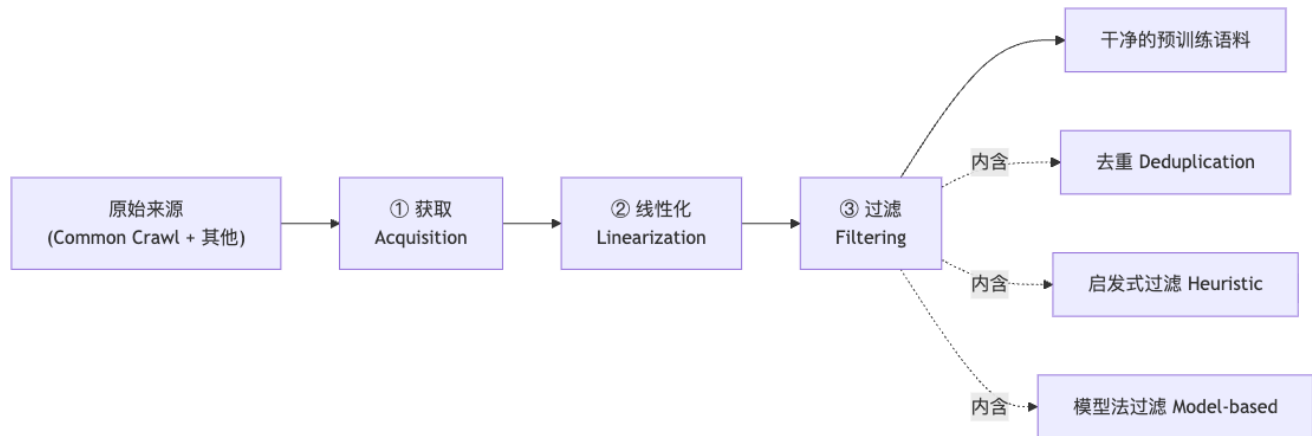
1. **token 量爆炸式增长**：从 GPT-3 的 0.24T 涨到 Qwen3 的 36T，5 年涨了 ~150 倍。2. **参数量不再盲目变大**：Chinchilla (2022) 证明此前模型“参数过大、数据不足”，提出在固定算力下应让 **token 数随参数数同步增长**（著名的 Chinchilla 最优：约每参数配 20 tokens）。之后大家转向“中等参数 + 海量数据”，OLMo2 仅 32B 参数却用 6T tokens 即是例证。

这就是为什么“数据工程”成了独立学问：没有足够多、足够好的 token，再大的模型也喂不饱。



5. 数据治理流水线：三大步骤总览

任何预训练语料的构建都可归结为三步，这是全讲的骨架，务必记牢：



步骤	做什么	一句话
① Acquisition 获取	决定”从哪些来源拿数据”	Common Crawl + 额外公开来源
② Linearization 线性化	把各种格式（HTML/PDF/Word）转成纯文本	“格式 → 纯文本”，看似平凡实则关键
③ Filtering 过滤	删掉低质、重复、有害内容	去重 + 启发式 + 模型法

下面逐步展开。

6. 步骤 ①：获取 (Acquisition)

6.1 为什么不能只用 Common Crawl?

CC 虽大，但有结构性缺陷（课件三点）：1. 覆盖问题：某些语言、领域在 CC 里严重不足 2. 高质量来源值得重复：像 Wikipedia 这种可靠来源，最好复制多份（上采样）3. 很多数据不在 CC 里：非 HTML 内容（Word、PDF、扫描件）CC 抓不到

所以实践中是 Common Crawl 打底 + 额外公开来源补充。

6.2 典型补充来源

来源类型	为什么要	代表
百科	开源、免费、专家/社区编辑、权威可靠、易获取	Wikipedia
书籍	质量极高、长文本、覆盖面广、教育性强（传记、教科书）	Project Gutenberg（最早的数字图书馆）、BookCorpus、Books1/2/3（已不可访问）
学术内容	高度专业、有学术严谨性	arXiv（LaTeX 格式）、S2ORC、PubMed
代码	对编程任务至关重要	The Stack、GitHub、BigQuery、StarCoder

来源类型	为什么要	代表
数学	对数学任务至关重要	OpenMathText、FineMath

[NOTE] 为什么书籍和论文这么宝贵？

网页文本多是短、碎、口语化的；而书籍/论文是长篇、结构化、逻辑连贯、知识密度高的。模型从中能学到长期依赖、严谨论证、专业知识——这是从网页垃圾里学不到的。代码和数学则分别对应模型的“编程”和“推理”能力，是专门补强的。

6.3 版权问题（不可回避）

课件专门列出版权争议：– Stack Overflow 宣布向 AI 巨头收费才能用其数据训练 – Books3（含大量受版权保护书籍）因侵权争议已被下架、不可访问 – 这也是 LLaMA 强调“只用公开可得且兼容开源的数据”的原因

[WARN] 合规是数据获取的硬约束

“技术上能抓”≠“法律上能用”。版权、隐私（个人信息）、服务条款都会限制可用数据。这也是工业界对数据来源讳莫如深的原因之一——怕引来诉讼。

7. 步骤 ②：线性化 (Linearization)

7.1 是什么

线性化 = 把任意格式的文档（HTML、PDF、Word...）转换成“纯文本 (plain text)“。

$$\text{HTML / PDF / Word / ...} \xrightarrow{\text{linearization}} \text{Plain Text} \quad (7.1)$$

听起来像“复制粘贴”一样简单，课件却反复强调 “It really Matters（这真的很关键）”、“Important”。

7.2 为什么“看似平凡却极其重要”

[WARN] 不同工具抽出的纯文本差异巨大

同一个 HTML 网页，用不同的正文抽取工具（boilerplate removal tool）处理，得到的纯文本可能天差地别——有的把导航栏、广告、页脚一起抽进来，有的把正文里的表格、公式弄丢。课件原话：“**Variability in plain text output across different tools（不同工具的纯文本输出差异性）**。”这种差异会直接传导到模型质量：抽得脏，模型学到一堆 HTML 残渣；抽得丢三落四，知识就缺失。

7.3 Common Crawl 的两种格式：WET vs WARC（高频考点）

Common Crawl 提供两种文件格式，这是本节最该记的知识点：

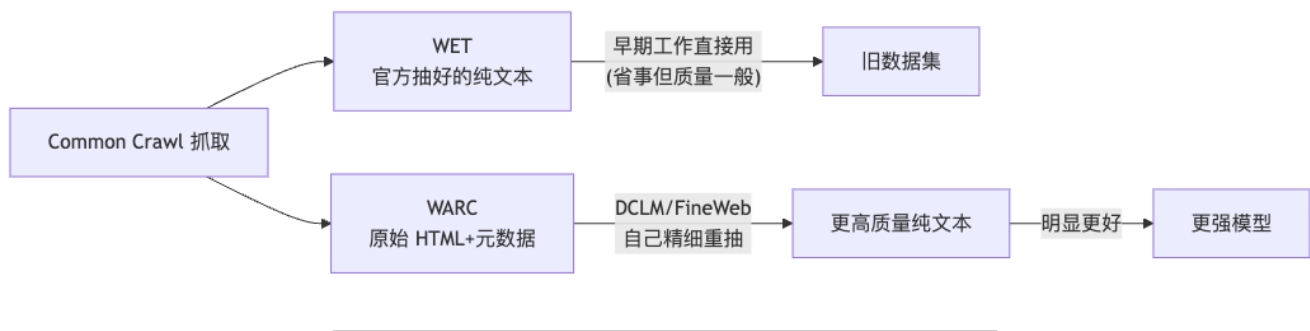
格式	全称	内容	谁用
WET	WARC Encapsulated Text	已经抽好的纯文本	多数早期工作直接拿 WET 当起点（省事）
WARC	Web ARChive	原始抓取数据（完整 HTML + 元数据）	DCLM、FineWeb 自己从 WARC 重新做线性化

关键结论 (DCLM / FineWeb 的发现):

DCLM 和 FineWeb 不用 CC 官方提供的 WET 纯文本, 而是从原始 WARC 自己重新抽正文, 效果明显更好。

[NOTE] 为什么自己抽更好?

CC 官方的 WET 是用通用工具批量抽的, 质量一般 (丢格式、留噪声)。FineWeb/DCLM 用更精细的正文抽取 (如 trafilatura 等) 从原始 HTML(WARC) 重抽, 能更好地保留正文、去掉导航/广告样板。这就是“线性化决定上限”的实证——同样的原始网页, 光改一个抽取步骤, 最终模型质量就有可观差距。这也是为什么课件把这步单拎出来强调。

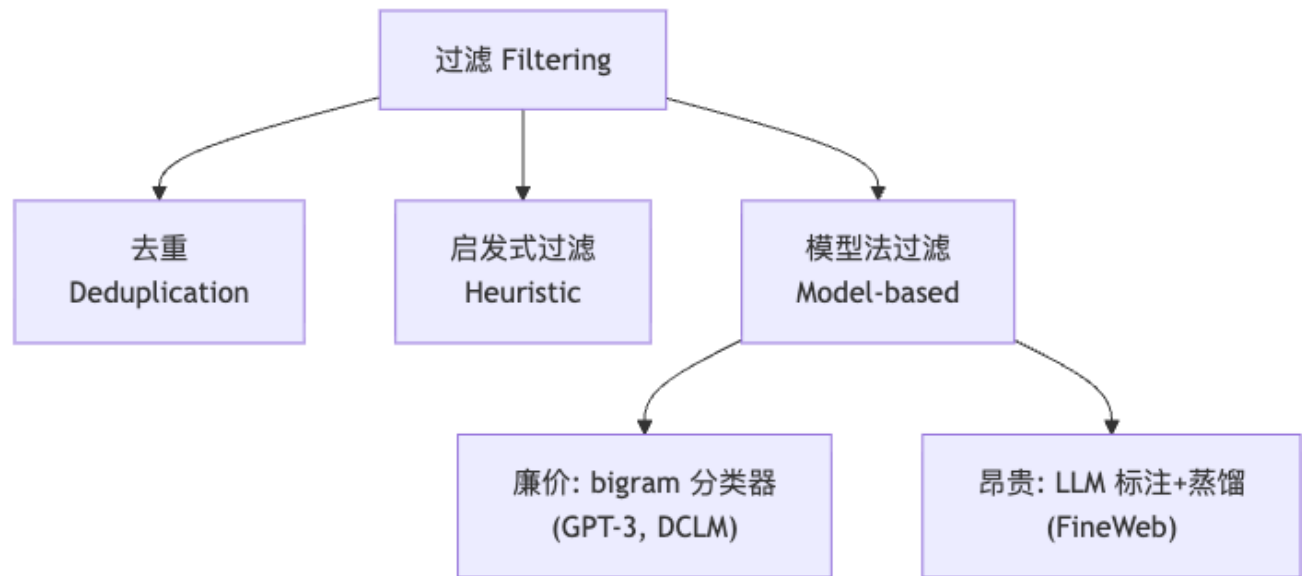


8. 步骤 ③: 过滤 (Filtering)

过滤是数据治理里最复杂、投入研究最多的一步。要过滤掉的东西包括: 语言 (非目标语言)、垃圾 (junk)、不良内容、低质量、重复等。

三大权威开源参考 (课件点名): – Dolma (Soldaini et al., ACL 2024) – FineWeb (Penedo et al., 2024) “为大规模获取最优质文本而蒸馏网络” – DCLM / DataComp-LM (Li et al., NeurIPS 2024) “寻找下一代训练集”

过滤分三类手段, 下面逐一展开:



8.1 去重 (Deduplication)

问题: Common Crawl 里充斥重复或近似重复的文本 (同一篇文章被多个网站转载、模板化页面等)。重复数据会让模型反复“背诵”, 浪费算力、损害泛化、还可能加剧记忆隐私风险。

标准做法：模糊去重 (Fuzzy Deduplication)，而非精确去重 (Exact Deduplication)。

[NOTE] 精确去重 vs 模糊去重

- 精确去重：只删完全一模一样的文档。但现实中“几乎一样、改了一个标点”的文档极多，精确去重抓不住。
- 模糊去重：删“足够相似”的文档。例如课件给的判据：两个文档若有 50% 的 13-grams（连续 13 个词的片段）重叠，就视为重复，删掉一个。

[EX] n-gram 重叠是怎么算的（给零基础读者）

“13-gram”就是文本里连续 13 个词组成的片段。把文档 A、B 各自切成所有 13-gram 的集合，看两个集合的交集占比。如果超过 50%，说明两篇文章大段雷同 → 判为重复。直接两两比较太慢，所以用 MinHash 等技巧快速估计集合相似度（见下）。

去重为什么“极其困难”（课件强调）：- DCLM 用了 8 页消融实验，FineWeb 用了 7 页专门讨论去重——足见其复杂。- 影响因素众多：

因素	选择
超参	n-gram 重叠百分比阈值定多少？
数据结构 / 算法	MinHash vs. 后缀数组 (Suffix Array) vs. 近似重复 Bloom 过滤 (BFF)
粒度	段落级？文档级？
比较范围	文档 vs 文档？还是文档 vs 整个语料？

[NOTE] 三种去重算法的直觉（自包含解释）

- MinHash：一种快速估计两个集合 Jaccard 相似度的技巧——对每个文档的 n-gram 集合取多个哈希的最小值组成“签名”，签名相同的比例 $\approx$ 真实相似度。让“两两比相似度”从 $O(N^2)$ 降到近线性。
- 后缀数组 (Suffix Array)：把全部文本拼成一个大字符串并对所有后缀排序，能高效找出语料中所有“重复出现的长子串”，适合精确长片段去重。
- BFF (near-duplicate Bloom Filtering)：用 Bloom 过滤器（一种省内存的概率型集合）记录见过的 n-gram，新文档若大量 n-gram 已见过即判重复。省内存、适合超大规模流式处理。

8.2 启发式过滤 (Heuristic Filtering)

就是 §2.4 讲过的 C4/T5 那套手写规则的延续：- 长度过滤：太短的行/页删掉 - 语言过滤：用语言识别器只留目标语言 - + 海量变体：各家在 T5 规则上加自己的规则（符号比例、重复行比例、平均行长等）

特点：廉价、可解释、规则化，但天花板有限——规则永远写不全，抓不住“语义上低质”的内容（语法通顺但毫无信息量的废话）。

8.3 模型法过滤 (Model-based Filtering)

用一个模型来判断“这篇文档质量高不高”。按成本从低到高：

(a) 廉价：Bigram 分类器 (GPT-3、DCLM)

[EX] DCLM 的 bigram 质量分类器

- 训练一个基于 bigram (二元词组) 特征的轻量分类器当“质量过滤器”- 正例 (高质量)：Wikipedia、OpenWebText2，外加指令格式数据 (OpenHermes 2.5) 和 ELI5 子版块高赞帖 - 负例 (低质量)：从语料里随机抽样 - GPT-3 的做法类似：正例用 Wikipedia、OpenWebText2、书籍思想和 GPT-3 的逻辑回归过滤一脉相承：“长得像 Wikipedia/优质论坛的就留下”。bigram 特征 + 简单分类器，跑得飞快，能扫海量数据。

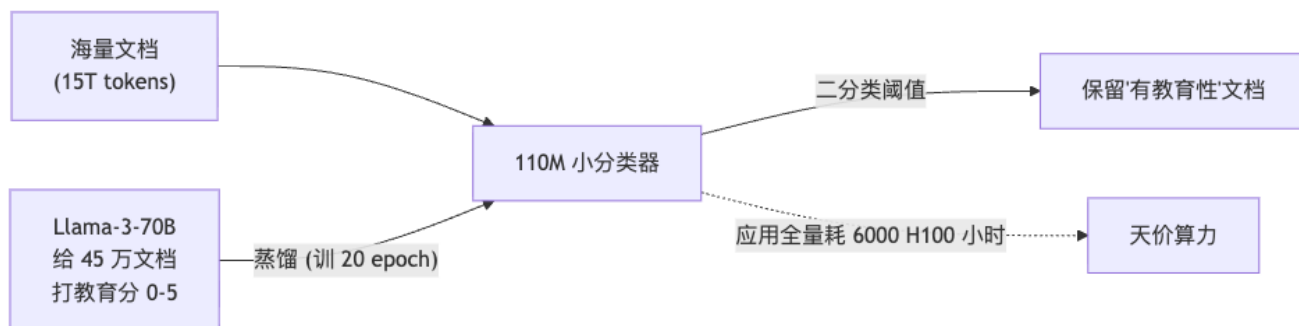
(b) 昂贵：LLM 标注 + 蒸馏 (FineWeb-Edu)

[EX] FineWeb 的“教育性”分类器（三步，需记忆）

核心思想：让一个强 LLM 标注“哪些文档有教育价值”，再把它判断蒸馏进一个小分类器，从而能廉价地扫全量数据。1. 第 1 步（标注）：提示 Llama-3-70B-Instruct 给文档的“教育性内容”打分，0—5 分。2. 第 2 步（蒸馏训练）：用这 45 万条 Llama-3 标注，微调一个 1.1 亿 (110M) 参数的 encoder-only Transformer，训 20 个 epoch。3. 第 3 步（应用）：把分数四舍五入到 0—5，设固定阈值转成二分类（是否“有教育性”），用这个小分类器去过滤全量数据。4. 成本：把分类器应用到 15T tokens 花了 6000 H100 GPU 小时——“光”过滤这一步就是天价算力。

[NOTE] 为什么不直接用大 LLM 扫全量？

让 70B 大模型逐篇给 15T tokens 打分，算力成本无法承受。所以走“大模型标注一小批 → 蒸馏成小模型 → 小模型扫全量”的范式：用昂贵模型造“金标准”，用廉价模型规模化执行。这是“模型法过滤”的精髓。



9. 测验解析：三类过滤的执行顺序（高频考点）

[EX] 课件原题（Quiz）

“实践中，多数工作先做启发式过滤，再去重，最后做模型法过滤。为什么遵循这个顺序？” (In practice, most work does heuristic filtering, then deduplication, then model-based filtering. Why do they follow this order?)

参考解析（从“成本”和“效果”两个角度）：

启发式过滤 → 去重 → 模型法过滤 (9.1)

1. 启发式过滤放最前：它最廉价（纯规则、 $O(n)$ 扫一遍），能先砍掉海量明显的垃圾（乱码、报错页、过短页）。先用便宜手段把数据量降下来，后面更贵的步骤要处理的数据就少了——省算力。
2. 去重放中间：在垃圾已去除后再去重，
 - 一方面去重本身比模型法便宜，应在昂贵步骤之前缩减数据；
 - 另一方面，先去重能避免“重复的低质内容污染模型法过滤器的统计/训练”，也避免对同一份重复内容反复跑昂贵的模型打分（省钱）。
3. 模型法过滤放最后：它最贵（要跑分类器甚至 LLM，如 FineWeb 的 6000 H100 小时）。理应放在数据量已被前两步大幅压缩之后，只对“已经比较干净、不重复”的数据做精细的质量判别，性价比最高。

[TIP] 一句话记忆

“先便宜后贵、先粗后精”：用廉价手段尽早削减数据量，把昂贵的模型法过滤留到最后只处理精简后的数据。顺序的本质是算力预算的最优分配。

10. 总结与展望

[TIP] 期末复习要点（本讲必记）

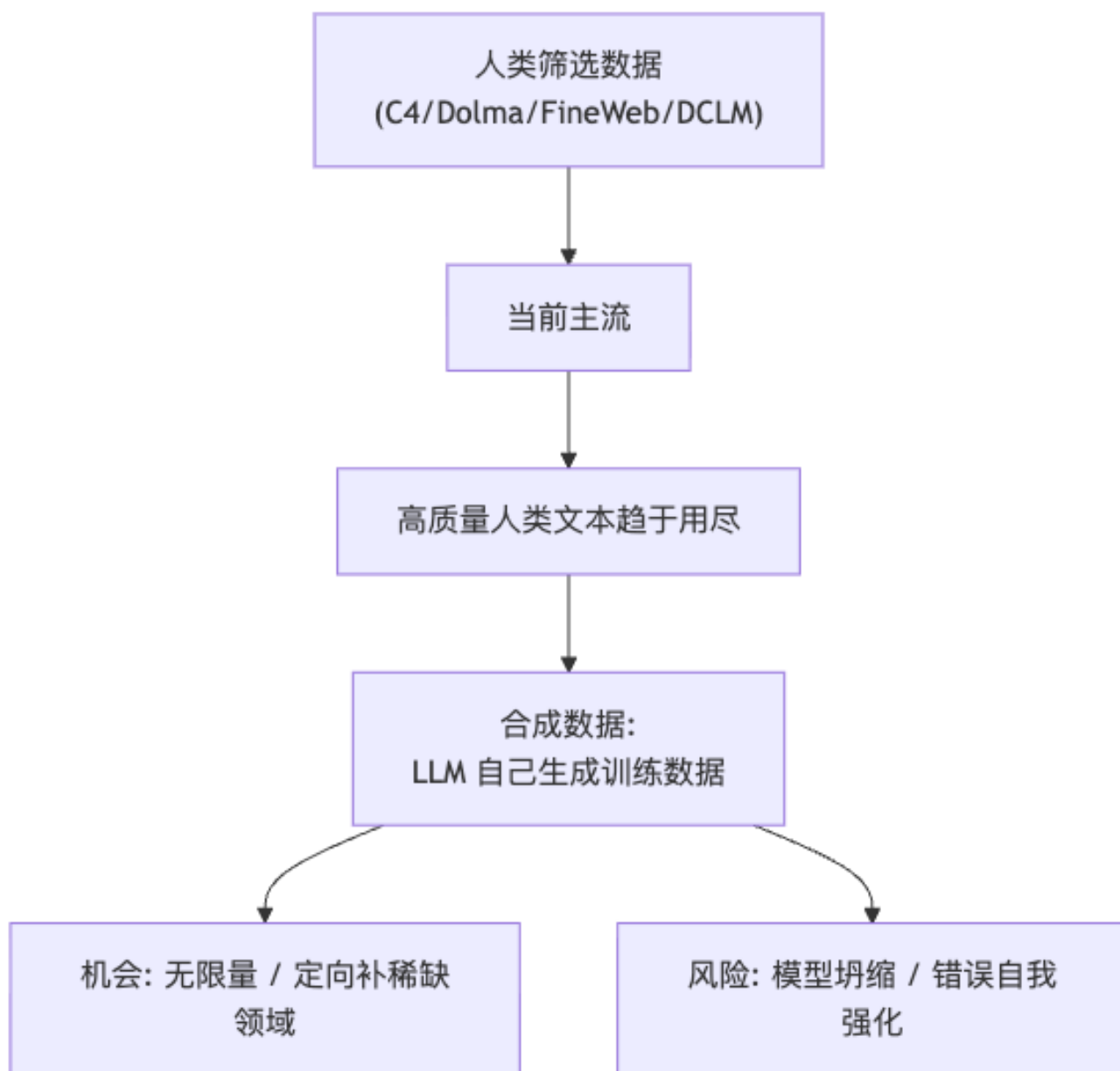
1. 数据 = 大模型的核心竞争力：规模 (scale) 与质量 (quality) 都极重要；工业界保密，开源社区逆向复刻。2. 演进主线：精选小语料 → WebText(Reddit 点赞过滤) → Common Crawl(量大但脏) → C4(启发式洗掉 90%) → GPT-3(分类器过滤 + 上采样) → Pile/Dolma/FineWeb/DCLM(开源军备竞赛, 已达 15T–30T tokens)。3. 三步流水线：Acquisition (获取) → Linearization (线性化) → Filtering (过滤)。4. 线性化看似平凡却关键：WET (官方纯文本) vs WARC (原始 HTML)；FineWeb/DCLM 从 WARC 自抽，效果更好。5. 过滤三类：去重 (模糊去重、50% 13-gram、MinHash/后缀数组/BFF)、启发式 (T5 规则)、模型法 (DCLM bigram 分类器；FineWeb 用 Llama-3 标注 + 蒸馏 110M 分类器，应用 15T 耗 6000 H100 小时)。6. 执行顺序：启发式 → 去重 → 模型法 (先便宜后贵、先粗后精)。7. 核心结论：“更少但更高质量的数据，反而性能更好” (DCLM)；“每一步都重要，连看似平凡的线性化和去重也是”。

课件原文总结要点：– 极其重要：规模与质量缺一不可 – 工业界因竞争原因披露极少 – 开源社区研究广泛且持续 – 每一步都重要，连看似平凡的（线性化、去重）也是 – 各项努力层层叠加：{C4, RefinedWeb...} → {DCLM, FineWeb} → {OLMo 2/3...}

展望——合成数据 (Synthetic Data)：

“超越人工筛选的数据：合成预训练数据，即用 LLM 生成新的训练数据。”

当高质量人类文本逐渐被“用尽”，下一个前沿是**让大模型自己生成训练数据**喂给下一代模型——这既是机会（无限量、可定向生成稀缺领域数据），也带来新挑战（模型坍塌 model collapse、错误自我强化等）。



11. 例题 (Worked Examples)

[EX] 例题 1: 判断过滤手段类别

问: 以下做法分别属于“去重 / 启发式 / 模型法”哪一类? (a) 删除所有含花括号 { 的页面; (b) 用 Llama-3 给文档打教育分再训小分类器; (c) 删除与其他文档 13-gram 重叠超 50% 的文档; (d) 用 langdetect 只留英语。
答: (a) 启发式; (b) 模型法; (c) 去重; (d) 启发式。记忆法: 写死的规则 = 启发式; 比相似度删重复 = 去重; 要“学/打分”= 模型法。

[EX] 例题 2：为什么 GPT-3 故意保留一些低分文档？

答：(1) 保持多样性，避免只剩“像 Wikipedia 的文本”导致风格单一；(2) 分类器本身有偏见（正例只来自几个高质量源），全盘相信会系统性丢掉某些有用但“不像正例”的内容；(3) 模型也需要见过一些“普通/低质”文本以具备鲁棒性。

[EX] 例题 3：WET 与 WARC 哪个更适合做高质量数据集？为什么？

答：WARC（原始 HTML+ 元数据）。因为可以用更精细的正文抽取工具自己做线性化，去掉导航/广告样板、更好保留正文；CC 官方的 WET 是通用工具批量抽的，质量一般。FineWeb/DCLM 实证：从 WARC 自抽比用 WET 明显更好。

[EX] 例题 4（估算题）：FineWeb 教育分类器的成本意味着什么？

问：应用 110M 分类器到 15T tokens 花 6000 H100 GPU 小时，说明了什么？答：即便是“廉价”的小分类器，在 15T tokens 这种规模上做一次全量推理也要天价算力（6000 H100 小时）。这解释了为什么必须走“大模型标注小批 → 蒸馏小模型 → 小模型扫全量”，更解释了为什么“模型法过滤要放在流水线最后”——必须先启用启发式 + 去重把数据量压下来。

12. Anki 记忆卡片

以下为 Q/A 形式，可直接导入 Anki（正面 Q，背面 A）。

#	Q（正面）	A（背面）
1	预训练数据治理的三大步骤？	获取 Acquisition → 线性化 Linearization → 过滤 Filtering
2	C4 是什么，洗掉了多少数据？	Colossal Clean Crawled Corpus (T5)，用启发式规则过滤掉约 90% 的 Common Crawl，剩 175B tokens
3	GPT-2 的 WebText 用什么信号过滤网页？	Reddit 外链且点赞 ≥3 的页面（用人类点赞当质量代理）
4	GPT-3 如何过滤 Common Crawl？	训练逻辑回归分类器（正例 = WebText/Wikipedia/书籍，负例 = 原始 CC），按阈值保留，并对高质量源上采样
5	WET 与 WARC 区别？谁更好？	WET = 官方抽好的纯文本；WARC = 原始 HTML+ 元数据。FineWeb/DCLM 从 WARC 自抽，效果更好
6	过滤的三类手段？	去重 Deduplication、启发式 Heuristic、模型法 Model-based
7	标准去重是精确还是模糊？给个判据	模糊去重 (Fuzzy)；如 13-gram 重叠 >50% 即判重复
8	三种去重算法/结构？	MinHash、后缀数组 (Suffix Array)、近似重复 Bloom 过滤 (BFF)
9	FineWeb 教育分类器三步？	① Llama-3-70B 给文档打教育分 0-5；② 用 45 万标注微调 110M encoder Transformer 20 epoch；③ 阈值转二分类扫全量（15T tokens 耗 6000 H100 小时）
10	三类过滤的执行顺序及原因？	启发式 → 去重 → 模型法；先便宜后贵、先粗后精，最优分配算力
11	“缩放律的另一半”指什么？	数据规模和模型规模同等重要；Chinchilla 指出参数与 token 需均衡增长
12	数据工程的下一个前沿？	合成数据（LLM 生成训练数据），风险是模型坍塌

#	Q (正面)	A (背面)
13	为什么不能只用 Common Crawl?	覆盖不足 (语言/领域)、高质量源需重复、非 HTML (PDF/Word) 抓不到
14	DCLM 的一句话结论?	“更少但更高质量的数据, 反而性能更好”

13. 记号与术语速查

术语	English	含义
词元	Token	模型处理文本的最小单位; 数据量以 token 计 ($1T=10^{12}$)
预训练语料	Pre-training Corpus	预训练阶段喂给模型的全部文本
通用爬虫	Common Crawl (CC)	每月公开抓取全网的非营利项目, 约 20TB/月
网页抓取	Web Scraping	程序自动抓取网页内容 (带 HTML)
线性化	Linearization	把 HTML/PDF/Word 等转成纯文本
WET / WARC	—	CC 的两种格式: 纯文本 / 原始 HTML+ 元数据
去重	Deduplication	删除重复或近似重复文档
模糊去重	Fuzzy Deduplication	按相似度 (如 n-gram 重叠) 删近似重复
n 元组	n-gram	连续 n 个词的片段
MinHash	MinHash	快速估计集合相似度的哈希技巧
后缀数组	Suffix Array	高效查找重复长子串的数据结构
BFF 启发式过滤	Bloom Filter Dedup Heuristic Filtering	用 Bloom 过滤器做近似去重 手写规则过滤 (长度/语言/符号等)
模型法过滤 上采样	Model-based Filtering Up-sampling	用分类器/LLM 判断质量 同一份高质量数据重复多次喂入
缩放律	Scaling Laws	性能随参数/数据/算力幂律提升
合成数据	Synthetic Data	用 LLM 生成的训练数据
巨型清洁爬取语料	C4	T5 提出的启发式清洗 CC 数据集
教育性分类器	Educational Classifier	FineWeb 用 LLM 标注蒸馏出的质量分类器

[NOTE] 数据来源

本笔记整理自 FNLP 课程课件 “**Data for Training LMs**” (Yansong Feng, WICT @ PKU, 2026-05), 课件改编自 Princeton COS 484、UCB CS 288、Stanford CS224N、Im-class (Yoav Artzi) 等。涉及数据集与模型: C4/T5、The Pile、RefinedWeb、Dolma、FineWeb、DCLM、GPT-2/3、Llama 2/3、OLMo 等。